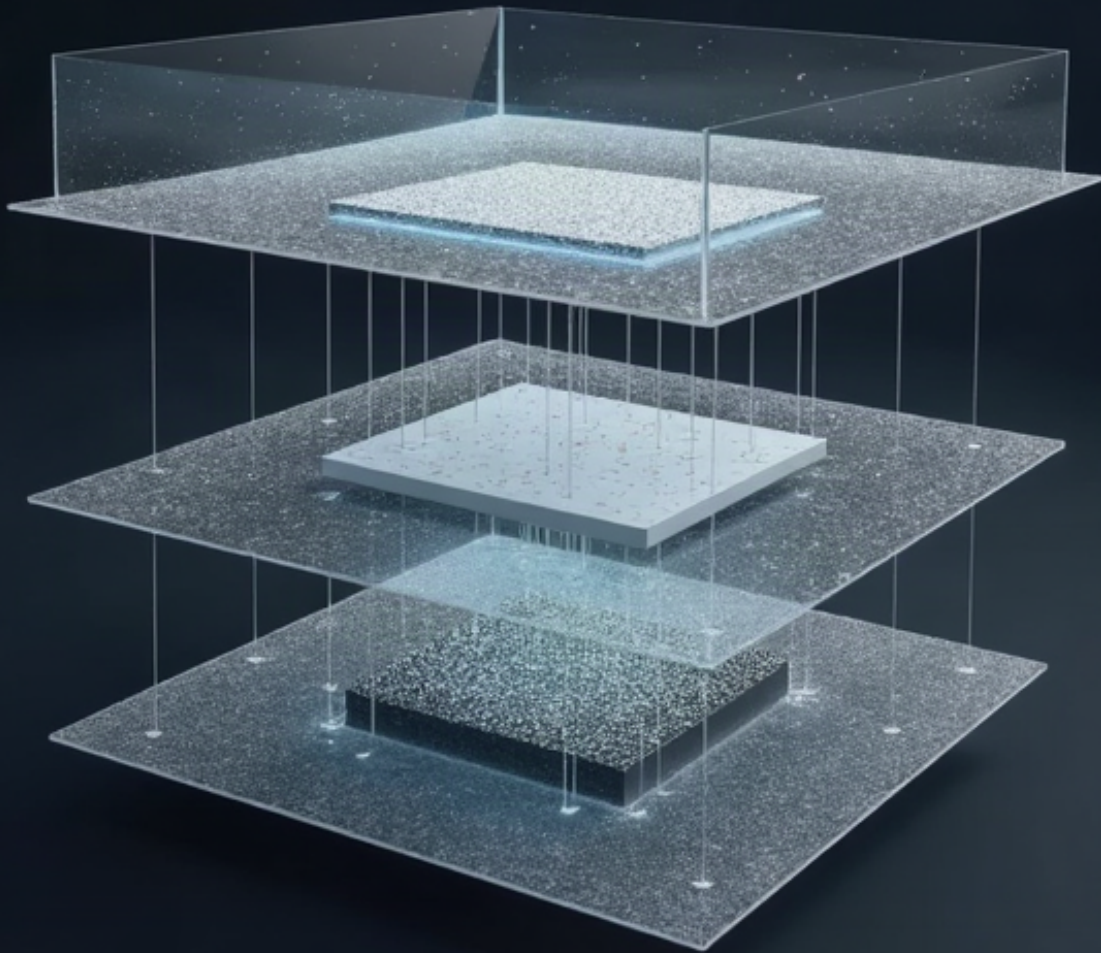


CLAD: Trust By Design

Governing Language Model Assertion
in Regulated Environments



David Callaghan

CLAD: Trust by Design

Governing Language Model Assertion in Regulated Enterprises

David Callaghan

`mr.david.callaghan@gmail.com`

ORCID: <https://orcid.org/0009-0006-4855-7774>

DOI: <https://doi.org/10.5281/zenodo.22729185>

September 12, 2026

Contents

I	The Governance Problem	1
1	Introduction	2
1.1	Premise: What Generation Does Not Establish	4
2	The Systems in Scope	8
2.1	What is in scope, and what is not	8
2.2	What the system computes	8
2.3	What the model returns is a distribution, not an answer	10
2.4	The pretraining objective	10
2.5	In-context learning, and why the prompt is the whole control surface	11
2.6	Instruction tuning and preference optimization	11
2.7	Three components that are absent	13
2.8	A floor, not merely a gap	13
2.9	What this predicts about better models	14
2.10	How this compares with other model families	14
II	What Goes Wrong	16
3	Hallucination as Ungrounded Assertion	17
3.1	Scope of the term	17
3.2	Assertion	18
3.3	Grounding	18
3.4	Why grounding needs a defeater condition	19
3.5	Five categories	20
3.6	The definition	22
3.7	What is claimed as new, and what is not	22
3.8	Error: a faithful report of a bad source is not a fabrication	23
3.9	Lucky: a right answer is not evidence about the system	24
3.10	Misapplied: authentic is not applicable	24
3.11	Contested: a contested record is not a grounded one	25
3.12	Aggregate Accuracy Does Not Identify Grounding	26
3.13	Grounding, Context, and Applicability	27
4	Form Invariance and Repeatability Both Fail	32
4.1	The map and what it does not respect	32
4.2	Every other component in the stack does better	32
4.3	What the measurements show	33
4.4	Why “train a normalizer” is not the answer	34
4.5	The validation consequence	34
4.6	The same failure, on a single input held fixed	35

4.7	Why this makes accuracy the wrong thing to certify	35
4.8	Why the two claims are one	36
III	What a System Must Supply	38
5	System Invariants	39
5.1	Behavioral invariants: what does not hold	39
5.2	System invariants: what must be supplied	40
5.3	Grounding Is Auditable; Accuracy Is Not	42
IV	The Target Architecture	46
6	A Case Traced End to End	47
7	Governed Retrieval: A Reference Architecture	49
7.1	The underspecified-request principle	49
7.2	Eight components	49
7.3	The flow	50
7.4	What each intervention buys	51
7.5	Structured evidence is the strongest case	52
7.6	What this costs	52
7.7	Retention against minimization	53
7.8	What this does not claim	53
7.9	Bounding the Request Space	54
8	Scale, Scope, and What Is Actually Tunable	58
8.1	What scale does, and what it cannot do	58
8.2	Reproducibility is a property of topology, not of size	58
8.3	Corpus scope changes what can be measured	59
8.4	What the dials actually buy	59
8.5	The conclusion that does not follow	60
V	Governance in Practice	62
9	Institutional Warrant and Tiered Reliance	63
9.1	Warrant	63
9.2	Reliance tiers	64
9.3	Three Teams, Three Budgets	65
10	An Evaluation Protocol	67
10.1	Methodological commitments	67
10.2	Attribution audit: the ungrounded assertion rate	68
10.3	Estimating evidence dependence, and why naive ablation fails	68
10.4	Abstention items: catching the lucky cell	69
10.5	Form-invariance audit: measuring I1 directly	70

10.6 Repeatability audit: measuring I7	70
10.7 What number is acceptable	71
VI The Control Model	73
11 The Architecture of Assurance	74
11.1 Axioms	75
11.2 Threat Model	79
11.3 Ontology (Primitive Sets and Pipeline)	82
11.4 Control Surface Decomposition	82
11.5 Governance Component Model	84
11.6 Enforcement Model	84
11.7 Failure Semantics	86
11.8 Interface Contracts	88
11.9 Audit Chain	88
11.10 Risk Model	90
11.11 Component Instantiation	92
11.12 Composition Algebra	93
11.13 Scope Limitations and Extension Points	93
11.14 Where the results are	94
11.15 Key takeaways	94
11.16 The Evidence Surface	95
VII The Control Layers	98
12 Responsible Prompting (EPG)	99
12.1 The Problem and the Approach	99
12.2 Principles	101
12.3 The Hierarchical Constraint Model	105
12.4 Constraint Authorship and Governance	114
12.5 Constraint Evaluability and Mandatory Decomposition	117
12.6 Conflict Resolution	121
12.7 Lateral Authority and Domain Isolation	123
12.8 EPG as a Governance Component	125
12.9 Operational Guidance	126
12.10 Key takeaways	130
13 Stopping Bad Outputs (ROC)	131
13.1 The Problem and the Approach	131
13.2 Principles	132
13.3 The Output Evaluation Model	133
13.4 Output Constraint Taxonomy	135
13.5 The Output Filtering Pipeline	137
13.6 Threat-Specific Controls	139
13.7 ROC as a Governance Component	142

13.8 Operational Guidance	143
13.9 Limitations	145
13.10 Key takeaways	146
14 Seeing the Whole Picture (MDR)	147
14.1 Purpose and Scope	147
14.2 Control Surfaces	148
14.3 Cross-Component Monitoring	149
14.4 Incident Detection and Classification	151
14.5 Incident Response Workflows	152
14.6 Integration Architecture	154
14.7 Operational Requirements	155
14.8 Limitations	157
14.9 Key takeaways	158
A Worked Examples	159
A.1 Prompt Governance (EPG)	159
A.2 Runtime Output Controls (ROC)	162
B Meeting Regulators	164
B.1 NIST AI Risk Management Framework (AI RMF 1.0)	164
B.2 EU AI Act (High-Risk AI Systems)	167
B.3 ISO/IEC 42001 (AI Management System)	168
B.4 HIPAA (Health Insurance Portability and Accountability Act)	168
B.5 SOX (Sarbanes-Oxley Act, Section 404)	170
B.6 GLBA (Gramm-Leach-Bliley Act)	170
B.7 NERC CIP (Critical Infrastructure Protection)	171
B.8 Cross-Framework Coverage Summary	172
B.9 Key takeaways	173
C A Validation Benchmark: Corpus and Case Design	174
C.1 Why a real corpus rather than a synthetic one	174
C.2 Sources	174
C.3 Case families	174
C.4 Configurations and measures	175
C.5 Reuse conditions	175
D Theorems, Lemmas, and Proofs	176
D.1 Syllogistic Arguments for Solution Validity	182
D.2 What must be true	183
D.3 Theorems stated with the requirements they justify	185
E The EPG and ROC Formal Models	188
E.1 Prompt Governance (EPG)	188
E.2 Runtime Output Controls (ROC)	193

F Templates & Classifier Specifications	197
F.1 EPG Constraint Library Template	197
F.2 ROC Output Classifier Specifications	199
Conclusion	201
Limitations and Threats to Validity	203
Related Work	205
Glossary	208
About the Author	214

Part I

The Governance Problem

1 Introduction

Consider a decision you have to be able to defend two years from now.

A clinical operations team asks a language model system whether a compound is approved for a paediatric indication. The system answers. Someone acts on the answer. Later—in an audit, an inspection, a dispute—the question arrives: what was that answer based on? Not whether the model is generally accurate. Which document, in which version, as of which date, under which jurisdiction, supported this specific claim.

Most current validation practice cannot answer that question, and the reason is not that the practice is careless. It is that the practice measures the wrong thing. The dominant approach evaluates a model by sampling questions with known answers and scoring the outputs against them—*reference-based evaluation*, in the terms of the natural-language-generation literature. It produces a number—accuracy, or its complement, often reported as a hallucination rate—and that number is then used as evidence of fitness for deployment.

The central argument of this book can be stated in one sentence. **In regulated settings, the relevant question is not merely whether an answer is accurate. It is whether the organization can reconstruct why this particular assertion was produced, identify the authoritative and versioned evidence on which it rests, establish that the evidence applies to the resolved case context, verify the asserted relation, and show that reliance was authorized under the institution’s governing rules.** Accuracy is one input to that question and settles none of it.

This book argues that reference-based evaluation cannot support the use it is put to, for two reasons that compound.

The first is a classification problem. Pooling every wrong answer into one metric conflates failures with different causes and different remedies. A system that faithfully reports an erroneous source has a data problem. A system that fabricates a plausible-looking figure has a generation problem. A system that correctly reports a regulation that was superseded last year, or that applies in a different jurisdiction, has a context-resolution problem. These are three separate defects, addressed by three separate teams, on three separate budgets. One number does not tell you which one to spend.

The second reason is worse, because it closes off the obvious remedy. The system applies no canonical form to its inputs. It computes a conditional distribution over token sequences given token sequences; its inputs are strings; and two semantically identical queries—a contraposed conditional, a quantity written in digits rather than words, an active rather than passive phrasing—are two distinct points in input space. Nothing in the architecture guarantees they map to the same output, and measurement confirms they frequently do not. A database normalizes before it compares. A compiler canonicalizes before it optimizes. This system does neither. Nor does the problem stop at paraphrase: the same input, re-asked in the same session, can produce a different assertion as context accumulates. So a passing test does not extend to materially identical requests, and does not securely cover even the string that was tested.

The consequence for validation is direct. If a test case passes, you have learned that one string produced an acceptable output on one occasion. You have not learned that the semantically identical string will, or that the same string will next week, or that the same string will later in the same conversation. A passing test does not generalize to the paraphrase, so the test suite does not certify the behavior it appears to certify. Accuracy is a property of a sample, and this

sample does not support the inference that validation requires of it.

That is a negative result, and on its own it would be unhelpful. The constructive claim is that a different property does not have this defect. *Grounding*—whether an assertion is supported by evidence applicable to the decision at hand, through a mechanism actually sensitive to that evidence—is a property of the relation between an assertion and an evidential base, not of a sampled output. It is stable under paraphrase in the way accuracy is not, because it is a fact about the pipeline rather than about a draw from it. It is therefore the property a governance regime can audit.

That claim is made precise later, in the sharpest form the argument takes: no accuracy figure identifies or bounds the grounding rate over the unit square. No accuracy figure constrains the rate at which a system asserts beyond its evidence, in either direction. A system can be highly accurate and almost entirely ungrounded, and the second fact is the one that predicts what happens when the questions change.

What this book is not arguing. Not that these systems are unreliable in a way that precludes use. I build them, in exactly the regulated settings this book is about, and they work. Not that accuracy is uninteresting; it is interesting, and it is the wrong thing to certify against. And not that better models will not help. They will, on some axes and not others, and which axes those are follows from the architecture itself.

Contributions. Six, in the order they appear.

1. **A redefinition of hallucination, indexed to a decision context.** Existing definitions ask whether an assertion is supported by a supplied source. This one asks the prior question—whether that source governs the case—by requiring support to be applicable at a resolved context of jurisdiction, valid time, material facts, and intended use, and by requiring the assertion to have been produced *because of* the evidence rather than merely to agree with it. The consequence is a failure class that faithfulness and attribution metrics score as clean by construction, and a definition evaluable at the point of assertion without a reference answer.
2. A five-category taxonomy following from it—SOUND, LUCKY, ERROR, FABRICATION, and MISAPPLIED—in which the fifth, assertion grounded in authentic but inapplicable evidence, is a distinct control failure with its own remedy and owner.
3. A proof that aggregate accuracy neither identifies nor bounds the grounding rate, so a validation program reporting accuracy alone cannot distinguish radically different reliance profiles.
4. An architectural account of why the failures are structural, grounded in the specific architecture and training regime these systems inherit, and an argument that the absence of a canonical form makes accuracy uncertifiable rather than merely incomplete.
5. A set of nineteen invariants separating what assertion behavior does not supply from what a system must be built to supply, each tied to a control.
6. A reference architecture for governed retrieval, a definition of institutional warrant, and an account of why the division of ownership across data, policy, and platform functions is itself a principal cause of failure.

Scope. This book is about one class of system: models that emit natural-language assertions by autoregressive sampling over a token vocabulary, fitted by next-token likelihood on large text corpora, and optionally post-trained on human preferences. That is the decoder-only transformer language model established at scale by the GPT-3 generation and everything built on that pattern since. It is not about deep learning generally. Generative adversarial networks and diffusion models are out of scope, as are encoder-only classifiers, rankers, and extractors, and the conventional predictive models that make up most of an enterprise analytics estate. Those components assert nothing, so grounding is not a coherent question about them—which is not to say they are easy to validate, since they share several of the problems described here. The boundary is stated precisely, with the reason it falls where it does, alongside the account of the architecture.

Structure. Chapter 2 identifies the systems in scope and derives, from their architecture and training regime, why the failures this book describes are structural. Chapter 3 redefines hallucination as ungrounded assertion, gives the five categories that follow, proves that aggregate accuracy does not identify the grounding rate, and defines the resolved context and the seven dimensions of grounding — along with why retrieval and citation do not supply them. Chapter 4 shows that neither form invariance nor repeatability holds, and what that costs a validation program. Chapter 5 turns those failures into a list: what these systems do not supply, what a system must be built to supply instead, and why grounding rather than accuracy is the property that can be measured.

The second half is constructive. Chapter 7 gives a reference architecture, Chapter 8 says what in it is actually tunable, Chapter 9 defines warrant with reliance tiers and the organizational analysis that goes with it, and Chapter 10 gives an evaluation protocol. Chapter 11 then builds the control model that generalizes the architecture — five surfaces, a governance component, an algebra for composing components — and the three chapters after it govern the surfaces one at a time: the prompt before the model runs, the output before it is delivered, and the system as a whole over time. A closing chapter on related work positions the argument.

Notation. Formal notation is used where it makes a claim checkable and avoided where it would only make one look rigorous. Two operators, five defined categories, one proposition with a two-line proof. Every definition is stated over observable behavior, so that any claim made here can be tested against a running system.

1.1 Premise: What Generation Does Not Establish

One premise carries the rest of the book, and it is deliberately modest. A system’s producing a statement is a different thing from that statement being supported by evidence, applicable to the case at hand, checked, and permitted for use. Any architect will grant the distinction in every other context: a report that renders is not a report that reconciles, and a value that populates a field is not a value that passed validation. The premise says only that generated text is subject to the same distinction, and that the generation event is not what closes it.

The argument below needs no position on what a model understands or represents internally. It needs the distinction stated formally, which is the business of this section.

1.1.1 Notation

M	the language model: a parameterized generative function
S	the deployed system: M together with its decoding, retrieval, policy, and control layers
x	an input—a prompt, a structured request, a conversation state
p, q	propositions or claims
c	the resolved context of a decision
E	an evidentiary basis
V	a verification procedure
I	an institution or accountable organization

The distinction between M and S is load-bearing rather than pedantic. Every property this book identifies as necessary for reliance is a property of S , and none is a property of M . A model has no version of an audit record, an authorization, or a resolved context; a system can have all three. Where the literature reports a result about models, I say M ; where the object of governance is at issue, I say S .

1.1.2 The resolved context

A great deal of enterprise information is true only relative to a governing frame. Call that frame the *resolved context* and write

$$c = \{j, t_v, f, u\}, \quad (1.1)$$

where

j	jurisdiction or governing regime: the legal, regulatory, or policy authority whose rules apply
t_v	<i>valid time</i> : the effective date the assertion is about—the as-of date against which the governing rules are to be assessed
f	material case facts: patient facts, customer facts, transaction facts, product facts, or whatever else is decision-relevant
u	intended use: the decision class, the reliance context, what the answer will be used to do

What these four have in common is what decides membership: each is something that must be settled about the *question* before the question is well posed. Ask whether a filing may use the accelerated pathway and you have asked nothing answerable until the regime, the as-of date, the case facts and the intended use are fixed. That is why c contains these and not other true and relevant things.

A second time dimension is required, and it fails that test, which is why it does not belong in c . Write t_k for *transaction time*: the state of the evidential base the system actually had access to—which corpus release, ingested to what date. A requester does not supply t_k and could not; it is a fact about the system at the moment it answered, not about the question it was asked. The distinction is the ordinary bitemporal one and it is not decoration.

t_v is a property of the *question*: which date's rules govern this decision. t_k is a property of the *answer's provenance*: what the system could have known when it answered. Correctness is assessed against t_v ; reconstruction requires t_k ; and their relation is checkable in *both* directions,

which is the whole reason two timestamps are needed:

$$t_k \prec t_v \implies \text{the base may omit instruments in force at } t_v \quad (\text{omission}), \quad (1.2)$$

$$t_k \succ t_v \implies \text{the base may contain instruments not in force at } t_v \quad (\text{anachronism}). \quad (1.3)$$

Omission is the familiar case: a rule amended before t_v but ingested after t_k leaves an assertion ungrounded in a way no inspection of the retrieved evidence will reveal, because the missing instrument is missing.

Anachronism is the other half and in regulated work it is the more dangerous of the two, because the answer is confidently wrong rather than incompletely right. A corpus current to today, answering a question as-of two years ago, will retrieve provisions that had not yet been made. Applying a rule to a transaction that preceded it is a familiar and serious error in its own right, and the system will produce a clean citation to a real instrument while committing it.

Neither failure is prevented by having a recent corpus, and both are prevented by the same condition. $\text{Applicable}(E, c)$ must be read to require *interval containment* on the temporal component of c : the instrument's own validity period includes t_v . That excludes the not-yet-enacted and the already-superseded together, and it is a filter on the evidence rather than a property of the corpus. Temporal validity is therefore part of applicability rather than a dimension of its own, which is why the conditions on grounded evidence, set out later, do not list it separately. Both directions are also detectable at the aggregate level without knowing what any amendment said: compare the corpus ingest watermark against the decision date, per source class, and flag either sign of the difference. Systems that record only one timestamp cannot perform the comparison, and most record only one.

$\text{Resolved}(c)$ holds when the components of (1.1) are explicit rather than assumed or inferred without validation. t_k , for the reason above, is not among them: it is recorded rather than resolved, and belongs to what the audit record must preserve. The distinction between explicit and inferred is the whole of it: a system that guesses the jurisdiction correctly has not resolved the context, because nothing in the record establishes that the guess was checked, and an auditor cannot distinguish a correct guess from a lucky one.

The truth predicate is correspondingly indexed. $\text{True}(p \mid c)$ is truth relative to a resolved context, and the relation that fails is:

$$\text{True}(p \mid j_1, t_1, f_1, u_1) \not\Rightarrow \text{True}(p \mid j_2, t_2, f_2, u_2). \quad (1.4)$$

1.1.3 The non-entailments

Write $\text{Pr}_M(\text{gen}(p) \mid x)$ for the probability that the model M , given input x , generates a token sequence expressing p . This is a generation event and nothing more — a fact about M 's output distribution. It is deliberately not called an assertion: $\text{Assert}_S(p)$, defined in Chapter 3, is a property of the deployed system S and is behavioral rather than probabilistic, for reasons given there. The two are kept in separate notation because conflating them is the error this section exists to block. The premise is that a high value of $\text{Pr}_M(\text{gen}(p) \mid x)$ does not establish any of the properties on which reliance depends:

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{True}(p \mid c), \quad (1.5)$$

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{Supports}(E, p, c), \quad (1.6)$$

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{Applicable}(E, c), \quad (1.7)$$

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{Verified}(V, p, c), \quad (1.8)$$

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{Warranted}_I(p \mid c). \quad (1.9)$$

Compactly, and in the order in which an institution needs them:

$$\Pr_M(\text{gen}(p) \mid x) \text{ high} \not\Rightarrow \text{True}(p \mid c) \not\Rightarrow \text{Supports}(E, p, c) \not\Rightarrow \text{Warranted}_I(p \mid c). \quad (1.10)$$

These are non-entailments, and the distinction matters because the claim is frequently misread as stronger than it is. They do not assert that the properties fail to co-occur. A model can and routinely does produce propositions that are true, well supported, applicable, verifiable, and fit for reliance. The claim is that the generation event is not what establishes those properties, so the generation event cannot be what an institution points to when asked to defend them. Each arrow in (1.10) marks a gap that something other than the model has to close.

Nor is the claim that the properties are unrelated. High generation probability is correlated with truth on the distribution the model was fitted to, sometimes strongly. Correlation on a training distribution is not entailment, and what concerns this book is what happens when the distribution of interest is not that one.

1.1.4 What follows

The premise is weak, and that is its value: it can be granted without conceding anything contentious, and it is sufficient for everything in this book.

Two clarifications, because both are easy to misread from the non-entailments alone. The claim is not that model output is unreliable. Output is often excellent, and the argument concerns what can be established about an individual assertion rather than what can be expected of a population of them. And the claim is not that the properties in (1.10) are rare. In a well-built system most assertions have all of them. The point is that they are established elsewhere, by components that can be pointed to.

That is the whole of the constructive program. If generation does not establish support, applicability, verification, or authorization, then those properties are supplied by something in S that is not M —and identifying, building, and measuring that something is what the rest of this book is about.

2 The Systems in Scope

Before classifying what these systems get wrong, it is worth being exact about which systems are meant. The argument that follows depends on specific properties of one architecture and one training regime, and it does not generalize to deep neural networks at large.

2.1 What is in scope, and what is not

In scope are systems with all four of the following properties.

1. They emit natural-language assertions—propositional content a reader can act on—as their output.
2. They produce that output by autoregressive sampling over a fixed token vocabulary, one token at a time, each conditioned on the preceding ones.
3. Their parameters are fitted by next-token likelihood on large text corpora.
4. They are optionally post-trained on human demonstrations and preference comparisons.

Concretely: the decoder-only transformer language models established at scale by the GPT-3 generation [4], extended by instruction tuning and preference optimization [15], and everything built on that pattern since. That is the point at which these systems became plausible components of a business process, and it is the regime current deployments inherit.

Out of scope, and worth naming because “deep learning” is often treated as one thing:

- **Generative adversarial networks** and **diffusion models**. Both are generative, and neither generates by autoregressive token sampling—an adversarial objective and an iterative denoising objective respectively—and neither emits propositional content that an organization relies on as an assertion. A synthesized image makes no claim that could be grounded or ungrounded.
- **Encoder-only and discriminative models**. A masked language model used as a classifier, a ranker, or an extractor produces a label or a score. The exclusion here is one of *tractability*, not of importance, and it should not be read as saying such systems raise no governance concern. A prediction of “high risk”, “eligible”, or “fraud likely” is routinely acted upon, and it needs provenance, applicability, validation, monitoring, and authorization exactly as an assertion does. What differs is that a finite, enumerable output space and a task-specific validation regime make the technical problem materially different, as Section 2.10 develops. This book is about systems whose primary output is open-ended natural-language assertion.
- **Conventional predictive and statistical models**. Regression, gradient-boosted trees, and the rest of the enterprise analytics estate are out of scope for the same reason: they do not assert.

2.2 What the system computes

The computation has four stages, and the first is where the canonical-form argument originates.

Tokenization. Text is segmented into tokens drawn from a fixed vocabulary $\mathcal{V}$, typically by byte-pair encoding. This is a purely lexical operation. It is worth pausing on what it means for the rest of the stack: `10`, `ten`, and `10.0` are distinct token sequences, as are `Q1 2024` and `first quarter of 2024`. No semantic normalization occurs at this boundary, or at any later one.

Embedding. Token identifiers are mapped to vectors by an embedding matrix $W_E \in \mathbb{R}^{|\mathcal{V}| \times d}$, and position information is added. An input of n tokens becomes a matrix in $\mathbb{R}^{n \times d}$.

Causal self-attention. Each of L layers computes, per head,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \Lambda\right) V, \quad \Lambda_{ij} = \begin{cases} 0 & i \geq j, \\ -\infty & i < j, \end{cases} \quad (2.1)$$

where Q, K, V are linear projections of the layer input, d_k scales the logits to keep the softmax out of saturation, and Λ is the causal mask that prevents position i from attending to positions after it [23]. Attention output is combined with a position-wise feed-forward transformation, with residual connections and normalization.

Unembedding and factorization. The final layer is projected to logits over $\mathcal{V}$ and normalized, giving $P(x_k \mid x_{<k}; W)$, where W denotes the fitted model parameters — distinct from Θ , which throughout this book is the set of inference configurations, and from $\theta \in \Theta$, one such configuration. The causal mask is what licenses reading the whole as an autoregressive factorization:

$$P(x_1, \dots, x_n; W) = \prod_{k=1}^n P(x_k \mid x_{<k}; W). \quad (2.2)$$

Now state the signature of the resulting map plainly, because everything that follows about canonical form rests on it:

$$M : \mathcal{V}^* \longrightarrow \Delta(\mathcal{V}^*), \quad (2.3)$$

a function from token sequences to distributions over token sequences. The signature is M 's and not S 's, and the distinction matters: a deployed system with typed requests, databases, tools, gates, and escalation paths is a control-flow structure over records and decisions, not a map from strings to strings. Everything this book asks for lives in the difference between the two. The domain is $\mathcal{V}^*$. Not propositions, not normalized queries, not parsed intents, not records with identity—token sequences. There is no slot in the parameterization for a proposition, and no stage between the tokenizer and the unembedding at which two encodings of the same claim are brought to a common form. This is not an omission that could be patched at the margins; it is what the architecture is.

There is direct evidence that content is stored in a form tied to its surface presentation rather than propositionally. Models trained on statements of the form “ A is B ” assert them reliably and fail to answer a query directed at B , even though the fact is the same fact [3]. The result is about what was learned rather than about how two phrasings of one request are treated—a distinction worth keeping, and this is not an instance of the canonical-form failure. What it shows is narrower than an absence of internal representation, and the narrower claim is the one this book needs. Whatever representations the model may learn—and Section 1.1 is deliberately agnostic about them—the architecture exposes no stable, inspectable, versioned, or contractually enforceable propositional record on which an institution can rely, and no guaranteed invariance

relation between formulations of the same fact. An assertion’s availability depending on the order in which the fact was encountered is evidence about the second of those, not about what is or is not encoded.

2.3 What the model returns is a distribution, not an answer

One property of (2.2) is easy to pass over because it is so familiar, and it carries more weight in what follows than any other single fact about these systems. The model does not compute an output. It computes a distribution over next tokens, and a separate decoding step turns that distribution into text.

Where the distribution is sampled — the ordinary configuration for open-ended generation — the same request can produce different text on different occasions. Temperature and nucleus parameters shape how much the output varies; they are knobs on the variance, not a switch that removes it. This is not a defect in a particular product, and it is not a misconfiguration. It is what a probability distribution over sequences is for. A system asked to generate fluent prose that never varies would be a worse generator, which is why no vendor ships one.

Two consequences follow, and both are load-bearing later.

The first is that a verified output is a statement about one draw. The check established what came back that time. It did not establish what the process will return the next time the same question is asked, because there was no single thing the process was going to return.

The second is that even holding the request fixed, the conditioning does not hold still. A session accumulates context, and (2.2) conditions on everything in the window. The characters a user types may be identical on the second occasion while the sequence the model conditions on is not. Greedy decoding narrows the first consequence considerably and does not remove the second, and it does not by itself deliver identical output run to run either — for reasons of accelerator arithmetic and request batching that are taken up where the measurement is specified.

None of this makes these systems unusable, and none of it is exotic. Plenty of non-deterministic computation is relied upon in regulated settings. It does mean that any assurance argument of the form *we tested it and it passed* has to say what it thinks the test established, and that question turns out to be harder to answer than it looks.

2.4 The pretraining objective

Parameters are fitted by minimizing cross-entropy on next-token prediction over a large corpus:

$$\mathcal{L}_{\text{pre}} = - \sum_k \log P(x_k \mid x_{<k}; W). \quad (2.4)$$

For the GPT-3 generation this was applied at a scale of order 10^{11} dense parameters over a corpus of order 10^{11} tokens drawn from filtered web crawl, curated web text, books, and encyclopaedic sources [4].

Read (2.4) as a specification and audit it for what is absent. The claim to make here is narrower than it is tempting to make. Fitting human text does make the model sensitive to truth-correlated structure—that is why these systems are useful—so it is wrong to say the objective is blind to truth. What (2.4) contains is no term that *rewards truth over plausible falsehood where the two diverge*. On the training distribution they mostly coincide and the loss cannot tell them apart; off it they separate and the loss remains indifferent. Beyond that, it

contains no term that is a function of evidence. No term that is a function of evidence, or of the provenance of the text being predicted. No term that is a function of the context in which an assertion might later be relied upon. And no term that is minimized by declining to produce a continuation—the objective is defined over every position, and an abstention is simply a different continuation with its own probability.

What (2.4) does reward, and rewards very effectively, is assigning high probability to continuations resembling the training distribution. A fluent falsehood in a familiar genre resembles the training distribution closely. This is not a criticism of the objective, which does what it was designed to do; it is an observation about scope. The system was fitted for *plausible continuation*, and plausible continuation and warranted assertion are different targets that coincide on the training distribution and separate off it.

One consequence is worth drawing now, because it is the origin of two of the five failure categories set out in the next chapter. Off the training distribution—an entity never described, a figure never published, a combination of jurisdiction and effective date that never co-occurred in the corpus—the system must still emit tokens. It has no representation of the fact that it is now extrapolating rather than reporting, because (2.4) gave it no reason to construct one. It produces the most probable continuation, which is fluent and which rests on nothing. Whether such a continuation happens to be true is a separate matter, and the distinction between those two outcomes is far less important than it looks.

2.5 In-context learning, and why the prompt is the whole control surface

The property of this generation of models that made enterprise use plausible was not raw perplexity. It was in-context learning: task behavior induced by examples and instructions placed in the prompt, with no gradient update [4]. That is what made a general-purpose model deployable against a specific business task without a training program per task, and it is why these systems entered production at all.

It also has a consequence that is easy to miss and directly relevant to governance. If task behavior is induced through the prompt, then the prompt is the control surface—and by (2.3) the prompt is a token sequence. All conditioning, including everything an architect would think of as configuration, policy, or scope, enters the computation as surface form and is processed by the same mechanism as the content. There is no separate typed channel for constraints.

This is the architectural root of the canonical-form failure. Other channels exist—system prompts, fine-tuning, adapters, logit biases, tool schemas—but the per-request control surface, the one that varies with what a user asks, is the prompt. When that surface is a string, and the map’s domain is strings, sensitivity to the form of the string is not a defect layered on top of the design. It is the design operating as specified.

2.6 Instruction tuning and preference optimization

Deployed systems add post-training. Supervised fine-tuning on demonstrations is followed, in the regime established by Ouyang et al. [15], by preference optimization: a reward model r_ϕ is fitted to human comparisons between candidate outputs, and the policy is optimized against r_ϕ with a divergence penalty against the supervised reference,

$$\max_W \mathbb{E}[r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}}(\pi_W(y | x) \| \pi_{\text{ref}}(y | x)). \quad (2.5)$$

Here x is the prompt as assembled and sent to the model, y a candidate output for it, π_W the policy being trained and π_{ref} the supervised reference it is held near, $\mathbb{D}_{\text{KL}}$ the Kullback–Leibler divergence between them, and β the weight on that penalty. y rather than o is used because the objective ranges over candidate outputs being compared, not over the one output finally delivered.

This changes the output distribution substantially, and it is why current systems are usable where their pretrained predecessors were not. For the purposes of this book, three properties of (2.5) matter.

The reward is a function of the output, not of the evidence. $r_\phi(x, y)$ is fitted to preferences over (x, y) pairs. Nothing in its domain is the evidential base E — the identified, versioned sources an assertion is supposed to rest on — nor the source version, nor the resolved context. An assertion and its ungrounded twin—identical text, different provenance—are indistinguishable to the reward model, and therefore receive identical optimization pressure.

The precise limitation is worth stating carefully, because the strong version is false. Preference optimization *can* select for the observable correlates of grounding: raters shown a well-cited answer and a bare one will prefer the former, and instructions to raters can make citation quality part of the comparison. What r_ϕ cannot do is *verify* the relation, because E is not in its domain. It can learn that citation-shaped output is preferred; it cannot learn that a citation supports the claim attached to it, except through whatever raters happened to check at labeling time—which does not generalize to evidence they never saw. So the pressure is toward the appearance of grounding, and the gap between appearance and relation is exactly where an auditor has to look.

Two qualifications keep the claim defensible. Deployed systems can place retrieved evidence, tool output, and structured context inside x , and rubric-based or process-supervised training can score intermediate steps rather than only final outputs—so the domain of the reward is a design choice rather than a fixed limitation. And the direction of pressure is an empirical tendency contingent on rater instructions, reward design, and task incentives, not a theorem: raters told to prefer abstention where evidence is absent will produce a reward that prefers it. The claim that survives is conditional. *Preference optimization alone does not establish a source-specific, version-specific, context-specific evidence relation for a live assertion, unless the training and deployment architecture explicitly represents and verifies that relation.*

The pressure is toward assertion. Human raters comparing outputs prefer answers that are fluent, confident, complete, and helpful. A hedged answer, a request for missing context, or a refusal reads as less helpful than a direct one, and will usually lose the comparison unless raters are specifically instructed otherwise. The gradient therefore points away from abstention. The sycophancy results are a measured consequence of this structure [20], not an incidental defect.

The result is better-calibrated form, not better-grounded content. Post-training teaches the surface conventions of careful assertion—citation shapes, hedging vocabulary, structured justifications—because those conventions are present in preferred outputs. Teaching the relation those conventions signal is harder and is bounded by what raters could verify: preference data can carry a signal about whether citations checked out on the examples that were labeled, and cannot carry one about evidence nobody looked at. So the conventions generalize and the relation does not. For an auditor this is the least comfortable finding in the section,

because the direction of the gap is unfavourable: post-training reliably improves how grounded an output *looks* and improves the grounding rate of (5.1) only incidentally, which widens rather than narrows the distance between apparent and actual auditability.

2.7 Three components that are absent

Stated as a gap analysis, three things a warranted-assertion system would contain are absent from the forward pass, and each maps to a category. These are claims about M , not about S . A deployed system may well have an evidence gate, a context resolution step, and an abstention branch—indeed the invariants stated later require all three. The point is that none of them comes from the model, so each must be built, and a system assembled without them inherits the gaps below by default rather than by choice.

No evidence gate. Nothing in the computation consults an evidential base, forms a judgment about whether it supports the candidate claim, and conditions emission on the answer. Retrieval places evidence *in the context*, which raises the probability of continuations consistent with it, but that is influence, not gating: by (2.3) the retrieved text is more tokens, conditioning a distribution, not a precondition on output. This is why ungrounded assertion remains available in retrieval-augmented systems, and why the requirement that an assertion be *sensitive* to its evidence — that it would not have been made had the evidence been absent — is a genuine constraint rather than a description of existing behavior.

No context resolution. Nothing establishes c before answering. “Is this approved?” is syntactically complete while leaving jurisdiction, effective date, material facts, and intended use unspecified, and there is no mechanism whose function is to notice the omission. This is the failure developed later under the name misapplied evidence.

No abstention primitive. There is no state corresponding to “I have not established this.” Abstention, where it occurs, is a generated continuation—the tokens of a refusal, produced because refusal-shaped text was probable in that context, not because a support check failed. This is why abstention behavior is sensitive to phrasing: it is an output, not a branch. Any system that must reliably decline needs the decision to sit outside M .

2.8 A floor, not merely a gap

It might be hoped that the ungrounded categories shrink toward zero with better training. There is a formal reason to expect otherwise. Kalai and Vempala [9] show that a calibrated language model must produce incorrect assertions at a positive rate on facts appearing rarely in its training distribution: a model well calibrated on the frequency of rare facts is thereby committed to generating some of them wrongly, and a model that avoided this would be miscalibrated in a way that costs it elsewhere.

The engineering reading is that the ungrounded cells have a floor above zero that is not a defect of any particular system, and that the floor is highest exactly where enterprise questions live—specific entities, specific dates, specific jurisdictions, all of them rare in a general corpus. If the generation path cannot be made not to produce ungrounded assertions, then the surrounding system must be able to *detect* them. That is the argument for measuring grounding directly rather than waiting for the model to improve.

2.9 What this predicts about better models

The analysis supports specific predictions, which matter for roadmap decisions because they are not the predictions usually made.

Scale and post-training reliably reduce outright fabrication on questions resembling the training distribution. They do not address a corpus that is wrong, which is a property of the data rather than the model. They do not address inapplicable evidence with the context it lacks. Capability gains can make a model more likely to *ask* for a missing jurisdiction or effective date, which helps; they cannot make asking reliable, and they cannot supply the answer, which has to come from the request or from a system that resolves it. And their effect on lucky guessing runs in the wrong direction: a better model guesses better, which raises accuracy while leaving the assertion exactly as disconnected from evidence as before. The next chapter gives these four outcomes names and assigns them owners.

One concession is owed here, and it is specific. A system that samples several candidate outputs and selects among them with a verifier does have a check outside the forward pass, and where the verifier is sound—a proof checker, a test suite, a constraint solver—that check is a genuine correctness oracle. Such a system satisfies something structurally similar to the external gate this book argues for, and it would be wrong to claim the mechanism is absent from systems that plainly have it.

What does not transfer is the availability of the oracle. Verification of that kind requires correctness to be cheaply decidable against a formal specification, which is why it works for mathematics, code, and constraint satisfaction. There is no oracle for whether a particular regulatory instrument is the governing one for this jurisdiction at this effective date, applied to this population, for this decision class. The question is not undecidable in some deep sense; it is simply not settled by computation over the assertion. It is settled by evidence and by authority. So outcome verification is a real mechanism that addresses a class of problems this book is not about, and its success there is not evidence about the class it is.

Longer context windows, retrieval, and tool use change what the system can do without changing (2.3). The domain remains $\mathcal{V}^*$; there is still no canonicalization stage; the reward still cannot see provenance. Retrieval is the most consequential of them for this book’s purposes, and its contribution is not primarily accuracy—it is that an identified evidential base is what makes the grounding rate observable at all.

2.10 How this compares with other model families

Section 2.1 excluded discriminative and predictive models on the ground that grounding is not a coherent question about a label. With the architecture now in view the comparison can be made properly, and it should be, because the exclusion is about *whether grounding applies* and not about whether those systems are easy to validate. It would be wrong to suggest that discriminative and predictive models are unproblematic, and the overlap with this book’s categories is worth naming, because it locates the argument honestly.

A classifier that has learned a spurious feature—the hospital identifier rather than the pathology, the formatting artifact rather than the content—is right for the wrong reason. That is the same defect this book calls a *lucky* assertion: correct output, no supporting relation, and no reason to expect the next case to go the same way. A classifier whose label flips under an imperceptible perturbation has failed a form-invariance requirement. A classifier degrading under distribution shift has exactly the problem of a sample that does not generalize. And per-instance

attribution is weak for most model families, which is a traceability failure. Enterprise ML monitoring exists because none of this is solved.

What is different is narrower and it is about whether the validation problem is *well posed*. For a classifier it is. The output space is finite and enumerable, so a complete reference set can in principle be constructed. Errors are label mismatches: atomic, countable, and unambiguous. The input space carries a metric, so perturbation robustness can be stated as a radius and, for some model classes, certified within it. Residual risk concentrates in distribution shift, which is a monitorable quantity with established practice around it.

None of those four properties holds for an assertion. The output space is $\mathcal{V}^*$, so there is no complete reference set to build. An error is not a label mismatch: an assertion can be right in one jurisdiction and wrong in another, adequate for a weaker claim than the one made, or true for reasons unconnected to its evidence. The equivalence classes on the input are semantic rather than metric, so there is no radius to certify and no coverage strategy against a class you cannot enumerate. And the assertion makes a claim, so its correctness is relative to a context that the input may never have specified.

Part II

What Goes Wrong

3 Hallucination as Ungrounded Assertion

The architecture of Section 2 produces failures with a specific shape: assertion is computed over surface form, with no evidence gate and no resolved context, so output can be fluent and detached from anything at once. The received vocabulary for those failures misdescribes them, and the misdescription has consequences for what gets measured.

The term *hallucination* is in universal use and is used in at least two incompatible ways. In popular and much industry usage it is truth-conditional: a hallucination is a false or fabricated output. In the natural-language-generation literature it generally is not. There, hallucination means content unsupported by the source, whether or not that content happens to be true, and the possibility of a factually correct hallucination is explicitly recognized [7, 13].

This book adopts the second convention and claims no priority for it. Separating support from truth is established work, and a definition that merely restated it would contribute nothing. The definition below departs from *both* conventions somewhere else, and it is worth being exact about where, because that departure is this book’s first contribution and it is narrower and more specific than “hallucination is not falsity.”

The source-based definition takes the source as given. It asks whether the assertion follows from the evidence supplied. It does not ask the prior question: whether that evidence *governs the decision at hand*. A clinical guideline may be authentic, authoritative, correctly retrieved, accurately quoted, and genuinely entailing of the assertion—and concern a different jurisdiction, a superseded period, or another patient population. Every faithfulness metric scores that assertion as clean, because the entailment holds. Definition 3.3 does not, because it requires support to be *applicable at a resolved context*.

That is the move: hallucination indexed to a decision context rather than to a source. It is the first of this book’s two central claims. The second, developed in the chapter that follows, is that the same architecture makes accuracy uncertifiable. They are connected, and that connection is drawn there too.

3.1 Scope of the term

One restriction has to be stated before the definitions, because the definition below is broader than ordinary usage and would otherwise convict a great deal that nobody thinks is defective. Under a requirement of applicable, traceable, dependence-bearing evidence, the following would all count as ungrounded: a statement of common knowledge; an inference a reader would regard as obvious; a fact the user supplied and the system restated; hypothetical or exploratory content; and a true claim from parametric memory in a setting where nobody expected an evidence record. Calling those hallucinations would be a misuse of the word.

So the term is used here in an operationally restricted sense: **an asserted proposition, presented as decision-relevant, that lacks an auditable and applicable evidentiary basis**. The restriction is carried by *presented as decision-relevant*, and it does two things. It excludes the benign cases above, which are not offered as grounds for consequential action. And it makes the reliance tier — how much weight the output is asked to carry — part of the definition’s scope rather than an afterthought: the same sentence can be unobjectionable at Tier 1 and a hallucination at Tier 3, because what changed is what it is being relied upon for.

Readers who find the appropriation of the word uncomfortable may substitute *unwarranted*

assertion or *reliance-ineligible assertion* throughout without affecting any argument. I keep “hallucination” because it is the word the field, industry, and regulators actually use, and a redefinition that abandons the term forfeits the conversation it is trying to correct. Nothing here is intended as a general taxonomy of undesirable model output.

3.2 Assertion

I will talk about the deployed system S rather than the model M in isolation. That is the thing being validated, and it is the thing whose failures have owners.

Definition 3.1 (Assertion). $\text{Assert}_S(p)$ holds iff system S , under input x , emits p in an unqualified, decision-relevant form under a stated output-interpretation policy.

The definition is behavioral rather than probabilistic, and the earlier temptation to define it by a sequence-probability threshold should be resisted for three reasons. The probability of a full proposition’s token sequence is small even when the system plainly asserts it; equivalent phrasings of one assertion carry different sequence probabilities, which reintroduces form dependence into the definition of assertion itself; and most deployed interfaces do not expose the relevant distribution. What makes an output an assertion is that it was emitted unqualified in a decision-relevant position, which an interpretation policy can specify and a reviewer can apply.

The policy has to separate commitment from mention: a system that lists p among possibilities, hedges it, or attributes it to a source without adopting it has not asserted it. Where a graded notion of commitment is wanted it should be introduced separately rather than folded into Assert_S . Nothing here requires a view about what M believes, per Section 1.1, and every predicate below is likewise defined over instrumentable behavior.

3.3 Grounding

Grounding is the property that carries the book, and it must mean considerably more than that a document was retrieved or a citation was emitted.

Definition 3.2 (Grounded evidence). For an evidential set E , proposition p , and resolved context c :

$$\begin{aligned} \text{Grounded}_E(E, p, c) \leftrightarrow & \text{Authentic}(E) \wedge \text{Versioned}(E) \wedge \text{Authoritative}(E, c) \wedge \text{Applicable}(E, c) \\ & \wedge \text{Supports}(E, p, c) \wedge \text{Sufficient}(E, p, c) \wedge \text{Traceable}(E, p, c). \end{aligned} \quad (3.1)$$

Definition 3.3 (Grounded assertion). $\text{Grounded}_S(p \mid c)$ holds iff there is some $E \subseteq E_S$ in the system’s evidential base such that

$$\text{Grounded}_E(E, p, c) \wedge \text{Sensitive}(S, p, E) \wedge \text{NoDefeater}(E_S, p, c), \quad (3.2)$$

where $\text{Sensitive}(S, p, E)$ holds iff, had E not supported p , the assertion would not have been produced, and

$$\text{NoDefeater}(E, E_S, p, c) \equiv \text{DefeatSearch}(E, E_S, p, c) = \emptyset. \quad (3.3)$$

DefeatSearch is a *specified search* over the governed base, not a quantification over its subsets. It returns evidence E' that is applicable at c , of standing at least equal to that of E , and that

either supports $\neg p$ or undermines E 's support for p :

$$\text{DefeatSearch}(E, E_S, p, c) = \{ E' \in \mathcal{Q}(E_S, c) : \text{standing}(E', c) \succeq \text{standing}(E, c) \\ \wedge [\text{Supports}(E', \neg p, c) \vee \text{undermines}(E', E, p, c)] \}. \quad (3.4)$$

The seven conjuncts of (3.1) are properties of the *evidence*; Sensitive is a property of the *system*; and NoDefeater is a property of the *evidential base as a whole*. All three kinds are required, and the separation matters operationally because they are checked by different means and fail for different reasons. The account of resolved context takes the evidence dimensions one at a time; here I note why the other two cannot be dropped.

3.4 Why grounding needs a defeater condition

NoDefeater closes a hole that the existential quantifier in (3.2) would otherwise open. Without it, grounding is satisfied whenever *some* applicable authoritative source supports p —even if another source of equal standing in the same base supports $\neg p$. That is not a corner case. Two regulators may differ; a standard and its national implementation may differ; a clinical guideline and a local protocol may differ; a group policy and a subsidiary policy may differ. A system that retrieves one side of such a disagreement and asserts it has not produced a grounded assertion. It has produced an arbitrary selection from a contested record.

Three features of (3.3) and (3.4) are deliberate, and each answers an objection the unbounded form invites.

It is a search, not a quantifier. $\mathcal{Q}(E_S, c)$ is the governed retrieval over the base under the applicability predicates of c —the same retrieval the system already performs, run in the contrary direction. Quantifying over subsets of a corpus would be neither computable nor useful: in a large regulatory base something can be read as supporting almost any negation. Bounding the condition to a specified search makes it a control that a system can execute and a reviewer can re-execute, and it aligns the definition with the governed-retrieval architecture. The cost is honest and worth stating: a defeater outside the search is a corpus-coverage failure rather than a loophole in the definition, and it is detected the way coverage failures are always detected—by widening the search, not by strengthening the logic.

What defeats is *unresolved* defeat. Institutions act amid disagreement constantly, and they do it legitimately, through precedence rules and escalation procedures that were established precisely because authorities conflict. A condition that made any equal-standing disagreement fatal to grounded assertion would forbid most of what regulated organizations lawfully do. So NoDefeater should be read as: no defeater that the institution's own precedence and escalation policy leaves unresolved. Where the policy determines an outcome, the conflict has been resolved and the assertion can be grounded on the surviving evidence, with the resolution recorded. Where the policy is silent or the authorities are incomparable under it, the assertion is *CONTESTED* and escalation is the correct output. The condition is therefore relative to a governed artifact, not absolute—and an institution with no precedence policy has not discovered that grounded assertion is impossible; it has discovered a missing deliverable.

Standing is an ordering, not a predicate. $\text{standing}(E, c)$ places evidence in a partial order of authority that the institution defines: statute above guidance, regulator above trade

body, group policy above local practice except where local is stricter. Contrary evidence of *lower* standing is not a defeater—it is resolved by precedence, which is what precedence is for. Only contrary evidence of equal or incomparable standing defeats. This matters practically: the unordered version would escalate every routine conflict, and a control that escalates routine cases is switched off. It also locates a deliverable: the precedence order is a governed artifact owned by the policy function, and its absence is why conflicts currently resolve by retrieval order.

Defeat comes in two kinds. The vocabulary is borrowed from work on defeasible reasoning, where the distinction is standard. A *rebutting* defeater supports the contrary claim. An *undercutting* defeater leaves the contrary claim untouched and attacks the support relation instead: the instrument was withdrawn, its scope was narrowed by a later provision, the passage was superseded in a revision the retrieval did not reach, or the extraction misread a table. Undercutting defeat is common in a governed corpus and invisible to a search for contradictions, because nothing contradicts anything—the support simply no longer holds. Both belong in (3.4).

Where DefeatSearch returns a result, the correct outcome is adjudication or escalation rather than assertion. Section 3.11 treats this as a category in its own right.

Every output of M is caused by training, so causal provenance cannot be the criterion—it would be satisfied by everything. And $\text{Grounded}_E(E, p, c)$ alone is satisfied whenever suitable evidence happens to exist somewhere in E_S , whether or not the assertion had anything to do with it. A system that retrieves an authoritative passage and then answers from parametric memory, coincidentally agreeing with it, satisfies (3.1) and is not grounded in any sense an auditor should accept. Sensitive is what excludes this, and its operational test is ablation: remove or contradict the putatively supporting evidence and observe whether the assertion survives.

Sufficient closes a different hole, and one a data-quality reviewer will find first. Evidence can be authentic, versioned, authoritative, applicable, and genuinely supporting, and still address only part of what the claim requires. A rule imposing five conditions is not satisfied by a source that speaks to three of them, and an assertion resting on such a source is grounded in every dimension except adequacy. Completeness is a standard data-quality dimension and its omission here would be conspicuous.

The two conjuncts are independent rather than redundant, and the reason should be stated because the redundancy objection is otherwise natural. Supports is a *positive* relation admitting degrees: the evidence favors p , at some strength, as Section 3.13.3 sets out. Sufficient is a *coverage* relation: the evidence addresses every condition the claim requires. Evidence can favor a partial claim strongly—entail it, even—while covering three of five conditions, so neither conjunct implies the other. Their remedies differ accordingly: inadequate support is fixed by retrieving *better*, insufficient coverage by retrieving *more*.

3.5 Five categories

Let $A_i \in \{0, 1\}$ record whether assertion i is true under its resolved context c_i , and $G_i \in \{0, 1\}$ whether it is grounded in the sense of Definition 3.3. Crossing the two gives four categories.

	A_i	G_i	Category	Meaning
(i)	1	1	SOUND	True assertion with authenticated, applicable, traceable evidentiary support
(ii)	1	0	LUCKY	True assertion lacking a sufficient auditable evidentiary basis
(iii)	0	1	ERROR	Grounded but false: the system accurately reported what applicable authoritative evidence says, and the evidence diverges from the world
(iv)	0	0	FABRICATION	False assertion without an auditable evidentiary basis

Table 3.1: The four categories, numbered (i)–(iv) for reference throughout. These are the four cells of a two-by-two *contingency table* on A_i and G_i , listed one per row rather than pivoted into a grid: accuracy is the proportion with $A_i = 1$, the grounding rate the proportion with $G_i = 1$. Reference-based evaluation observes A_i only.

Because grounding requires applicability, a fifth category is analytically distinct and, in regulated deployment, the most consequential.

	Condition	Category	Meaning
(v)	Authentic(E) $\wedge$ Authoritative(E, c') $\wedge$ $\neg$ Applicable(E, c)	MISAPPLIED	Authentic and perhaps authoritative evidence that does not govern c : wrong jurisdiction, time period, population, policy, product, or decision use

Table 3.2: The fifth category. Operationally this is a subset of $G_i = 0$, since Definition 3.3 requires applicability. It is kept separate because it identifies a different control failure with a different owner.

A sixth follows from NoDefeater for the same reason:

- (vi) CONTESTED — the governed defeater search returns applicable evidence of equal or incomparable standing on the other side. No selection is grounded, whichever side the system asserts and whichever turns out to be right.

Like MISAPPLIED, this is operationally a subset of $G_i = 0$: the no-defeater condition of Definition 3.3 fails, so the assertion is ungrounded and falls in the right-hand column of Table 3.1. It is kept separate for the same reason—the control that fails is different, the remedy is different, and the owner is different. There is no double counting in either case.

The relationship between Table 3.2 and Table 3.1 should be stated explicitly, since it is a place where a reader may reasonably suspect double-counting. Grounding as defined requires

applicability, so a misapplied assertion is *operationally ungrounded*: $G_i = 0$, and it falls in the right-hand column. It remains analytically distinct because the underlying control failure is different in kind. In cells (ii) and (iv) no evidence was brought to bear at all. In the misapplied case authentic evidence was retrieved, cited, and genuinely relied upon—and its scope did not match the assertion’s resolved context. The first is a generation failure. The second is a binding failure. Work on the generation path will rarely find it, because the information that would reveal the mismatch—which regime governs, which version was in force at t_v —is metadata about the evidence rather than content of the assertion, and is usually not in the context at all.

3.6 The definition

Definition 3.4 (Hallucination, error, misapplied evidence).

$$\text{Hallucination}(p \mid c) =_{\text{def}} \text{Assert}_S(p) \wedge \neg \text{Grounded}_S(p \mid c), \quad (3.5)$$

$$\text{Error}(p \mid c) =_{\text{def}} \text{Assert}_S(p) \wedge \text{Grounded}_S(p \mid c) \wedge \neg \text{True}(p \mid c), \quad (3.6)$$

$$\text{Misapplied}(p \mid c) =_{\text{def}} \text{Assert}_S(p) \wedge \text{Supports}(E, p, c') \wedge \text{Authentic}(E) \wedge \neg \text{Applicable}(E, c), \quad (3.7)$$

$$\text{Contested}(p \mid c) =_{\text{def}} \text{Assert}_S(p) \wedge \neg \text{NoDefeater}(E, E_S, p, c). \quad (3.8)$$

Note what (3.5) omits: **any condition on the truth of p** . A hallucination is an assertion unsupported by applicable evidence, and that is all. Truth and falsity distinguish successes from errors; they do not distinguish hallucination from sound assertion.

Two consequences follow immediately, and both invert the standard usage. A false assertion faithfully reporting an applicable source is *not* a hallucination; it is an error, and the fault lies in the evidence. A true assertion resting on nothing *is* a hallucination, and remains one however often it turns out right. The next three subsections defend each of these, and the defense is in each case an argument about remedies rather than about words.

The definition also has a property that neither convention has: it is *evaluable at the point of assertion*, without a reference answer. That is what makes this an engineering claim rather than a terminological preference. A truth-conditional definition can only be applied offline, against known answers. A source-based definition can be applied at runtime but only relative to whatever was retrieved. Definition 3.3 is applied at runtime against the evidence *and* the context, which is what a gate needs.

3.7 What is claimed as new, and what is not

Groundedness, provenance, factuality, attribution, and contextual relevance all have substantial literatures, and nothing here is offered as a discovery in any of them. The claim is narrower, and stating it precisely is worth a paragraph because a claim pitched wider would be false.

This book does not treat hallucination as synonymous with factual error, and does not claim priority for that separation. It formalizes the relevant failure as the absence of an auditable relation between an assertion, an authenticated and *contextually applicable* evidentiary basis, and a verification pathway that an institution has authorized. Four components of that formalization are, so far as I am aware, not present in existing definitions:

1. **Applicability as a conjunct.** Support is indexed to $c = \{j, t_v, f, u\}$, so evidence that entails the assertion but does not govern the case fails the definition. Faithfulness metrics take the source as given and cannot express this condition.

2. **Dependence rather than entailment.** Sensitive requires the assertion to have been produced *because of* the evidence, not merely to be consistent with it. A system that retrieves an authoritative passage and answers from parametric memory satisfies entailment and fails this.
3. **Authority and version as conditions on support.** Whether a source has standing for this decision class, and which version of it applied at t , are treated as preconditions of valid support rather than as metadata.
4. **Defeat by contrary authority.** NoDefeater makes grounding a property of the evidential base rather than of a selected subset of it, so an assertion drawn from one side of a live disagreement between applicable authorities is not grounded. Faithfulness and attribution metrics evaluate an assertion against the source it cites and have no way to express this.
5. **The non-identification result.** Proposition 3.1 establishes that no accuracy figure constrains the resulting rate, which is what makes the redefinition consequential for validation rather than expository.

Taken together these separate four cases that pooled accuracy and conventional source-faithfulness metrics conflate: lucky truth, grounded error, fabrication, and contextual misapplication. The compact form of the claim is that factual accuracy is insufficient for institutional reliance, because an assertion must also possess a reconstructable and contextually applicable evidentiary lineage—and that a metric which cannot distinguish those four cases cannot support a reliance decision.

3.8 Error: a faithful report of a bad source is not a fabrication

A corpus contains, consistently and across several reference works, an incorrect effective date for a regulatory provision. The system reports that date. Ablation confirms the assertion is driven by those documents: remove them and it stops.

Reference-based evaluation scores this as a hallucination, because the output is false. That classification discards a distinction the organization needs. The system did what a reporting component is supposed to do: it reported what its evidence said, through a mechanism sensitive to that evidence. The defect is in the evidence.

One clarification is owed first, because the cell looks over-full as often stated. If grounding requires Supports and Sufficient, how can a grounded assertion be false? Several candidate causes turn out not to belong here. A misread passage means the evidence does not in fact support the claim, so Supports fails and the assertion is ungrounded. An invalid inference is the same. Overstating a source fails Sufficient. Those are FABRICATION in the sense of (3.5), not ERROR, and their remedy is on the generation path.

ERROR is therefore narrower than it first appears, and the narrowness is the point: it is the case where the system accurately reported applicable authoritative evidence and *the evidence itself diverges from the world*. Grounding, as defined, is institutional evidentiary grounding rather than truth-tracking, so the cell is exactly the gap between the two. It is the only failure the model and the pipeline cannot be blamed for, and the only one that no amount of generation-path work will reduce.

The distinction is not terminological, it is budgetary. Cell (iii) is addressed by work on the evidential base—source selection, authority ranking, curation, retrieval from systems of record

rather than secondary literature. Cell (iv) is addressed by work on the generation path—evidence gating, abstention policies, refusal thresholds. Different teams, different skills, different funding cycles. A metric that pools them reports a number on which no one can act.

There is a normative point as well. We do not say that a careful analyst fabricated when she accurately reported a source that later proved wrong. We say she was misled, and we look at her sources. Machine assertion should be held to the same distinction, in both directions.

3.9 Lucky: a right answer is not evidence about the system

Now the reverse. The system is asked for a figure absent from its evidential base—the headcount of a small subsidiary. The generation path, running on surface regularities about how entities of this type and sentences of this genre continue, produces 8,200. That happens to be right.

Reference-based evaluation scores this as a success. Every property that makes fabrication dangerous is nonetheless present, and the precise defect is worth stating because it is what makes the case more than a curiosity: had the true figure been 3,400, the same process would have produced the same output. The assertion does not depend on the fact it reports. It is not a measurement; it is a draw from a distribution that happened to land on the answer.

The consequence for validation is the one that matters. A user who receives a cell-(ii) output, checks it, and finds it correct has acquired *no* evidence about the system. The check confirmed the output, not the process, and the process will behave identically on the next question, where the draw may land elsewhere. Lucky answers are not repeatable performances. Grounding is—and that asymmetry is what makes it the auditable property.

This also explains why a benchmark that rewards cell (ii) does more than fail to inform. Scoring lucky answers as successes applies optimization pressure toward confident guessing and away from abstention, which by Section 2.6 is a direction the training regime already favors. The metric compounds a bias the architecture supplies.

3.10 Misapplied: authentic is not applicable

The category pooled metrics hide, and in my experience the most common serious failure in regulated deployment.

The system asserts that a treatment is indicated for a condition. The assertion is driven by clinical guidance that says exactly that, and ablation confirms it. But the guidance concerns an adult population and the case is paediatric. Or a system asserts a capital treatment on the basis of a rule that governs a different jurisdiction. Or it cites a policy that was superseded before the decision date. Or it applies a product rule to a different product class.

Without the context index these are cell (iii): grounded, false, therefore charged to a defective evidential base. That verdict is wrong twice over. The base was not defective—the guidance is accurate for its population, the foreign rule is correctly stated, the superseded policy was valid when issued. And the implied remedy is wrong: curating the corpus will not help, because nothing in it is incorrect.

What failed is applicability, and the remedy is context resolution: establishing c before answering, checking currency against t , checking jurisdiction, material facts, and intended use, and abstaining or qualifying when the applicable evidence has not been identified. These examples are illustrations of the failure shape rather than reports of measured incidents; the underlying generalization error is developed below.

3.11 Contested: a contested record is not a grounded one

The failure that the existential quantifier hides, and the one most likely to be misclassified as an error.

Two authoritative sources in the base disagree. A group accounting policy and a statutory requirement in one jurisdiction reach different treatments. A clinical guideline and a local formulary differ on a first-line therapy. A standard and its national implementation diverge on a threshold. The system retrieves one, and asserts it. Ablation confirms the assertion rests on that source; the source is authentic, versioned, authoritative, applicable, sufficient, and traceable.

Every dimension of (3.1) passes, and the assertion is not fit for reliance. What the system has done is resolve a policy conflict by retrieval order, which is to say arbitrarily, and it has produced a record showing a single clean citation with no indication that the question was contested at all. That is worse than an unsupported answer, because the audit trail actively conceals the problem: a reviewer sees one authoritative source and an assertion that follows from it.

Note that the defect is independent of which side is correct. If the retrieved source happens to state the treatment a reviewer would have chosen, nothing has been established—the selection was still arbitrary, and the next case may go the other way. This is the lucky cell again, at the level of source selection rather than token generation.

The remedy is neither corpus work nor generation work. It is *adjudication*: a rule that determines which authority governs when authorities conflict, and an escalation path for cases the rule does not settle. That is a policy artifact, and its absence is why contested cases surface as apparently well-grounded assertions.

The categories exist to route work, and this is what they are for. Four remedies, across three owners:

Category	Diagnosis	Remedy	Typical owner
ERROR	applicable evidence, but wrong	source authority, curation, systems of record	data
MISAPPLIED	evidence inapplicable at c	context resolution, currency and scope binding	policy / risk
CONTESTED	applicable authority on both sides	precedence rules, adjudication, escalation path	policy / risk
LUCKY, FABRICATION	no evidential connection	evidence gating, abstention, refusal thresholds	platform

Table 3.3: A single pooled hallucination rate cannot distinguish these rows, and therefore cannot tell an organization which of four budgets to spend. The division of ownership is itself a principal source of failure.

3.12 Aggregate Accuracy Does Not Identify Grounding

This result is often described as an independence result, and that word will mislead anyone who takes it in its statistical sense. What follows is a *non-identification* result, not a claim of statistical independence. It shows that an aggregate accuracy figure does not determine, and without further assumptions does not bound, the grounding rate. It does not show that the two are probabilistically independent in deployed populations, and they may well be correlated—a system built to ground its assertions will plausibly be more accurate too. The distinction matters because the weaker claim is the one a validation regime turns on: an assessor who accepts an accuracy figure as evidence about grounding is relying on an entailment that does not hold, whatever the empirical association happens to be.

The redefinition of Section 3.6 is not merely a useful reclassification. It supports a formal result about what aggregate accuracy can and cannot tell an assessor—and the result is what makes the redefinition consequential rather than semantic.

3.12.1 The result

For a population of n assertions, define

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^n A_i, \quad \text{GroundingRate} = \frac{1}{n} \sum_{i=1}^n G_i. \quad (3.9)$$

Proposition 3.1 (Non-identification). *Let $\alpha, \gamma \in [0, 1]$ with $n\alpha$ and $n\gamma$ integral. There is a population of n assertions with $\text{Accuracy} = \alpha$ and $\text{GroundingRate} = \gamma$. Aggregate accuracy therefore does not identify the grounding rate, and supplies no bound on it in the absence of further assumptions; the same holds in the other direction.*

Proof. Let $n_{\text{sound}}, n_{\text{lucky}}, n_{\text{err}}, n_{\text{fab}}$ be the counts in the four cells of Table 3.1, so that $n\alpha = n_{\text{sound}} + n_{\text{lucky}}$ and $n\gamma = n_{\text{sound}} + n_{\text{err}}$. Choose any integer

$$n_{\text{sound}} \in [\max(0, n\alpha + n\gamma - n), \min(n\alpha, n\gamma)],$$

an interval containing an integer for every admissible α, γ , and set $n_{\text{err}} = n\gamma - n_{\text{sound}}$, $n_{\text{lucky}} = n\alpha - n_{\text{sound}}$, and $n_{\text{fab}} = n - n\alpha - n\gamma + n_{\text{sound}}$. Each is a non-negative integer by the choice of interval, and the four sum to n . $\square$

The result is elementary, and saying so is the point rather than a hedge. It is the standard fact that the row and column totals of a two-by-two table do not determine the four cells inside it. Those four cells are the categories of Table 3.1 — a contingency table on A_i and G_i — and the two totals are the two rates: accuracy counts the cells with $A_i = 1$, the grounding rate those with $G_i = 1$. Fixing both totals still leaves the interior free, which is what the interval in the proof measures. The proof is three lines of arithmetic because there is nothing harder to do. Its form is the ordinary one for a non-identification argument — exhibit a family of data-generating configurations consistent with the same observed value — and here the family covers the whole unit square, so accuracy constrains the grounding rate not weakly but not at all.

What makes an elementary result worth stating formally is that the entailment it denies is asserted constantly and informally. A validation regime that reports accuracy alone is reporting one margin and drawing conclusions about the other, and the objection *surely the number tells us something* is answered not by a sophisticated argument but by an arithmetic one that cannot be negotiated with.

The construction requires only that grounding not entail truth, which cell (iii) supplies: an assertion can rest on authentic, applicable, authoritative evidence and still be false, through misreading, invalid inference, or evidence adequate for a weaker claim than the one asserted.

The result is arithmetically simple and strategically significant, and the two should not be confused. For a population of generated assertions, any feasible aggregate accuracy can coexist with any feasible grounding rate, because the population may contain varying proportions of grounded-true, ungrounded-true, grounded-false, and ungrounded-false assertions. Consequently pooled accuracy cannot identify or bound the frequency of auditable grounding.

One limitation should be stated before the corollary, because a reader is entitled to ask for it. The construction establishes *logical* independence: no accuracy figure entails anything about the grounding rate. It does not establish that the two are empirically uncorrelated in deployed systems, and they may well be—a system built to ground its assertions will plausibly be more accurate too. The claim is about what an accuracy measurement *licenses*, not about what is typical. That is sufficient for the argument, because a validation regime rests on entailment rather than on correlation: an assessor who accepts an accuracy figure as evidence of grounding is relying on a relation that does not hold, whatever the empirical association happens to be in the population they did not measure.

Corollary 3.1. *A system measured at 95% accuracy is consistent with a grounding rate of 0, and a system measured at 40% accuracy is consistent with a grounding rate of 1.*

3.12.2 What it implies for a validation program

What follows is a claim about validation programs rather than about models. A program that reports accuracy without measuring per-assertion grounding cannot distinguish between radically different reliance profiles—between a system whose correct answers rest on identifiable authoritative evidence and one whose correct answers are draws that happened to land well. Those two systems have the same score and different futures, and the difference appears the moment the question distribution moves.

Two further readings matter for anyone choosing between releases. Since neither rate constrains the other, an intervention that raises accuracy tells you nothing about grounding; and optimizing against reference answers applies pressure toward confident guessing, which raises accuracy by moving mass into the lucky cell where $G_i = 0$. A leaderboard improvement is fully consistent with a system that has become more willing to assert without support.

3.13 Grounding, Context, and Applicability

Definition 3.2 listed seven conditions on evidence and left them unexplained. This section takes them in turn and identifies the generalization error that makes applicability the hardest of the seven to enforce. The resolved context c that four of them depend on was defined in Section 1.1.2.

3.13.1 Illicit universalization

Relation (1.4) names a failure that deserves its own label, because treating it as ordinary inaccuracy leads to the wrong remedy.

Illicit universalization is the treatment of a claim that holds under one jurisdiction, time, population, policy, or use as though it held generally, because the relevant context was never resolved. The claim is not fabricated. It is not even, in its own frame, wrong. It has been detached from the frame that made it assessable, and then asserted into a frame where its truth

value is simply a different question.

This is worth insisting on because it reframes a large fraction of what gets reported as hallucination. Many apparent failures are not invented facts. They are unresolved-reference failures, temporal-validity failures, jurisdiction failures, or applicability failures: information that is correct somewhere, stated without a valid binding to the context in which reliance is proposed. The diagnostic question is not “is this true?” but “true *where, when, for whom, and for what use?*”—and a system that cannot represent the question cannot be said to have answered it.

The data-architecture analog is exact, and worth stating because it makes the failure recognisable rather than novel. A measure without its dimension keys is not a fact; it is a number. Revenue is meaningless without period, entity, and currency, and no data architect would accept it into a warehouse unqualified. Illicit universalization is the same defect at the level of an assertion: a claim detached from the dimensions that make it assessable, presented as though those dimensions were universal rather than merely unstated. What ordinary dimensional modeling enforces by schema, these systems do not enforce at all, because by (2.3) there is no schema to enforce it with.

There is a structural reason this failure is native to the architecture rather than incidental. By (2.3) the system’s input is a token sequence, and a token sequence carries no context index. A question that omits its jurisdiction is not, to the system, an incomplete question; it is a complete string. Nothing in (2.4) rewards noticing that a binding is missing, and by Section 2.6 preference optimization actively rewards answering anyway. The system is not failing to resolve context. It has no representation of context to resolve, and no mechanism that could register the omission.

3.13.2 The seven dimensions

Most of what follows will be familiar. These are terms of art, and they are used here in their established senses rather than redefined: Authentic is authentication, Authoritative is the best-evidence preference for the instrument over an account of it, Applicable is relevance and materiality — indirectness, in the evidence-grading frameworks used in medicine — Sufficient is completeness, Traceable is lineage or, in a proceeding, chain of custody, and Versioned is currency, which in a temporal database is the pair of valid and transaction time. A data quality practitioner will recognize six of the seven on sight, and an archivist will recognize authenticity as the first of the four qualities a record must have to be relied upon.

The one that has no counterpart in data quality is Supports, and the reason is structural rather than interesting: data quality assesses data, and whether a source establishes a particular claim is not a property of the source. Its nearest established relatives are the weight an item of evidence carries and, in this literature, what attribution metrics operationalize.

The claim made for the list is therefore not that the dimensions are new. It is the partition, given at the end of this section.

Authentic. The source is what it purports to be. In practice this means retrieval from a controlled corpus whose contents have known origin, not from a scrape whose provenance is a URL. Authenticity is a property an organization establishes by ingestion controls, and it is the dimension most familiar from ordinary data governance.

Versioned. The source version, its effective date, and the corpus state t_k at retrieval can all be identified. A regulation without an effective date is not evidence about what was required at a decision date; it is evidence about what some document said at some point. Both timestamps are needed, for different purposes: the effective date establishes validity at t_v , and t_k establishes what the system could have seen. This is the dimension that permits replay, and without it the record itself cannot support reconstruction: whatever an investigator eventually pieces together from archives and change logs, it will not be a reconstruction from the system’s own evidence.

Authoritative. The source has the standing required for the decision type. A vendor summary of a regulation and the regulation itself may agree on content and differ entirely in standing, and which one is required is a function of the decision, not of the content. Authority is a relation between a source and a decision class, not a scalar attached to a document—which is why an authority ranking that is global rather than use-conditional will misclassify.

Applicable. The source governs or materially bears on c . Read component-wise, this requires the source to govern in jurisdiction j , to be in force at valid time t_v —interval containment, per Section 1.1.2, so neither not-yet-enacted nor already-superseded—to bear on the material facts f , and to cover the intended use u . This is where (1.4) bites, and it is the dimension standard retrieval most reliably fails, for reasons developed in Section 3.13.4.

Supports. The source establishes p at the strength asserted. Section 3.13.3 distinguishes the grades of support this covers.

Sufficient. The source addresses every condition the claim requires, not merely some of them. A rule imposing five conditions is not satisfied by evidence speaking to three, and an assertion resting on partial evidence is grounded in every other dimension while being inadequate in the one that matters. Sufficiency and support are separated because their remedies differ: insufficient support is fixed by retrieving *more*, inadequate support by retrieving *better*.

Traceable. A reviewer can reconstruct the evidence-to-assertion relation: which passage, which version, retrieved at which corpus state, supporting which claim. Traceability is a property of the record rather than of the evidence, and it is what turns the other six into an auditable finding rather than an assertion about the system’s internals.

Note that four of the seven—Authoritative, Applicable, Supports, and Sufficient—are relations to c rather than properties of the document. A corpus can be perfectly authentic, fully versioned, and completely traceable while failing all four, and a governance program that treats grounding as a corpus property will build exactly that system. The no-defeater condition of (3.3) is stronger still: it is a relation to the *entire base*, so no examination of a single retrieved source can establish it.

3.13.3 Entailment and support

$\text{Supports}(E, p, c)$ covers a range, and collapsing it is a common source of overclaiming. At the strong end,

$$\text{Entails}(E, p, c) \Rightarrow \text{Supports}(E, p, c), \quad (3.10)$$

where E together with the governing rules of c settles p . A capital calculation that follows deterministically from a rule and a set of positions is of this kind, and so is a policy question whose answer is stated in the policy.

Most enterprise questions are weaker. Evidence may make p substantially more likely without settling it; it may support a qualified version of p but not the unqualified assertion; it may support p for a class of cases of which c is plausibly but not demonstrably a member. None of this is a defect. It is the normal condition of decision-making under real evidence.

The requirement is not that every assertion be entailed. It is that the kind and strength of support be *explicit and proportional to the consequence of reliance*. A system that reports “supported” without distinguishing entailment from suggestive correlation has not given a reviewer what they need, and has created the specific failure in which an assertion adequate for one decision class is relied upon for another. This is cell (iii) in its most defensible form: the evidence was authentic, applicable, and authoritative, and it was adequate for a weaker claim than the one asserted.

3.13.4 Why retrieval and citation are not enough

The anticipated objection to this entire section is that retrieval-augmented generation [10] already addresses it. Retrieval does help, materially, and no argument here counts against using it. It does not establish grounding in the sense of Definition 3.2, and the gap is specific.

Standard retrieval ranks passages by semantic similarity to the query and places the top results in the context window. Consider what that procedure establishes against the six dimensions. It can establish authenticity, if the corpus is controlled. It can establish traceability, if the retrieval event is logged. It establishes nothing about versioning unless version metadata is carried and used, nothing about authority unless authority is represented and conditioned on the decision type, and—critically—nothing about applicability, because semantic similarity to the query is not applicability to the context.

That last point deserves to be stated as a slogan, because it is the practical core of this book:

Retrieval is not applicability. Citation is not entailment. And authoritative evidence is not automatically governing evidence for the case at hand.

The nearest existing measurement is worth naming, since retrieval evaluation does score something adjacent. Context-relevance metrics assess whether retrieved evidence is relevant *to the query*. Applicability asks whether it *governs the case*. A paediatric question that retrieves adult guidance scores as highly relevant—the topic matches exactly—and the guidance does not govern. Relevance to a query and authority over a decision are different relations, and only the first is measured.

Each clause of the slogan names a distinct substitution that systems make routinely. A passage about paediatric dosing and a passage about adult dosing are near-neighbours in embedding space; that is what makes similarity search work, and it is exactly why similarity search will surface the wrong one. A citation demonstrates that a document was consulted, not that it supports the claim attached to it, and citation-to-claim alignment is a separate check that standard pipelines do not perform. And a source may be the most authoritative document in the corpus and still not govern this decision.

Nor does Sensitive come for free. Placing evidence in the context raises the probability of consistent continuations; it does not make the assertion conditional on the evidence having supported it. By Section 2.7 retrieved text is more tokens, not a precondition, and a system

may retrieve an authoritative passage and then answer from parametric memory, agreeing with it or not.

The conclusion is not that retrieval fails. It is that retrieval supplies one of six dimensions reliably, two more if metadata is carried and logged, and none of the three that are relations to *c*. The remaining work is architecture.

4 Form Invariance and Repeatability Both Fail

One qualification belongs in the first paragraph, because the heading is stronger than what follows and the difference is the point. Training on paraphrase-rich text does apply pressure toward treating equivalent formulations alike, and embeddings do place many paraphrases near one another. What is absent is not similarity; it is a *guarantee*—an enforced, inspectable, versionable canonical form of the kind every other component in the stack supplies. A learned tendency and an enforced invariant differ in exactly the way that matters for validation, and the rest of this section is about that difference.

One property does most of the damage, and it is worth naming before it is formalized. Call it *form invariance*: materially equivalent requests should produce equivalent decision-relevant outcomes. It is the property every other component in an enterprise stack supplies without being asked, it is the property this architecture does not supply, and its absence is what breaks validation. It is stated as invariant I1 in Chapter 5, where the full set is tabulated and where the numbering used throughout is defined; this section explains why it fails, what follows for a validation program, and why the obvious remedy does not work. The failure has a one-sentence statement: the system is keyed on the surface form of its input, not on the content.

4.1 The map and what it does not respect

Section 2.2 established the signature: S maps $\mathcal{V}^*$ to distributions over $\mathcal{V}^*$. The domain is token sequences.

Now consider the equivalence relation that validation cares about. Write $x \sim_c x'$ when x and x' encode *materially equivalent* requests under the same resolved context c —same question, same governing frame, same material facts, same intended use. This relation partitions the input space. What a test suite needs is that decision-relevant behavior be constant on those classes:

$$x \sim_c x' \implies \text{EquivalentDecisionRelevantOutcome}(x, x', c). \quad (4.1)$$

Two things about (4.1) are deliberate. It is stated over *material* equivalence under a resolved context, not over semantic equivalence in the abstract, because the equivalence that matters is domain- and context-relative. And it requires equivalent decision-relevant *outcomes*, not identical text. Nobody needs the system to produce the same sentence twice. What an institution needs is that materially equivalent requests not yield incompatible assertions, incompatible evidence bases, or incompatible decision-relevant conclusions without an identifiable reason.

Nothing in the architecture delivers (4.1). There is no normalization pass, no canonical form, no quotient map. By Section 2.2 the tokenizer is lexical, the embedding is of tokens, and no later stage brings two encodings of one claim to a common form. Two members of a $\sim_c$ class are two different points in $\mathbb{R}^{n \times d}$ with different distributional histories, and (2.4) contains no term penalizing divergence between them. The map is free to send them anywhere, and sometimes does.

4.2 Every other component in the stack does better

The failure is easy to overlook because it is unlike anything else in an enterprise architecture.

- A database normalizes before it compares. Results do not depend on whether a predicate was written NOT (a OR b) or (NOT a) AND (NOT b); the planner canonicalizes, and equivalence is decidable in the algebra. The comparison should not be overdrawn: equivalence for the full query language is not decidable either, and floating-point aggregation can differ across parallel plans. What a database supplies is a canonical form for the fragment that matters and a documented statement of where determinism is and is not guaranteed. Neither is available here.
- A compiler canonicalizes before it optimizes. $x*2$ and $x+x$ converge.
- A cache keyed correctly hashes normalized content, not raw bytes. Keying a cache on the unnormalized string is a familiar bug, and its symptom is familiar too: identical requests, different answers, no error anywhere.

The last case is the closest, and it is barely an analogy. The system behaves like a cache keyed on the raw string: materially identical requests are distinct keys, and nothing in the pipeline would notice.

The deeper version of the point is architectural, and it is the one that should concern anyone who has built a system of record. A canonical record requires stable identity, a normalized or governed representation, declared dependencies, lineage, and reproducible derivation. A model's latent representation has none of these properties: it has no stable identity for an entity across paraphrases, no inspectable normalized form, no declared dependency on a source, and no reproducible derivation path from evidence to output. It is therefore not a candidate system of record, whatever its accuracy. Few architects would assert that it is; the error is subtler and commoner, and consists in treating model *output* as authoritative without requiring the lineage that any other authoritative source in the enterprise would have to supply.

Insofar as materially equivalent formulations *are* treated alike—and often they are—that is a contingent achievement of training coverage. The training distribution contained many paraphrase pairs, so the map is approximately constant on well-covered classes. This degrades exactly where prediction says it should: as formulations become rarer, longer, more domain-specific, more like the queries a specialist actually asks. The behavior is best where the guarantee is needed least.

4.3 What the measurements show

The prediction is testable and has been tested.

The cleanest case is format sensitivity. Holding semantic content fixed and varying only formatting choices—separators, casing, spacing, the shape of an enumeration—produces large spreads in task performance [19]. This is the result to show a sceptical colleague: the content of the request is unchanged by construction, the variations are choices a prompt author makes arbitrarily and does not record, and behavior moves anyway. Aggregate performance cannot shift unless individual answers changed, so a spread across formats is a lower bound on the rate at which materially equivalent requests receive different treatment.

Framing sensitivity gives the direction, and is the second clean case. When a prompt signals what the user would prefer to hear, assertion shifts toward it [20]. The content of the question is unchanged; the assertion is not.

Consistency under paraphrase, measured directly, falls well below what a content-directed process would produce [6], and models will contradict themselves within a single generation

[14]. The second of these bears on repeatability as well as on form invariance: the instability is not only across sessions and phrasings but inside one continuous output, where any notion of a maintained position would have to apply if it applied anywhere.

4.4 Why “train a normalizer” is not the answer

The natural engineering response is to canonicalize the input: put a normalizer in front of the model and the problem goes away. It does not, for five reasons that compound.

Material equivalence is context-sensitive. Whether two requests are equivalent depends on *c*. “Is this reportable?” asked of two transactions identical but for jurisdiction are not materially equivalent requests, and no context-free normalizer can know that.

It depends on domain ontology. Deciding that two phrasings pick out the same product class, the same patient population, or the same instrument type requires the domain’s own identity conditions. That is ontology work, not string work.

Paraphrases carry material distinctions. In regulated language, surface differences are frequently the point. “Indicated for” and “may be used in” are not paraphrases. “Shall” and “should” are not paraphrases. A normalizer aggressive enough to collapse genuine paraphrases will collapse these too, and the failure will be silent and consequential.

General semantic equivalence is not reliably decidable. Unlike relational algebra, natural language offers no decision procedure for equivalence, and any learned approximation inherits the properties of the thing it approximates, including sensitivity to form.

A latent representation is not an inspectable canonical record. Even a normalizer that worked would produce an internal representation that no reviewer can read, version, or diff—which fails the requirement it was introduced to satisfy.

The conclusion is not that normalization is useless. Structured intake, controlled vocabularies, and typed request objects all help, substantially, and belong in the architecture. The conclusion is that the remedy is not *semantic* normalization of free text. It is explicit context binding, structured domain representations where they can be built, governed source corpora, deterministic or inspectable policy controls where the decision permits them, and a preserved evidence-and-policy path for every consequential assertion. That is what governed retrieval is for.

4.5 The validation consequence

Now the argument this book exists to make. State the validation inference explicitly and it fails visibly.

Let $T = \{(x_1, a_1), \dots, (x_n, a_n)\}$ be a test suite: inputs paired with acceptable outputs. Suppose S passes—for each i , $S(x_i)$ is acceptable. The inference validation needs is:

For every x that a user might actually send, if $x \sim x_i$ for some tested x_i , then $S(x)$ is acceptable.

That inference is valid only if (2.3) is constant on $\sim$ -classes, which is form invariance. Form invariance fails. So the inference is invalid, and the test suite does not license the conclusion drawn from it.

Three aggravating features, each of which an engineer will recognize as worse than the bare failure.

The classes are unbounded. You cannot enumerate the paraphrases of a question, so you cannot fix the problem by testing more members of each class. Coverage is not a strategy against an infinite equivalence class; the best you can do is sample it, which returns you to sampling.

The failures are silent. A form-invariance violation raises nothing. There is no exception, no warning, no anomaly in the logs. The system returns a fluent, well-formed, differently-wrong answer with the same confidence as the right one. Contrast a schema violation or a type error, which announce themselves.

4.6 The same failure, on a single input held fixed

Everything so far concerns *different* inputs that mean the same thing. The second failure is worse, because it does not need two inputs.

Repeatability — invariant I7 — is the property that the same request, in the same session, yields the same assertion. It does not hold, and by Section 2.3 it cannot be assumed to: the output is a draw, and the sequence conditioned on grows as the session does, so the request that was tested and the request as it later arrives are not the same point in input space even when the characters are identical.

The consequence is worth stating separately from the paraphrase argument, because it is a strictly stronger claim. Form invariance failing means a passing test does not *extend* to materially identical requests — the guarantee has a boundary, and the boundary is narrower than anyone assumed. Repeatability failing means the guarantee does not securely cover *the string that was tested*. There is no boundary to find, because the test result was a statement about one draw and the next draw is a different event.

For a validation program these compose badly. Enumerate more paraphrases and you address form invariance without touching repeatability. Re-run the same case a hundred times and you estimate a per-input distribution without learning anything about the neighboring inputs real users will send. Neither remedy reaches the other failure, and no amount of either produces the thing a deployment decision wants, which is a claim about behavior rather than about a sample of it.

4.7 Why this makes accuracy the wrong thing to certify

Put the two halves together.

Accuracy is estimated by sampling: draw inputs, compare outputs to expected answers, report the proportion correct. The estimate is a statement about the distribution the inputs were drawn from. Using it to certify deployment requires that the estimate transfer to the inputs that will actually arrive—and the failure of form invariance is precisely the claim that it does not transfer across semantic equivalence, which is the dimension along which real inputs differ from test inputs.

The conclusion should be stated carefully, because the strong form is indefensible and invites dismissal. Reference-based accuracy is not worthless and it does not establish nothing. It provides evidence *conditional on* a specified evaluation distribution, task operationalization, prompt representation, model version, and serving configuration. Every empirical validation regime generalizes from finite samples under such conditions, and a classifier is in the same position with respect to distribution shift unless its input domain and robustness properties are bounded.

The claim is narrower and it is about which conditions can be specified. In free-form consequential interaction, the prompt representation is chosen by the user and is not recorded; the equivalence class of materially identical requests is unbounded and cannot be enumerated; and the map is not constant on that class. So the conditions under which the accuracy estimate would transfer are not merely unmet but unstateable, and the generalization that broad consequential reliance requires is unavailable — not because the measurement is wrong, but because what it is conditional on cannot be pinned down. The number is a correct measurement of how the system performed on those strings, and that is a smaller thing than it is usually taken for.

Notice, finally, what this does to the cell-(ii) case of Section 3.9. A lucky assertion is one that would have been produced whatever the fact had been. Verifying it tells you about the draw, not the process. The failure of form invariance generalizes that observation from lucky assertions to *all* assertions: any verified output tells you about the draw. Cell (ii) was the special case where the disconnection is total; form-invariance failure is the same disconnection showing up as instability across the input space, and non-repeatability is it showing up on a single input held fixed.

That is the link between the two halves of this book. The taxonomy of Section 3 says that a correct answer can be produced for no reason. This section says that even a correct answer produced for a good reason carries no guarantee of recurrence, because nothing holds the system to the content rather than the string. Both point at the same missing property, and the next section names it.

4.8 Why the two claims are one

Section 3 redefined hallucination as ungrounded assertion. This section argued that there is no canonical form, so a passing test establishes nothing about a materially equivalent input. The two are the same architectural fact seen from two sides, and it is worth saying how, because each makes the other consequential.

Why do these systems hallucinate in the redefined sense? Because assertion probability is computed over formulations. A string can be made probable by surface statistics with no route through evidential support for its content, so assertion and evidence come apart—which is exactly $\neg \text{Grounded}_S(p \mid c)$ while $\text{Assert}_S(p)$ holds. Ungrounded assertion is what the absence of a content-level representation looks like from the output side.

And why does the absence of a canonical form matter, rather than being an interesting curiosity about paraphrase robustness? Because of what Section 3.9 showed about a lucky assertion: verifying it tells you about the draw, not the process. Form-invariance failure generalizes that from lucky assertions to *all* assertions. Any verified output tells you about the draw, because nothing holds the system to the content rather than the string. The lucky cell was the special case where the disconnection is total; form-invariance failure is the same disconnection showing up as instability across the input space.

So the redefinition identifies the defect and the canonical-form argument explains why you

cannot test your way out of it. Together they force one conclusion: the property to measure is not whether assertions turn out right, but whether they were grounded when made.

Part III

What a System Must Supply

5 System Invariants

Sections 2 and 4 argue from architecture. This section argues from contract, which is the form a reviewer can check without agreeing with me about mechanism.

If you were specifying a component whose assertions you intended to rely on, you would write down invariants. Two groups are needed, and the distinction between them is the book’s practical claim in compressed form. The first group states properties of assertion *behavior*: things you would assume without thinking about them from a database, a rule engine, or a calculation service. All of them fail, and none can be made to hold by work on M . The second states properties the *system* must supply. None of them holds by default, and all of them can be engineered.

5.1 Behavioral invariants: what does not hold

	Invariant	Statement	Holds	What breaks without it
I1	Form invariance	$x \sim_c x'$ implies equivalent decision-relevant outcome	No	Tests do not generalize past the wording tested
I2	Closure	$\text{Assert}_S(p)$ and $\text{Assert}_S(p \rightarrow q)$ imply $\text{Assert}_S(q)$	No	Conclusions the system’s own premises entail are unavailable
I3	Decomposition	$\text{Assert}_S(p \wedge q)$ implies $\text{Assert}_S(p)$	No	Summaries assert what their parts deny
I4	Consistency	Not both $\text{Assert}_S(p)$ and $\text{Assert}_S(\neg p)$	No	Two users, or one user twice, get opposite answers
I5	Self-report fidelity	Claims about what it knows track what it asserts	No	Confidence cannot be used to route or gate
I6	Correctness	$\text{Assert}_S(p)$ implies p	No	Output cannot be relied on unchecked
I7	Repeatability	Same input, same session, same assertion	No	A verified answer is not a verified behavior

Table 5.1: Behavioral invariants. I6 is the one everybody already knows fails, and it is the least interesting: nobody deploys these systems believing output is true by construction. I1 and I7 are the ones that invalidate validation.

One reading to head off first. These are invariants, so *holds* is a universal claim and a single counterexample defeats it. “No” in the third column therefore means the property is not guaranteed, not that behavior is always inconsistent. In practice these systems satisfy I1–I4 on most inputs most of the time, which is precisely why the failures are hard to catch in testing and why a specification cannot rely on them. An invariant that holds usually is not an invariant; it is a tendency, and nothing downstream can be built on it.

Three further observations, since the pattern matters more than the entries.

I6 failing is priced in; I1 failing is not. Every deployment I have seen assumes the system is sometimes wrong and builds review around it. Almost none assumes that a reviewed, approved, tested behavior will not reproduce under a rephrasing. The first is a known unknown that review handles. The second invalidates the review.

I2–I4 share one cause. These are not three independent defects. There is no component whose function is to maintain the set of asserted claims—nothing that closes it under consequence, decomposes conjunctions, or checks it for contradiction. By Section 2.2 the asserted “set” is not a maintained data structure at all; it is whatever successive forward passes emit. Absent such a component, I2–I4 have nothing to be true of.

I5 failing has a specific and costly consequence. A great deal of proposed architecture routes on model-reported confidence: escalate when the model says it is unsure, proceed when it says it is confident. I5’s failure means the routing signal is produced by the same process as the content and is not a measurement of it. Verbalized confidence can be trained to better aggregate calibration [11], and models have some capacity to evaluate their own outputs [8]; neither delivers the per-claim reliability a gate requires. Aggregate calibration and per-claim boundary recognition are different properties, and only the second supports a decision about *this* answer.

That last point generalizes, and it is the shape of the whole argument: a property that holds on average over a distribution is not a property you can gate on for an individual case. Accuracy is such a property, as Proposition 3.1 shows. So is calibration.

5.2 System invariants: what must be supplied

The constructive claim. These are conditions on S , not on M . None holds by default; each is achievable by architecture; and together they are what must be true for a response to cross from ordinary generative assistance into consequential decision support.

Two points about the second table.

I17 is the load-bearing one. Without a controlled-failure path, every other system invariant degrades into a preference. A system that detects missing applicability and answers anyway has not enforced I11; it has measured it. And by Section 2.7 the gate cannot live inside M , because generated refusal is a continuation rather than a branch. I17 is therefore an architectural requirement, not a prompt.

Proportionality, not maximalism. Not every use requires all ten. Tiering makes the controls proportional to the consequence of reliance, and a drafting assistant needs none of I13–I19. What does not follow is the converse: a system cannot claim auditability because it reports an accuracy score, emits citations, or carries a generic disclaimer. Those are compatible with failing every invariant in Table 5.2.

	Invariant	Requirement	Control that supplies it
I8	Source identity	Every cited source has stable identity and established authenticity	Controlled corpus, ingestion authentication
I9	Temporal validity	Source version and effective date are recorded and compared against t	Version metadata, corpus release pinning
I10	Context binding	c is explicit before assertion, or explicitly marked unknown	Intake resolution, typed request objects
I11	Applicability	Evidence is filtered on scope, not similarity alone	Metadata-aware retrieval, scope predicates
I12	Traceability	Each consequential claim links to the evidence supporting it	Claim decomposition, claim-level citation
I13	Support verification	The cited evidence is checked to support the claim at the strength asserted	Entailment checks, policy logic, human review by tier
I14	Reproducibility	The assertion can be re-derived from recorded inputs and versions	Snapshot pinning, deterministic replay
I15	Authorization	Reliance is permitted at the applicable tier, with duties separated	Tiered policy, approval gates, access control
I16	Reconstructability	The full decision path is retained for the required period	Audit record, retention policy
I17	Controlled failure	Unmet conditions produce qualification, refusal, or escalation—not an unqualified answer	Output gating outside M
I18	Sufficiency	The evidence set is checked for coverage of every condition the claim requires	Required-field or checklist verification against the rule
I19	Defeater search	A scoped search for contrary or undermining authority of equal or higher standing is performed and recorded	Governed defeater query; institution’s precedence order

Table 5.2: System invariants. I8–I9 are ordinary data governance. I10–I11 are the context work that Section 3.10 shows is most often absent. I12–I13 are what distinguishes grounding from citation. I14–I16 are what an audit actually asks for. I17 is what makes the others enforceable rather than advisory. I18–I19 correspond to the coverage and no-defeater conditions of Definition 3.3, and exist so that every condition in the definition has a system obligation that establishes it.

5.3 Grounding Is Auditable; Accuracy Is Not

5.3.1 The asymmetry

Section 5 said what does not hold. This section says which property can be measured instead, and why that property rather than the obvious one.

Accuracy and grounding are both properties of assertions, and they differ in a way that decides which one a governance regime can use.

To check whether an assertion is *correct*, you need to know the answer. Sometimes you do: a deterministic calculation can be recomputed, a figure reconciled against a system of record, a constraint evaluated, an outcome observed later. Where correctness is cheaply decidable it should be checked directly, and such claims should not be routed through a language model at all.

The difficulty is the residue, and the residue is what these systems are used for: open-world questions whose answers are not available at the moment of decision. There, correctness can be established only against a reference built in advance—on questions somebody already had the answer to, which is to say not the question that was asked. You cannot check correctness on the query a user just sent, because if you could answer it you would not have sent it to the system.

To check whether an assertion is *grounded*, you do not need to know the answer. You need to know what evidence the system used and whether that evidence is applicable at *c*. Both are properties of the system’s own execution. Condition (G1) is a check on the retrieved evidence: is it authentic, current at *t*, of the right jurisdiction and use? Condition (G2) is a perturbation test: ablate or contradict the evidence and see whether the assertion survives. Neither consults a reference answer.

This has a consequence out of proportion to how obvious it sounds once stated. Grounding can be checked *at runtime, per assertion, on live traffic*. Accuracy cannot. That is the difference between a metric and a gate. A metric tells you, after the fact, how a sample went. A gate decides whether this particular assertion is allowed out. Everything a regulated deployment actually needs—blocking, escalating, qualifying, logging a reconstructable basis—requires a gate, and only grounding supplies one.

It would be convenient to claim that grounding is stable under paraphrase where accuracy is not, and that claim would be false. A rephrased request can retrieve different evidence, resolve a different context, decompose into different claims, return a different verification outcome, or engage a different precedence rule. The grounding *status* of the resulting assertion can change, and governed retrieval does not repair form sensitivity.

What grounding supplies is different and, for a governance regime, more useful. It is auditable *per execution*. However the request was phrased, the system can inspect the evidence, the resolved context, the verification result, the defeater search, and the authorization path for the assertion that was actually emitted. Form-invariance failure therefore becomes *visible and attributable* rather than silent: two paraphrases that produce different assertions produce two traces, and the traces say which condition diverged and why. That is what accuracy cannot do, because a changed answer with no trace is indistinguishable from a changed answer with a good reason.

So the pathology of Section 4 is not repaired by measuring grounding. It is made observable, which is the most any measurement can offer against a property the architecture does not guarantee.

5.3.2 Determinism is not the requirement

Before the measurement itself, one reading has to be blocked. The asymmetry above may suggest that auditable systems must be deterministic, and grounding auditable because generation is not. That reading is false and it would misdirect the remedy.

Plenty of non-deterministic computation is relied upon in regulated settings and audited without difficulty. Monte Carlo valuation, randomized algorithms, and sampling-based statistical procedures all produce different output on different runs, and all are defensible. What makes them so is not determinism but three other properties: the run is *reproducible*, because the seed and the state are recorded and the computation can be replayed; the error is *bounded*, because a convergence or confidence statement is available; and there is a *specification* saying what the computation was supposed to approximate.

Generation can satisfy the first of those, but only under conditions worth spelling out, because they are frequently not the conditions of a commercial deployment. Section 2.3 established that the output is sampled; what matters here is how far that can be constrained. Greedy decoding removes sampling variance. It does not by itself produce identical output run to run. Floating-point addition is not associative; the order in which a reduction is performed on an accelerator depends on which kernel is selected; and kernel selection depends on batch shape. A request batched with different neighbours can therefore yield slightly different logits, and where two candidate tokens are close, a different token. Sparse expert routing under capacity limits introduces the same batch dependence more directly. Behind a hosted endpoint, the served model version may change without notice.

Reproducibility is therefore an architectural achievement rather than a property of the technology. It requires pinned model versions, controlled batching or single-request inference, deterministic kernel selection, recorded decoding parameters, and a stateless request. Those conditions are available to an organization that self-hosts or contracts for them, and largely unavailable to one consuming a shared public endpoint. This is why I14 appears in Table 5.2 as something a system must supply rather than something it has, and a validation program that assumes replay works should test that assumption rather than inherit it.

The point for this section is that non-determinism is not the *irreducible* gap. It can be engineered away, at a cost that is ordinary architecture.

The gap is that reproducibility over a single input buys very little when the set of inputs you need to reason about is a semantic equivalence class you cannot enumerate. Replaying one string tells you what that string does. It says nothing about the materially equivalent string a different user will send tomorrow, and by Section 4 there is no bound to appeal to in its place. The contrast with adversarial robustness is instructive: there, perturbations live in a metric space, so a claim of the form “behavior is stable within ε ” is both meaningful and sometimes certifiable. Paraphrase admits distance measures—embedding cosine, edit distance, entailment scores—but none with the property the certification argument needs: that everything within ε of a request is materially equivalent to it, and everything materially equivalent is within ε . Semantically identical requests can be far apart in embedding space and materially different requests can be adjacent, which is why the corresponding guarantee cannot be stated even approximately.

So the requirement is reproducibility plus a characterization of behavior over the class of inputs that matter. The first is engineering and this book treats it as such. The second is the one with no known general solution, which is why Section 5.3 argues for measuring grounding—a property of the individual assertion and its evidence—rather than attempting to certify behavior

over a class that cannot be enumerated.

The distinguishing property is worth stating once, because the whole argument turns on it: the systems in scope produce, as their primary output, statements that people treat as claims about the world, and they produce them by sampling token sequences. The first half is why the question of grounding arises at all. The second half, developed in Section 2.2, is why it cannot be answered by inspecting the model.

Two boundary cases. Multimodal systems are in scope to the extent that their text output is produced as in (1)–(4), and out of scope for their image or audio output. Retrieval-augmented and tool-using systems built on models in scope are squarely in scope—indeed they are the systems this book is chiefly about, since Section 5.3.5 argues that only such systems can be measured at all.

Later developments—longer contexts, sparse routing, tool use, and reinforcement training on verifiable outcomes—change capability substantially and change none of the structural properties identified below, a claim I return to in Section 2.9.

5.3.3 What independence implies for a validation program

Proposition 3.1 established that accuracy and the grounding rate are not linked by any entailment over the feasible region. Read as a statement about validation programs rather than about models, it says something sharper than “accuracy is incomplete.” It says accuracy is *uninformative* about the property that governs reliance: a reported accuracy figure is consistent with every grounding rate, so no accuracy threshold, however high, constitutes evidence that a system asserts within its evidence.

The asymmetry above explains why this is not symmetrical bad news. Both properties are worth knowing, and only one of them can be established for the assertion in front of you at the moment you need to decide about it.

5.3.4 The measurement target

The quantity to measure is the rate at which assertion outruns applicable evidence—which, by Definition 3.4, is the hallucination rate:

$$\text{hallucination rate} =_{\text{def}} \Pr(\text{Hallucination}(p \mid c) \mid \text{Assert}_S(p)) = \Pr(\neg \text{Grounded}_S(p \mid c) \mid \text{Assert}_S(p)). \quad (5.1)$$

This is worth pausing on, because it is where the redefinition earns its keep. Under the standard truth-conditional definition, a hallucination rate can only be estimated against reference answers, offline, on questions whose answers are already known. Under Definition 3.4 it is estimated from the system’s own execution: what evidence was used, and did it apply. Same word, same intuition, and a quantity that can be computed on live traffic and used to stop an assertion. The redefinition is what converts a retrospective score into an operable control. This is (3.5) turned into a statistic. It is estimated by attribution audit rather than comparison against reference answers, and the framework for attributing generated statements to identified sources is established [17]: sample assertions, and for each determine whether it is attributable to identified evidence that is applicable at c .

One further decomposition is needed, because the ungrounded rate as defined pools causes with different owners. An assertion can be ungrounded because no relevant evidence exists in the controlled corpus; because it exists and retrieval failed to surface it; because it was surfaced and excluded for want of applicability metadata; because it was available and the generation path ignored it; or because policy correctly excluded it and the system answered anyway. Those are

five distinct control failures—corpus coverage, retrieval recall, metadata completeness, gating, and policy enforcement—and only the last two are properties of the generation path. An audit that cannot separate them will attribute a corpus gap to the model.

Report it decomposed, not pooled. The three rows of Table 3.3 have different owners, so a single figure defeats the purpose of having measured anything:

- ungrounded rate—no evidential connection (cells ii, iv);
- misapplied rate—evidence present but inapplicable at c (cell v);
- error rate—grounded and false (cell iii), reported against the corpus rather than the system.

5.3.5 Where this can be measured, and where it cannot

One constraint is essential, and it is the point at which this book’s argument becomes an argument about architecture rather than about metrics.

The evidential base has two parts and they are not equally tractable.

For **retrieval-provided evidence**, condition (G2) is directly testable. The evidence is identified, it is present in the context, and you can ablate it, contradict it, or perturb it and observe whether the assertion survives. The test is cheap, automatable, and runs on live traffic.

For **parametric evidence**—whatever the training corpus contributed to the weights—it is not. “Had E not supported p ” ranges over counterfactual training corpora, and evaluating it properly means retraining. Training-data attribution methods approximate the counterfactual, and at frontier scale they remain a research problem rather than an instrument. The difference between the two cases is one of kind, not degree.

So (5.1) is well defined for any system and estimable only for systems that make their evidential base explicit at inference time. An unaugmented model’s ungrounded assertion rate is a perfectly good quantity that nobody can currently measure.

This is not a limitation of the definition. It is a requirement on the system, and it is the strongest practical conclusion of this book. If the rate at which a system asserts beyond its applicable evidence is the quantity that governs whether you may rely on it, then the system has to be built so that the quantity is observable: an identified, versioned, context-resolved evidential base at the point of assertion, not a training corpus somewhere behind it. Section 2 sets out what that requires.

Part IV

The Target Architecture

6 A Case Traced End to End

The last three chapters said what these systems do not supply and what a system must supply instead. Before turning to how such a system is built, it is worth watching a single case go wrong, because the failure is not dramatic and that is the point.

This chapter traces one request through every condition, showing what each invariant produces as a record and where the assertion would have been stopped. The case is constructed to exercise applicability, both time dimensions, and the defeater search at once; it is an illustration of the failure shapes described above rather than a report of a measured incident.

The request. An operator asks: “*Can we rely on the accelerated pathway for this filing?*” A system with a free-form interface will answer. Nothing in the sentence is ill-formed.

Note what *this filing* is doing. The request does not arrive cold: the filing has been discussed in earlier turns of the session, which is how these questions are actually asked, and it is why the operator sees nothing missing in the sentence. Some of what *c* needs is therefore recoverable from the conversation — the entity, the product, plausibly the filing date.

That recovery is not resolution, and the distinction is the one drawn in Section 1.1.2: Resolved(*c*) requires the components to be explicit rather than inferred without validation. A value carried forward from an earlier turn is an inference from context, and by Section 2.3 the context is a token sequence the model conditions on, not a validated field. If the operator mentioned two filings, or mentioned the entity before the product changed, the carried-forward value may be the wrong one, and nothing in the record will show which happened. The case is made harder, not easier, by the request being well situated in a conversation.

Context resolution (I10). Every component of *c* is either unstated or inferred rather than established. The system resolves, or refuses:

<i>j</i>	jurisdiction of the filing — <i>not stated</i> ; two candidate regimes apply to the entity
<i>t_v</i>	valid time — the filing date, 14 months ago, <i>not stated</i> ; the operator assumed today
<i>f</i>	material facts — entity class and product type, <i>partially stated</i>
<i>u</i>	intended use — a binding submission, not an internal estimate, <i>not stated</i>

Four of four components unresolved, inferred, or wrong — and note that *partially stated* is not a partial pass: a material-facts component that is half established leaves the applicability test half evaluable, which is to say not evaluable. Under I10 this request does not proceed; it returns a resolution prompt. **Most deployments stop being defensible here**, and nothing downstream can recover it, because a well-grounded answer to the wrong question is indistinguishable in the record from a well-grounded answer.

Retrieval under applicability (I11). With *c* resolved, retrieval filters on scope rather than similarity alone. Similarity search returns guidance for both jurisdictions and for the current period; the applicability predicates admit only instruments governing *j* whose validity interval

contains t_v . Two of the five highest-similarity passages are excluded: one for jurisdiction, one for anachronism—it postdates the filing by four months and would have supported the answer the operator expected.

Temporal checks (I9). The corpus watermark t_k is today; t_v is 14 months ago. By (1.3) the base may contain instruments not in force at t_v , which is what the previous step caught. Had t_k predated t_v , (1.2) would flag possible omission instead, and the correct action would be to decline rather than to answer from a base known to be incomplete.

Support and sufficiency (I12, I13, I18). The surviving passage supports eligibility for the pathway. Claim decomposition splits the response into four atomic claims; three link to that passage. Coverage checking against the enumerated conditions of the rule finds the passage addresses four of five: it is silent on a filing-history condition. Sufficiency fails. The correct output is a qualified answer naming the unaddressed condition, not an unqualified yes.

Defeater search (I19). The governed contrary query returns a supervisory statement of equal standing, in force at t_v , narrowing the pathway for this entity class. It does not contradict the primary passage—it *undercuts* it, by removing the entity from scope. A search for contradictions would have found nothing. NoDefeater fails, and the case is **CONTESTED** in the sense of Section 3.11 unless the institution’s precedence order separates the two instruments. Here it does not: both are of the same standing.

Gating (I17). Three conditions have failed—applicability caught two candidate sources, sufficiency is unmet, and a defeater exists. The gate returns an escalation with the unresolved question stated, not an answer with caveats.

What the record contains (I14, I15, I16). Resolved c ; the corpus release and t_k ; every retrieved passage with source identity, version, and validity interval; the applicability exclusions and their reasons; claim-level links; the coverage gap; the defeater and its standing; the gate decision; and the escalation recipient.

What each metric would have said. This is the point of the exercise.

Measurement	Verdict	Why
Reference-based accuracy	possibly correct	The unqualified answer may match a reference answer written without c
Faithfulness / attribution	clean	The citation is accurate and the passage does entail the claim
Context relevance	high	Topic matches exactly
Ungrounded assertion rate	hallucination	Applicability, sufficiency, and no-defeater all fail

Three of the four established measurements pass a case that should not have produced an assertion at all. That is the book’s argument in one row.

7 Governed Retrieval: A Reference Architecture

Section 5.3.5 established a requirement rather than a preference: the grounding rate is measurable only where the evidential base is explicit at inference time. Section 3.13.4 established that standard retrieval supplies one of seven grounding dimensions reliably and none of the three that are relations to *c*. This section sets out what closes the gap.

The distinction I will use throughout:

Standard retrieval retrieves evidence. *Governed* retrieval establishes the controlled conditions under which retrieved evidence may support an institutional assertion.

This is architecture, not prompt engineering. None of the eight components below can be implemented as an instruction to the model, and Section 2.7 says why: instructions enter as tokens and are processed by the same mechanism as the content, so an instruction to abstain is a probabilistic influence on a continuation rather than a precondition on output.

7.1 The underspecified-request principle

Before the components, the requirement that motivates the first two of them.

Natural language is an incomplete specification for consequential action.

A fluent request routinely leaves unresolved: the referents; the jurisdiction; the effective time; the applicable policy; the objective; the requester’s authority; the constraints; the side effects; the reversibility; the threshold at which action should be taken; and whether the requester wants analysis, a recommendation, execution, or a binding determination.

The point is not that natural language is a poor interface. It is an excellent interface, which is why these systems are worth building. The point is that a fluent request cannot be treated as a *complete operational specification* in a regulated decision or execution workflow, and that the gap between the two is invisible in the request itself. A well-formed sentence does not announce which of its parameters were never bound.

Two consequences. The system must force or validate material bindings before crossing into consequential action—which is I10. And the boundary at which it crosses has to be an explicit architectural decision, because by Section 2.6 the training regime supplies pressure to answer rather than to ask.

7.2 Eight components

1. Authoritative corpus control. Sources are approved, authenticated, and governed. Each carries stable identity, version, effective date, jurisdiction, authority level, and applicability metadata, and sources can be pinned to a corpus release or a decision-time snapshot. This supplies I8, I9, and the substrate for I11 and I14. It is the component most familiar from ordinary data governance and, in my experience, the one most often assumed to be sufficient on its own.

2. Context resolution before assertion. Material variables— j , t , f , u —are resolved, collected, or explicitly marked unknown before an assertion is produced. Where material context is missing the system requests it, qualifies the output, declines to produce a decision-ready assertion, or escalates. This supplies I10 and is the only defense against illicit universalization (Section 3.13.1). It is also the component with the highest product cost, because it makes the interaction less immediately satisfying, and it is therefore the first to be cut.

3. Metadata-aware retrieval. Retrieval filters or ranks on scope constraints, not on semantic similarity alone: jurisdiction predicates, effective date windows, population or product class, decision type. The retrieval event and the resulting evidence set are retained. This supplies I11, and it is the direct answer to the observation of Section 3.13.4 that a paediatric and an adult dosing passage are near-neighbours in embedding space.

4. Claim decomposition. Complex outputs are decomposed, where feasible, into atomic claims, each with linked evidence and an explicit support relation. This supplies I12. A response-level citation list does not permit checking the third sentence, and the third sentence is frequently the one relied upon.

5. Evidence-to-assertion verification. Citation presence is insufficient. The system checks whether the cited evidence supports the claim, and whether it does so under c and at the strength asserted (Section 3.13.3). Controls may combine deterministic rules, structured policy logic, source checks, independent model review, and human validation, proportioned to risk. This supplies I13 and is where the Sensitive condition of Definition 3.3 is actually enforced rather than assumed.

6. Output gating. If evidence is missing, stale, conflicting, inapplicable, or insufficient for the strength of claim requested, the system does not present an unqualified assertion as decision-ready. It returns bounded information, an explicit uncertainty, a request for missing context, or an escalation path. This supplies I17, and it must sit outside M .

7. Audit record and replay. Retain the prompt or structured input; the resolved context; source identifiers and versions; retrieved passages; model version; prompt or template version; system configuration; verification results; human interventions and approvals; the final response; and timestamps with retention identifiers. This supplies I14 and I16, and it is what answers the question posed at the start of Section 1.

8. Authorization and tiered reliance. Permitted use depends on the consequence of reliance, not only on model quality. Higher-consequence uses require stronger constraints, independent validation, and where appropriate a qualified human decision-maker with the standing to be accountable. This supplies I15, and the tiers of reliance that make it proportionate are developed with the definition of warrant.

7.3 The flow

raw generative output → context resolution → controlled retrieval → applicability filtering
→ evidence-to-claim verification → authorization or escalation → auditable decision support

Two things about this sequence carry most of its value, and neither is the ordering.

Each arrow is a *stopping point*. Context resolution can return a request for missing bindings; applicability filtering can exclude every candidate source; verification can find that the evidence does not reach the claim; the defeater search can find contrary authority; and authorization can decline. A pipeline in which the only outcome is an answer has implemented the boxes without the arrows, and it is the arrows that make the conditions enforceable rather than advisory.

Each arrow is also a *record*. What the audit trail contains is not a summary of the final answer but the sequence of determinations that produced it: which context was resolved and how, which sources survived filtering and which were excluded and why, what the verification returned, what the defeater search returned, and who authorized the result. That is what makes the flow reconstructable months later by someone who was not present, which is the requirement Section 1 opened with.

The sequence is not a confidence gradient. It is a sequence of distinct conditions, and an assertion that satisfies six of them is not ninety per cent warranted; it is unwarranted for a reason that the record should name.

7.4 What each intervention buys

Common interventions address different invariants, and conflating them is how programs spend a great deal and move one number. The table below is best read as a purchasing guide: the left column is what a program funds, the middle column is what that purchase actually delivers against Table 5.2, and the right column is what it is frequently assumed to deliver and does not.

Two features of it are worth pointing out before reading it. Retrieval appears with two distinct entries in the middle column because it does two unrelated things: it reduces fabrication, which is why it is usually funded, and it supplies an identified evidential base, which is what makes (5.1) measurable at all and is the more durable of the two. And no single row delivers more than a handful of invariants, which is the practical form of an argument about ownership: there is no purchase that makes a system auditable, only a set of purchases that between them cover the conditions.

Intervention	Buys	Does not buy
Standard retrieval	I8; makes (5.1) measurable; reduces FABRICATION	I10–I11, I13; form invariance; applicability
Better base model	fewer fabrications on covered topics	I8–I19; may worsen LUCKY by guessing better
Verifier loops	I13 on checkable claims; some of I2–I4	grounding, unless the verifier checks evidence rather than form
Persistent memory	a precondition for I7 and I14	either of them—storage is not retrieval discipline
Context resolution	I10, and the substrate for I11	anything on the generation path
Output gating	I17, and thereby enforceability of the rest	correctness of the evidence itself

Table 7.1: Retrieval does double duty in most deployments: it is funded to improve accuracy, and its more durable contribution is making the grounding rate observable at all.

The row worth dwelling on is the second. A better base model improves the numbers most programs report while leaving every system invariant untouched, and its effect on the lucky cell runs the wrong way: a model that guesses better produces more cell-(ii) assertions and scores higher for doing so.

7.5 Structured evidence is the strongest case

The components above are described in terms of retrieved passages, which understates what is available. The highest-assurance form of E in an enterprise is not a document at all. It is a *query against a system of record*, with the query text, the parameters, the result set, and the as-of state all recorded.

Structured evidence satisfies the grounding conditions more cleanly than text retrieval does, because the evidence has a schema. Authenticity follows from the source system’s own controls. Versioning is the table’s temporal model rather than a document revision. Applicability becomes a set of predicates over columns—jurisdiction, effective date range, product class, population—evaluated rather than judged. Sufficiency becomes a completeness check over required fields. Traceability is the recorded query. And the defeater check of (3.3) becomes a query for contradicting rows rather than a search for contradicting prose.

The practical consequence is a preference order that most retrieval-first designs invert: where a claim can be answered from a governed table, it should be, and the model’s role is to render the result rather than to find it. Text retrieval is the fallback for claims whose evidence exists only as prose, and it inherits every judgment that the schema would otherwise have decided.

7.6 What this costs

The controls are not free and a proposal that does not say so is not usable. The multipliers can be given structurally, without benchmarks, because they follow from how many model calls and retrievals each control requires.

Let a response contain m consequential claims, each with up to k candidate supporting sources. Generation is one call. Claim decomposition is one call plus $O(m)$. Support verification is $O(mk)$ calls where it is model-based, and $O(1)$ per claim where the check is deterministic. The defeater search is one additional retrieval per claim, $O(m)$. Coverage checking is $O(m)$ against an enumerated rule and unbounded against a rule that must be read out of prose. So the dominant term is mk , and the design levers follow directly: reduce claims per response, reduce candidate sources per claim, or replace model verification with deterministic checks. A ten-claim response over three candidate sources each is on the order of thirty verification calls against one generation call—which is the difference between a system that costs what a chat costs and one that does not.

Storage grows with retained evidence rather than with traffic, and Section 7.7 constrains what may lawfully be kept.

Tiered reliance exists for this reason and not only for governance tidiness. If every request bore the full control set the economics would not work, and a design that applies Tier 3 controls to Tier 1 traffic will be abandoned rather than descoped.

Component	Cost profile	Notes
Corpus control, metadata	one-off plus upkeep	Ontology and curation effort; no per-request cost
Metadata-aware retrieval	negligible per request	Predicate filtering is cheaper than the embedding search it constrains
Context resolution	one round trip, or a form	Latency and interaction cost, not compute
Claim decomposition	multiplies calls	Linear in claims per response
Support verification	multiplies again	The dominant cost. Deterministic checks are cheap; model-based entailment is per claim per source; human adjudication is the expensive tail
Defeater search (I19)	one extra retrieval	The governed contrary query, run under the same applicability predicates. Cheap, and it needs the precedence order to exist
Coverage checking (I18)	per claim	Cheap where the rule's conditions are enumerated; expensive where they must be inferred from prose
Audit record	storage, growing	See Section 7.7 on what may lawfully be kept

Table 7.2: Cost profile of the controls. The expensive items are per-claim verification and human adjudication, which is precisely why tiering makes them proportional to consequence.

7.7 Retention against minimization

The audit record of component 7 conflicts with another control regime, and the conflict should be stated rather than discovered in review. Prompts, resolved contexts, and retrieved passages routinely contain personal or special-category data. Data minimization and erasure obligations cut directly against the retention that reconstructability requires.

Four resolutions, none complete on its own. Retain references, identifiers, and hashes in place of content wherever the content is recoverable from a governed source, so the audit record points at evidence rather than copying it. Segregate the audit store and control access to it separately from the application. Align retention to the retention schedule of the *decision* the assertion supported, rather than choosing a new period for the AI system. And where erasure is exercised, preserve the structure of the record—that a claim was made, on what evidence class, under what authority—while removing the identifying content.

The point for this book is narrower than the resolutions. Reconstructability is a requirement in tension with minimization, the tension is a design decision rather than an implementation detail, and it requires a fifth function: privacy and records management, which joins the three that own the rest.

7.8 What this does not claim

Governed retrieval does not make these systems infallible, and nothing above should be read as proposing that it does. Cell (iii) survives it entirely: a controlled, versioned, applicable,

authoritative corpus can still be wrong, and no amount of architecture converts a bad source into a good one. Verification is relative to the checks implemented and the assumptions they encode. Human review introduces its own failure modes and its own capacity limits. And the floor of Section 2.8 does not move.

The function of the architecture is narrower and worth stating precisely. It transforms otherwise unlicensed generated text into a controlled, inspectable input to an accountable institutional process—one whose failures are attributable, whose evidence is identifiable, and whose reasoning can be reconstructed after the fact by someone who was not present. That is a lower bar than reliability and a much higher bar than accuracy.

7.9 Bounding the Request Space

Section 4 argued that the absence of a canonical form is what makes accuracy uncertifiable, and Section 4.4 argued that training a semantic normalizer is not the remedy. There is a third option, and it is the one an architect reaches for first: if the model will not canonicalize its input and cannot be taught to, *do not give it free-form input at all*.

Restricting the interface is not a workaround. It is the direct application of the diagnosis: canonicalization moves upstream of the model, into a schema, where it is inspectable and versioned. This section works through what that buys, what it does not, and where the residual risk goes—because it does not disappear.

7.9.1 The construction

Replace free text with a typed request. Instead of “is this approved for children?” the interface accepts an object:

<code>intent</code>	INDICATION_CHECK (from a closed set)
<code>subject_id</code>	a key into a system of record
<code>indication</code>	a coded value
<code>jurisdiction</code>	required
<code>as_of</code>	required
<code>population</code>	required, from a controlled vocabulary

The object is rendered into a fixed template by code the organization controls and versions. Requests are stateless: one request, one fresh context.

The fields are not arbitrary, and the discipline that selects them is the point rather than the example. Every component of *c* appears, because Section 1.1.2 established that a claim whose context is unresolved is not yet evaluable—so jurisdiction, valid time, and intended use are required fields, and a request missing any of them does not validate. The material facts *f* enter as keys into systems of record rather than as free text, so that the entity under discussion is identified rather than described. The intent is drawn from a closed set, because the decision class determines which sources have standing under standing and therefore what counts as authoritative. And coded values replace prose wherever a controlled vocabulary exists, since that is what allows applicability to be evaluated as a predicate in Section 7’s third component rather than judged.

What the schema encodes, in other words, is the set of bindings that would otherwise have to be guessed. Anything the type system requires is a thing the system can no longer silently assume.

7.9.2 What this fixes

Against Table 5.2, the gains are specific and substantial.

I1 becomes satisfied by construction. Not because the model acquired form invariance, but because the opportunity to violate it has been removed. Two materially equivalent requests are now *the same request*: the equivalence class of Section 4 has exactly one member, and the canonical form is supplied by the schema.

The request space becomes enumerable. It is a product of typed fields rather than $\mathcal{V}^*$. It may be combinatorially large, but it is *stratifiable*: coverage can be designed by jurisdiction, date band, population, and intent, with known sampling properties. That is ordinary test design. The validation problem becomes well posed in the sense of Section 2.10—the same sense in which it is well posed for a classifier.

I10 is enforced by the type system rather than detected. A request without a jurisdiction does not validate. The misapplied category of Section 3.10 does not need to be caught downstream, because the binding it depends on is a precondition of submission. This is the single largest gain, and it is available with no machine learning at all.

I7 becomes achievable. Statelessness removes context accumulation, which is the dominant source of within-session drift. Combined with the reproducibility conditions of Section 5.3.2, replay becomes a realistic control rather than an aspiration.

7.9.3 What this does not fix

The output space is still $\mathcal{V}^*$. Bounding the input bounds the input. The model can still emit any string, so ungrounded assertion survives untouched: a confident figure resting on nothing is exactly as available as before. Grounding is I13 and no amount of input discipline supplies it.

Errors are still not label mismatches. If the response is prose, it can be partly right, right for the wrong reason, or adequate for a weaker claim than the one made. The atomic countable error of the classifier case does not follow from a typed input.

7.9.4 Where the problem goes

This is the part that decides whether the design is an improvement or a disguise. **Who fills in the request?**

If a person translates their need into the typed object, the ambiguity has not been eliminated; it has been moved into that translation, and it is unrecorded unless the system captures it.

If a language model performs the translation—which is what is usually built, because it preserves the interface that made the technology attractive—then the unbounded input space has been reintroduced at the parsing stage. Form-invariance failure now determines *which request object is constructed*. A paraphrase can produce a different jurisdiction, a different population, or a different intent, and everything downstream will process that object correctly and produce a well-grounded answer to the wrong question.

This is worse than the original failure in one specific respect: it is less visible. An ungrounded assertion at least looks like an assertion, and the audit record shows what evidence was and was

not used. A misparsed request produces a record that is internally consistent and complete. Every control passes. The evidence applies to the context in the record, and the context in the record is not the user’s context.

The mitigation is not technical. It is to **render the resolved request back and require confirmation before proceeding**, which converts a silent failure into a visible one and puts the binding decision where accountability already sits. This is the pattern enterprise systems have always used before consequential execution, and the argument of this book is that a natural-language front end does not earn an exemption from it. Where confirmation is impractical, the parse should be treated as part of the assertion for audit purposes: recorded, versioned, and measured for accuracy against reviewed samples like any other component.

7.9.5 Constraining the output, and what that does and does not buy

The natural complement is to constrain the output as well: grammar-constrained decoding into a schema, with an enumerated verdict field, required evidence references, and an explicit `INSUFFICIENT_EVIDENCE` variant.

This is worth doing and its benefits should be stated exactly, because it is easy to credit it with more than it delivers.

It *does* guarantee that the output parses, that the output space is enumerable, and that errors become atomic and countable again—restoring on the output side the well-posedness that typed requests restore on the input side. It *does* make abstention a first-class value rather than a hoped-for phrasing, and it makes per-field evidence linkage (I12) structurally enforceable rather than conventional.

It does *not* satisfy I17. A grammar constrains which strings can be emitted; it does not determine which the model selects. The choice between a substantive verdict and the abstention variant is made by the same next-token computation as everything else, subject to the same pressures identified in Section 2.6. Constrained decoding makes abstention *expressible*, not *correctly triggered*.

I17 is therefore satisfied by the check, not by the schema. The evidence conditions of Definition 3.2 are evaluated outside the model, and the gate must be able to *override* a substantive verdict the model produced—replacing it with a qualification, a refusal, or an escalation—when those conditions are not met. A system that trusts the model’s own choice of variant has implemented a vocabulary for abstention and no gate.

7.9.6 Where this is practical

The design suits high-volume, repetitive, well-typed decisions: eligibility determinations, coding against a rulebook, standard disclosures, reconciliations, completeness checks. The user is an operator working a queue against a known decision class, and the intent set is small and stable.

It suits exploratory work badly, and that matters, because exploration is a large part of why these systems were adopted. An analyst pursuing an unanticipated question has no intent to select.

Two costs deserve naming.

The intent taxonomy is an ontology project. Enumerating decision classes, coding their fields, and maintaining controlled vocabularies is the same work as building a conformed dimensional model, with the same effort profile and the same tendency to be underestimated.

An incomplete taxonomy fails quietly. A question outside the intent set does not raise an error. It is routed to the nearest available intent, which is a misparse of the kind described in Section 7.9.4. Twenty intents may cover most of the volume, and the residual tail is disproportionately where the unusual, high-consequence cases live. A closed intent set therefore needs an explicit *no matching intent* outcome and a route for it, or the constraint will silently manufacture the failure it was introduced to prevent.

7.9.7 The effect on what the system is

One consequence should be faced directly, because it changes how the component should be described and governed.

If the input is typed, the output is schema-constrained, and the evidence conditions are checked outside the model, then it is fair to ask what the model is contributing. Frequently the honest answer is: retrieval, summarization, and the rendering of a determination into readable language—while the determination itself is made by deterministic rules over structured evidence.

That is a legitimate and valuable architecture, and it is one enterprises already know how to validate. But it *reclassifies* the system. The model is a retrieval and presentation layer over a decision service, not the decision-maker, and the assurance argument should say so. A program that describes such a system as “AI-driven decisioning” is overstating the model’s role and, more importantly, misdirecting its own validation effort toward the component that is doing the least consequential work.

There is a further consequence worth facing, because it follows from combining this section with Section 7.5 and a reader will combine them. If structured evidence is the strongest available form of *E*, and typed requests plus constrained output are what reach the higher reliance tiers, then the architecture that best satisfies this book’s conditions is one in which the model does comparatively little of the deciding. Taken to its conclusion, that is an argument for routing decidable claims *away* from the model altogether.

I take that to be a feature rather than an embarrassment. A framework that identifies which decisions do not need a language model is more useful than one that assumes every decision does. What remains for the model, once the decidable residue is routed away, is the genuinely open-textured part: synthesis across heterogeneous sources, questions whose evidence exists only as prose, interpretation where the governing text is ambiguous, and interaction with people who cannot state their need in a schema. That is a smaller claim about the technology than the field usually makes, and a more defensible one—and it is the part where the measurement problem of Section 5.3 genuinely bites, because it is the part no schema can decide.

Tiered reliance is where this resolves. Bounding the request space is not a general answer to the problem of ungrounded assertion; it is the mechanism by which a narrow, well-understood decision class can be moved to a higher reliance tier than free-form interaction can support. Free-form interfaces are appropriate at Tiers 0 to 2. Reaching Tier 3 and above is, in practice, an exercise in bounding what may be asked and what may be answered.

8 Scale, Scope, and What Is Actually Tunable

The architectural choices of Sections 7 and 7.9 were presented as though the model were fixed. It usually is not. The question an architect faces early is whether to consume a large general-purpose model as a service or to operate a smaller model over a defined domain, and the two options are routinely compared on accuracy. This section argues that accuracy is close to irrelevant to the comparison, identifies the two variables that do govern it, and declines a tempting conclusion that does not follow.

8.1 What scale does, and what it cannot do

Scale improves accuracy. That is not in dispute and nothing here argues otherwise. What matters is what an accuracy improvement licenses, and by Proposition 3.1 it licenses nothing about the grounding rate. The two quantities are not linked by any entailment, so a model that scores better tells you it scores better.

There is a stronger point available, and it follows from Section 2.4 rather than from any claim about how well these systems work. The objective (2.4) contains no term that is a function of truth. Accuracy gains from scale are therefore gains in modeling the training distribution, which correlates with truth to the degree the corpus is truthful. They are not convergence toward truth as an optimization target, because no term in the loss distinguishes a true continuation from a plausible false one where the two diverge. Scale sharpens the estimate of the distribution; it does not introduce a distinction the objective never drew.

This is worth stating carefully, because it is easy to overstate in either direction. The claim is not that a larger model is not more often right—it is. The claim is that being more often right on a distribution is a different property from standing in an auditable relation to evidence, and that no amount of the first produces the second. The mechanism that would produce grounding is absent from the architecture, per Section 2.7, and adding parameters does not add a mechanism.

8.2 Reproducibility is a property of topology, not of size

Invariant I14 is where the comparison is usually miscast. Reproducibility does not scale, in either direction, because it is not a function of the model at all. It is a function of who operates the inference.

Section 5.3.2 listed the conditions: pinned model versions, controlled batching or single-request inference, deterministic kernel selection, recorded decoding parameters, statelessness. Every one of those is a property of the serving stack. A parameter count appears in none of them.

What is true, and is the defensible core of the common intuition, is that the *typical consumption pattern* for large models makes those conditions harder to obtain. A shared multi-tenant endpoint batches your request with other traffic, so batch composition is outside your control; sparse expert routing under capacity limits makes that batch composition affect your output; and the served version may change without notice. None of that follows from the model being large. It follows from the model being operated by someone else.

The distinction is not pedantic, because it changes what an architect should do about it. “Prefer a small model” is not actionable advice and is sometimes wrong. “Obtain version pinning,

deterministic serving, and stateless requests—by self-hosting, by contract, or by choosing a deployment that offers them” is actionable, and it is available for open-weight models of any size.

8.3 Corpus scope changes what can be measured

Here the comparison is genuinely asymmetric, and in a way that has nothing to do with parameter count and everything to do with what the evidential base contains.

Two of this book’s measurements require establishing a *negative*: that the evidence does not support a claim. The abstention items of the evaluation protocol need it by construction—an item whose correct behavior is silence is only well formed if you can establish that the answer is absent from what the system can draw on. And the ungrounded assertion rate (5.1) needs it for the parametric half of E , which is why Section 5.3.5 concluded that half is largely unmeasurable.

Negative claims about a bounded, enumerated, versioned corpus are tractable. You can determine that a controlled document set does not address a question, by search over an enumerated and versioned index rather than by inference about what a model absorbed. For a corpus of any size that is work rather than inspection, and it is tractable in the way any completeness check over a known population is tractable. Negative claims about a web-scale training corpus are not tractable with current attribution methods, and the gap is one of kind rather than maturity: the question is what a model did *not* absorb, and attribution techniques estimate influence of what it did.

A related asymmetry follows from Section 2.8. The statistical floor on ungrounded assertion is highest for facts that are rare in the training distribution—and *rare* is only definable relative to a known corpus. For a domain corpus you can characterize coverage and therefore estimate where the floor sits. For a frontier training set you cannot, so the floor is present, consequential, and unquantified.

So bounding the corpus does not merely improve accuracy on the domain. It changes which properties of the system are *measurable at all*, which is a different and more important kind of gain. It is the same move as Section 7.9, applied to the evidence rather than the request: enumerability is what makes negative statements available, and negative statements are what auditing requires.

8.4 What the dials actually buy

Collecting the choices available to an architect against the invariants they affect:

The ordering is the practical conclusion of this section, and it is somewhat counterintuitive: the variable that dominates procurement discussion has the smallest *direct* effect on whether the resulting system can be audited.

One indirect path is real and should not be buried in a table. Where an in-scope model performs the context resolution or request construction of Section 7.9.4—translating free text into a typed request—its capability bears directly on I10, and a misparse there produces the most dangerous failure in the architecture: a complete, internally consistent record answering the wrong question. So model capability does matter, at exactly one point, and the conclusion to draw is not that a larger model fixes it. It is that the parse must be made *reviewable*, per Section 7.9.4, because a control that depends on model capability is a control whose failure rate is unknown. Better capability narrows that failure rate without bounding it; confirmation bounds it.

Choice	Invariants	Effect on auditability
Typed request interface	I1, I7, I10	Large. Supplies the canonical form and enforces context binding
Controlled, versioned corpus	I8, I9, I11	Large. Precondition for every evidence condition
Bounded, enumerable corpus	enables I13, I17 testing	Large. Makes negative evidence claims and abstention items possible
Schema-constrained output plus external gate	I12, I13, I17	Large. Atomic errors; abstention enforced rather than expressed
Operational control of inference	I14, I7	Moderate to large. Version pinning, batching, statelessness
Claim decomposition	I12	Moderate. Claim-level rather than response-level linkage
Model parameter count	none directly	No direct effect. Affects accuracy, which by Prop. 3.1 bounds nothing. One indirect path matters—see below

Table 8.1: Architectural choices ranked by effect on auditability. Model size is the dial most often debated and the one with the least direct effect.

8.5 The conclusion that does not follow

It is tempting to conclude that small domain models can be governed and large general-purpose models cannot. That does not follow, and it is worth saying why, because the error would misdirect real spending.

Every condition in Table 5.2 is a property of the system surrounding the model. None mentions parameter count, training corpus size, or provenance of the weights. A large model placed inside a governed architecture—typed requests, controlled corpus, external evidence gate, defeater search, audit record, pinned version—satisfies those conditions exactly as a small model would, because the conditions are about the architecture. Conversely, a small model consumed through a shared endpoint with no version pinning and free-form input satisfies almost none of them.

The two genuine axes identified above are *operational control of the inference stack* and *enumerability of the evidential base*. Both correlate with model size in current practice, and neither is caused by it. Self-hosted open weights give operational control at any scale. A retrieval corpus can be bounded and versioned whatever model reads it.

The claim I would defend is therefore narrower than the intuition and more useful than it: **you cannot govern inference you neither operate nor contract for, and you cannot make negative evidence claims about a corpus you cannot enumerate.**

The contractual half deserves emphasis, because enterprises govern third-party processors routinely and the instruments are familiar. What would otherwise be configuration becomes a term, and the terms are nameable: version pinning with a defined support window; advance notice of model and serving changes; a deterministic or single-tenant serving option; recorded decoding parameters; retention of request and response for a stated period; audit rights over the serving configuration; and restrictions on sub-processing. An organization that cannot obtain those has not discovered a property of large models—it has discovered the terms it accepted.

The practical conclusion is a procurement checklist rather than a prohibition, and it is the one place in this book where the binding control is legal rather than technical.

One asymmetry does survive, and it is worth stating because it is where the intuition is right. Scale is offered as a solution to accuracy, and accuracy is not the constraint. So an organization that responds to a grounding problem by buying a larger model has bought an improvement it cannot use for the purpose it needs, and has usually given up operational control to obtain it. That is not an argument against large models. It is an argument against treating model capability as the lever when the binding constraint is architectural.

Part V

Governance in Practice

9 Institutional Warrant and Tiered Reliance

The architecture of Section 7 serves a condition I have so far used without defining. This section defines it and makes the controls proportional to consequence.

9.1 Warrant

Grounding is necessary for reliance and not sufficient. An assertion can rest on authentic, applicable, authoritative evidence, be traceable to it, and still be something the institution is not entitled to act on—because nobody checked it, or because nobody with the standing to authorize reliance did so.

Definition 9.1 (Institutional warrant).

$$\begin{aligned} \text{Warranted}_I(p \mid c) \leftrightarrow & \text{Resolved}(c) \wedge \text{Grounded}_S(p \mid c) \\ & \wedge \text{Verified}(V, p, c) \wedge \text{Authorized}(I, p, c). \end{aligned} \quad (9.1)$$

where

- $\text{Resolved}(c)$: the material context variables of (1.1) are explicit rather than assumed or inferred without validation;
- $\text{Grounded}_S(p \mid c)$: Definition 3.3—which unpacks to the six evidence conditions of (3.1) plus Sensitive, so authenticity, versioning, authority, and applicability are preconditions of valid support rather than separate requirements;
- $\text{Verified}(V, p, c)$: an appropriate procedure has checked the assertion, its evidence relation, and the controls the decision class requires;
- $\text{Authorized}(I, p, c)$: the institution permits reliance at the relevant risk tier, possibly subject to human review or approval.

Three properties of (9.1) matter for how it is used.

It is a conjunction, not a score. There is no aggregation of the four conjuncts into a confidence figure, and an assertion satisfying three of them is not three-quarters warranted. This is the same point as the flow in Section 7: these are distinct conditions with distinct owners and distinct failure records.

It is a property of a process, not of a model. Nothing in (9.1) is a property of model weights, a benchmark score, or a generated response. Authorized in particular is a fact about an institution’s governing rules and delegations, and no improvement to M can supply it. This is why warrant cannot be procured: a vendor can sell you a component that makes grounding measurable, and cannot sell you an authorization.

It is revocable and context-relative. (9.1) is not a claim that p is certain, or even that p is true. It is the condition under which an institution may responsibly act on p for a defined purpose, knowing that it may be wrong and that the record will show what was relied upon and why. That is the standard ordinary institutional decision-making already meets, and the standard machine assertion currently does not.

9.2 Reliance tiers

Controls should be proportional to the consequence of reliance. The table below is deliberately compact: its purpose is to show that permissible reliance is set by consequence rather than by model quality, and that argument does not require a fully specified control catalog. Any real program will need its own tiering, calibrated to its regulator and its risk appetite.

Tier	Use	Required	Invariants engaged
0	Unwarranted generative assistance: brainstorming, drafting	Clear framing as unverified; no reliance claim	none
1	Informational assistance	Visible uncertainty; no reliance claim	I17
2	Evidence-linked assistance	Sources visible; user retains verification duty	I8, I9, I12
3	Governed decision support	Context-bound evidence, verification controls, audit trail, authorized human reliance	I8–I19
4	Constrained execution	Deterministic guardrails, approvals, reversibility, monitoring, full records	I8–I19 plus action controls
5	Autonomous consequential action	Narrowly specified settings, stringent formal controls, continuing assurance, explicit authority to act	all, plus independent assurance

Table 9.1: Reliance tiers. The tier is set by the consequence of reliance, not by model quality. Tier 5 is included for completeness and should be approached with considerable caution in regulated settings; nothing in this book argues that current systems are ready for it.

Assignment to a tier needs criteria, and while a full consequence taxonomy is out of scope, the dimensions are nameable: *reversibility* of the action the assertion supports; *magnitude* of the exposure if it is wrong; *breadth*, meaning how many parties are affected and whether they consented; *detectability*, whether an error would surface before harm; and *availability of a competent human check* in the time the decision allows. A use case scoring badly on reversibility and detectability belongs at a higher tier than its nominal risk rating suggests, because those two dimensions determine whether anything can be done after the fact.

The principle the table exists to enforce is a negative one. Permissible reliance does not rise because the model got better. It rises when evidence, verification, constraints, authorization, and accountability rise together and in proportion to what is at stake. A system that improves its benchmark score has not thereby earned a higher tier, and a system that has implemented I8–I19 has not thereby earned Tier 5.

Two boundaries are worth marking explicitly, because they are where programs drift. The step from Tier 2 to Tier 3 is the step at which the institution rather than the user assumes the verification duty, and it requires everything in Table 5.2; systems are frequently described as Tier 3 while implementing Tier 2. And the step from Tier 3 to Tier 4 is the step from assertion

to action, at which reversibility and blast radius become first-order design questions rather than operational details.

9.3 Three Teams, Three Budgets

Everything above is stated as though an organization were a single agent that could decide to satisfy Table 5.2. It is not, and the way it is divided is itself a principal cause of the failures this book describes. This section is the least formal in the book and, for practitioners, possibly the most useful.

9.3.1 The division

Responsibility for the conditions of warrant is distributed across organizational functions that were not designed to hold it jointly. In financial services the fourth row below is a second-line function with an independent reporting line and a supervisory mandate; in other sectors the equivalent may sit in quality assurance, clinical governance, or internal audit. Wherever it sits, it is the function that performs the independent challenge this book argues for, and it is the one most often absent from discussions of these systems.

Function	Typical responsibility	Owns	Failure if left in isolation
Data / knowledge	Source ingestion, curation, lineage, metadata, quality	I8, I9	Authentic documents with no policy applicability or output control
Policy / risk / compliance	Rules, jurisdiction, decision authority, approvals, procedures	I10, I15	Governing policy with no control over how assertions are retrieved, generated, or verified
Platform / AI engineering	Models, prompts, orchestration, retrieval, evaluation, monitoring	I11–I14, I17	Answer quality, latency, and usability with no evidentiary authority or institutional authorization
Model risk / independent validation	Independent challenge, validation, and approval of models and their use	I13, I15, I16	Validating M while the risk sits in S : a well-documented model, an unexamined evidence path

Table 9.2: Four functions, and the invariants each is positioned to supply. No function owns a complete path from source to authorized reliance.

These are not merely three technical workstreams. They typically have separate budgets, separate incentives, separate release cadences, separate accountability structures, and separate systems of record. A data team is measured on lineage coverage and freshness. A compliance function is measured on policy completeness and audit findings. A platform team is measured on answer quality, latency, adoption, and cost. None is measured on the grounding rate of (5.1), because it spans all three.

9.3.2 The unit of failure is the handoff

The consequence is that failures concentrate at the boundaries rather than inside any function's remit. A system may have:

- excellent data lineage and weak policy binding—I8 and I9 without I10;
- well-specified governing policy and uncontrolled retrieval—I10 without I11;
- high benchmark accuracy and no per-assertion provenance—nothing in Table 5.2 at all, with a validation report that looks complete;
- citations without applicability validation—I12 without I11, which is precisely cell (v);
- human review without a reproducible evidence path—review that cannot be re-performed and therefore cannot be audited, which is I13 without I14;
- a validated model inside an unvalidated system—documentation of *M*'s training, evaluation, and limitations, with no examination of context resolution, applicability logic, defeater checking, or authorization. This is the characteristic failure of the fourth row, and it is the most dangerous because it produces a formal approval.

Each of these is a system in which every function has done its job. The ungoverned handoff is the operational unit of failure, and it is invisible from inside any one function's metrics.

This also explains a pattern worth naming, because it is the most expensive form of the problem. Cell (v)—misapplied grounding—sits exactly on the boundary between the data function, which supplies authentic versioned sources, and the policy function, which knows which sources govern which decisions. The metadata required to enforce applicability is a policy artifact stored in a data system and consumed by a platform component. It is nobody's deliverable, and it is the single most common thing I find missing.

9.3.3 What follows

Two conclusions, and the first is uncomfortable for how these programs are usually organized.

Evidentiary controls must be architecture and cross-functional invariants, not after-the-fact review tasks. A review step owned by one function cannot establish a property that requires artifacts from two others, and adding reviewers does not change this. Table 5.2 is a specification for a system, and a specification that spans three budgets needs an owner who spans them.

A fifth function appears once retention is considered. Privacy and records management own the tension described in Section 7.7: the audit record that reconstructability requires is in direct conflict with minimization and erasure obligations, and neither the data, policy, platform, nor validation function is positioned to resolve it. That handoff has the same structure as the others and is usually discovered later.

And the grounding rate is the right cross-functional metric precisely because no single function can move it alone. Accuracy can be improved by the platform team without reference to the others, which is part of why it is the number that gets reported. (5.1) cannot: improving it requires corpus work, policy metadata, and pipeline control together. A metric that only improves when three functions cooperate is an unusually honest instrument for a problem whose failures live between them.

10 An Evaluation Protocol

The preceding sections argued for a property and described an architecture that makes it observable. This section is the operational residue: what to run, in what order, and what the numbers mean.

Four measurements, in the order I would add them to an existing evaluation pipeline. None replaces accuracy; all of them answer questions accuracy cannot.

10.1 Methodological commitments

Before the measurements, the decisions that determine whether any of them are comparable. A protocol that leaves these implicit produces numbers that cannot be read across systems or across time.

Unit of analysis. The *atomic consequential claim*, not the response and not the sentence. A response containing four claims yields four observations, and a sentence carrying two claims yields two. This choice follows from I12: if evidence links at claim granularity then measurement must too, or the metric will average a supported claim against an unsupported one in the same sentence.

Decomposition and its validation. Claim decomposition is itself a fallible step and must not be assumed correct. Validate it on a held-out sample against human decomposition, report agreement, and report the rate at which decomposition merges or splits claims wrongly, because those errors propagate into every downstream measure.

Labeling and adjudication. Authenticity and versioning can be determined mechanically from metadata. Authority, applicability, support, sufficiency, and defeat require judgment, and therefore require raters with domain competence, a written adjudication procedure for disagreement, and a reported agreement statistic. State which measure of agreement and on what sample; a grounding rate without it is one rater’s opinion at scale.

The instrument has its own error rate. If applicability is labeled by a model or an entailment classifier rather than by people, the error rate of that instrument bounds the precision of every rate it produces, and must be reported alongside. A grounding rate cannot be more accurate than the process that assigned it.

Sampling and strata. Sample from deployment-representative traffic, stratified by reliance tier, decision class, source class, and user population, with strata reported. Uniform sampling over traffic will under-represent exactly the high-consequence low-volume cases the framework exists to catch.

Correlated observations. Claims within one response share a context, a retrieval, and a resolved c , so they are not independent. Confidence intervals must account for clustering by response; treating claims as independent observations will overstate precision, often substantially.

Costs of over-control. Report false refusals and unnecessary escalations alongside the grounding rate, together with human-review load and latency. A system that improves its grounding rate by declining more has not necessarily improved, and Section 10.4 makes the same point about abstention: the pair is the measurement.

Baselines. Report against at least three configurations — generation without retrieval, retrieval by similarity alone, and governed retrieval — on identical cases. A single system’s grounding rate is uninterpretable; the framework’s claim is about the *separation* between configurations, which is what Appendix C is designed to measure.

10.2 Attribution audit: the ungrounded assertion rate

What it measures. Equation (5.1), decomposed by the three rows of Table 3.3.

Procedure. Sample assertions from deployment-representative traffic—not from a curated question set, since the point is to measure behavior on the distribution you actually serve. For each sampled assertion:

1. Identify the evidence the system had available, from the execution trace.
2. **(G1)** Determine whether any of it supports the assertion and is applicable at c : authentic, current at t , right jurisdiction and intended use. Failure here with support present is the misapplied rate.
3. **(G2)** Perturb: remove the putatively supporting evidence, or replace it with evidence for a contrary claim, and re-run. If the assertion survives unchanged, it was not resting on that evidence. Failure here is the ungrounded rate.
4. Where the assertion is grounded and false, record it against the *corpus*, not the system.

Honest limits. Step 2 requires judgment about applicability, which means human raters or a well-validated entailment model, and agreement on attribution is imperfect. Report the rate with its inter-rater agreement, not as a point estimate. And per Section 5.3.5, step 3 works for retrieval-provided evidence and does not work for parametric evidence: say which you measured.

10.3 Estimating evidence dependence, and why naive ablation fails

The sensitivity condition of Definition 3.3 is the one measurement in this book that requires a designed intervention rather than an inspection, and the obvious design does not work. Stating why is necessary, because the obvious design is what a reader will otherwise implement.

The confound. The natural test is ablation: remove the putatively supporting evidence and see whether the assertion survives. But removing a passage changes the token sequence, and Section 4 argued at length that behavior moves under semantically inert input variation. So a changed assertion after ablation cannot be attributed to the loss of evidential support rather than to the perturbation itself. An unchanged assertion is no cleaner: it is consistent with dependence on the evidence and equally consistent with reproduction from parametric memory, which is the case the condition exists to exclude. Naive ablation is confounded in both directions by the book’s own central argument.

The controlled design. Four requirements, and none is optional.

1. **Substitute, do not delete.** Replace the passage with a length-matched, structurally similar passage on an unrelated topic, so that the input is perturbed comparably without the evidence being present.
2. **Run an evidence-preserving control.** Substitute a *differently worded but equally supporting* passage. If the assertion changes under that too, the item has no measurement power and should be discarded rather than scored.
3. **Report the contrast.** The quantity of interest is the difference between the change rate under evidence-removing substitution and the change rate under the evidence-preserving control. A raw flip rate without the control is uninterpretable.
4. **Prefer contradiction where the case allows it.** Substituting a passage that asserts the contrary holds length and structure closest to the original while inverting the evidential content, and it additionally exercises the defeater path of I19.

What the protocol yields, and what it does not. It yields a graded *dependence estimate* under fixed model, decoding, context, and request conditions, not a proof of causal dependence. Redundant supporting evidence in the retrieved set will mask dependence on any single passage, so the unit of intervention should be the supporting *set* rather than the individual document. Stochastic decoding requires repetition and a stated decision rule for what counts as a changed assertion. And the error properties of the protocol itself — false positives from residual form sensitivity, false negatives from redundancy — must be reported alongside the estimate.

This is more expensive than the earlier framing in Section 5.3.5 suggested, and the correction matters: grounding is measurable at runtime for the evidence-applicability conditions, which are inspections, and measurable only by designed intervention for the dependence condition. A program that treats the second as cheap will produce a number that does not mean what it says.

10.4 Abstention items: catching the lucky cell

This is the single most useful addition to an existing suite, and the one that most directly changes what the suite selects for.

What it measures. Whether the system asserts in the absence of evidence.

Procedure. Construct items whose answers are *absent from the accessible evidence*, where the only correct behavior is abstention or an explicit statement that the basis has not been established. Then—and this is the part that inverts normal practice—**penalize correct answers on these items**. A system that answers such an item correctly has demonstrated only that its guessing distribution happened to align with the world, which is cell (ii), and rewarding it selects for exactly the decoupling of assertion from evidence that Section 2 identified as the root defect.

Construction. Straightforward where you control the retrieval corpus: hold out documents and ask about their contents. Harder for parametric knowledge, where establishing absence is the same intractable problem as before. Practical approach: build abstention items over a controlled corpus, and treat the resulting abstention rate as a property of the retrieval-grounded configuration rather than of the model.

The coverage constraint, which is not optional. Penalizing true answers on abstention items creates an obvious incentive: abstain more. Measured alone, the statistic is maximized by a system that never answers anything. So it must be reported *jointly with coverage*—the proportion of answerable items the system does answer—and read as a curve rather than a point, exactly as risk and coverage are traded off in selective prediction. A system that improves its abstention score while its coverage falls has not improved; it has become more cautious, which may or may not be what was wanted. The pair is the measurement. Either number alone is gameable in opposite directions.

Why it matters more than it looks. Most suites contain no items whose correct answer is silence. A system therefore never encounters a case where answering is penalized, and no optimization pressure toward abstention exists anywhere in the development loop. Adding these items is the cheapest available change to what the process selects for.

10.5 Form-invariance audit: measuring I1 directly

What it measures. How far the system is from invariant I1—a number for the property Section 4 says invalidates test suites.

Procedure. Build a battery of formulation pairs that are semantically equivalent and syntactically distinct: contraposed conditionals, negation duals (“no A are B ” / “all A are non- B ”), quantities in words against digits, active against passive, reordered conjunctions, identity statements in both directions. For each pair, measure divergence in assertion behavior at matched output probability. Aggregate to a single *form divergence* statistic—in the terms of the NLP evaluation literature, a measure of paraphrase robustness.

Two cautions. Matching output probability across paraphrases is not trivial, since the probability is itself formulation-sensitive; state the matching method. And control for the possibility that a paraphrase is harder for reasons of tokenization rather than logical form, or the statistic will measure the wrong thing.

The use that matters. Run the battery across model checkpoints, post-training stages, and inference-compute budgets. This answers a question the field currently assumes an answer to: whether additional scale, training, or inference-time computation moves systems toward form invariance, or merely broadens the set of formulations handled well. Falling divergence is genuine progress on I1. Rising accuracy with flat divergence is broader coverage, which does not restore the validation inference of Section 4.5. I am not aware of a systematic published measurement of this, which is itself worth noting given how much rests on the assumption.

10.6 Repeatability audit: measuring I7

What it measures. Whether a verified answer is a verified behavior.

Procedure. Re-issue identical inputs at multiple points within a session, across sessions, and after intervening turns of unrelated content. Report the proportion of inputs whose assertion changes, and report it separately for the three conditions, because they fail for different reasons: within-session drift indicates context accumulation, cross-session variation indicates a serving or version problem, and sensitivity to intervening unrelated turns indicates that conversational state is influencing content it should not touch.

Why it belongs alongside the form-invariance audit. I1 and I7 are the two invariants that invalidate a test suite, and they do so in complementary ways. Form-invariance failure means a passing test does not extend to the paraphrase; repeatability failure means it does not

securely cover even the string that was tested. Measuring the first without the second gives a misleading reassurance, because a low form divergence over inputs that are individually unstable is not evidence of anything.

What is acceptable. Unlike the other three measurements, this one has a defensible target: for a system intended to reach Tier 3, repeatability under fixed version and pinned decoding should be effectively total, because the conditions of Section 5.3.2 make it achievable. A non-trivial repeatability failure at that tier is not a tolerance to be negotiated; it is a sign that one of those conditions is not actually in place, and the audit's real value is in finding out which.

The measurement is cheap—no raters, no reference answers—and it directly quantifies how much a passing test case is worth.

10.7 What number is acceptable

A measurement regime without thresholds is not a control regime, and the objection is fair. There is no universal threshold, and I will not invent one; but the method for deriving a local threshold is available and it does not require new data.

The threshold is set by two things: the consequence tier of Section 9, and what control sits downstream of the assertion. Those two bound the answer from opposite directions.

Where a competent reviewer examines every assertion before it is relied upon, a high ungrounded rate is tolerable and the metric is not a safety measure at all—it is a *workload* measure, predicting how much reviewer time the system will consume and whether the review is a rubber stamp. A system whose ungrounded rate exceeds what reviewers can actually check has silently converted a control into a formality, and the grounding rate is how that is detected.

Where the assertion is relied upon directly, the tolerable rate is bounded by something the institution already knows. Every regulated firm has an accepted error rate for the equivalent manual process: a sampling standard for file reviews, a tolerance for unevidenced decisions found in quality assurance, an internal audit threshold that triggers a finding. Those numbers exist, they were negotiated with a regulator or set by policy, and they are the right anchor—because the question a supervisor will ask is not whether the system is good in the abstract but whether it is at least as well evidenced as the process it replaced. Anchoring to the incumbent standard also makes the comparison honest in the direction that usually favors the system: manual processes frequently cannot produce the evidence trail this book requires either.

Two cautions. Do not set a threshold on pooled accuracy and infer a grounding threshold from it, which Corollary 3.1 forbids. And set the abstention and coverage pair jointly, per Section 10.4, or the threshold will be met by a system that declines to answer.

Collecting the four measurements with the two framing questions, a validation report that supports sign-off contains, at minimum:

Statistic	Answers	Caveat to state
Ungrounded assertion rate	how often it asserts without support	retrieval-grounded or parametric; rater agreement
Misapplied rate	how often support is inapplicable at c	which components of c were checked
Error rate (grounded, false)	corpus quality	attributed to sources, not the system
Abstention rate on held-out items	whether silence is available	corpus-controlled configuration only
Form divergence	what a passing test is worth	probability-matching method
Repeatability	whether verification persists	within- and cross-session
Accuracy	performance on this sample	does not bound any of the above (Cor. 3.1)

Table 10.1: Accuracy stays on the report. It moves from the headline to a line item with a stated scope, which is what Corollary 3.1 licenses.

Part VI

The Control Model

11 The Architecture of Assurance

Every later chapter of this book is an instance of one idea developed here: that the *evidence* an AI system produces about its own governance can be made provable, even though its outputs cannot. The distinction is the whole point, and it is easy to lose. What follows proves properties of the audit record — that it is complete over the governed surfaces, that it is tamper-evident, that a missing interaction is detectable — conditional on a set of assumptions enumerated at the end of this chapter. It proves nothing about whether any particular output is compliant; the chapters on ungrounded assertion and canonical form explain why no such proof is available.

This chapter builds the foundation — the axioms Clad rests on, the surfaces at which an interaction can be governed, the model of a governance component, and the algebra by which components compose.

By the end of the chapter you will be able to:

- state the axioms Clad’s guarantees depend on, and recognize when a deployment violates them;
- decompose an AI interaction into its five control surfaces and classify each surface’s governability;
- describe a governance component as the tuple $g = (S, C, E, A, R)$ and read the interface contract between two components;
- explain what the composition algebra guarantees when components are deployed together.

Clad formalizes a governance boundary that makes risk, responsibility, and evidentiary state explicit for regulated AI use. Rather than promising perfect prevention, it focuses on provability: it guarantees that every governed interaction produces auditable evidence of which rules were in effect, which evaluations occurred (or failed), and what artifacts and versions participated in the interaction. That provability is the core compliance deliverable for regulators and internal auditors.

Clad’s architecture combines enforcement points, a Global Interaction Log to prevent undetectable bypasses, and tamper-evident audit chains with independent signing and Supervisor-mediated degraded records. Together these elements ensure that failures and deviations are visible and investigable, enabling traceability, root-cause analysis, and remediation. The framework formalizes the residual-risk reality: governance reduces but does not eliminate risk from stochastic model behavior and external unknowns, which must be documented, monitored, and mitigated with companion controls.

This meta-framework is the normative baseline for the component chapters that follow: implementers must map controls to the specified control surfaces, satisfy the architectural preconditions for composability, and adopt evidence formats that support independent verification and regulatory review. The three components developed in Part IV — Enterprise Prompt Governance (EPG), Runtime Output Controls (ROC), and Monitoring, Detection and Response (MDR) — are each independently rigorous but derive their scope, interfaces, and composability properties from the model built here. Those three abbreviations are used throughout the rest of the book: EPG governs the prompt before the model runs, ROC governs the output before it is delivered, and MDR watches the whole system over time.

11.1 Axioms

The following axioms are accepted as true about the world in which this governance system operates. They are not proven; they are the foundation on which all subsequent definitions, lemmas, and theorems rest. If any axiom is invalid in a specific deployment context, the theorems that depend on it must be re-evaluated.

Axiom 1 (Interaction Model). An AI interaction is described by the tuple:

$$\forall i \in I : i = (x, u, M, \theta, o)$$

where:

- $x \in X$ - the assembled prompt, including any retrieved context, conversation history, and system instructions
- $u \in U$ - the user input that triggered the interaction
- $M \in \mathcal{M}$ - the model identifier and version (a specific artifact, not an endpoint or family name)
- $\theta \in \Theta$ - the inference configuration: temperature, top-p, top-k, sampling parameters, tool availability, and any other parameters that affect the output distribution
- $o \in O$ - the model output

Scope limitation: This axiom models a single, non-agentic interaction turn. Multi-turn conversations are modeled as ordered sequences of interactions where each p_k includes relevant prior context from interactions i_1 through i_{k-1} . Agentic workflows (tool use, multi-model chains) are modeled as directed acyclic graphs of interactions where one interaction’s output feeds into another’s prompt. These extensions preserve the per-interaction governance properties but introduce additional composition concerns addressed in §12.

Governance scope claim: This tuple captures the elements within the governance scope of this framework. Elements outside this tuple — such as retrieval index state, external tool behavior, system-level routing, agent orchestration logic, and infrastructure configuration — may influence outputs but are classified as external factors per the three-tier model below. The framework does not claim that (x, u, M, θ, o) is a complete causal description of the output; it claims that these are the elements the governance system observes, constrains, and audits.

Three-tier element classification:

Definition (Element Classification). All factors that may influence an AI interaction’s output are classified into exactly one of three tiers:

Tier 1 — Modeled and Observable: Elements in the interaction tuple (x, u, M, θ, o) . These are within governance scope: constrainable, evaluable, auditable.

Tier 2 — Known but Unmodeled: Elements that are identified as influencing outputs but are not captured in the interaction tuple. Examples:

- RAG index state and ranking parameters
- External tool implementations and their behavior
- System-level routing, caching, and load balancing
- Agent orchestration logic and internal scratchpads
- Infrastructure differences across replicas

These contribute documented ungoverned risk (Theorem 4, in the theorems appendix). Each deployment must enumerate its Tier 2 elements.

Tier 3 — Unknown / Emergent: Factors not yet identified that may influence outputs. These contribute undocumented residual risk. The governance system cannot account for Tier 3 elements by definition, but it must acknowledge their possible existence.

What “prompt” includes: The assembled prompt x is the complete input to the model at inference time. In RAG architectures, p includes retrieved documents (though the retrieval process itself is Tier 2). In multi-turn conversations, p includes conversation history. In systems with tool-use definitions, p includes tool schemas. The governance system governs p as assembled — the assembly process itself is within prompt governance scope insofar as it determines the content of p , but upstream factors that influence assembly (index state, retrieval ranking) are Tier 2.

Statelessness assumption: This axiom assumes stateless conditional generation: the model’s output distribution is fully determined by (p, u, θ) with no hidden internal state. Models with persistent memory, non-reset hidden states, or recurrent reasoning chains that carry state across invocations violate this assumption. For such models, the internal state must be captured in p or θ , or the model must be classified as having reduced auditability on the temporal dimension (Axiom 5).

Axiom 2 (Non-Determinism). Governance systems must not assume deterministic model output. The output is sampled from a conditional distribution parameterized by the prompt, user input, model, and inference configuration.

$$M : X \times U \times \Theta \rightarrow \Delta(O)$$

$$o \sim M(x, u, \theta)$$

Formally: governance must be correct under the assumption that

$$\neg \forall (x, u, M, \theta) : |\text{support}(M(x, u, \theta))| = 1$$

The output distribution $M(x, u, \theta)$ may have non-singleton support, meaning different outputs are possible for identical inputs. The degree of non-determinism is influenced by θ but cannot be assumed to be zero for any configuration.

Implication: No governance mechanism operating solely on prompts can guarantee specific output properties. This is not a limitation of the governance system; it is a property of the governed artifact. Governance reduces the probability of non-compliant outputs; it does not eliminate it.

Note on near-deterministic configurations: Some inference configurations (e.g., temperature ≈ 0) produce near-deterministic output. The axiom does not claim high variance — it claims that governance systems must not *depend* on determinism, because (a) floating-point non-determinism in GPU computation prevents true determinism even at temperature 0, and (b) model updates can change output distributions without changing the API contract.

Axiom 3 (Guarantee Independence, Conditional). Each governance component’s formal guarantees hold independently of whether other components are deployed, **provided that the following infrastructure preconditions are met:**

$$\forall g \in G, \forall G' \subseteq G :$$

$$\text{guar}(g) \text{ holds in deployment}(\{g\}) \leftrightarrow \text{guar}(g) \text{ holds in deployment}(G')$$

GIVEN preconditions P1, P2, P3:

P1 (Infrastructure Stability):

Infrastructure shared across components (storage, networking, compute) maintains its availability, capacity, and integrity guarantees independent of the number of deployed components. Adding a component does not degrade shared infrastructure below the thresholds required by existing components.

P2 (Pipeline Non-Interference):

No component UNILATERALLY prevents another component from receiving its governed artifacts, EXCEPT when blocking is a declared governance action (fail-closed posture per §7).

Specifically:

- No component silently drops or redirects interactions.
- A component MAY block an interaction (fail-closed) if and only if:
 - (a) the block is a declared governance action, AND
 - (b) the blocking event is recorded in the audit chain (as a degraded record or a policy-enforcement record), AND
 - (c) downstream components are notified of the block so they can record their own “interaction blocked upstream” entry.

This exemption resolves the conflict between fail-closed posture (§7) and pipeline non-interference: blocking IS interference, but it is AUTHORIZED, RECORDED interference — distinguishable from silent bypass or malicious short-circuiting.

P3 (Surface Isolation with Read-Only Cross-Flow):

Components’ EVALUATION GUARANTEES are determined solely by artifacts within their own governed surfaces. However, components MAY receive read-only data from other surfaces via soft requirements (R_soft) to enhance effectiveness.

Formally, P3 distinguishes two types of cross-surface data flow:

ALLOWED — Read-only cross-surface data (soft dependencies):

- A component g_2 may read artifacts from S_{g_1} (e.g., ROC reads prompt context from EPG). This does NOT violate P3 because:
- g_2 ’s guarantee $\text{guar}(g_2)$ holds even without this data (R_soft)
 - g_2 does not modify S_{g_1} artifacts
 - The data flow is one-directional and read-only

PROHIBITED — Shared mutable state affecting evaluation:

- Components must not share writable state that any component’s evaluation function E_g depends on for its guarantee. Examples:
- Two components writing to the same constraint store
 - A component modifying artifacts another component evaluates
 - Shared caches whose state affects evaluation outcomes

Infrastructure services (GIL, Supervisor, KMS) are shared but are NOT governance components — they do not evaluate constraints or produce evaluation records. Their availability is covered by P1 (Infrastructure Stability), not P3.

What this claims: Given P1-P3, EPG’s guarantee (every prompt is evaluated against all applicable constraints and a complete audit record is produced) does not require ROC to be present. ROC’s guarantee does not require EPG. The guarantees are independent.

What this does NOT claim: It does not claim that governed *artifacts* are independent — prompts causally influence outputs. It does not claim that infrastructure is automatically

stable under composition — P1 must be verified. It does not claim that pipeline interference is impossible — P2 must be architecturally enforced.

If preconditions are violated: If P1 fails (e.g., adding ROC causes audit storage to degrade), $\text{guar}(g)$ for other components may be compromised. If P2 fails (e.g., ROC short-circuits requests before EPG processes them), $\Phi(\text{EPG})$ is violated. If P3 fails (e.g., components share mutable constraint state), evaluation determinism is compromised. Each violation must be detectable and reported (see §6: Enforcement Model).

Why the conditional claim is sufficient: Phased adoption requires that deploying component A alone provides A’s full guarantees. P1-P3 are architectural requirements that a competent infrastructure team can verify and maintain. The preconditions are testable, not aspirational — they can be validated through capacity testing (P1), pipeline tracing (P2), and architecture review (P3).

Status note: This axiom is a *design goal* that requires architectural enforcement, not a fact about the world. Downstream theorems that depend on Axiom 3 are conditional on P1-P3. This conditionality is explicit in each dependent proof.

Axiom 4 (Observability). All elements of an AI interaction that are relevant to governance must be observable and recordable by the governance system.

$\forall e \in \text{elements}(i)$ governed by some $g \in G$:

$\exists \text{observe}(e) : e \rightarrow \text{record}(e)$

where $\text{record}(e)$ is capturable, storable, and retrievable at any future time t

The governance scope is bounded by the observability boundary:

$\text{governable}(e) \rightarrow \text{observable}(e)$

$\neg \text{observable}(e) \rightarrow \neg \text{governable}(e)$

Implication: If the full assembled prompt is not logged, it cannot be governed or audited. If inference parameters are not recorded, they cannot be constrained. If model version is not tracked, temporal audit is impossible. Observability is a precondition for governance, not a consequence of it.

Contrapositive (critical for architecture): Any interaction element that is not observable is, by definition, outside the governance scope. The governance system must explicitly declare its observability boundary and acknowledge that elements beyond it contribute ungoverned risk.

Axiom 5 (Temporal Identity). Models, constraints, configurations, and all governed artifacts are versioned entities whose identity includes their state at a specific point in time.

$\forall x \in \{M, C, \Theta, P\}, \forall t_1, t_2 \in T$:

$x(t_1)$ and $x(t_2)$ are potentially distinct artifacts

$\wedge \text{identity}(x, t) = (x, \text{ver}(x, t))$

Specifically for models:

A model endpoint updated between interactions i_1 and i_2 is treated as two distinct models for governance purposes:

$m(t_1) \neq m(t_2)$ if $\text{ver}(m, t_1) \neq \text{ver}(m, t_2)$

even if $\text{endpoint}(m, t_1) = \text{endpoint}(m, t_2)$

Implication: Audit records must capture the version of every governed element at the time of the interaction. “Which model was used?” is not answered by an endpoint URL — it is answered by a versioned model identifier. “Which constraints were in effect?” is not answered

by the current constraint set — it is answered by the versioned constraint set at interaction time.

Practical constraint: This axiom requires that all governed elements expose version information. If a model provider does not expose model version (only an endpoint), the governance system must either (a) record whatever version signal is available (e.g., response headers, model ID strings) and document the limitation, or (b) classify the model as having reduced auditability on the temporal dimension.

11.1.1 Design Requirement: Audit Linkability

This is an engineering requirement on the implementation rather than a property of the world, so it is stated here as a design requirement rather than an axiom.

Requirement (Audit Linkability). Audit records produced by different governance components for the same interaction must share a propagated interaction identifier enabling chain composition.

$\forall g_1, g_2 \in G, \forall i \in I :$

$A_{g_1}(i).interaction_id = A_{g_2}(i).interaction_id$

The identifier must be:

- Generated at interaction initiation (before any component processes it)
- Propagated through the pipeline without modification
- Included in every component’s audit record
- Resistant to spoofing, duplication, and accidental collision

IMPLEMENTATION NOTE: This is a distributed tracing problem with well-established solutions (e.g., W3C Trace Context, OpenTelemetry). The meta-framework does not prescribe an implementation — it requires the property.

11.1.2 Design Requirement: Global Interaction Log

This requirement addresses the “Ghost Chain” problem: Theorem 3a proves that existing records cannot be tampered with, but it does not prove that every interaction *has* a record. A bypassed or failed enforcement point could leave an interaction completely unrecorded — indistinguishable from “no interaction occurred.”

Together these four properties make every interaction that enters the pipeline either fully governed, governed with degraded records, or detectable as a gap. The Ghost Detection theorem states this precisely, and states what it does not cover.

11.2 Threat Model

A governance framework without an explicit threat model is incomplete for regulated industries. This section enumerates the adversary types, in-scope threats, and explicitly out-of-scope threats. All theorems and guarantees in subsequent sections are conditional on this threat model — they hold against the in-scope threats and make no claims about out-of-scope threats.

11.2.1 Adversary Types

Definition (Adversary Classification). A1 — External User (Adversarial): Capability: crafts inputs u to elicit non-compliant outputs. Motivation: data exfiltration, policy bypass, boundary probing. Knowledge: may have partial knowledge of prompt structure, constraints, and model behavior through iterative probing.

A2 — Insider (Malicious or Compromised): Capability: authors or modifies constraints, attestations, or prompts within their authorized scope. Motivation: weaken governance, create backdoors, exfiltrate data. Knowledge: full knowledge of governance structure within their domain.

A3 — Compromised Vendor / Supply Chain: Capability: provides models with hidden behaviors (backdoors, data leakage triggers, biased outputs under specific inputs). Motivation: espionage, competitive advantage, sabotage. Knowledge: full knowledge of model internals (opaque to governance).

A4 — Regulator as Adversarial Auditor: Capability: requests complete audit evidence for any interaction, tests claims against evidence, probes for gaps. Motivation: verify compliance, identify violations. Knowledge: full knowledge of governance claims and framework structure.

11.2.2 In-Scope Threats

These threats are addressed by the governance components defined in this framework. For each, the responsible component is identified.

T1 — Policy Drift and Misconfiguration:

Constraints change over time without trace; model or inference config changes without governance awareness.

Addressed by: EPG (constraint versioning), MDR (drift monitoring), Axiom 5 (temporal identity).

T2 — Incomplete or Absent Audit Trail:

Interactions processed without evaluation or logging; audit records missing, incomplete, or tampered with.

Addressed by: Audit chain (§8), Audit integrity (§8.4), Component guarantees (§5.2), Failure semantics (§5.3).

T3 — Constraint Authoring Abuse (Insider — A2):

Malicious or negligent constraint authors weaken governance by creating trivially satisfiable constraints, disabling constraints for specific projects, or decomposing semantic constraints in bad faith.

Addressed by: EPG (RBAC, separation of duties, dual control for constraint authorship, decomposition attestation). This chapter defines the requirement; EPG specifies controls.

ADDITIONAL REQUIREMENT (Decomposition Verification):

Semantic constraint decompositions (mechanical + procedural) must be subject to periodic validation ensuring the decomposed checks actually prevent the behavior the original semantic constraint intended to prevent. This requires:

- Independent red-team testing of decomposed constraints
- Periodic re-attestation by domain experts other than the original decomposition authors
- Automated regression testing with known-bad inputs that mechanical checks must catch

Without this, a formally “passing” decomposition can functionally

fail — the “auditing a lie” problem. EPG must specify the verification protocol and minimum testing cadence.

T4 — Governance Bypass:

Interactions routed around governance components (shadow AI, direct API access, local deployments outside governed platform).

Addressed by: Enforcement model (§6), MDR (bypass detection).

T5 — Adversarial User Input (A1):

Users craft inputs designed to elicit non-compliant outputs, extract data, or probe policy boundaries.

Addressed by: ROC (output evaluation/filtering), MDR (input monitoring, rate limiting, anomaly detection).

Note: prompt governance (EPG) reduces but cannot eliminate this risk per Axiom 2 and Theorem 4.

T6 — Component Failure During Interaction:

A governance component becomes unavailable mid-interaction, leaving interactions ungoverned.

Addressed by: Failure semantics (§5.3).

11.2.3 Acknowledged Threats – Out of Meta-Framework Scope

These threats are real but addressed by companion documents or external disciplines. The meta-framework explicitly does not claim to address them.

T7 — Prompt Injection (Direct and Indirect):

Untrusted content (user input, retrieved documents, tool outputs) embedded in the prompt overrides governance instructions.

Scope: ROC (runtime output controls) and MDR (input sanitization, output classification). The meta-framework acknowledges this as R_input risk. EPG can require “prompt must include injection defenses” as a constraint, but cannot prevent injection at the framework level.

T8 — Model Jailbreaking:

Adversarial inputs that cause the model to ignore its instructions.

Scope: ROC (output filtering), MDR (red-team testing, monitoring).

The meta-framework models this as part of Axiom 2 (non-determinism) and Theorem 4 (irreducible residual risk). It cannot be solved at the governance level — it requires model-level and output-level controls.

T9 — Model Poisoning / Supply Chain Compromise (A3):

Models with hidden backdoors or biased training data.

Scope: External to this framework entirely. Model governance (training data audits, model cards, bias testing) is a distinct discipline. MDR can verify model selection criteria but cannot inspect model internals (§13.2).

T10 — Training Data Leakage / Memorization:

Models reproduce training data (PII, copyrighted content) in outputs.

Scope: ROC (output filtering for PII/sensitive content), MDR (monitoring for data leakage patterns). The meta-framework classifies model internals as γ = external.

T11 — Cross-Tenant Data Leakage (Multi-Tenant Providers):

In shared model deployments, data from one tenant influences outputs for another.

Scope: Infrastructure security (outside this framework). MDR can monitor for anomalous cross-tenant patterns.

11.2.4 Threat Model Completeness Statement

Definition (Threat Model Scope). This framework’s guarantees are valid against threats T1-T6. They are NOT valid against threats T7-T11 without deployment of the companion controls specified for ROC and MDR.

Any claim of “complete governance” must be qualified: “Complete within the meta-framework’s threat model (T1-T6), with residual risks from T7-T11 addressed by companion controls.”

11.3 Ontology (Primitive Sets and Pipeline)

11.3.1 Primitive Sets

I	= set of all AI interactions	- the fundamental unit of governance
P	= set of all prompts	- assembled instructions sent to the model
U	= set of all user inputs	- runtime input from end users
M	= set of all models	- AI models as versioned artifacts
Θ	= set of all inference configurations	- parameters affecting output distribution
O	= set of all outputs	- raw model outputs
O'	= set of all delivered outputs	- outputs after post-processing/filtering
$T = (\mathbb{R}, \leq)$	- totally ordered time domain	- for versioning and temporal audit
Σ	= set of all control surfaces	- partitioned elements of the interaction
G	= set of all governance components	- the building blocks of the solution
V	= set of all audit records	- evaluation evidence

11.3.2 The AI Interaction Pipeline

An AI interaction is a temporally ordered sequence of transformations:

Definition (AI Interaction Pipeline). An interaction $i \in I$ is a tuple $i = (x, u, M, \theta, o, o', t)$ where:

$p \in P$	- the assembled prompt (including retrieved context, conversation history, system instructions, tool schemas)
$u \in U$	- user input at runtime
$m \in M$	- the model as a versioned artifact: $\text{identity}(m, t)$
$\theta \in \Theta$	- inference configuration (temperature, top-p, tool availability, sampling parameters)
$o \in O$	- raw model output: $o \sim M(x, u, \theta)$
$o' \in O'$	- delivered output: $o' = \text{filter}(o)$ where $\text{filter} \in \text{runtime controls}$
$t \in T$	- timestamp of the interaction

The pipeline has a causal ordering: x precedes u (prompt exists before user input arrives) (p, u, θ) precedes o (output is conditioned on all three) o precedes o' (filtering follows generation)

11.4 Control Surface Decomposition

11.4.1 Definition

Definition (Control Surface). A control surface $S \in \Sigma$ is a subset of the interaction pipeline elements that can be governed. Each surface has:

- $\text{elements}(S) \subseteq \{p, u, m, \theta, o, o'\}$ — which pipeline elements it covers
- $\text{governability}(S) \in \{\text{full}, \text{partial}, \text{external}\}$ — degree of control
- $\text{owner}(S) \rightarrow G$ — which governance component is responsible

11.4.2 Surface Partitioning

Definition (Governance Surface Partition). The interaction pipeline decomposes into five control surfaces:

σ_{prompt}	$= \{p\}$	- prompt content and structure
σ_{input}	$= \{u\}$	- user input at runtime
σ_{config}	$= \{m, \theta\}$	- model selection and inference configuration
σ_{output}	$= \{o\}$	- raw model output
σ_{delivery}	$= \{o'\}$	- post-processing and delivery

These are exhaustive and disjoint: $S_{\text{prompt}} \cup S_{\text{input}} \cup S_{\text{config}} \cup S_{\text{output}} \cup S_{\text{delivery}} = \{p, u, m, \theta, o, o'\} \forall S_i, S_j : i \neq j \rightarrow S_i \cap S_j = \emptyset$

11.4.3 Governability Classification

Definition (Governability). A control surface S has governability class $\text{gov}(\sigma)$:

$\text{gov}(\sigma) = \text{full}$ iff $\exists$ mechanism that deterministically constrains all elements of S prior to or during interaction execution, AND all elements of S are observable (Axiom 4)

$\text{gov}(\sigma) = \text{partial}$ iff $\exists$ mechanism that influences but cannot deterministically constrain elements of S , OR some elements are observable but not fully constrainable

$\text{gov}(\sigma) = \text{external}$ iff elements of S are outside the solution's observability boundary entirely

Classification of surfaces:

$\text{gov}(\sigma_{\text{prompt}})$	$= \text{full}$	- prompts are authored within constraints before execution; fully observable and constrainable
$\text{gov}(\sigma_{\text{input}})$	$= \text{partial}$	- input validation constrains form (length, format, injection patterns) but not semantic content; observable but not fully constrainable
$\text{gov}(\sigma_{\text{config}})$	$= \text{partial}$	- model selection and inference parameters (θ) are fully constrainable; model internals/weights are external. Mixed governability within this surface. Decomposition: $\gamma(\text{model selection}) = \text{full}$ $\gamma(\theta) = \text{full}$ $\gamma(\text{model internals}) = \text{external}$
$\text{gov}(\sigma_{\text{output}})$	$= \text{partial}$	- output is stochastic (Axiom 2); influenced by σ_{prompt} and σ_{config} governance, not determined by it; observable post-generation
$\text{gov}(\sigma_{\text{delivery}})$	$= \text{full}$	- post-processing/filtering is deterministic and fully controllable; observable

11.5 Governance Component Model

11.5.1 Component Definition

Definition (Governance Component). A governance component $g \in G$ is a tuple $g = (S_g, C_g, E_g, A_g, R_g)$ where:

- | | |
|---|---|
| $S_g \subseteq \Sigma$ | - the control surfaces this component governs |
| C_g | - the constraint set this component enforces |
| $E_g : \text{artifact}(S_g) \times C_g \rightarrow V$ | - the evaluation function (produces audit records) |
| $A_g \subseteq V$ | - the audit records this component produces |
| R_g | - the requirements this component has of other components
(may be $\emptyset$ for independently deployable components) |

Precondition (from Axiom 4): $\forall s \in S_g : \text{observable}(s) = \text{true}$ A component can only govern surfaces whose elements are observable.

11.5.2 Component Guarantees

Definition (Component Guarantee). A component g provides guarantee $\text{guar}(g)$ iff:

- | | |
|---|--|
| $\forall i \in I_{\text{governed_by_}g} :$ | |
| E_g is total over C_g | - every constraint is evaluated
(completeness) |
| $\wedge E_g$ is deterministic | - same inputs produce same
evaluation (repeatability) |
| $\wedge \text{Audit}_g(i)$ is immutable | - audit records cannot be
altered post-hoc |
| $\wedge \text{Audit}_g(i)$ contains <code>interaction_id</code> | - enables chain composition
(Design Requirement) |
| $\wedge \text{Audit}_g(i)$ contains <code>ver(x, t)</code> for all
governed elements x (Axiom 5) | - all governed elements are
version-stamped |

11.6 Enforcement Model

The governance framework describes what SHOULD happen. The enforcement model defines HOW compliance is ensured at runtime. Without enforcement, governance is advisory — which contradicts the “inviolable” property required for regulated industries.

11.6.1 Enforcement Points

Definition (Enforcement Point). An enforcement point is a location in the interaction pipeline where governance constraints are evaluated and non-compliant interactions are blocked, modified, or flagged.

$$EP = \{ ep_1, ep_2, \dots, ep_n \}$$

Each enforcement point has:

- $\text{location}(ep) \in \text{pipeline stages}$ — where in the pipeline it operates
- $\text{mode}(ep) \in \{\text{blocking, flagging}\}$ — whether it can stop interactions
- $\text{component}(ep) \rightarrow g$ — which governance component operates it
- $\text{bypass_resistance}(ep) \in \{\text{strong, moderate, weak}\}$

11.6.2 Required Enforcement Architecture

Definition (Enforcement Architecture). For the governance system to provide its guarantees, the following enforcement properties must hold:

EA1 (Chokepoint Enforcement): All AI interactions **MUST** pass through governed enforcement points. No path from user input to model invocation may bypass EPG. No path from model output to user delivery may bypass ROC.

Formally: $\forall i \in I_{\text{governed}} :$

$\exists ep \in EP : \text{component}(ep) = g_EPG \wedge i \text{ passes through } ep$
 $\wedge \exists ep' \in EP : \text{component}(ep') = g_ROC \wedge i \text{ passes through } ep'$

EA2 (Identity Binding): Every model invocation is bound to a governed project identity. Anonymous or unattributed model calls are blocked by default. This prevents shadow AI and ungoverned deployments.

Formally: $\forall i \in I :$

$\exists \text{project_id}(i) \in \text{governed_projects}$
 $\vee i \text{ is blocked}$

EA3 (Bypass Detection): The system **MUST** detect and alert on interactions that circumvent governance enforcement points. Detection mechanisms include:

- Network-level monitoring for direct model API calls
- API key / credential management restricting model access
- Anomaly detection for ungoverned interaction patterns

Formally: $\forall i \in I_{\text{ungoverned}} :$

$P(\text{detect}(i)) \geq \text{detection_threshold} \text{ (deployment-specific)}$

EA4 (Enforcement of Constraint Authorship): Constraint creation, modification, and deletion require:

- Authenticated identity with authorized scope
- Separation of duties: author $\neq$ sole approver
- Audit trail of all constraint changes

This addresses insider threat T3 from §2.2.

11.6.3 Governability Conditioned on Enforcement

Full governability of a surface is therefore conditional on enforcement, not a property of the surface alone: without a chokepoint, a surface classified as fully governable is only fully governable for the interactions that happen to pass through the governed path. The Conditional Full Governability theorem states the condition.

11.6.4 Enforcement Failure Modes

See §7 (Failure Semantics) for how enforcement point failures are handled.

11.7 Failure Semantics

Governance components can fail. The framework must define what happens when they do. A framework silent on failure semantics does not say what its guarantees mean during an outage, which is when they matter most.

11.7.1 Failure Postures

Definition (Failure Posture). Each governance component and enforcement point declares a failure posture:

$\text{fail_posture}(g) \in \{\text{fail-closed}, \text{fail-open-flagged}, \text{fail-open}\}$

fail-closed: If the component cannot evaluate or log, the interaction is BLOCKED. No interaction proceeds without governance. Appropriate for: high-risk workloads, regulated data, safety-critical.

fail-open-flagged: If the component cannot evaluate or log, the interaction proceeds but is FLAGGED as ungoverned in the audit trail with a degraded- state indicator. A separate monitoring alert is generated. Appropriate for: medium-risk workloads where availability matters.

fail-open: If the component cannot evaluate or log, the interaction proceeds silently. NOT RECOMMENDED for any regulated workload. This posture exists to model legacy/uncontrolled deployments.

11.7.2 Degraded State Audit Records

Definition (Degraded Audit Record). When a component g fails during interaction i , a degraded audit record is produced:

```
A_g_degraded(i, t) = {
  interaction_id  : identifier(i)
  timestamp      : t
  component      : g
  status         : DEGRADED
  failure_reason  : description of failure
  posture_applied : fail_posture(g)
  action_taken   : {blocked, proceeded_flagged, proceeded_silent}
}
```

This record is written to the audit chain in place of the normal $\text{Audit}_g(i, t)$, preserving chain completeness even during failures.

11.7.3 Component Guarantee Under Failure

Definition (Component Guarantee Under Failure). A component g provides guarantee $\text{guar}(g)$ iff:

For all interactions i where g is operational:

E_g is total over C_g	- completeness
$\wedge E_g$ is deterministic	- repeatability
$\wedge \text{Audit}_g(i)$ is immutable	- tamper resistance
$\wedge \text{Audit}_g(i)$ contains interaction_id	- chain linkage
$\wedge \text{Audit}_g(i)$ contains $\text{ver}(x, t)$ for all governed elements x	- version stamps

For all interactions i where g has failed:

$A_g_degraded(i, t)$ is produced	- failure is recorded
$\wedge$ fail_posture(g) is enforced	- declared posture applies
$\wedge$ monitoring alert is generated	- failure is detected

guar(g) is thus a guarantee about BOTH normal and failure modes. The component either governs the interaction fully OR records that it could not and applies the declared failure posture.

11.7.4 Supervisor Signing Protocol for Degraded Records

Added after Round 2 review identified that a crashed component cannot call KMS to sign its own degraded record, breaking the cryptographic chain (AI2-AI3).

Definition (Governance Supervisor). A Governance Supervisor is an infrastructure service that:

- Monitors the health of all governance components
- Produces and signs degraded audit records on behalf of failed components
- Operates independently of any individual governance component
- Has its own KMS-managed signing key (distinct from component keys)

Protocol (Degraded Record Signing). When component g fails during interaction i :

1. The Supervisor detects g 's failure (via health check or timeout).
2. The Supervisor generates $A_g_degraded(i, t)$ containing:
 - The failure reason and posture applied
 - The Supervisor's identity (not g 's)
 - A signature using the Supervisor's KMS key
3. The degraded record includes:
 - signed_by : supervisor_id (NOT component g)
 - signing_key : KMS_key_supervisor
 - on_behalf_of : g
4. The Merkle chain continues: the degraded record's chain_hash is computed from the predecessor, maintaining AI2 integrity.

Property (Chain Continuity Under Failure). The audit chain remains cryptographically intact even when a component fails, because:

- The Supervisor signs on behalf of the failed component
- The chain_hash links through the degraded record
- Independent verifiers can distinguish Supervisor-signed records from component-signed records (different key identity)
- A Supervisor-signed record is weaker evidence than a component- signed record (the Supervisor did not perform evaluation), but it is stronger than a gap in the chain (which is undetectable without the GIL)

11.8 Interface Contracts

11.8.1 Contract Definition

Definition (Interface Contract). An interface contract between components g_1 and g_2 is a tuple $K(g_1, g_2) = (\text{provides}, \text{requires}, \text{handoff})$ where:

- $\text{provides}(g_1 \rightarrow g_2)$ — what g_1 makes available to g_2
- $\text{requires}(g_2 \leftarrow g_1)$ — what g_2 needs from g_1 to function optimally
- $\text{handoff}(g_1, g_2)$ — the data and identifiers exchanged at the boundary

A contract is SATISFIED iff: $\text{provides}(g_1 \rightarrow g_2) \supseteq \text{requires}(g_2 \leftarrow g_1)$

11.8.2 Contract Optionality

Definition (Hard vs. Soft Requirements). $R_{\text{hard}}(g) \subseteq R_g$ — requirements without which g cannot produce $\text{guar}(g)$

$R_{\text{soft}}(g) \subseteq R_g$ — requirements that enhance g 's effectiveness but are not necessary for $\text{guar}(g)$

For independently deployable components (required by Axiom 3): $R_{\text{hard}}(g) = \emptyset$

For dependent components:

$R_{\text{hard}}(g) \neq \emptyset$ — the component requires another component's output

11.8.3 Handoff Protocol

Definition (Handoff). A handoff between g_1 and g_2 for interaction i is:

```
handoff( $g_1, g_2, i$ ) = {
  interaction_id  : identifier( $i$ )           - shared linking key
  artifact       : output( $g_1, i$ )          - the governed artifact
  artifact_hash  : hash(artifact)           - integrity verification
  audit_ref      : pointer( $A_{g_1}(i)$ )      - reference to  $g_1$ 's audit record
  constraint_ctx :  $C_{g_1}$                   - the constraints  $g_1$  enforced
  version_manifest : {ver( $x, t$ ) |  $x \in$ 
                                elements( $S_{g_1}$ )} - version stamps of all governed
                                                    elements (Axiom 5)
}
```

11.9 Audit Chain

11.9.1 Audit Record Definition

Definition (Component Audit Record). For component g and interaction i at time t :

```
Audit_g( $i, t$ ) = {
  interaction_id  : identifier( $i$ )
  timestamp      :  $t$ 
  component      :  $g$ 
  surface        :  $S_g$ 
  evaluations     : { ( $c, \text{ver}(c, t), \text{eval}(c, \text{artifact}(i))$ ) |  $c \in C_g$  }
  artifact_hash  : hash(artifact( $i$ ))
  version_manifest : { ( $x, \text{ver}(x, t)$ ) |  $x \in \text{elements}(S_g)$  } - Axiom 5
  predecessor    : pointer( $A_{g'}(i, t)$ ) |  $\perp$ 
  observability_note : any elements of  $S_g$  not fully observable - Axiom 4
}
```

11.9.2 Chain Composition

Definition (Audit Chain). An audit chain for interaction i is an ordered sequence of component audit records:

$$\text{chain}(i) = [A_{g_1}(i, t), A_{g_2}(i, t), \dots, A_{g_n}(i, t)]$$

where:

$$\begin{aligned} \forall k \in \{2, \dots, n\} : A_{g_k}.\text{predecessor} &= A_{\{g(k-1)\}} && - \text{linked} \\ \wedge \forall k : A_{g_k}.\text{interaction_id} &= \text{identifier}(i) && - \text{same interaction} \\ \wedge \forall k : \text{artifact integrity is verifiable via hash chain} &&& - \text{tamper-evident} \end{aligned}$$

11.9.3 Audit Integrity Properties

An artifact hash alone is insufficient for tamper resistance in regulated industries: a malicious administrator or compromised system could rewrite records and recompute the hashes. The properties below close that gap.

Definition (Audit Integrity Requirements). The audit system must satisfy the following integrity properties:

AI1 (Immutable Storage): Audit records, once written, cannot be modified or deleted by any party, including system administrators, for the duration of the regulatory retention period. Implementation approaches: WORM storage (e.g., S3 Object Lock, Azure Immutable Blob), append-only databases, write-once volumes.

AI2 (Cryptographic Chain Integrity): Each audit record A_{g_k} includes a cryptographic hash of its predecessor record, forming a Merkle-like chain:

$$A_{g_k}.\text{chain_hash} = \text{hash}(A_{g(k-1)}.\text{chain_hash} \parallel A_{g_k}.\text{content})$$

Tampering with any record in the chain invalidates all subsequent chain_hash values, making modification detectable.

AI3 (Record Signing): Each audit record is signed by the producing component using a key managed by an independent key management service (KMS):

$$\text{Audit}_g(i, t).\text{signature} = \text{sign}(\text{KMS_key_g}, \text{Audit}_g(i, t).\text{content})$$

No component holds its own signing key. Key management is independent of component operation, preventing a compromised component from forging records.

AI4 (Independent Verification): An external auditor or independent system can verify the integrity of the audit chain without relying on any governance component:

- Chain_hash values can be independently recomputed
- Signatures can be verified against KMS public keys
- Record completeness can be verified against interaction_id generation logs

AI5 (Chain Truncation Detection): Whole-interaction deletion is detected by AI4's comparison against interaction_id generation logs. PARTIAL truncation is not: dropping ROC's record while keeping EPG's leaves a chain whose every remaining link verifies and whose every remaining signature is valid, under an interaction_id that is present as expected.

The audit system MUST therefore record, per interaction, the set of components its risk tier requires, and verification MUST compare the chain against that set:

$$\begin{aligned} \forall i \in I_{\text{governed}} : \\ \text{expected_components}(\text{tier}(i)) &\subseteq \{g \mid A_g \in \text{chain}(i)\} \\ \vee \exists \text{degraded_record}(g, i) &\text{ for each } g \text{ in the difference} \end{aligned}$$

A chain shorter than its tier requires, with no degraded record accounting for the gap, is a truncation and MUST be investigated on the GIL3 ghost pathway. Without this check the cryptographic chain provides no protection against a truncating adversary, because truncation is not modification.

AI6 (Retention and Jurisdiction): Audit records must be:

- Retained for the regulatory minimum period applicable to the deployment context (e.g., 6 years for SOX, as required by HIPAA, as specified by sector regulators)
- Stored in jurisdictions compliant with applicable data residency requirements (GDPR, state privacy laws)
- Protected when they contain PII/PHI: encrypted at rest, access-controlled, with PII minimization where possible (e.g., hashing identifiers in logs rather than storing plaintext)

These properties are what make the chain tamper-evident: modification of any record is detectable by an independent verifier even if immutable storage is bypassed for one of them. The Tamper-Evident Audit Chain theorem states the result and the case it does not cover, which is deletion.

11.10 Risk Model

11.10.1 Risk Attribution

Definition (Compliance Risk). For an interaction i , define compliance risk $R(i)$ as the probability that the delivered output o' violates a compliance requirement.

By the pipeline model: $R(i) = P(o' \text{ violates requirement} \mid p, u, m, \theta)$

This risk is attributable to multiple surfaces:

$$R(i) = R_prompt(i) + R_input(i) + R_config(i) + R_output(i) + R_delivery(i)$$

where each R_{S_k} represents risk attributable to surface S_k :

R_prompt	- risk from inadequate prompt constraints
R_input	- risk from adversarial or edge-case user input
R_config	- risk from inappropriate model selection or inference parameters (e.g., high temperature for safety-critical task)
R_output	- risk from raw output content (model behavior)
$R_delivery$	- risk from post-processing errors

Modeling Simplification (Risk Additivity). The additive decomposition $R(i) = \sum R_{S_k}$ is a modeling simplification, not a claim of statistical independence. In practice, risks interact: a weak prompt constraint combined with high temperature ($R_prompt \times R_config$) may produce higher risk than either alone. The additive model is used for ATTRIBUTION (which surface contributes what risk) not for precise QUANTIFICATION. Implementations requiring precise risk measurement should use empirical methods, not this decomposition.

ADDITIONAL RISK: Infrastructure Dependencies

The governance infrastructure itself introduces risk:

R_GIL	— risk from GIL unavailability (fail-closed blocks all governed interactions; single critical-path dependency)
$R_supervisor$	— risk from Supervisor unavailability (degraded records cannot be signed; chain integrity weakened)

These are operational risks of the governance system, not compliance risks of the AI interaction. They must be managed through standard

HA/DR practices for critical infrastructure.

RISK-TIERED GOVERNANCE: Not all interactions require the same governance intensity. Deployments **SHOULD** define risk tiers that bind data classification to governance posture:

Tier: Critical (PHI, PCI, financial reporting)

fail_posture: fail-closed (mandatory)

audit: full chain with AI1-AI6

constraints: full EPG + ROC + MDR

Tier: Standard (internal tools, non-sensitive data)

fail_posture: fail-open-flagged

audit: full chain, sampling permitted for volume management

constraints: EPG + ROC, MDR optional

Tier: Low (development, sandbox, experimentation)

fail_posture: fail-open-flagged or fail-open

audit: interaction logging, chain optional

constraints: enterprise-level EPG only

The specific tier definitions are deployment-specific. The meta- framework requires that tiers exist and that the binding between data classification and governance posture is itself a governed, auditable constraint (subject to EA4 change control).

11.10.2 Governance as Risk Reduction

Definition (Risk Reduction). A governance component g with guarantee $\text{guar}(g)$ reduces the risk attributable to its governed surface:

$$R_Sg(i \mid g \text{ deployed}) \leq R_Sg(i \mid g \text{ not deployed})$$

The residual risk after deploying g is: $R_Sg_residual = R_Sg(i \mid g \text{ deployed})$

11.10.3 Prompt Governance Risk Reduction Syllogism

SYLLOGISM 1 (Prompt-to-Output Risk Transfer):

Major Premise:

The probability distribution of model outputs is conditioned on the prompt and inference configuration.

Formally: $o \sim M(x, u, \theta)$, therefore $P(o \mid p_1, u, \theta) \neq P(o \mid p_2, u, \theta)$ in general.

Minor Premise:

Prompt governance constrains prompts to satisfy properties that reduce the probability of non-compliant outputs.

Formally: $x \models C^*(\text{project}) \rightarrow P(o \text{ violates} \mid p, u, \theta) \leq P(o \text{ violates} \mid p', u, \theta)$ where x' is an ungoverned prompt.

Conclusion:

Prompt governance reduces the probability of non-compliant outputs for the class of violations attributable to prompt deficiency.

Formally: $R_prompt(i \mid EPG \text{ deployed}) < R_prompt(i \mid EPG \text{ not deployed})$

Note: This does NOT claim R_input , R_config , R_output are reduced by prompt governance. Those risks require their own governance components.

11.11 Component Instantiation

11.11.1 Solution Component Map

Definition (Solution Components). The Clad instantiates the following:

```

g_EPG = (
  S      = { $\sigma_{\text{prompt}}$ },
  C      = constraint hierarchy (deontic),
  E      = mechanical evaluation  $\cup$  procedural attestation,
  A      = prompt audit records,
  R_hard =  $\emptyset$                                 - independently deployable
  R_soft = {constraint_ctx from organizational governance}
)

g_ROC = (
  S      = { $\sigma_{\text{output}}$ ,  $\sigma_{\text{delivery}}$ },
  C      = output constraint set,
  E      = output evaluation function,
  A      = output audit records,
  R_hard =  $\emptyset$                                 - independently deployable
  R_soft = {handoff from g_EPG}                  - enhanced by prompt context
)

g_MDR = (
  S      = { $\sigma_{\text{input}}$ ,  $\sigma_{\text{config}}$ },
  C      = monitoring rules, thresholds, model selection policies,
  E      = anomaly detection, compliance drift analysis,
           model/config constraint evaluation,
  A      = monitoring and incident records,
  R_hard =  $\emptyset$                                 - independently deployable
  R_soft = {audit records from g_EPG, g_ROC}
)

```

11.11.2 Interface Contract Instantiation

```

K(g_EPG, g_ROC) = {
  provides(g_EPG  $\rightarrow$  g_ROC):
    - The assembled prompt x and its cryptographic hash
    - The effective constraint set  $C^*(\text{project})$ 
    - The prompt audit record  $\text{Audit\_EPG}(i, t)$  with version_manifest
    - The interaction identifier
  requires(g_ROC  $\leftarrow$  g_EPG):
    - (soft) Prompt context for output evaluation
    - (soft) Constraint set for contextual output assessment
  handoff:
    - interaction_id: shared identifier (Design Requirement)
    - artifact: the assembled prompt x
    - artifact_hash: hash(p)
    - audit_ref: pointer to  $\text{Audit\_EPG}(i, t)$ 
    - constraint_ctx:  $C^*(\text{project})$ 
    - version_manifest: {ver(m, t), ver( $\theta$ , t), ver(c, t)  $\forall$  c}
}

K(g_ROC, g_MDR) = {

```

```

provides(g_ROC → g_MDR):
- The output audit record Audit_ROC(i, t)
- Any flagged violations or anomalies
- The delivered output o' and its hash
requires(g_MDR ← g_ROC):
- (soft) Output audit records for trend analysis
- (soft) Violation flags for incident correlation
handoff:
- interaction_id: same identifier
- artifact: delivered output o'
- artifact_hash: hash(o')
- audit_ref: pointer to Audit_ROC(i, t)
- violation_flags: set of flagged constraint violations
}
K(g_EPG, g_MDR) = {
provides(g_EPG → g_MDR):
- The prompt audit record Audit_EPG(i, t)
- Constraint version history for drift detection
requires(g_MDR ← g_EPG):
- (soft) Prompt audit records for root cause analysis
handoff:
- interaction_id: same identifier
- audit_ref: pointer to Audit_EPG(i, t)
- constraint_versions: {ver(c, t) | c ∈ C*(project)}
}

```

11.12 Composition Algebra

11.12.1 Component Composition Operator

Definition (Composition). Define the composition operator $\oplus$ on governance components:

```

g1 ⊕ g2 = (
  S      = Sg1 ∪ Sg2,
  C      = Cg1 ∪ Cg2,
  E      = Eg1 ∪ Eg2,
  A      = compose(Ag1, Ag2) via shared interaction_id,
  R_hard = R_hard(g1) ∪ R_hard(g2) \ {} {mutual provisions}
)

```

11.13 Scope Limitations and Extension Points

11.13.1 Single-Turn Interaction Scope

This meta-framework models single, non-agentic interaction turns. The following patterns require extension:

Multi-turn conversations:

Modeled as: sequence $[i_1, i_2, \dots, i_n]$ where p_k includes context from prior interactions. Per-interaction governance holds. Cross-turn governance (e.g., “the conversation as a whole must not reveal X”) requires additional constraints on the sequence, not yet formalized.

Agentic / tool-use workflows:

Modeled as: DAG of interactions where $\text{output}(i_k) \rightarrow \text{prompt}(i_j)$.

Per-interaction governance holds at each node. Graph-level governance (e.g., “the agent must not take action X without authorization”)

requires constraints on the DAG, not yet formalized.

Multi-model chains:

Modeled as: pipeline $[m_1, m_2, \dots, m_n]$ where each model processes the prior model’s output. Per-interaction governance holds per model.

Chain-level governance (e.g., “no PII may flow from model 1 to model 2”) requires constraints on inter-model data flow, not yet formalized.

Per-interaction governance (applying EPG/ROC/MDR to each individual turn or node) is directly supported by this framework. However, cross-interaction governance (constraints that span multiple turns, nodes, or models) requires additional formal machinery not yet defined. The framework does NOT claim “architectural compatibility” with these patterns in their full generality — it claims that its per-interaction guarantees hold at each node, while acknowledging that graph-level and sequence-level governance is an open problem.

Additionally, multi-turn and agentic patterns may introduce stateful behavior (KV caches, agent scratchpads, persistent memory) that violates Axiom 1’s statelessness assumption. For such patterns, either the state must be captured in p or θ at each turn, or the deployment must accept reduced auditability for the stateful elements (classified as Tier 2).

11.13.2 Model Internals

Model internals (weights, training data, fine-tuning provenance) are classified as external to the governance scope ($\gamma = \text{external}$ in `S_config`). Governance of model internals (model cards, training data audits, bias testing) is a distinct discipline not addressed by this framework. Interface point: `g_MDR` can verify that a model meets selection criteria (e.g., “model must have a published model card with bias assessment”) without governing the internals themselves.

11.14 Where the results are

This chapter states the model: the surfaces an interaction presents, the component that governs one, the contracts between components, and the algebra by which they compose. What is *proved* over that model — sixteen theorems, lemmas and corollaries, their proofs, and the twenty-two assumptions every one of them depends on — is collected in the theorems appendix.

The separation is deliberate. Deciding whether to adopt this framework needs the model and the guarantees in prose, which is this chapter. Building it, or auditing someone else’s build, needs the statements with their preconditions in one place, which is the appendix. Results are referred to by designation throughout — Theorem 4, Lemma 2, Axiom 5 — and those designations are stable.

11.15 Key takeaways

- Clad’s guarantees rest on the axioms of §1 and the two design requirements derived from them — audit linkability and the Global Interaction Log. Where a deployment cannot satisfy an axiom, the theorems that depend on it must be re-evaluated.
- An AI interaction is governed at five control surfaces (§4); each surface carries a governability class that bounds what any component can promise.

- A governance component is the tuple $g = (S, C, E, A, R)$ (§5), and components compose under a well-defined algebra (§12) that preserves each component's guarantees when they are deployed together.
- Governance reduces but does not eliminate residual risk (Theorem 4); what it guarantees is provable evidence, not perfect prevention.

The three chapters of Part IV instantiate this model, one control surface at a time. They begin with the prompt — the surface an enterprise fully controls before the model ever runs.

11.16 The Evidence Surface

The surface partition of Section 11.4 is exhaustive, and Theorem 1 proves it. This section argues that the theorem is true and the axiom it rests on is incomplete, and that closing the gap requires a sixth surface rather than a wider reading of an existing one.

The argument is put here, at the end of the chapter, because it is a critique of the foregoing rather than a qualification of it. Nothing above needs to be withdrawn.

11.16.1 What the theorem actually proves

Theorem 1 establishes that every element of every interaction is assigned to exactly one control surface. Its first proof step is the load-bearing one: *by Axiom 1, every interaction is described by (x, u, M, θ, o)* . The partition is then exhaustive over that tuple, and the conclusion follows.

So the theorem's scope is the tuple, and its truth is relative to the tuple. It proves completeness of the partition, not completeness of the tuple. Axiom 1 is careful about this and says so directly: it claims governance scope rather than completeness, and concedes that elements outside the tuple “may influence outputs” while declining to model them.

Among the elements named as outside: *retrieval index state*. The three-tier classification places it in Tier 2, known but unmodeled, contributing documented ungoverned risk.

11.16.2 Why that placement does not hold for retrieval-augmented systems

Tier 2 is the right classification for infrastructure that perturbs an output without being part of what the output claims. Load balancing, caching, replica differences: an interaction is not *about* its own routing, and nothing an auditor asks concerns which replica served the request.

Retrieved evidence is not of that kind. When a system answers a regulated question from a retrieved instrument, the instrument is not a factor that influenced the output. It is the *basis of the assertion* — the thing an auditor is asking about when they ask why. Placing it in Tier 2 makes the governance framework unable to represent the one element the audit is for.

The chapter's own architecture shows the consequence. The prompt surface governs the assembled artifact, and the evaluation engine operates on that assembled text rather than on the materials that composed it; the composition process is invisible to the evaluation layer. So retrieved evidence enters the governed tuple as opaque tokens inside x . Two prompts of identical text, one assembled from a current authoritative instrument and one from a superseded foreign-jurisdiction instrument, are the same governed artifact under this model. They evaluate identically, produce identical audit records, and are distinguishable only by information the framework has declared out of scope.

That is not a gap in enforcement. It is a gap in *representation*, and it cannot be closed by strengthening any of the five surfaces, because none of them has a term for a source.

11.16.3 The sixth surface

Adding evidence therefore requires amending Axiom 1 rather than extending Definition 3.2. Write the amended tuple

$$i = (x, u, M, \theta, o, E) \quad (11.1)$$

where E is the evidential basis available to the interaction: the identified, versioned sources the assertion is permitted to rest on. The partition then gains

$$\sigma_{\text{evidence}} = \{E\} \quad (11.2)$$

and Theorem 1 is restated over the six-element tuple, where it holds by the same proof.

Its governability class is worth stating precisely, because it is the reason the surface is worth adding:

$$\text{gov}(\sigma_{\text{evidence}}) = \begin{cases} \text{full} & \text{retrieval-provided evidence, identified at inference time} \\ \text{external} & \text{parametric evidence absorbed in training} \end{cases} \quad (11.3)$$

This is the only surface in the framework whose class is *split by deployment choice rather than fixed by the technology*. The prompt surface is governable because prompts are constructed; model internals are external because they are not observable. Evidence is either, depending on whether the architecture makes it explicit — which turns a governability classification into a design requirement, and is the strongest architectural conclusion in this book.

11.16.4 What the new surface requires

Three things the existing five do not supply.

An applicability relation. The output-evaluation machinery of the runtime control layer is defined over $\text{applicable}(R_{\text{ROC}}, i)$, and the completeness theorem for that layer asserts its audit record is total over exactly that set. The function is used and never defined anywhere in this framework. For constraints the omission is survivable, because applicability of a rule to an interaction is settled by level and domain. For evidence it is not: whether a source governs a case is the substance of the question, and it is developed in Chapter 3.13 as a relation to a resolved context of jurisdiction, valid time, material facts, and intended use.

A second time axis. Axiom 5 gives temporal identity over a single totally ordered domain, which is sufficient for asking which rule version was in force when an interaction ran. Evidence needs two axes: the time a source was valid, and the time the system could have known it. Their divergence is a detectable failure in both directions, and neither is expressible with one axis.

A dependence condition. The prompt surface can establish that required text was present and the output surface that prohibited text was absent. Neither can establish that an assertion *rested on* the evidence cited for it. The framework’s own worked example makes the point: a citation check fails, the prompt is reinforced, the model returns the same analysis with citations attached, and the second evaluation passes. Nothing in the five surfaces can distinguish that from grounded assertion, because a presence check on the artifact is the only instrument available.

11.16.5 Consequence for the guarantees

The composed guarantee of Section 11.12 is unaffected in what it claims and narrowed in what it covers. Every theorem in this chapter remains true over the tuple it quantifies over. What changes is the residual-risk accounting: evidence moves from Tier 2 undocumented influence to a governed surface with its own class, its own audit fields, and its own failure modes — which means the risk attributable to it becomes nameable instead of being absorbed into the irreducible remainder.

The conditions on σ_{evidence} are the grounding conditions of Chapter 3; the invariants that make them checkable, and the measurement that establishes them, are stated where the other system invariants are. This section claims only that the surface belongs in the partition, and that its absence was a scoping decision made before retrieval-augmented deployment was the normal case.

Part VII

The Control Layers

12 Responsible Prompting (EPG)

Chapter 11 built the model: five control surfaces, a governance component as the tuple $\mathbf{g} = (\mathbf{S}, \mathbf{C}, \mathbf{E}, \mathbf{A}, \mathbf{R})$, and an algebra for composing components. Part IV instantiates that model one surface at a time. We begin where an enterprise has the most control — the prompt, the instructions sent to a model before it ever runs. This is the surface governed by Enterprise Prompt Governance (EPG), the component $\mathbf{g_EPG}$.

By the end of the chapter you will be able to:

- explain why ungoverned prompting is a compliance and operational risk, not just a quality problem;
- build a hierarchical constraint model (enterprise $\succ$ department $\succ$ project) and reason about how constraints inherit and tighten;
- decompose a semantic constraint into mechanically evaluable checks plus procedural attestation, and resolve conflicts between constraints;
- map EPG onto the component model of Chapter 11 and state the enforcement conditions its guarantees depend on.

12.1 The Problem and the Approach

12.1.1 The Problem: Unstructured AI Prompting in Regulated Industries

Generative AI adoption is accelerating across Fortune 500 enterprises. Healthcare systems deploy diagnostic assistants, financial institutions build investment research tools, and energy companies automate compliance documentation. Yet beneath this progress lies a fundamental governance gap: prompt engineering remains ad-hoc, unstructured, and ungoverned.

The current state is unsustainable. Teams across an enterprise build prompts independently, with no shared governance framework, no compliance review process, and no visibility into what instructions are actually being sent to AI models. A marketing team crafting customer service responses, a legal team building contract analysis tools, and an operations team automating incident reports operate in isolation. Each makes autonomous decisions about what an AI system should do, how it should behave, and what constraints it should respect.

This absence of prompt governance creates compounding failure modes that threaten both regulatory compliance and operational integrity:

Compliance violations. Without governed prompts, enterprises expose themselves to direct regulatory violations. A healthcare chatbot built without HIPAA-compliant prompt constraints may instruct the model to include protected health information (PHI) in responses. A financial advisory tool lacking SOX-aligned prompt boundaries may generate investment recommendations that constitute unauthorized financial advice. These are not hypothetical risks. They represent the predictable outcome of deploying AI systems where the instructions sent to models are neither reviewed nor constrained by compliance requirements.

Inconsistency at scale. When identical business questions receive wildly different quality answers depending on which team built the prompt, the enterprise loses operational coherence. One customer service team’s prompt produces accurate, policy-aligned responses while another

team’s prompt for the same function generates contradictory answers. This inconsistency erodes trust in AI systems and creates liability when different parts of the organization make conflicting commitments to customers, regulators, or business partners.

Shadow AI proliferation. The absence of practical governance frameworks drives teams to build AI tools outside official channels. When the path to governed AI deployment is unclear or burdensome, teams bypass governance entirely. They stand up ungoverned chatbots, deploy unaudited analysis tools, and integrate AI capabilities that never touch the compliance review process. Each shadow AI system represents an unauditable risk vector. The enterprise has no inventory of what AI systems exist, no understanding of what instructions those systems operate under, and no mechanism to audit their behavior.

The PoC-to-production gap. Proof-of-concept AI projects routinely demonstrate compelling value, yet fail to graduate to production deployment. The cause is not technical infeasibility. It is governance infeasibility. Without a disciplined prompt governance framework, enterprises cannot answer the questions that gate production deployment: What constraints govern this system? How do we know those constraints are enforced? What happens when requirements change? Who approved these instructions? The absence of answers keeps valuable AI capabilities trapped in pilot purgatory.

These failure modes share a common root cause: enterprises treat prompts as code, not as governance artifacts. Prompts are instructions that determine AI system behavior, yet they receive none of the governance rigor applied to other behavior-determining artifacts like access control policies, data retention rules, or incident response procedures. The result is predictable: ungoverned instructions produce ungoverned behavior.

12.1.2 What EPG Provides

Enterprise Prompt Governance (EPG) is a constraint-based governance framework for managing AI prompts in regulated enterprises. It addresses the failure modes outlined above through a deliberate architectural choice: EPG defines boundaries within which teams build prompts. It does not author prompts.

This distinction is foundational. EPG is not a prompt dictation system. It does not maintain a central library of approved prompts. It does not require teams to select from pre-written templates. Instead, EPG establishes a constraint hierarchy that teams must satisfy when building prompts for their specific use cases. A healthcare department might inherit enterprise-wide constraints prohibiting PHI disclosure, add department-specific constraints requiring clinical disclaimer language, then allow individual project teams to craft prompts that satisfy both constraint sets while addressing their unique business requirements.

Hierarchical constraint inheritance implements this model. Constraints flow from enterprise governance teams to department leads to project teams. Each level adds specificity without removing protections established upstream. An enterprise-level constraint might require “no financial advice” across all AI systems. A finance department constraint might add “all numerical outputs must cite data sources.” A trading desk project constraint might further specify “volatility calculations must reference the last 90 days of market data.” The project team builds prompts that satisfy all three constraint levels, but retains full flexibility in how those prompts accomplish their specific business objective.

Full audit trail provides compliance teams with version-stamped, tamper-evident records of every prompt evaluation. When a regulator asks “what constraints governed this AI system on March 15, 2026,” the enterprise produces a cryptographically verifiable record showing the exact

constraints in effect, the prompt that was evaluated, the evaluation outcome, and the governance chain that established those constraints. This audit capability transforms AI governance from a trust-based model to a verification-based model.

Risk management posture frames EPG’s purpose with precision. EPG reduces the likelihood of violations, enables detection when violations occur, and ensures remediation processes execute when needed. It does not eliminate risk. Risk elimination would require prohibiting useful AI deployment. Instead, EPG implements a defensible risk management posture: governed boundaries that balance operational value against regulatory requirements, combined with detection and remediation capabilities that respond when boundaries are crossed.

12.1.3 How EPG Fits the Governance Solution

EPG is one component of the Clad framework, a multi-layered framework for governing AI systems across their full operational lifecycle. EPG specifically governs `S_prompt`, the control surface representing instructions sent to AI models. This is one of three primary control surfaces:

`S_prompt` (EPG, this chapter): The instructions, context, and guidance provided to AI models that shape their behavior. Prompt governance establishes what an AI system should do before it acts.

`S_output` (ROC): The content produced by AI models before delivery to end users or downstream systems. Output governance ensures what an AI system produces meets quality, safety, and compliance requirements.

`S_input` and `S_config` (MDR): User input validation and model selection/configuration governance, plus cross-component monitoring that detects when AI systems deviate from expected behavior patterns. (Chapter 11 decomposes this into two formal surfaces — `S_input` for user input and `S_config` for model selection and inference parameters — but operationally they are managed by a single component.)

These components interact but remain independently valuable. An enterprise may deploy EPG to govern prompt development while output controls remain under construction. A financial services firm might implement runtime output controls immediately to prevent unauthorized trading recommendations while building out comprehensive prompt governance over subsequent quarters. This independence-with-integration property is formally proven in the meta-framework: each component provides local guarantees that compose into system-wide guarantees when multiple components are deployed together.

The integration points are deliberate. When EPG marks a prompt as compliant with constraint `C_healthcare_phi`, runtime output controls can reference that evaluation to inform their own risk assessment. When monitoring systems detect anomalous model behavior, they can trigger EPG re-evaluation of the active prompts. The components form a coherent governance pipeline while remaining independently deployable.

12.2 Principles

Enterprise Prompt Governance rests on five foundational principles. Each principle establishes a critical design decision that shapes how EPG operates in production environments. Understanding these principles is essential for both executive leadership evaluating EPG’s strategic fit and technical teams implementing EPG in complex organizational contexts.

12.2.1 Governance is Constraint, Not Prescription

EPG defines what prompts must not violate. It does not define what prompts must say.

This distinction separates EPG from template-based prompt management systems. A template system maintains a library of approved prompts and requires teams to select from that library. EPG establishes constraints that prompts must satisfy and allows teams to craft prompts that meet those constraints while addressing their specific business requirements.

Consider the analogy of building codes. A municipal building code constrains how you build a structure. It specifies maximum heights, minimum fire safety measures, required load-bearing capacities, and mandatory accessibility features. It does not design your building. An architect working within those constraints retains full creative freedom to design a hospital, an office tower, or a residential complex. The building code ensures safety and compliance without prescribing architectural vision.

EPG operates identically. An enterprise constraint might prohibit disclosure of personally identifiable information (PII) in AI-generated outputs. That constraint says nothing about whether a team is building a customer service chatbot, a document summarization tool, or an incident analysis assistant. It establishes a boundary that all three use cases must respect while leaving the prompt design decisions to the teams who understand their specific business context.

Formally, constraints are predicates—functions that map a prompt to a boolean outcome:

$$C : P \rightarrow \{\text{true}, \text{false}\}$$

A constraint evaluates a prompt and returns true if the prompt satisfies the constraint, false otherwise. Constraints are not generators. They do not produce prompts. They validate prompts produced by other means.

This constraint-based model enables EPG to scale across diverse use cases without requiring centralized prompt authoring teams. Governance teams define boundaries. Project teams build within those boundaries. The governance layer remains constant while the application layer adapts to thousands of distinct business requirements.

12.2.2 Execution is Always Local

No organization executes an “enterprise-level prompt.” Every prompt that reaches a production AI model is executed by an end user through a specific project deployment.

This principle reflects operational reality. When a customer service representative uses an AI assistant to draft a response, they are executing a prompt that was built by the customer service project team, constrained by customer service department policies, and further constrained by enterprise-wide governance requirements. But the execution event occurs at project scope. The user does not directly interact with enterprise or department constraints. They interact with a project-level system that was built to satisfy those constraints.

EPG’s three-level hierarchy—enterprise, department, project—reflects this execution model. The hierarchy exists for governance composition, not execution routing. Constraints flow downward through the hierarchy during prompt development. Execution flows upward through the hierarchy during audit reporting. But the actual execution event occurs at project level.

Formally, for any prompt x that executes in production:

$$\forall x \in X_{\text{executed}} : \text{level}(x) = \text{project}$$

This property has significant architectural implications. EPG does not maintain enterprise-level or department-level execution environments. Those levels exist as constraint definition layers, not execution layers. When a project team deploys a prompt to production, they deploy

a project-level artifact that has been validated against the complete constraint set inherited from department and enterprise levels. But the deployment itself is scoped to the project.

This design choice prevents a common governance anti-pattern: the shared service model where a central team attempts to build and operate AI systems on behalf of the entire enterprise. Shared service models fail for AI because prompt requirements are irreducibly context-dependent. A customer service prompt for healthcare claims processing cannot be genericized into a universal customer service prompt. The domain knowledge, regulatory constraints, and user expectations differ fundamentally from customer service prompts in retail, financial services, or telecommunications. EPG recognizes this reality by placing execution authority at project level while maintaining governance authority at higher levels.

12.2.3 Risk Management, Not Risk Elimination

EPG reduces the likelihood of governance violations, enables detection when violations occur, and ensures remediation processes execute when needed. It does not eliminate violations.

Risk elimination would require prohibiting AI deployment. Any system that generates novel outputs carries inherent risk of producing outputs that violate policy, even when that system operates under well-designed constraints. A healthcare chatbot built with comprehensive HIPAA-aligned prompt constraints may still generate a response that inadvertently reveals protected health information if the model encounters an adversarial input pattern. A financial analysis tool governed by strict disclosure requirements may still produce a statement that could be construed as unauthorized advice if the model misinterprets an ambiguous user query.

EPG implements a defensible risk management posture instead of pursuing unattainable risk elimination. Consider the analogy of Sarbanes-Oxley (SOX) compliance in financial systems. SOX does not prevent financial fraud. It establishes controls that make fraud detectable and traceable. When fraud occurs, SOX-compliant systems provide the audit trail required to identify what happened, who was involved, and what corrective actions are needed. The compliance posture is not “fraud never occurs.” It is “when fraud occurs, we detect it, document it, and remediate it.”

EPG provides equivalent capabilities for prompt governance:

Likelihood reduction. Constraint validation during prompt development catches violations before deployment. A prompt that includes prohibited language triggers evaluation failure, preventing that prompt from reaching production.

Detection enablement. Full audit trails record every prompt evaluation, providing compliance teams with the data required to detect patterns of constraint violations, identify teams that consistently struggle with specific requirements, and flag anomalous prompt behaviors that warrant investigation.

Remediation assurance. Version-stamped constraint sets and tamper-evident evaluation records ensure that when violations are detected, remediation teams can identify exactly which constraints were violated, which prompts caused violations, and which governance policies need strengthening.

Formally, EPG guarantees audit completeness, not universal compliance:

$$\forall x \in X_{\text{executed}} : \exists \text{evaluation_record}(x, C_{\text{effective}}, \text{timestamp}, \text{result})$$

Every executed prompt has a corresponding evaluation record showing which constraints were in effect, when the evaluation occurred, and whether the prompt satisfied those constraints. The system does not guarantee that $\text{result} = \text{true}$ for all prompts. It guarantees that the evaluation occurred and the result is recorded.

This distinction is critical for executive stakeholders evaluating EPG. EPG is not an insurance policy against AI governance failures. It is a risk management framework that transforms ungoverned AI deployment into governed AI deployment, providing the visibility and control required to operate AI systems in regulated environments with defensible risk posture.

12.2.4 Audit the Artifact, Not the Process

The auditable unit in EPG is the assembled prompt evaluated against the effective constraint set. EPG does not audit whether teams followed prescribed development processes. It audits whether the prompts those teams produced satisfy governance requirements.

This principle reflects a fundamental insight about governance enforcement: process compliance is a proxy for outcome compliance, and proxies fail. A team might meticulously follow a prescribed prompt development process yet produce a prompt that violates critical constraints. Conversely, a team might bypass the prescribed process entirely yet produce a fully compliant prompt. If governance systems audit process rather than artifact, they reward the first team and punish the second, despite the first team producing a governance violation and the second team producing a compliant system.

EPG inverts this model. The evaluation question is not “did the team use the approved template?” but rather “does the assembled prompt satisfy the effective constraints?” A project team might build prompts through template composition, manual authoring, programmatic generation, or any hybrid approach. EPG remains agnostic to the construction process. It evaluates the construction output.

This artifact-centric audit model has significant practical advantages:

It accommodates innovation. When a team develops a novel prompt construction technique—using retrieval-augmented generation to inject domain context, employing few-shot learning to shape model behavior, or leveraging chain-of-thought prompting to improve reasoning quality—EPG evaluates the resulting prompt against constraints without requiring governance approval for the construction technique itself.

It reduces governance overhead. Process-based governance requires maintaining detailed procedural documentation, training teams on those procedures, and auditing procedural compliance. Artifact-based governance requires defining constraints and evaluating prompts. The latter scales far more efficiently across diverse teams with varying technical capabilities.

It provides objective evidence. A process audit produces subjective assessments: “the team substantially followed the procedure” or “the team deviated from step 4.” An artifact audit produces objective verdicts: “the prompt satisfies constraint C” or “the prompt violates constraint C.” Objective verdicts support consistent enforcement and defensible audit responses.

This principle shapes EPG’s technical architecture in concrete ways. EPG’s evaluation engine operates on assembled prompts, not on the source materials that contributed to those prompts. If a prompt is constructed by combining a base template, dynamic user context, and retrieved knowledge base fragments, EPG evaluates the final assembled text that will be sent to the AI model. The composition process is invisible to the evaluation layer.

12.2.5 Constraints Inherit Downward, Authority Scopes Laterally

Constraints flow downward through the hierarchy inviolably. Authority to define constraints flows laterally across domains at each level.

The downward inheritance rule is strict: a constraint established at enterprise level applies to all departments and all projects. A constraint established at department level applies to all

projects within that department. Lower levels can add constraints—tightening the boundary—but cannot remove constraints established upstream. This rule ensures that enterprise-wide governance requirements cannot be bypassed by department or project teams, even inadvertently.

The lateral scoping rule is equally critical: authority to define constraints is partitioned by domain. Legal teams govern legal compliance constraints. Security teams govern data protection and access control constraints. Clinical teams govern healthcare-specific constraints. Finance teams govern financial reporting and disclosure constraints. Each domain establishes constraints within its area of authority, and those constraints apply across all levels of the hierarchy.

This dual-axis model—hierarchical inheritance, lateral domain authority—prevents both under-governance and over-governance pathologies. Without hierarchical inheritance, departments could exempt themselves from enterprise security policies by simply not adopting those policies at department level. Without lateral domain authority, a single central governance team would need expertise across legal, security, clinical, financial, and operational domains—an impossible requirement that leads to either governance bottlenecks or governance gaps.

Consider a concrete example. An enterprise security team establishes a constraint prohibiting PII in test environments. This constraint applies enterprise-wide across all departments and projects. Separately, the healthcare compliance team establishes a constraint requiring HIPAA disclaimer language in all patient-facing AI systems. This constraint applies to the healthcare department and all its projects, but does not apply to the finance or operations departments. A project team building a patient portal inherits both constraints and cannot weaken either. The project team can add project-specific constraints—for example, requiring that appointment scheduling responses include cancellation policy information—but those additions only apply within that project.

Formally, the effective constraint set at any level is the union of constraints defined at that level and all levels above:

$$C_effective(project_p) = C_enterprise \cup C_department(p) \cup C_project(p)$$

This union operation ensures constraint sets can only grow, never shrink, as we move down the hierarchy. The lateral domain scoping determines which constraints appear in each C_level set, but the inheritance rule determines how those sets combine.

This principle directly addresses a concern executives frequently raise about AI governance frameworks: will governance become a centralized bottleneck that stalls innovation? EPG's answer is architectural: domain authority distributes governance decisions to the teams with relevant expertise, while hierarchical inheritance ensures those distributed decisions compose into coherent enterprise-wide policy. The result is governance that scales across hundreds of project teams without requiring every decision to flow through a central approval committee.

12.3 The Hierarchical Constraint Model

The hierarchical constraint model is the conceptual and technical foundation of EPG. This section provides the detailed specification required to implement EPG in production environments while maintaining sufficient business context for executive stakeholders to understand the model's architectural choices.

12.3.1 Three Governance Levels

EPG structures constraints across three levels: enterprise, department, and project. This structure mirrors the corporate organizational hierarchy that exists in regulated enterprises, providing

intuitive ownership mapping that aligns governance authority with existing reporting relationships and domain expertise.

Enterprise level establishes organization-wide constraints that apply to every AI system regardless of business unit or use case. Enterprise constraints typically address cross-cutting concerns: data protection policies, regulatory compliance baselines, acceptable use boundaries, and brand protection requirements. The enterprise governance team—often a cross-functional body reporting to the CIO, CISO, or Chief Compliance Officer—owns constraint definition at this level.

Typical enterprise constraints include:

- Prohibition of PII in test and development environments
- Required data minimization practices for customer information
- Prohibited content categories (hate speech, self-harm, violence)
- Geographic data residency requirements
- Required disclosure of AI system involvement in customer interactions

Enterprise constraints establish the floor beneath which no department or project can operate. They represent the organization's non-negotiable governance requirements.

Department level translates enterprise policy into domain-specific constraints. A department in this context represents a business unit with distinct regulatory, operational, or domain requirements. Healthcare, finance, legal, human resources, and customer service functions typically map to separate departments in the EPG hierarchy, though the exact mapping should mirror the enterprise's existing organizational structure.

Department leaders—Chief Medical Information Officers, Chief Investment Officers, General Counsels, Chief Human Resources Officers—own constraint definition at this level. They inherit all enterprise constraints and add department-specific requirements that reflect their domain's regulatory environment, operational standards, and risk tolerance.

Typical department constraints include:

- Healthcare: HIPAA compliance requirements, clinical disclaimer language, scope limitations preventing diagnosis or treatment advice
- Finance: FINRA suitability standards, required disclosure language, prohibition of specific financial advice
- Legal: Attorney-client privilege protection, required confidentiality notices, prohibition of legal conclusions
- Human Resources: Equal employment opportunity compliance, anti-discrimination constraints, required privacy disclosures

Department constraints refine enterprise policy without contradicting it. They add specificity appropriate to the domain while preserving all protections established at enterprise level.

Project level represents individual AI deployments—the specific systems that execute prompts in production. A project might be a customer service chatbot, a document analysis tool, a code generation assistant, or an incident report automation system. Project teams

include software engineers, data scientists, product managers, and business stakeholders who understand the specific use case and can define constraints that ensure the AI system serves its intended purpose without overstepping appropriate boundaries.

Project teams inherit constraints from both enterprise and department levels. They add project-specific constraints that address the unique requirements of their use case:

- A patient FAQ chatbot might add constraints requiring redirection to healthcare providers for medical questions
- A contract analysis tool might add constraints requiring flagging of non-standard clause language
- An investment research assistant might add constraints requiring citation of data sources for quantitative claims

Project constraints complete the constraint hierarchy. They represent the most specific governance layer—constraints that apply to a single AI system serving a specific business function.

This three-level structure provides clear ownership boundaries. Enterprise teams define organization-wide policy. Department leaders translate policy into domain practice. Project teams implement systems that satisfy both. Each level operates with appropriate autonomy within its scope while remaining accountable to constraints established upstream.

12.3.2 Constraint Types (Deontic Modalities)

EPG employs a deliberately minimal logical framework consisting of exactly two operators that carry logical force, plus one governance annotation that provides metadata without affecting evaluation.

Obligation $O(\phi)$ expresses that property ϕ must hold. When a constraint declares $O(\text{data_minimization_clause})$, the system requires that the evaluated prompt includes language implementing data minimization principles. Obligations impose positive requirements—things that must be present or true.

Prohibition $F(\phi)$ expresses that property ϕ must not hold. When a constraint declares $F(\text{pii_in_test_environments})$, the system requires that the evaluated prompt does not permit personally identifiable information in test or development contexts. Prohibitions impose negative requirements—things that must be absent or false.

Governance annotation $P_{\text{meta}}(\phi)$ records that property ϕ was considered by the governance team and no restriction was imposed. When a constraint set includes $P_{\text{meta}}(\text{industry_terminology})$, this signals “we explicitly evaluated whether to restrict industry-specific terminology and decided not to.” This annotation serves governance coordination purposes but carries no logical force in constraint evaluation.

These three constructs relate in specific ways:

$$F(\phi) \equiv O(\neg\phi)$$

A prohibition is logically equivalent to an obligation of the negation. Prohibiting PII disclosure is equivalent to obligating non-disclosure of PII. EPG maintains both operators for expressiveness—it is often clearer to state “PII must not appear in logs” than to state “the absence of PII in logs must hold”—but treats them as interchangeable during evaluation.

Critically, P_{meta} is not the classical deontic permission operator P . In standard deontic logic, permission relates to obligation through the axiom:

$$O(\phi) \rightarrow P(\phi)$$

This axiom states that if something is obligatory, it is also permitted. But this axiom does not apply in EPG because P_meta is a governance annotation, not a logical operator. The relationship between obligation and annotation is undefined because they operate in different semantic spaces. Obligations and prohibitions participate in constraint evaluation, inheritance, and conflict detection. Annotations do not.

This design choice prevents deontic paradoxes that arise when permission is treated as a logical operator:

Ross’s paradox. In classical deontic logic, if mailing the letter is permitted, and mailing the letter entails burning the letter or mailing it, then burning the letter or mailing it is permitted. This allows deriving permission for actions that should not be permitted through disjunction.

Free choice permission. The statement “you may have coffee or tea” should permit having coffee and permit having tea, but classical permission operators struggle to capture this intuition without allowing over-permissive inferences.

EPG sidesteps both paradoxes by refusing to treat permission as a logical operator. $P_meta(\phi)$ does not participate in logical inference. It cannot be combined with other constraints through boolean operators. It does not propagate through inheritance rules. It exists solely to document governance decisions for audit and coordination purposes.

This design choice has a second critical benefit: it ensures contradiction detection remains complete. EPG can definitively determine whether two constraints contradict each other by checking whether any property has both an obligation and a prohibition:

$$\text{Contradiction} \equiv \exists \phi : O(\phi) \in C_effective \wedge F(\phi) \in C_effective$$

If obligation and prohibition could be derived through complex logical inference involving permissions, boolean combinations, and conditional constraints, contradiction detection would become undecidable. By restricting the logical language to atomic obligations and prohibitions, EPG ensures contradictions can be detected in linear time through simple set intersection.

12.3.3 3.2a Property Language Restrictions

The properties ϕ that appear in obligations and prohibitions are atomic identifiers drawn from a controlled vocabulary. EPG does not allow boolean composition of properties within constraints.

This means the following are well-formed constraints:

$O(hipaa_disclaimer)$
 $F(pii_in_logs)$
 $O(data_source_citation)$

But the following are not well-formed constraints:

$O(hipaa_disclaimer \wedge patient_consent)$
 $F(pii_in_logs \vee phi_in_test_data)$
 $O(data_minimization \rightarrow anonymization)$

Properties are atomic. They cannot be combined with AND, OR, or IMPLIES operators. This restriction is not a limitation of EPG’s technical implementation. It is a deliberate design choice that prevents three classes of problems:

Paradox prevention. Boolean combinations of properties enable the deontic paradoxes mentioned above. By prohibiting these combinations, EPG eliminates entire classes of logical inconsistency.

Completeness guarantees. Contradiction detection remains polynomial-time computable

when properties are atomic. Introducing boolean operators would make contradiction detection NP-hard or undecidable depending on the operators allowed.

Implementation clarity. Composite constraints like $O(\phi \wedge \psi)$ raise questions about partial satisfaction. If a prompt satisfies ϕ but not ψ , does it partially satisfy the constraint? Should partial satisfaction block deployment? Atomic constraints eliminate ambiguity: a constraint is either satisfied or violated.

When a governance requirement genuinely involves multiple properties, EPG expresses this through multiple atomic constraints:

```
O(hipaa_disclaimer)
O(patient_consent_verification)
```

This representation makes explicit that both properties are independently required. The evaluation system checks each constraint separately and reports violations independently, providing clearer audit feedback than a single composite constraint would provide.

Contrary-to-duty obligations—requirements that activate when another requirement is violated—are similarly expressed as separate atomic constraints with documented relationships rather than as conditional logical expressions. If enterprise policy requires that when PII appears in an error log, it must be redacted within 24 hours, this is expressed as:

```
F(pii_in_error_logs)
O(redaction_procedure_documented)
```

The relationship between these constraints is recorded in governance metadata, not encoded in logical operators. This approach maintains the simplicity of the evaluation engine while providing sufficient expressiveness for real-world governance requirements.

12.3.4 Inheritance Rules (Strengthening)

Constraints propagate down the hierarchy according to strict inheritance rules that ensure lower levels cannot weaken protections established at higher levels.

Obligations propagate down inviolably. If $O(\phi)$ appears in the enterprise constraint set, $O(\phi)$ appears in the effective constraint set for every department and every project. Department and project teams cannot remove, modify, or weaken enterprise-level obligations.

Prohibitions propagate down inviolably. If $F(\phi)$ appears in the enterprise constraint set, $F(\phi)$ appears in the effective constraint set for every department and every project. Department and project teams cannot remove, modify, or weaken enterprise-level prohibitions.

Annotations do not propagate. If $P_{\text{meta}}(\phi)$ appears at enterprise level, this annotation does not appear in department or project effective constraint sets. Annotations are governance metadata scoped to the level where they are defined. They do not participate in inheritance.

Lower levels can add obligations or prohibitions. A department can introduce $O(\psi)$ or $F(\psi)$ even if neither ψ nor $\neg\psi$ appear in the enterprise constraint set. This addition tightens the constraint boundary for that department and all projects within it. The addition is valid provided it does not create a contradiction with inherited constraints.

Lower levels cannot introduce weakening constraints. A department cannot add $P_{\text{meta}}(\phi)$ if $O(\phi)$ or $F(\phi)$ was inherited from enterprise level. The annotation has no effect on evaluation—the obligation or prohibition remains in force—but the contradictory intent signal would indicate a governance coordination failure warranting review.

The effective constraint set at any level is computed as:

$$C_{\text{effective}}(\text{project}) = \{c \mid c \text{ is } O \text{ or } F, \text{ defined at project, department, or enterprise level}\}$$

This set excludes all P_meta annotations regardless of where they appear. Annotations are available for governance audit and coordination review but do not participate in prompt evaluation.

The inheritance model implements a monotonic strengthening property: moving down the hierarchy can only add constraints, never remove them. Formally:

$$C_effective(enterprise) \subseteq C_effective(department) \subseteq C_effective(project)$$

This property ensures that no project team can accidentally or intentionally weaken governance protections established at higher levels. The most permissive constraint set in the organization is the enterprise constraint set. Every department and project constraint set is either equal to or stricter than enterprise constraints.

This monotonic property has significant operational value. Enterprise governance teams can define organization-wide policy with confidence that their constraints will propagate to every AI system regardless of how many intervening layers exist in the organizational hierarchy. Department leaders can refine policy for their domain without risk of undermining enterprise baselines. Project teams can implement systems knowing that as long as they satisfy their effective constraint set, they are automatically in compliance with all higher-level policies.

12.3.5 Permission Semantics

EPG operates with a two-operator logical system where only obligations $O(\phi)$ and prohibitions $F(\phi)$ carry logical force. The governance annotation $P_meta(\phi)$ exists in a separate semantic space and serves governance coordination purposes rather than evaluation purposes.

This design choice creates nuanced semantics around permission that must be clearly understood:

P_meta has no evaluation effect. When constraint evaluation occurs, the evaluation engine considers only obligations and prohibitions. Annotations are invisible to the evaluation process. A prompt is compliant if it satisfies all obligations and violates no prohibitions, regardless of what annotations exist in the constraint set.

P_meta has no inheritance effect. Annotations do not propagate down the hierarchy. If the enterprise constraint set includes $P_meta(clinical_terminology)$, department and project constraint sets do not inherit this annotation. Each level defines its own annotations independently.

P_meta has no role in conflict detection. The system checks for contradictions by identifying properties that have both an obligation and a prohibition. Annotations do not participate in this check. $P_meta(\phi)$ and $O(\phi)$ do not contradict each other because they operate in different semantic spaces.

However, P_meta does differ from complete absence of constraint:

P_meta signals explicit consideration. When an annotation $P_meta(\phi)$ appears in a constraint set, this documents that the governance team at that level explicitly evaluated whether to impose a restriction on ϕ and decided not to. This is governance metadata that appears in audit trails and informs governance review processes.

P_meta triggers tension detection. When a higher level includes $P_meta(\phi)$ and a lower level includes $F(\phi)$, the system flags this pattern as a governance tension warranting review. The lower-level prohibition is valid—inheritance rules permit tightening—but the combination suggests possible misalignment between governance levels that should be investigated.

This leads to a critical semantic property: EPG is not permissive-by-default. The absence of $F(\phi)$ does not mean ϕ is permitted. In classical deontic logic with a permission operator, if

something is not prohibited, it is permitted. EPG rejects this equivalence. The absence of $F(\phi)$ means nothing definitive about the permissibility of ϕ . Only an explicit $P_{\text{meta}}(\phi)$ signals “we considered ϕ and chose not to restrict it.”

This design choice prevents a common governance failure mode. In permissive-by-default systems, teams interpret the absence of prohibition as permission to proceed. When governance teams fail to enumerate every possible risk, teams deploy systems that technically satisfy the written policies but violate the governance team’s intent. EPG’s design prevents this failure mode by decoupling “not prohibited” from “permitted.”

Consider the distinction in practice:

- **Absence of constraint.** Enterprise constraint set includes no mention of industry-specific terminology. Department teams cannot infer permission to use unrestricted terminology. They must either seek explicit permission (resulting in P_{meta} annotation) or proceed with understanding that future governance review might introduce restrictions.
- **Explicit annotation.** Enterprise constraint set includes $P_{\text{meta}}(\text{industry_terminology})$. Department teams can see that governance explicitly considered and chose not to restrict terminology. If a department later introduces $F(\text{industry_terminology})$, this will be flagged for review as a tension pattern, but the prohibition remains valid.

This semantic model supports a key governance workflow: progressive policy refinement. Early in EPG adoption, enterprise constraint sets may be sparse, containing only the highest-confidence, most critical restrictions. As governance teams gain experience with EPG, they add annotations documenting decisions about edge cases, ambiguous requirements, and domain-specific concerns. These annotations provide increasingly precise guidance to lower levels while maintaining the logical simplicity of the two-operator evaluation system.

The tension detection mechanism works as follows:

$$\text{Tension} \equiv P_{\text{meta}}(\phi) \text{ at level } L_{\text{higher}} \wedge F(\phi) \text{ at level } L_{\text{lower}}$$

When this pattern is detected, EPG generates a governance review notice. The notice does not block deployment—the prohibition is valid under inheritance rules—but alerts governance coordinators to investigate whether the lower-level team misunderstood enterprise intent or whether enterprise policy should be revised to explicitly prohibit ϕ rather than annotating it as unrestricted.

This mechanism helps prevent governance drift, where lower levels gradually introduce restrictions that undermine the flexibility enterprise governance teams intended to preserve.

12.3.6 Worked Example: Healthcare Constraint Inheritance

A concrete example demonstrates how the hierarchical constraint model operates in a regulated healthcare enterprise.

Enterprise Level

The enterprise governance team, reporting to the CISO and Chief Compliance Officer, establishes organization-wide constraints:

```
O(hipaa_disclaimer)
F(pii_in_logs)
P_meta(clinical_terminology)
```

The first constraint obligates that all customer-facing AI systems include disclaimer language clarifying HIPAA compliance scope. The second constraint prohibits logging personally identifiable information to system logs, test databases, or analytics platforms. The third annotation

documents that the governance team explicitly considered whether to restrict clinical terminology in AI responses and decided not to impose blanket restrictions, recognizing that medical terminology is often necessary for effective healthcare communication.

Department Level: Patient Services

The Patient Services department, led by the Chief Medical Information Officer, inherits enterprise constraints and adds department-specific requirements:

Inherited: $O(\text{hipaa_disclaimer})$, $F(\text{pii_in_logs})$

Added: $F(\text{clinical_terminology})$, $O(\text{empathetic_tone})$, $O(\text{provider_referral_required})$

The department inherits both the disclaimer obligation and the PII logging prohibition. These constraints remain in full force and cannot be modified at department level.

The department adds three new constraints:

$F(\text{clinical_terminology})$

This prohibition restricts the use of medical jargon in patient-facing communications. While the enterprise governance team chose not to restrict clinical terminology organization-wide—recognizing that provider-facing tools may require precise medical language—the Patient Services department determines that patient-facing systems should avoid terminology that may confuse or alarm patients.

The combination of $P_{\text{meta}}(\text{clinical_terminology})$ at enterprise level and $F(\text{clinical_terminology})$ at department level triggers a governance tension flag. This is not an error. The inheritance rules permit departments to tighten constraints. But the tension flag prompts governance review to ensure the department team understands enterprise intent and that the prohibition aligns with organizational goals. The review confirms that the restriction is appropriate for patient-facing systems even though enterprise policy leaves terminology unrestricted for provider-facing tools.

$O(\text{empathetic_tone})$

This obligation requires that patient-facing AI communications employ empathetic language that acknowledges patient concerns and emotional state. Healthcare interactions frequently involve stress, uncertainty, and vulnerability. Department policy requires that AI systems reflect this context in their communication style.

$O(\text{provider_referral_required})$

This obligation requires that AI responses to clinical questions include explicit referral to healthcare providers. The system must not only refuse to provide diagnosis or treatment advice—which would be captured in a prohibition constraint—but must actively direct patients toward appropriate clinical resources.

The effective constraint set for any Patient Services project is now:

$$C_{\text{effective}}(\text{Patient Services}) = \{ \\ O(\text{hipaa_disclaimer}), \\ F(\text{pii_in_logs}), \\ F(\text{clinical_terminology}), \\ O(\text{empathetic_tone}), \\ O(\text{provider_referral_required}) \\ \}$$

Project Level: FAQ Bot

The Patient Services FAQ Bot project team builds an AI system that answers common patient questions about appointment scheduling, insurance coverage, prescription refills, and

facility information. The project inherits all enterprise and department constraints and adds project-specific requirements:

Inherited: O(hipaa_disclaimer), F(pii_in_logs), F(clinical_terminology), O(empathetic_tone),
O(provider_referral_required)
Added: O(scope_to_faq_topics), O(appointment_scheduling_context)

The project team adds two constraints that narrow the AI system's scope:

O(scope_to_faq_topics)

This obligation requires that the FAQ bot's prompts include explicit boundaries limiting responses to the defined FAQ topic areas. Questions outside this scope should trigger a polite redirection rather than an attempt to answer with uncertain information. This constraint prevents scope creep where a narrowly-designed FAQ system begins attempting to answer complex questions it was not designed to handle.

O(appointment_scheduling_context)

This obligation requires that appointment-related responses include context about cancellation policies, rescheduling procedures, and patient rights. This information is legally required in certain jurisdictions and operationally important for patient satisfaction.

The project team cannot weaken any inherited constraints. If the team attempted to remove F(clinical_terminology)—perhaps arguing that some technical terms are unavoidable when discussing insurance coverage—the constraint evaluation would reject the project constraint set as invalid. The prohibition was established at department level and cannot be removed at project level.

The effective constraint set for the FAQ Bot project is:

$$C_{\text{effective}}(\text{FAQ Bot}) = \{ \\ \text{O(hipaa_disclaimer)}, \\ \text{F(pii_in_logs)}, \\ \text{F(clinical_terminology)}, \\ \text{O(empathetic_tone)}, \\ \text{O(provider_referral_required)}, \\ \text{O(scope_to_faq_topics)}, \\ \text{O(appointment_scheduling_context)} \\ \}$$

This constraint set represents the complete governance boundary for the FAQ Bot. When the project team builds prompts for the FAQ Bot, EPG evaluates those prompts against this seven-constraint set. A prompt that satisfies all seven constraints is compliant and can be deployed to production. A prompt that violates any constraint is rejected and must be revised.

Constraint Set Evolution

The hierarchical model accommodates governance evolution. Consider three scenarios:

1. **Enterprise tightening.** The enterprise governance team adds F(external_data_sources) prohibiting AI systems from incorporating data from external APIs without explicit security review. This prohibition immediately propagates to all departments and projects. Existing projects must re-evaluate their prompts against the expanded constraint set. Projects that were compliant yesterday may be non-compliant today if their prompts reference external data sources.

2. **Department refinement.** The Patient Services department adds O(spanish_language_support) requiring that patient-facing systems provide bilingual support. This obligation applies to all Patient Services projects but does not affect other departments. The FAQ Bot project

inherits the new obligation and must update its prompts to satisfy the requirement.

3. Project adaptation. The FAQ Bot project adds `F(insurance_claim_status)` prohibiting the FAQ bot from attempting to answer questions about specific claim status, which requires authenticated access to patient records that the FAQ bot is not designed to handle. This prohibition applies only to the FAQ Bot project and does not affect other Patient Services projects.

Each evolution point demonstrates the hierarchical model’s flexibility. Governance can tighten at any level without disrupting other levels. Constraints flow downward automatically without requiring manual propagation. Project teams retain autonomy to add project-specific constraints while maintaining compliance with all upstream requirements.

This worked example demonstrates the hierarchical constraint model operating across all three levels with realistic healthcare governance requirements, showing how obligations, prohibitions, and annotations compose to create defensible AI governance boundaries that balance regulatory compliance, operational requirements, and project-specific needs.

12.4 Constraint Authorship and Governance

The hierarchical constraint model defines what constraints exist and how they flow. This section defines who can author constraints, what controls prevent abuse, and how changes are managed. These controls directly address the insider threat — a governance framework is only as strong as its resistance to the people who operate it.

12.4.1 Role-Based Access Control

Constraint authorship is scoped by governance level and domain. Each level has designated roles:

Enterprise level. The central governance team — typically the CISO office, legal department, and chief compliance officer — authors organization-wide constraints. These teams define the security, legal, and regulatory baseline that all AI systems must satisfy. Enterprise-level authorship requires enterprise governance credentials and is visible to all downstream levels.

Department level. Department governance leads, appointed by department heads and approved by enterprise governance, author department-specific constraints. A Chief Medical Information Officer authors clinical AI constraints; a Chief Investment Officer authors financial advisory constraints. Each governance lead has an authorized domain scope — the set of governance properties they may create constraints for. A clinical governance lead cannot author marketing constraints; a marketing governance lead cannot author clinical constraints.

Project level. Project leads, appointed by their department and operating within the boundaries established by enterprise and department constraints, author project-specific constraints. Project authorship is the most narrowly scoped: project leads can only add constraints that tighten governance for their specific use case.

Formally, every agent has a scope function: $\text{scope}(\text{agent}) \rightarrow \text{set of domains}$. An agent can only create or modify constraints whose property belongs to their authorized domains. The system enforces this at every lifecycle event — creation, modification, deletion, and approval.

12.4.2 Dual Control with Orthogonal Approval

No constraint enters the governance system through a single actor. Every constraint lifecycle event requires two independent parties: one to propose and one to approve.

Basic invariant. For all non-emergency constraints: `author(c) ≠ approver(c)`. The person who writes the constraint cannot be the person who approves it. Both must have the constraint’s domain within their authorized scope. This prevents a single compromised or negligent actor from unilaterally modifying governance.

Enhanced control for Critical tier. For constraints classified as Critical — those governing PHI, PII, financial advice, safety-critical operations, or other high-risk properties — the system requires orthogonal approval:

- The approver must belong to a different organizational reporting line than the author: `org_unit(author(c)) ≠ org_unit(approver(c))`
- For ALL Critical-tier constraints at any governance level (enterprise or department), at least one approver must be from a central, non-business-unit function: the CISO office, central risk management, or compliance. This breaks the departmental chain of command entirely. A department head cannot capture both author and approver for Critical constraints regardless of how they structure their team’s reporting lines.

Collusion limitation. Dual control with orthogonal approval mitigates single-actor abuse and same-unit collusion. It does not prevent cross-unit collusion by determined insiders. Two individuals from different reporting lines who agree to approve each other’s malicious constraints will bypass this control. Compensating controls address this residual risk: independent periodic audit of constraint changes (sampling-based, risk-tiered), anomaly detection on constraint modification patterns (frequency, scope, timing), and mandatory review of high-impact constraint changes by the cross-domain governance body (§7.4).

12.4.3 Cross-Domain Review Trigger

When a proposed constraint could interact with constraints from another governance domain, cross-domain review is triggered. The trigger conditions are:

- The constraint’s property overlaps with a property governed by another domain’s existing constraints (detected by controlled vocabulary intersection)
- The constraint applies to a governance level where another domain has existing constraints on related properties (detected by property ontology relationships)
- Automated conflict detection identifies potential tension (see §6.4)

Cross-domain review involves representatives from all affected domains. It is the primary mechanism for conflict prevention — catching constraint interactions at authoring time before they reach production. The process and escalation path are detailed in §6 (Conflict Resolution).

12.4.4 Change Control and Audit Trail

All constraint lifecycle events are versioned and signed:

- **Creation:** who authored, who approved, when, which domain, what property, what governance level
- **Modification:** what changed, who proposed the change, who approved, previous version preserved, rationale documented

- **Deletion or deprecation:** who proposed, who approved, replacement constraint (if any), effective date, impact assessment

Constraint version history is subject to the same audit integrity properties as interaction audit records — immutable storage (AI1), cryptographic chain integrity (AI2), independent record signing (AI3), and independent verification capability (AI4). When a regulator asks “what constraints were in effect on March 15,” the enterprise produces a cryptographically verifiable record of the exact constraint set, including every modification that led to that state.

This connects to the meta-framework’s Axiom 5 (temporal identity): constraints are versioned entities whose identity includes their state at a specific point in time. It also connects to enforcement requirement EA4: constraint creation, modification, and deletion require authenticated identity with authorized scope.

12.4.5 Emergency Constraint Modification ("Break Glass")

Urgent situations — an active compliance violation discovered in production, a regulatory order requiring immediate response, a security incident demanding rapid containment — require governance to act faster than the standard dual-control process allows. The break-glass protocol provides this speed within hard bounds that prevent abuse.

Single-actor authorization. A single authorized enterprise-level actor may impose or modify a constraint immediately. The modification is flagged as `emergency(c) = true` in the audit trail.

Tightening only. Break-glass modifications may only add obligations or prohibitions. They may not weaken, remove, or relax existing O or F constraints, and they may not convert existing constraints to P_meta annotations. This prevents emergency powers from being used to loosen governance protections. An emergency that requires loosening governance is not an emergency that break-glass should address — it requires the standard deliberative process.

Mandatory expiration. Every break-glass constraint carries a time-to-live (TTL) calibrated to industry norms:

- Healthcare and life sciences: maximum 72 hours
- Financial services (high-frequency environments): maximum 8 hours
- Energy and infrastructure: maximum 24 hours

Deployment-specific TTLs may be shorter but never longer than these defaults.

Automatic reversion. If post-hoc dual-control review with orthogonal approval is not completed before TTL expiry, the emergency constraint is automatically reverted to the last known-good dual-controlled state. The reversion is itself an audited event. This prevents emergency constraints from persisting indefinitely — a common vector for configuration drift in regulated environments.

Rate limiting. No more than a deployment-specific maximum (recommended: 3) simultaneous break-glass modifications per domain. This prevents emergency powers from being used for bulk unauthorized governance changes.

Immediate conflict scan. Every break-glass constraint triggers an immediate, prioritized CONTRADICTION scan against all existing constraints at affected levels. An emergency constraint that contradicts an existing constraint is flagged immediately — it still takes effect (the

emergency may require it), but the contradiction is visible in the audit trail and must be resolved during post-hoc review.

The formal exception to the dual-control invariant is explicit:

$\forall c : \neg \text{emergency}(c) \rightarrow \text{author}(c) \neq \text{approver}(c)$ [standard dual control]
 $\forall c : \text{emergency}(c) \rightarrow \text{single-actor permitted, TTL enforced, tightening-only}$

12.5 Constraint Evaluability and Mandatory Decomposition

Every constraint in the governance system must be evaluable — it must be possible to determine, with a stable and repeatable answer, whether a given prompt satisfies the constraint. This requirement is non-negotiable for regulated industries where a compliance auditor may ask “was this prompt compliant?” and expect a definitive answer. This section defines the evaluability taxonomy, the mandatory decomposition protocol for constraints that resist direct evaluation, and the formal controls that prevent decomposition from becoming an accountability escape hatch.

12.5.1 Two Evaluability Classes

Every constraint in the governance system belongs to exactly one of two classes:

Mechanically evaluable constraints (C_m) can be checked programmatically. Their evaluation is deterministic, automated, and repeatable. The same prompt evaluated against the same mechanical constraint produces the same result every time, regardless of who runs the evaluation or when. Examples: PII pattern detection (regex or NER scan), required section presence (string or semantic similarity check), keyword and terminology checks, prompt length limits, required disclaimer presence.

Procedurally evidenced constraints (C_p) require qualified human attestation. Their evaluation is deterministic given the evidence — a signed attestation from a qualified reviewer is either present or absent, yielding a binary compliance determination. Examples: domain expert review (“a medical affairs officer has reviewed this prompt and attests it does not instruct diagnostic behavior”), legal sign-off, adversarial testing attestation (“red-team testing has been completed and results reviewed”).

No constraint exists outside this taxonomy. If a constraint cannot be classified as either mechanically evaluable or procedurally evidenced, it cannot enter the governance system in its current form.

12.5.2 Why Semantic Constraints Are Prohibited as First-Class Elements

Consider the constraint “prompts must not produce medical advice.” This is a semantic constraint — evaluating it requires human judgment about what constitutes “medical advice.” Two qualified evaluators may disagree. The same evaluator may reach different conclusions on different days. This non-determinism is fatal for audit completeness.

If a regulator asks “was this prompt compliant with your medical-advice prohibition on March 15?” and the answer depends on who you ask or when you ask them, the audit trail is not a trail — it is an opinion. Regulated industries require stable, verifiable compliance determinations. Non-deterministic evaluation cannot provide them.

Therefore, semantic constraints are prohibited as first-class elements in the governance system. They must be decomposed into mechanical and procedural components before they can be operationalized.

12.5.3 Mandatory Decomposition Protocol

Decomposition transforms a semantic constraint into a set of evaluable constraints through a structured five-step process:

Step 1: Identify semantic intent. A domain expert articulates the full intent of the semantic constraint. For “must not produce medical advice,” the intent might include: no diagnostic language, no treatment recommendations, no dosage guidance, no prognostic statements, explicit disclaimers, and explicit redirection to qualified providers.

Step 2: Decompose into evaluable components. The expert separates the intent into what can be automated (mechanical checks) and what requires human judgment (procedural attestations). Diagnostic language patterns can be scanned automatically; the adequacy of the overall framing requires expert review.

Step 3: Independent review. A second domain expert — not the original author — reviews the decomposition for adequacy. Does the set of mechanical and procedural components sufficiently capture the original semantic intent? Are there gaps?

Step 4: Formal attestation. The decomposition is formally attested: “this set of mechanical and procedural constraints is a sufficient representation of the original semantic intent.” The attestation is signed and versioned.

Step 5: Registration. The decomposition is registered in the constraint system. The original semantic constraint is preserved as an intent record (C_s) — a documentation artifact that captures the governance intent but is never evaluated directly.

12.5.4 Formal Separation: Intent Records vs Operational Constraints

The formal model distinguishes three sets of constraints:

C_s (semantic constraints / intent records): preserved as documentation, never evaluated directly, not part of the effective constraint set $C^*(\text{level})$. These capture what the governance team intended.

C_m (mechanically evaluable constraints): part of $C^*(\text{level})$, evaluated by the mechanical satisfaction relation $\models_m$. These capture the automatable aspects of governance intent.

C_p (procedurally evidenced constraints): part of $C^*(\text{level})$, evaluated by the procedural satisfaction relation $\models_p$. These capture the judgment-dependent aspects of governance intent.

The mapping $\text{decomp}: C_s \rightarrow (\mathcal{P}(C_m) \times \mathcal{P}(C_p))$ links each intent record to its operational decomposition. Only C_m and C_p appear in the effective constraint set, satisfaction relations, and audit records. The formal model maintains a clean separation: intent records document what governance means; operational constraints implement what governance checks.

12.5.5 Formal Soundness Relationship

The goal of decomposition is to establish a soundness relationship: if all mechanical checks pass and all procedural attestations are satisfied, then the original semantic constraint is satisfied.

Formally:

$$\begin{aligned} \text{Soundness}(c_s) &\Leftrightarrow \forall p, \text{evidence} : \\ &(\forall c_m \in \text{decomp_m}(c_s) : p \models_m c_m) \wedge \\ &(\forall c_p \in \text{decomp_p}(c_s) : \text{evidence} \models_p c_p) \\ &\rightarrow p \models_s c_s \end{aligned}$$

This is the goal, not a guarantee. The system distinguishes two operational states:

Soundness_claimed(c_s): the decomposition author asserts the implication holds, attested by a domain expert. This is a professional judgment backed by attestation.

No_counterexample_found(c_s): red-team testing and regression testing have not falsified the soundness claim. This is empirical evidence that the decomposition works for known test cases.

The system does not equate these. It seeks to falsify soundness claims through adversarial testing. Inability to falsify is evidence, not proof. This mirrors the scientific method: a hypothesis that survives rigorous testing is credible but not proven.

Completeness is explicitly not claimed. It is possible that a prompt satisfies the original semantic intent while failing a mechanical check (the decomposition is over-tight). This is acceptable — it means some compliant prompts may be conservatively rejected, which is the safer failure mode in regulated environments.

12.5.6 Residual Gap Controls

When soundness cannot be fully demonstrated — when the decomposition captures most but not all of the semantic intent — the decomposition documents a residual gap. The residual gap is the portion of the original semantic intent not fully captured by the mechanical and procedural components.

The residual gap is not an escape hatch. The following controls prevent it from becoming one:

Coverage requirement. Each semantic intent is broken into a finite set of aspects. For Critical-tier constraints, a minimum coverage threshold applies. Core aspects must be captured in C_m or C_p and may not be deferred to the residual gap. For a constraint prohibiting PHI disclosure, direct PHI pattern detection must be in C_m. An organization cannot leave PHI detection entirely in the residual gap and claim governance.

Maximum gap threshold. If a decomposition’s residual gap exceeds the allowable threshold, the decomposition is rejected and the constraint cannot be operationalized until a better decomposition is found. For Critical-tier constraints, the threshold is quantitative: the decomposition must demonstrate a minimum pass rate against the Gold Standard Prompt Set and regression test suite (deployment-specific, recommended minimum: 95% for PHI/safety constraints, 90% for other Critical constraints). For Standard and Low tiers, the threshold is qualitative, assessed by the cross-domain governance body. A documented gap in a Critical constraint is not merely a risk acknowledgment — without meeting the quantitative threshold and pairing with proportional compensating controls, it is a governance failure that blocks deployment.

Compensating control requirement. Every residual gap must be paired with a specific, actionable compensating control. “We acknowledge the gap” is not sufficient. A valid compensating control is concrete and measurable: “mechanical checks cover an estimated 80% of diagnostic language patterns; compensating control: 10% human-in-the-loop review of outputs for this project, managed by medical affairs, with monthly reporting on findings.”

Cross-domain escalation. Any decomposition with a residual gap for a Critical-tier constraint must be reviewed and approved by the cross-domain governance body, not just the owning domain. This prevents a single domain from accepting inadequate decompositions for high-risk constraints.

Automatic human-in-the-loop trigger. For Critical-tier constraints with documented residual gaps, the Governance Supervisor automatically mandates a human-in-the-loop (HITL)

review rate proportional to the gap severity. If mechanical checks cover an estimated 80% of the semantic intent, a minimum 20% HITL review rate applies to outputs from prompts governed by that constraint. A documented gap without a proportional HITL rate is a governance failure, not an acceptable risk acknowledgment.

12.5.7 Decomposition Verification

Initial attestation is necessary but not sufficient. Decompositions must be periodically validated to ensure they remain adequate as models evolve, regulations change, and adversarial techniques improve.

Risk-based sampling. Not all constraints require the same verification intensity:

- Critical constraints (PHI, financial advice, safety-critical): full red-team testing, non-author re-attestation annually, regression suite maintained and run on every decomposition change
- Standard constraints: sampling-based verification — a random subset is re-attested each cycle, regression suite maintained
- Low-risk constraints: regression suite only, re-attestation triggered by material changes

Verification mechanisms:

- Red-team testing: adversarial inputs designed to violate the semantic intent while passing the mechanical checks. If an adversary can craft a prompt that the mechanical checks approve but that clearly violates the original intent, the decomposition is inadequate.
- Non-author re-attestation: periodic review by domain experts other than the original decomposition authors. Fresh eyes catch assumptions that become invisible to the original team.
- Automated regression testing: known-bad test cases that mechanical checks must catch. If a previously-detected violation pattern starts passing mechanical checks (due to check updates or configuration changes), the decomposition is flagged for revision.

Decomposition change triggers. Any change to C_m or C_p for a decomposition triggers automated rerun of CONTRADICTION detection with all existing constraints at affected levels, heuristic rerun of tension detection for overlapping domains, and regression test suite execution.

12.5.8 Role of LLM-Assisted Evaluation

Large language models may be used as development tooling within the decomposition process: triaging which semantic constraints are hardest to decompose, flagging potential gaps in proposed decompositions, and prioritizing which constraints need human review most urgently.

LLMs are not audit mechanisms. Non-deterministic evaluation cannot serve as audit evidence. LLMs are not attestors. “An AI evaluated this and found it compliant” is not a qualified human judgment. LLM evaluation exists outside the formal governance model. It informs the humans who author decompositions and perform attestations. It does not replace them.

12.6 Conflict Resolution

When multiple governance domains author constraints at the same level, those constraints may interact in ways that create tension or outright contradiction. A marketing domain’s obligation for conversational tone and a legal domain’s prohibition on promissory language are each individually reasonable, but satisfying both simultaneously is harder than satisfying either alone. This section defines the two-phase conflict resolution model: prevent conflicts at authoring time (Phase 1), and resolve escaped conflicts at evaluation time (Phase 2).

12.6.1 Why Conflicts Occur

Conflicts arise because governance is multi-stakeholder. The legal team, the security team, the marketing team, and the clinical safety team each author constraints from their domain perspective. Each set of domain-specific constraints is internally consistent. But the combined constraint set — the union of constraints from all domains at a given level — may contain interactions that no single domain anticipated.

Tension is not the same as contradiction. Two constraints are in contradiction when no prompt can satisfy both: $O(\phi)$ and $F(\phi)$ for the same atomic property ϕ . Two constraints are in tension when both can be individually satisfied, but satisfying both simultaneously is harder than satisfying either alone. Contradiction is a logical property that can be detected mechanically. Tension is a practical property that requires judgment to assess.

12.6.2 Phase 1: Conflict Prevention at Authoring

When a new constraint is proposed, the system performs automated conflict detection before the constraint can be registered.

Contradictions are hard blocks. If the proposed constraint contradicts an existing constraint at the same level, it cannot be registered. The author must modify the proposed constraint or work with the other domain to resolve the contradiction before resubmitting.

Tensions trigger cross-domain review. If the system detects potential tension between the proposed constraint and existing constraints from other domains, cross-domain review is triggered (§4.3). Representatives from all affected domains must review the interaction and choose a resolution before the constraint is registered.

Resolution options at authoring:

- **Modify constraints.** One or both constraints are revised to eliminate the tension. This is the preferred resolution.
- **Add a precedence rule.** A precedence annotation is added to the Enterprise Precedence Table (§6.5): “when these two constraints are both applicable and create tension, constraint A takes precedence.” The rationale is documented.
- **Accept the tension** — restricted by risk tier:
- **Critical tier: not permitted.** Tension in Critical-tier constraints must resolve to either modification or a precedence rule. Unresolved tension in Critical-tier constraints is a governance failure.
- **Standard tier:** permitted with cross-domain governance body approval and documented compensating controls.
- **Low tier:** permitted with owning domain approval.

12.6.3 Phase 2: Automated and Arbiter Resolution at Evaluation

If a conflict surfaces during prompt evaluation that was not caught at authoring, runtime resolution proceeds in three stages designed to keep production systems running while ensuring governance accountability:

Stage 1: Enterprise Precedence Table. The system consults the Enterprise Precedence Table (§6.5). If the conflicting constraints have a registered precedence rule, the rule is applied automatically. No human intervention is needed. This handles known conflict patterns in high-volume production flows without committee lag.

Stage 2: Default global priority. If no specific precedence rule exists for this constraint pair, the system applies the default domain priority ordering established at the enterprise level — for example, Safety > Legal > Security > Compliance > Business. The higher-priority domain’s constraint takes precedence. The interaction proceeds without blocking. The conflict event is simultaneously flagged for post-hoc arbiter review to create a permanent specific rule.

Stage 3: Arbiter escalation (non-blocking). The cross-domain governance arbiter reviews the conflict post-hoc to create a permanent resolution — either a new precedence rule in the Enterprise Precedence Table or a constraint modification through the standard authoring process (§4). Arbiter action is required but does not block the interaction. The default global priority already resolved it at runtime.

Anti-circularity requirement. The arbiter’s resolution must introduce a structural fix — a precedence rule or constraint modification — that prevents the same conflict from recurring. Arbiter resolutions that do not produce a permanent fix are rejected. This prevents an escalation loop where the same conflict is repeatedly flagged without resolution.

12.6.4 Formal Conflict Detection

CONTRADICTION is a formal predicate, mechanically detectable, and complete for the atomic property fragment:

$$\text{CONTRADICTION}(c_1, c_2) \equiv c_1 = O(\phi) \wedge c_2 = F(\phi) \text{ for some atomic } \phi \in \Phi$$

Since properties are atomic and independent (§3.2a), pairwise contradiction detection is complete. Any unsatisfiable pair of O/F constraints over independent atomic properties is detected by this predicate. Multi-constraint unsatisfiability where each pair is individually satisfiable but the set as a whole is not does not arise for the supported fragment — atomic O/F constraints over independent properties are jointly satisfiable if and only if they are pairwise satisfiable.

TENSION is an advisory classification, not a formal predicate. It is not sound, not complete, and no theorem or formal guarantee depends on it. It is a governance process aid. Detection heuristics include: the same atomic property referenced by constraints from different domains, properties related via the enterprise property ontology, and LLM-assisted analysis flagging potential semantic interactions (advisory, non-authoritative). Tensions are soft flags that trigger cross-domain review. They do not block constraint registration.

All formal guarantees in this framework are based on **CONTRADICTION** detection only. **TENSION** detection is operationally valuable but carries no formal weight.

12.6.5 Enterprise Precedence Table

The Enterprise Precedence Table is a machine-readable priority matrix maintained at the enterprise level. It serves two functions:

Default domain ordering. A global priority ranking across governance domains — for example, Safety > Legal > Security > Compliance > Business. This ordering is applied automatically during Phase 2, Stage 2 when no specific precedence rule exists. It prevents production systems from blocking on unknown conflicts while maintaining a defensible governance posture (safety and legal always take precedence over business preferences).

Specific constraint-pair rules. Known tension patterns are registered with explicit precedence decisions and documented rationale. When two specific constraints conflict, the table specifies which takes precedence and why. These rules are created through the standard authoring process or by arbiter resolution during Phase 2.

The precedence table is itself a governed, versioned, auditable artifact. Changes follow the standard authorship process (§4) with enterprise-level dual control. The table is the mechanism that transforms conflict resolution from a committee process into an automated, auditable system — essential for high-volume production AI systems where committee-speed resolution would constitute a denial of service.

12.7 Lateral Authority and Domain Isolation

Hierarchical inheritance governs how constraints flow vertically — from enterprise to department to project. Domain isolation governs how constraints are bounded horizontally — preventing one governance domain from interfering with another. Without lateral isolation, a marketing governance lead could inadvertently (or intentionally) weaken the legal team’s constraints. This section defines the domain model, scope partitioning, isolation guarantees, and the mechanisms for detecting indirect cross-domain effects.

12.7.1 Domain Model

A domain is a bounded area of governance authority. Typical domains include legal, security, data privacy, marketing, clinical safety, and financial compliance. Domains are defined at the enterprise level by central governance — individual departments cannot create or modify domain definitions.

Each domain has three defining attributes:

- A designated set of authorized agents (authors and approvers) who may operate within the domain
- A defined scope of properties from the enterprise controlled vocabulary (§3.2a) that the domain may govern
- A set of governance levels at which the domain may author constraints

Property vocabulary governance is critical for domain isolation. Adding a new property to the controlled vocabulary requires enterprise governance approval. If the new property falls in a region where multiple domains have scope, cross-domain review of the property definition is triggered before it can be used in constraints. This prevents a subtle form of gaming: creating new fine-grained properties to avoid overlap detection with existing domains.

12.7.2 Scope Partitioning

Direct overlap is detected formally through set intersection over canonical property identifiers. If $\text{scope}(D_1) \cap \text{scope}(D_2) \neq \emptyset$, the domains have overlapping authority over some properties.

For the overlap region:

- The overlap is explicitly defined and documented by enterprise governance
- Constraints governing properties in the overlap region require approval from all overlapping domains (joint authorship)
- No single domain can unilaterally create, modify, or delete constraints in the overlap region

For non-overlapping domains — where $\text{scope}(D_1) \cap \text{scope}(D_2) = \emptyset$ — complete isolation is enforced by scope checks at every constraint lifecycle event.

Indirect interaction is an acknowledged limitation. Actions in one domain’s scope may materially affect risk in another domain’s scope without sharing any property identifiers. A marketing constraint `0(use_layman_terms_exclusively)` could suppress a legal constraint `0(standard_legal_disclaimer)` if layman reformulation removes required legal language. These interactions are not detected by set-intersection scope analysis because the properties are formally separate.

Indirect interactions are partially detected through tension heuristics (§6.4) and impact analysis (§7.3). The formal Scope Isolation theorem (Section E.1.14) covers direct overlap only and explicitly states this limitation.

12.7.3 Domain Isolation Guarantee and Global Impact Analysis

Direct isolation is enforced by the constraint authorship system. An agent in domain D_1 cannot create, modify, or delete constraints in domain D_2 ’s exclusive scope. Shared-scope constraints require approval from all overlapping domains. These scope checks execute at every lifecycle event — creation, modification, deletion, and approval.

Indirect effect detection operates through Global Impact Analysis. When a new constraint is proposed, it is tested against a Gold Standard Prompt Set maintained by each domain. The Gold Standard set contains representative prompts that are known to satisfy each domain’s critical constraints. Impact is measured: does the new constraint cause any Gold Standard prompt to fail a constraint from another domain?

If yes, the new constraint is flagged for cross-domain review and treated as if it were in shared scope — regardless of whether the properties formally overlap. If no, the constraint proceeds through normal authoring.

Impact analysis is empirical, not exhaustive. It detects known interaction patterns but cannot prove absence of indirect effects. This is explicitly acknowledged in the formal model.

Adversarial diversity requirement. Gold Standard Prompt Sets must be constructed or audited by a party independent of the constraint authors in that domain. This prevents “thinning” the set to allow specific constraints to pass impact analysis undetected. The independent party may be the cross-domain governance body, an internal red team, or an external assessor — but it must not be the team proposing the new constraint.

12.7.4 Cross-Domain Governance Body

For enterprises with multiple overlapping domains, a standing cross-domain governance body is required for Critical-tier deployments. Its composition includes one representative from each domain, chaired by enterprise governance, plus an independent compliance representative not affiliated with any single domain.

Responsibilities:

- Resolve constraint tensions flagged by cross-domain review
- Serve as the arbiter for Phase 2 conflict resolution (§6.3)
- Periodically review the domain model and scope partitioning for adequacy
- Approve changes to domain definitions and scope boundaries
- Review and approve residual gaps for Critical-tier decompositions (§5.6)

Governance risk. The cross-domain governance body can centrally override domain isolation by redefining domain scopes. This concentration of authority is itself a governance risk. Compensating controls include: scope redefinition requires notification to all affected domains plus a minimum 5-business-day review period; scope changes are subject to audit integrity properties (AI1-AI6); the body’s composition must include diversity across reporting lines with no single business unit holding a majority.

The Scope Isolation theorem in Section E.1.14 holds relative to fixed domain definitions and explicitly does not limit the governance body’s scope-redefinition authority.

12.8 EPG as a Governance Component

EPG does not exist in isolation. It is one component of the Clad framework, formalized in Chapter 11. This section maps EPG’s concepts onto that component model, establishing how EPG participates in the broader governance architecture.

12.8.1 EPG as Component `g_EPG`

EPG instantiates the governance component `g_EPG` defined in the meta-framework:

- **Surfaces:** $S = \{S_prompt\}$ — EPG governs the prompt surface only
- **Constraints:** $C =$ the hierarchical constraint set $C^*(project)$ defined in this chapter
- **Evaluation:** $E =$ mechanical evaluation ($\models_m$) combined with procedural attestation ($\models_p$)
- **Audit:** $A =$ prompt audit records (evaluation records per meta-framework §9)
- **Hard requirements:** $R_hard = \emptyset$ — EPG is independently deployable. It provides its full guarantees without any other governance component present.
- **Soft requirements:** $R_soft = \{organizational\ governance\ context\}$ — EPG’s evaluation effectiveness improves when ROC provides output context and MDR provides monitoring data, but its guarantees do not depend on them.

12.8.2 Enforcement Requirements

EPG requires the meta-framework’s enforcement architecture to achieve full governability of the prompt surface:

- **EA1 (Chokepoint enforcement):** All AI interactions must pass through EPG enforcement points. No path from user input to model invocation may bypass EPG.

- **EA2 (Identity binding):** Every model invocation is bound to a governed project identity. Anonymous or unattributed model calls are blocked.

Without EA1 and EA2, the governability classification of `S_prompt` is partial rather than full — EPG’s guarantees apply only to interactions that voluntarily pass through it. Shadow AI and direct API access bypass governance entirely.

12.8.3 Audit Chain Participation

EPG produces evaluation records (component-signed) for each governed interaction. These records include the interaction identifier, constraint evaluations with version stamps for every applicable constraint, a cryptographic hash of the assembled prompt, and a version manifest of all governed elements.

Records are signed with EPG’s KMS key and hash-chained per the meta-framework’s audit integrity properties (AI2-AI3). During EPG failure, the Governance Supervisor produces process records (supervisor-signed) that maintain chain continuity without evaluation content — a weaker form of evidence that documents the failure rather than the evaluation.

12.8.4 Interface Contracts

EPG provides data to companion components through the meta-framework’s interface contract mechanism:

- **EPG to ROC:** the assembled prompt, the effective constraint set $C^*(\text{project})$, the prompt audit record, and the interaction identifier. These are soft requirements for ROC — they enhance ROC’s evaluation effectiveness but are not required for ROC’s guarantees.
- **EPG to MDR:** the prompt audit record and constraint version history. These enable monitoring systems to detect governance drift and perform root-cause analysis on incidents.

12.8.5 Failure Posture

EPG declares a failure posture per risk tier, consistent with the meta-framework’s failure semantics:

- **Critical workloads:** fail-closed. No prompt proceeds to the model without governance evaluation. If EPG is unavailable, the interaction is blocked.
- **Standard workloads:** fail-open-flagged. If EPG is unavailable, the interaction proceeds but is flagged as ungoverned in the audit trail.
- **Low-risk workloads:** configurable per deployment.

The binding between data classification and failure posture is itself a governed constraint — subject to the same authorship, dual control, and audit requirements as any other constraint.

12.9 Operational Guidance

This section translates EPG’s formal model into practical adoption guidance for enterprises. It covers phased rollout, risk-tiered application, change management, constraint libraries, tooling, and the framework’s explicit limitations.

12.9.1 Phased Adoption

EPG adoption follows a maturity progression. Each phase provides independently valuable governance; later phases build on earlier ones.

Phase 1 (3-6 months): Enterprise baseline. Establish enterprise-level constraints covering security, legal, and compliance baselines. Deploy the controlled property vocabulary. Create the Enterprise Precedence Table with default domain ordering. Stand up the cross-domain governance body. Deploy constraint authoring and evaluation tooling.

Phase 2 (6-12 months): Department governance. Add department-level constraints for the highest-risk business units first. Adopt the first constraint libraries (§9.4). Establish Gold Standard Prompt Sets and impact analysis for each domain. Begin decomposition verification cadence for Critical-tier constraints.

Phase 3 (12-18 months): Full coverage. Extend governance to all departments and project-level constraints. Establish full verification cadence across all risk tiers. Risk-tiered governance is fully operational. Constraint change management pipelines are mature.

Phase 4 (18+ months): Optimization. Develop organization-specific constraint libraries. Contribute to cross-organization library sharing. Conduct maturity assessments. Optimize governance overhead based on operational data.

12.9.2 Risk-Tiered Application

Not all AI interactions require the same governance intensity. EPG maps to three risk tiers:

Critical (PHI, PCI, financial advice, safety-critical operations): full constraint evaluation, fail-closed posture, orthogonal dual control with central approver, full verification cadence (annual red-team, continuous regression), accept-tension prohibited, complete audit chain with HITL trigger for residual gaps.

Standard (internal tools, non-sensitive business processes): full constraint evaluation, fail-open-flagged posture, standard dual control, sampling-based verification, complete audit chain.

Low (development, sandbox, experimentation): enterprise constraints only, fail-open-flagged or configurable posture, standard dual control, regression-only verification, interaction logging.

12.9.3 Constraint Change Management

Constraints are governance artifacts with production impact. They require the same change management discipline as production code.

Staging environment. New or modified constraints are tested against the Gold Standard Prompt Set (§7.3) and the regression suite before deployment to production. A constraint that breaks existing Gold Standard prompts is caught before it affects live interactions.

Canary deployment. For high-volume production systems, new constraints can be deployed to a subset of interactions first, with monitoring for unexpected failures or conflicts. If the canary shows elevated failure rates or unexpected tension patterns, the deployment is halted.

Rollback. Every constraint change has an automated rollback path to the previous known-good version. If a newly deployed constraint causes production issues, the system can revert to the prior state within minutes, not hours.

Change windows. For Critical-tier workloads, constraint changes follow a defined change window with notification to all affected project teams. Surprise constraint changes in production are a governance failure, not a governance feature.

12.9.4 Constraint Libraries

Pre-decomposed, industry-vetted constraint packages reduce the per-organization authoring burden. A “HIPAA Prompt Constraints v1.0” package, for example, provides a set of enterprise-level constraints derived from the HIPAA Privacy Rule and Security Rule, with pre-attested decompositions, known residual gaps, and regression test suites.

Libraries are versioned using semantic versioning (major.minor.patch), maintained by domain experts, and subject to the same governance lifecycle as custom constraints.

Local verification gate. Organizations cannot simply import a library and activate it. They must:

- Review each constraint and decomposition against their specific technical context
- Attest that the decompositions are adequate for their deployment (vendor decomposition may not match local model behavior)
- Run the library’s regression test suite against their actual models and prompts. This is mechanically enforced: the system verifies that the test suite has been executed and passed on local infrastructure before transitioning library constraints from “Registered” to “Active.” Paper attestation without test execution is not permitted.
- Minimum coverage: local test execution must cover at least the library’s defined regression suite in full. Organizations may add local test cases but may not skip library-defined cases.

Liability. The adopting organization bears responsibility for constraint adequacy, not the library maintainer. The library provides a starting point; local attestation provides accountability.

12.9.5 Tooling Recommendations

Effective EPG deployment requires tooling support across several dimensions:

- Constraint authoring IDE with conflict detection and controlled vocabulary enforcement
- Automated mechanical evaluation engine with regression test management
- Procedural attestation workflow system with evidence capture and signing
- LLM-assisted triage for decomposition adequacy and tension detection
- Gold Standard Prompt Set management and impact analysis tooling
- Dashboard for constraint coverage, evaluation results, verification status, and audit compliance
- Enterprise Precedence Table editor with change control integration

12.9.6 Limitations

EPG provides governance of the prompt surface. This section states explicitly what EPG does and does not guarantee.

What EPG guarantees (formal commitments):

- Audit completeness: every governed prompt is evaluated against every applicable constraint, with a tamper-evident, version-stamped record
- Contradiction freedom: no pair of constraints in the effective set is contradictory (for the atomic property fragment)
- Scope isolation: no agent can create or modify constraints outside their authorized domain scope (under fixed domain definitions)
- Decomposition coverage: every semantic intent has a registered decomposition (existence, not quality)

What EPG does not guarantee:

- Output correctness: EPG governs prompts, not model outputs. A governed prompt can still produce non-compliant output due to model behavior, user input, or inference configuration. Output governance requires runtime output controls.
- Content accuracy: EPG ensures prompts include required elements and exclude prohibited elements. It does not ensure the model's response is factually correct.
- Model behavior: EPG cannot prevent hallucination, jailbreaking, or emergent model behavior. These require model-level and output-level controls.

What EPG provides as governance-process tools (operationally valuable, not formally proven):

- Tension detection: advisory heuristics that flag potential constraint interactions. Valuable for governance practice but not mathematically guaranteed to be sound or complete.
- Impact analysis: empirical testing of indirect cross-domain effects via Gold Standard Prompt Sets. Detects known patterns but cannot prove absence of indirect effects.
- Decomposition verification: red-team testing and re-attestation that seek to falsify soundness claims. Provides empirical confidence, not logical proof.
- Collusion detection: periodic audit sampling and anomaly detection on constraint changes. Catches patterns of abuse but cannot prevent determined adversaries.

The formal model uses a restricted deontic fragment — atomic properties with two operators (O and F) — that does not capture all real-world governance complexity. This is a deliberate design choice for tractability, not an oversight. The restricted fragment enables complete contradiction detection, a property that would be undecidable in a richer logic.

EPG is one component of a complete governance solution. It must be complemented by runtime output controls (ROC), monitoring and response (MDR), and organizational controls (training, role-based access management, change management processes) to achieve comprehensive AI governance.

12.10 Key takeaways

- EPG governs S_prompt by defining the *boundaries* teams build within, not the prompts themselves. Constraints flow enterprise $\succ$ department $\succ$ project, each level tightening but never loosening what upstream established (§3).
- Semantic constraints become auditable by mandatory decomposition into mechanical checks plus procedural attestation (§5); conflicts resolve by a defined precedence (§6); authority is bounded laterally so no domain can weaken another’s protections (§7).
- As a component, EPG is g_EPG with hard requirements $R_hard = \emptyset$ — it is independently deployable and delivers its full guarantees alone, while composing cleanly with ROC and MDR (§8).
- EPG governs the prompt, not the output. A compliant prompt can still yield a non-compliant output, because models are stochastic. That gap is exactly what runtime output controls address.

The formal model, worked examples, and constraint-library template for EPG are collected in the formal-model appendix, the worked-examples appendix, and the templates appendix.

13 Stopping Bad Outputs (ROC)

Chapter 12 governed the prompt — the surface an enterprise fully controls before the model runs. But a governed prompt does not guarantee a governed output: models are stochastic, and the same instructions can produce different results. This chapter governs the other side of the model — the output, before it reaches a user — through Runtime Output Controls (ROC), the component `g_ROC`.

By the end of the chapter you will be able to:

- explain why prompt governance alone leaves irreducible residual risk on the output surface;
- distinguish deterministic (hard) from classifier-based (soft) output evaluation, and reason about their different evidentiary properties;
- design a risk-tiered pass / flag / block pipeline that trades evaluation depth against delivery latency;
- select controls for the output-side threats EPG defers — prompt injection, jailbreaking, PII/PHI leakage, and cross-tenant isolation.

13.1 The Problem and the Approach

13.1.1 The Problem: Ungoverned Model Outputs

Enterprise Prompt Governance (EPG, Chapter 12) ensures that the instructions sent to AI models are compliant, consistent, and auditable. But a governed prompt does not guarantee a governed output. AI models are stochastic — the same prompt can produce different outputs across invocations. A perfectly governed prompt can still yield an output that discloses protected health information, provides unauthorized financial advice, reproduces copyrighted training data, or responds to a jailbreak attempt with content the prompt explicitly prohibited.

This is not a theoretical concern. It is a mathematical certainty, formalized in the meta-framework as the Irreducible Residual Risk theorem: even with all governance components deployed, residual compliance risk is non-zero. The question is not whether non-compliant outputs will occur, but whether the enterprise can detect them before they reach end users, contain them when they do, and trace them back to root cause.

Runtime Output Controls (ROC) addresses this gap. Where EPG governs what goes into the model, ROC governs what comes out.

13.1.2 What ROC Provides

ROC is a runtime evaluation and filtering layer that operates between model output generation and output delivery to end users or downstream systems. It provides:

Hybrid output evaluation. Hard output constraints — such as “never include Social Security numbers in responses” — are evaluated using deterministic checks identical in character to EPG’s mechanical evaluations. Soft output constraints — such as “detect potential medical advice” — are evaluated using classifier-based scoring with configurable thresholds. Both tiers produce auditable evaluation records, but with explicitly different evidentiary properties.

Risk-tiered timing. Critical workloads (PHI, PCI, safety-critical operations) receive real-time blocking evaluation: no output reaches the end user until ROC has evaluated it. Standard and low-risk workloads receive outputs immediately, with asynchronous evaluation and post-hoc flagging of violations. This balances safety against latency.

Threat-specific controls. ROC addresses the threats that EPG explicitly deferred: prompt injection detection (T7), jailbreak detection (T8), PII/PHI filtering (T10), training data leakage detection (T10), and cross-tenant data isolation (T11).

Full audit chain participation. ROC produces output audit records — component-signed, hash-chained, and version-stamped — that compose with EPG’s prompt audit records and MDR’s monitoring records into the complete audit chain defined by the meta-framework.

13.1.3 How ROC Fits the Governance Solution

ROC instantiates component `g_ROC` from the meta-framework, governing `S_output` (raw model output, governability $\gamma = \text{partial}$) and `S_delivery` (delivered output after filtering, governability $\gamma = \text{full}$). It is independently deployable — its guarantees hold without EPG or MDR present — but its evaluation effectiveness improves with EPG context (the prompt and its constraint evaluations inform output risk assessment).

The three-component governance pipeline is: EPG governs the prompt (before inference) $\rightarrow$ the model produces output $\rightarrow$ ROC evaluates and filters the output (before delivery) $\rightarrow$ MDR monitors patterns across interactions (continuous). Each component provides independent guarantees that compose into system-wide coverage.

Relationship to EPG’s formal model. EPG uses a deontic constraint framework (obligations O and prohibitions F over atomic properties from a controlled vocabulary Φ). ROC uses a different evaluation model — deterministic rules, classifier-based scoring, and composite logic — because output evaluation is fundamentally different from prompt constraint evaluation. Outputs are dynamic, stochastic artifacts that require probabilistic detection, not static artifacts that can be evaluated against binary predicates. The two components share the property vocabulary Φ and compose at the audit chain level (shared `interaction_id`, hash-chained records), but they do not share a constraint evaluation logic. This is by design, not an inconsistency.

13.2 Principles

ROC shares the meta-framework’s foundational principles (risk management over elimination, audit the artifact, composable governance). Three additional principles are specific to output controls.

13.2.1 Evaluate What Was Produced, Not What Was Intended

EPG ensures the prompt requests compliant behavior. ROC evaluates whether the model actually produced compliant output. These are different questions with different answers. A prompt that forbids medical diagnosis may still receive a diagnostic response if the model ignores, misinterprets, or is jailbroken past the instruction. ROC’s evaluation target is the output artifact itself — the actual tokens the model generated — not the prompt that requested them.

13.2.2 Honest Probabilism Over False Determinism

Some output evaluations are deterministic: a Social Security number regex either matches or it doesn’t. Others are probabilistic: a classifier scoring whether text constitutes “medical advice”

produces a confidence score, not a binary answer. ROC does not pretend probabilistic evaluations are deterministic. It explicitly distinguishes two evaluation tiers — deterministic and classifier-based — with different audit properties. A deterministic evaluation is hard evidence (“this output contained an SSN”). A classifier-based evaluation is scored evidence (“this output scored 0.87 on the medical-advice classifier, above the 0.80 threshold”). Both are auditable. Neither is falsely promoted to the other’s evidentiary standard.

13.2.3 Speed Is a Safety Property

In output governance, latency is not just a performance concern — it is a safety and availability property. A real-time blocking evaluator that takes 10 seconds per output transforms a responsive AI system into an unusable one. An unusable governed system drives users to ungoverned alternatives (shadow AI). Therefore, ROC explicitly models the tradeoff between evaluation depth and delivery latency through risk-tiered timing: real-time blocking for Critical workloads where safety outweighs speed, and asynchronous evaluation for Standard/Low workloads where speed enables adoption and adoption enables governance.

13.2.4 Defense in Depth, Not Defense in Duplication

ROC does not duplicate EPG’s work. EPG ensures the prompt is well-formed. ROC ensures the output is compliant. The two components address different failure modes: EPG addresses governance process failures (wrong instructions). ROC addresses model behavior failures (wrong outputs despite right instructions). When EPG provides prompt context to ROC, ROC uses it to improve evaluation accuracy — but ROC’s guarantees do not depend on EPG being present.

13.3 The Output Evaluation Model

EPG evaluates prompts — static artifacts authored before inference. ROC evaluates outputs — dynamic artifacts generated by a stochastic model at inference time. This fundamental difference shapes the entire evaluation model.

13.3.1 Two Evaluation Tiers

ROC uses a hybrid evaluation model with two tiers, each with distinct properties:

Tier 1: Deterministic evaluation. Rule-based checks that produce binary pass/fail results. A regex for Social Security numbers either matches or doesn’t. A keyword blocklist either flags or doesn’t. A structural check for required disclaimers either finds them or doesn’t. These evaluations are deterministic, repeatable, and produce the same evidentiary standard as EPG’s mechanical checks (C_m). They are the output-side equivalent of EPG’s mechanical evaluation.

Formal property: $\text{eval_d} : \mathcal{O} \times \mathcal{R} \rightarrow \{\text{pass}, \text{fail}\}$ where $\mathcal{O}$ is the output and $\mathcal{R}$ is the rule. The evaluation is a total function — every output has a definite result for every rule.

Tier 2: Classifier-based evaluation. ML classifiers that produce confidence scores. A PII detection model scores the likelihood that an output contains personally identifiable information. A toxicity classifier scores the likelihood of harmful content. A medical-advice detector scores the likelihood that text constitutes clinical guidance. These evaluations are probabilistic — the same output evaluated twice by the same classifier may produce slightly different scores due to model non-determinism.

Formal property: $\text{eval_c} : \mathcal{O} \times \mathcal{K} \rightarrow [0, 1]$ where $\mathcal{O}$ is the output and $\mathcal{K}$ is the classifier.

The evaluation produces a confidence score, not a binary result. A configurable threshold τ_K converts the score to a governance decision: $\text{score} \geq \tau_K \rightarrow \text{flag}$ and $\text{score} < \tau_K \rightarrow \text{below_threshold}$.

Critical distinction: A classifier score below threshold is NOT a “pass” in the same sense as a deterministic evaluation pass. It means “the classifier did not reach the threshold of suspicion at the configured sensitivity.” The audit record must use quantized satisfaction notation — $o \models_{\tau} c$ — meaning “output o satisfied constraint c at threshold τ .” This is weaker evidence than deterministic satisfaction ($o \models c$) and the audit record must make this distinction explicit. Calling a below-threshold score a “pass” is regulatory dishonesty.

13.3.2 Threshold Governance

Classifier thresholds are governance artifacts, not implementation details. The threshold τ_K for each classifier K determines the tradeoff between false positives (compliant outputs incorrectly flagged) and false negatives (non-compliant outputs incorrectly passed). This tradeoff has direct regulatory and operational consequences:

- A PHI classifier threshold set too low floods the review queue with false positives, making governance operationally unsustainable.
- A PHI classifier threshold set too high lets real PHI disclosures through, creating regulatory exposure.

Therefore, thresholds are subject to the same governance controls as EPG constraints: they are versioned, owned by a governance domain, require dual-control modification, and are included in the audit record for every evaluation. When a regulator asks “why did your system approve this output?” the answer includes not just the classifier score but the threshold in effect at that time and who set it.

Risk-tiered threshold defaults:

- Critical workloads: conservative thresholds (lower τ , more flags, fewer misses)
- Standard workloads: balanced thresholds
- Low-risk workloads: permissive thresholds (higher τ , fewer flags, more throughput)

Threshold monotonicity invariant. Consistent with EPG’s constraint inheritance principle (Chapter 12; lower levels can only tighten, never loosen), classifier thresholds must satisfy: $\forall l_2 \prec l_1 : \tau(k, l_2) \leq \tau(k, l_1)$. A department may lower a threshold (tighten, more flags) but may never raise it above the enterprise default. A project may lower it further but never above the department’s level. This prevents governance weakening through threshold manipulation.

Threshold policy bounds for Critical tier. For Critical-tier constraints, each classifier has a policy-defined maximum threshold (τ_{max}) that cannot be exceeded without an exceptional process (board-level risk exception, documented and time-limited). This prevents insiders from gradually raising thresholds until classifiers are effectively disabled. Threshold changes and their empirical flag rates must be monitored centrally and reported to MDR. A sustained decrease in flag rates without corresponding improvement in classifier precision is an anomaly that triggers governance review.

13.3.3 Audit Properties by Tier

The two tiers produce audit records with explicitly different evidentiary properties:

Deterministic evaluation records state: “This output was evaluated against rule R. The result was pass/fail. This is a deterministic, repeatable finding.”

Classifier-based evaluation records state: “This output was scored by classifier K (version V) with threshold τ_K . The score was S. The governance decision was flag/below_threshold. This is a probabilistic finding with the stated confidence.” A classifier “below_threshold” result is not proof of compliance — it is one input to a broader control set. For regulated constraints (PHI, financial advice), organizations must maintain external classifier validation reports linked in the classifier governance profile; auditors should reference those reports alongside per-event records.

Both records are component-signed, hash-chained, and version-stamped per the meta-framework’s audit integrity properties (AI1-AI6). But consumers of audit records — regulators, auditors, incident responders — must understand that a classifier-based “pass” is not the same evidentiary standard as a deterministic “pass.” The audit record makes this distinction explicit.

13.3.4 The Relationship Between Prompt Constraints and Output Controls

EPG constrains what the prompt instructs. ROC constrains what the output contains. These are complementary but independent:

- EPG’s `F(medical_diagnosis)` ensures the prompt does not instruct the model to diagnose. ROC’s medical-advice classifier checks whether the output actually contains diagnostic content regardless of what the prompt said.
- EPG’s `0(hipaa_disclaimer)` ensures the prompt includes a disclaimer instruction. ROC’s disclaimer detector checks whether the output actually includes the disclaimer.
- EPG may have no constraint on a topic, but ROC independently blocks outputs containing PII patterns.

When EPG context is available (via the interface contract), ROC uses it to improve evaluation: if the prompt was flagged for tension between conversational tone and regulatory language, ROC may apply stricter classifiers to the output. But ROC’s core guarantees — that every output is evaluated and a complete audit record is produced — hold without EPG context.

13.4 Output Constraint Taxonomy

Output constraints are organized into three classes based on their evaluation mechanism. Every output constraint belongs to exactly one class.

13.4.1 Deterministic Output Constraints (`O_d`)

Rule-based checks with binary outcomes. These are the output equivalent of EPG’s mechanically evaluable constraints (`C_m`).

Examples:

- Pattern matching: SSN, credit card number, medical record number detection via regex or NER

- Blocklist enforcement: prohibited terms, competitor names, restricted phrases
- Structural checks: required disclaimer presence, required confidence disclosure, format compliance
- Length and format: output length limits, required JSON schema compliance, encoding validation

Properties: deterministic, repeatable, low latency, high confidence. Audit records carry full evidentiary weight.

13.4.2 Classifier-Based Output Constraints (O_c)

ML classifier evaluations with confidence scores. These have no direct equivalent in EPG — they are unique to output governance because outputs exhibit semantic properties that resist rule-based detection.

Examples:

- PII/PHI detection: NER-based classifiers trained on medical records, financial data, personal identifiers
- Toxicity and harm scoring: classifiers detecting hateful, violent, or dangerous content
- Medical advice detection: classifiers distinguishing informational content from diagnostic or prescriptive language
- Financial advice detection: classifiers detecting personalized investment recommendations vs. general market commentary
- Prompt injection detection: classifiers identifying outputs that suggest the model's instructions were overridden
- Copyright/memorization detection: classifiers detecting verbatim reproduction of training data

Properties: probabilistic, threshold-dependent, variable latency (depends on classifier complexity), confidence-scored. Audit records carry scored evidentiary weight (confidence + threshold).

Each classifier has a governance profile:

- Classifier identity and version (Axiom 5 — temporal identity)
- Threshold τ (governed artifact, dual-control modification)
- Known false positive and false negative rates at the configured threshold
- Validation cadence (how often the classifier is retested against updated benchmark data)

13.4.3 Composite Output Constraints (O_x)

Evaluations that combine deterministic and classifier-based checks into a single governance decision. These address scenarios where neither tier alone is sufficient.

Example: “Output must not disclose PHI” decomposes into:

- Deterministic: regex scan for SSN, MRN, date-of-birth patterns (O_d)
- Classifier: NER model for contextual PII detection — names in medical context, indirect identifiers (O_c)
- Composite decision: flag if EITHER the deterministic check fails OR the classifier score exceeds threshold

Composite constraints require explicit decision logic: how do the component evaluations combine? Options include ANY-flag (flag if any component flags — conservative), ALL-flag (flag only if all components flag — permissive), or weighted scoring. The decision logic is itself a governed artifact.

13.5 The Output Filtering Pipeline

ROC operates as a pipeline between model output generation and delivery. The pipeline has four stages, executed in order.

13.5.1 Stage 1: Deterministic Evaluation

All deterministic output constraints (O_d) are evaluated against the raw model output. This stage is fast (milliseconds) and produces binary results.

Short-circuit rule: If any deterministic constraint fails in Critical tier, the pipeline decision is BLOCK immediately — Stage 2 and 3 results are not awaited. This prevents latency from classifier evaluation delaying a deterministic block.

If any deterministic constraint fails:

- Critical tier: output is BLOCKED immediately (short-circuit). Proceed to Stage 4 (fall-back).
- Standard/Low tier: failure is FLAGGED in the audit record. Pipeline continues.

13.5.2 Stage 2: Classifier-Based Evaluation

All applicable classifier-based output constraints (O_c) are evaluated. Classifiers score the output; scores are compared to configured thresholds.

Critical tier: ATOMIC BUFFERED EVALUATION. The complete output is buffered before any delivery begins. Streaming is disabled for Critical S_delivery. Classifiers run on the complete buffered output synchronously. No tokens reach the end user until all evaluations complete. Latency budget is deployment-specific (recommended: 500ms-2s maximum). If classifiers exceed the latency budget, the timeout is treated as a failure: ROC defaults to block + safe response. Delivering an unevaluated output on timeout is never permitted for Critical tier.

Standard tier: POST-HOC AUDIT MONITORING. Output is delivered immediately. Classifiers run asynchronously. If a violation is detected post-delivery, the violation is

logged and an alert is generated. This is NOT a preventative control on S_delivery — it is a detection/audit function. The output cannot be un-delivered; the incident record enables after-the-fact remediation. Organizations must understand that Standard-tier classifier evaluation provides monitoring evidence, not real-time protection.

Low tier: SAMPLED POST-HOC AUDIT. Only a deployment-specific percentage of outputs are evaluated by classifiers. Minimum sampling floor must be defined per data classification and cannot be reduced below the floor without independent review. This reduces computational cost but provides only statistical coverage.

Minimum sampling and threshold governance. Sampling rates and thresholds for each risk tier are tied to a central data classification registry. Moving a workload from Critical to Standard or Low requires independent review (not just the owning team’s decision). This prevents insider abuse: a malicious actor cannot reclassify sensitive workloads to avoid evaluation.

13.5.3 Stage 3: Composite Evaluation and Decision

Composite constraints (O_x) aggregate results from Stages 1 and 2. The governed decision logic produces a final evaluation for each composite constraint.

The pipeline produces one of three outcomes for each output:

- **PASS:** all constraints satisfied. Output proceeds to delivery.
- **FLAG:** one or more soft constraints triggered (classifier threshold exceeded). Output may proceed with a flag in the audit record, or may be held depending on risk tier and flag severity.
- **BLOCK:** one or more hard constraints failed (deterministic violation) or Critical-tier classifier threshold exceeded. Output does not proceed. Stage 4 activates.

13.5.4 Stage 4: Blocked Output Handling (Fallback)

When an output is blocked, the system must respond — silence is not acceptable to the end user. Fallback options, configurable per constraint and per risk tier:

Safe response substitution. The blocked output is replaced with a pre-approved safe response: “I’m unable to provide that information. Please contact [qualified resource].” Safe responses are themselves governed artifacts — authored, reviewed, and versioned under EPG constraints.

Redaction with post-redaction re-evaluation. For outputs where only a portion is non-compliant (e.g., an otherwise useful response that contains one SSN), the non-compliant portion is redacted and the remainder is delivered with a redaction notice. Redaction rules are deterministic and auditable. For Critical-tier PHI constraints, the composite PHI constraint (O_x) must be re-run on the redacted output. If the redacted output still flags (because residual context enables re-identification — e.g., rare disease + small location), the output is blocked entirely and a safe response is substituted. Redaction is only acceptable for PHI when the post-redaction re-evaluation passes.

Retry with modified prompt. The system re-invokes the model with a modified prompt that reinforces the violated constraint. The retry is itself a governed interaction (registered in the GIL, evaluated by EPG). For Critical workloads: maximum 1 retry. If the retry output is semantically similar to the original (similarity hash comparison), skip the retry and proceed directly to safe response substitution — repeated attempts against model memorization or inherent behavior waste resources and increase exposure. For Standard/Low: maximum 2 retries.

Escalation. For violations that cannot be handled by substitution, redaction, or retry, the interaction is escalated to a human reviewer. The output is held (not delivered) until the reviewer makes a disposition decision. Escalation is the last resort and should be rare in a well-tuned system.

Every fallback action is recorded in the audit trail: what was blocked, why, which fallback was applied, and what was ultimately delivered (or not delivered) to the end user.

Block-rate monitoring. Sustained high block/fallback rates are themselves an operational incident. ROC's block and substitution metrics must feed MDR or a central dashboard. When block rates exceed a baseline threshold (deployment-specific), an investigation is triggered: the root cause may be model drift, prompt governance failure, classifier miscalibration, or adversarial probing. Frequent safe-response substitution degrades user trust and drives users to ungoverned alternatives.

13.6 Threat-Specific Controls

Chapter 11 identified five threats (T7-T11) that EPG deferred to ROC and MDR. This section defines ROC's controls for each.

13.6.1 T7: Prompt Injection Detection

Threat: Untrusted content embedded in the prompt (via user input, RAG-retrieved documents, or tool outputs) overrides governance instructions, causing the model to produce output that violates constraints.

ROC's role: Detect outputs that suggest the model's instructions were overridden, regardless of whether the injection was in the prompt (EPG's concern) or the output (ROC's concern).

Controls:

- **Deterministic (O_d):** pattern detection for common injection signatures in outputs — instruction echoing (“As an AI assistant, I have been told to ignore...”), role-switching markers, system prompt leakage.
- **Classifier-based (O_c):** injection detection classifier trained on known injection/exfiltration output patterns. Scores the likelihood that the output was influenced by injected instructions rather than the governed prompt.
- **Behavioral:** compare output against the prompt's stated constraints. If EPG context is available, ROC can evaluate whether the output satisfies the constraints EPG imposed. An output that violates a constraint the prompt explicitly enforced is a strong signal of injection.
- **Cross-surface consistency check (Critical tier, requires EPG context):** compare the semantic intent of the output against the semantic intent of the prompt using a summarization-to-intent classifier. If $\text{Intent}(\text{output}) \notin \text{Intent}(\text{prompt})$, the output is flagged as potentially injection-influenced. This detects subtle injection where the model shifts topic or persona without triggering overt pattern detectors.

Limitation: Prompt injection is fundamentally an arms race. New injection techniques emerge regularly. ROC's controls reduce risk but cannot eliminate it. Classifier retraining cadence and red-team testing frequency must match the evolving threat landscape.

13.6.2 T8: Jailbreak Detection

Threat: Adversarial user inputs cause the model to ignore its safety instructions, producing content the prompt explicitly prohibited.

ROC’s role: Detect outputs that indicate the model has been jailbroken — content that a properly-instructed model would not produce.

Controls:

- Classifier-based (O_c): jailbreak detection classifier scoring the likelihood that the output represents a safety boundary violation. Trained on red-team datasets of successful jailbreak outputs.
- Constraint violation detection: if EPG context is available, ROC checks whether the output violates any constraint that the prompt was evaluated against. An output violating F(medical_diagnosis) when the prompt passed EPG’s F(medical_diagnosis) check is a jailbreak indicator.
- Rate-based detection: repeated outputs from the same user session that progressively approach constraint boundaries suggest probing behavior. ROC flags these patterns for MDR escalation.

Limitation: Jailbreak techniques evolve faster than classifiers can be retrained. ROC provides a defense layer, not an impenetrable barrier. The meta-framework’s Theorem 4 (Irreducible Residual Risk) applies directly here.

13.6.3 T10: PII/PHI and Training Data Leakage

Threat: The model reproduces protected health information, personally identifiable information, copyrighted content, or other sensitive training data in its outputs.

ROC’s role: Detect and prevent sensitive data from reaching end users.

Controls:

- Deterministic (O_d): regex and pattern matching for structured PII (SSNs, credit card numbers, medical record numbers, phone numbers, email addresses). High precision, low false-positive rate.
- Classifier-based (O_c): NER models for contextual PII detection (names in medical context, dates of birth adjacent to diagnoses, indirect identifiers). Higher recall than regex but with false-positive cost.
- Composite (O_x): for Critical-tier HIPAA/PCI workloads, combine deterministic + classifier into a conservative ANY-flag composite. Flag if either component detects potential PII.
- Verbatim reproduction detection: compare output against known copyrighted or sensitive corpus segments using similarity scoring. This addresses training data memorization — the model reproducing passages it was trained on.

Redaction capability: When PII is detected in an otherwise useful output, ROC can redact the PII elements (replace with “[REDACTED]” or type-appropriate placeholders) rather

than blocking the entire output. Redaction preserves utility while removing the compliance violation.

PHI-first policy for HIPAA-scope Critical workloads. For any HIPAA-covered workload classified as Critical, the following mandatory controls apply:

- Any classifier PHI score above a conservative alert threshold (τ_{alert} , lower than the standard flag threshold) must trigger either block/redact or human review routing. “Flag only” without blocking or human review is not acceptable for PHI in Critical tier.
- Documented PHI residual risk must be paired with compensating controls, including a minimum percentage of human-in-the-loop review on outputs — analogous to EPG’s residual gap controls (Chapter 12, §5.6).
- The “flag and continue” path available for Standard/Low tiers does not apply to PHI in Critical tier. Every PHI suspicion above τ_{alert} is either blocked, redacted (with post-redaction re-evaluation), or routed to human review.

Audit specificity: PII detection audit records include the type of PII detected (SSN, name, MRN, etc.), the detection method (regex, classifier, composite), and the action taken (block, redact, human-review, flag). This granularity enables HIPAA breach assessment — regulators need to know not just that a violation occurred, but what type of data was exposed and to whom.

13.6.4 T11: Cross-Tenant Data Leakage (Monitoring Signal, Not Control)

Threat: In multi-tenant AI deployments (shared model endpoints), data from one tenant influences outputs for another.

ROC’s role: MONITORING SIGNAL ONLY. Cross-tenant isolation is fundamentally a model deployment and infrastructure concern — tenant-specific models, data isolation in fine-tuning, KV-cache separation. ROC cannot prevent cross-tenant contamination at the model level. It can only detect certain leakage patterns in outputs after the fact.

ROC does NOT claim T11 as a control it enforces. It provides detection signals that feed MDR for investigation:

- Tenant context binding: each interaction carries a tenant identifier. ROC compares output content against the tenant’s known data domain. Anomalies (Company B’s products appearing in Company A’s output) are flagged to MDR.
- Deterministic (O_d): if tenant-specific identifiers are known (product names, customer IDs, internal terminology), pattern matching detects their presence in the wrong tenant’s outputs. These are passed to MDR as incident signals.
- Statistical detection is MDR’s responsibility, not ROC’s. Subtle cross-tenant patterns (style transfer, knowledge leakage without explicit identifiers) require trend analysis across many interactions.

This is explicitly a monitoring function, not a governance control. ROC’s audit record notes the tenant-consistency check result but does not claim to “govern” cross-tenant isolation. Infrastructure controls (model isolation, data segregation, tenant-specific deployments) are the actual controls for T11.

13.7 ROC as a Governance Component

13.7.1 ROC as Component g_ROC

ROC instantiates the meta-framework's g_ROC:

- **Surfaces:** $S = \{S_output, S_delivery\}$
- **Constraints:** $C_ROC = O_d \cup O_c \cup O_x$ (deterministic, classifier-based, composite)
- **Evaluation:** $E = \text{deterministic evaluation (eval_d)} \cup \text{classifier evaluation (eval_c)} \cup \text{composite logic}$
- **Audit:** $A = \text{output audit records (evaluation records per meta-framework §9)}$
- **R_hard:** $\emptyset$ — independently deployable
- **R_soft:** $\{\text{prompt context from EPG, constraint set } C^*(\text{project}) \text{ from EPG}\}$

13.7.2 Interface Contracts

EPG $\rightarrow$ ROC (soft requirements): EPG provides the assembled prompt, its effective constraint set, and the prompt audit record. ROC uses this context to improve evaluation accuracy — for example, applying stricter classifiers when the prompt was flagged for tension, or checking output compliance against specific EPG constraints. Without EPG context, ROC still evaluates outputs against its own constraint set but with reduced contextual precision.

ROC $\rightarrow$ MDR (soft requirements): ROC provides the output audit record, any flagged violations or anomalies, the delivered output and its hash, and violation flags. MDR uses this for trend analysis, incident correlation, and compliance drift monitoring.

13.7.3 Audit Chain Participation

ROC produces output audit records for every governed interaction. Each record contains:

- Interaction identifier (shared with EPG and MDR via GIL)
- The raw output hash and the delivered output hash (may differ if redaction occurred)
- Evaluation results for every applicable output constraint, with tier designation (deterministic or classifier-based)
- For classifier evaluations: classifier version, score, threshold, and governance decision
- For blocked outputs: the fallback action taken and the content ultimately delivered
- Predecessor pointer to EPG's audit record (if available)

Records are component-signed with ROC's KMS key, hash-chained per AI2, and subject to all meta-framework audit integrity properties (AI1-AI6). During ROC failure, the Governance Supervisor produces process records per meta-framework §7.4.

13.7.4 Failure Posture

ROC's failure posture is risk-tiered, consistent with the meta-framework:

- **Critical:** fail-closed. No output is delivered without ROC evaluation. If ROC is unavailable, the output is held and a safe fallback response is delivered to the user.
- **Standard:** fail-open-flagged. Output is delivered but flagged as ungoverned. Asynchronous evaluation resumes when ROC recovers.
- **Low:** fail-open. Output is delivered. The missed evaluation is logged.

ROC's fail-closed posture has a different cost profile than EPG's: EPG blocks a prompt before model invocation (no compute wasted). ROC blocks an output after model invocation (compute is consumed, output is generated but not delivered). This means ROC failures are more expensive operationally — the model has already done the work. This motivates aggressive HA/DR investment for ROC in Critical workloads.

13.8 Operational Guidance

13.8.1 Phased Adoption

Phase 1 (3-6 months): Deploy deterministic output controls (O_d) for the highest-risk constraints: PII pattern detection, blocklists, required disclaimer checks. These are low-latency, low-false-positive, and immediately valuable. Deploy for Critical workloads first.

Phase 2 (6-12 months): Add classifier-based controls (O_c) for key threat areas: PII/PHI NER, medical advice detection, financial advice detection. Tune thresholds against production data. Establish classifier governance (versioning, validation cadence, threshold ownership).

Phase 3 (12-18 months): Deploy composite constraints (O_x) combining deterministic and classifier outputs. Extend coverage to Standard workloads. Implement full fallback pipeline (safe response, redaction, retry, escalation).

Phase 4 (18+ months): Deploy threat-specific advanced controls (injection detection, jailbreak detection, memorization detection). Continuous classifier improvement based on production findings. Cross-component optimization with EPG and MDR.

13.8.2 Classifier Governance

Classifiers are not static tools. They are governed artifacts requiring lifecycle management:

- **Versioning:** every classifier has a version identifier recorded in audit trails (Axiom 5)
- **Validation cadence:** classifiers are periodically retested against updated benchmark datasets. Critical classifiers (PII, medical advice): quarterly. Standard classifiers: semi-annually.
- **Threshold management:** thresholds are governed artifacts with dual-control modification. Threshold changes follow the same authorship process as EPG constraint changes.
- **Drift detection:** classifier performance is monitored against baseline metrics. If precision or recall degrades beyond defined thresholds, the classifier is flagged for retraining or replacement.

- **Red-team testing:** classifiers are regularly tested with adversarial inputs designed to bypass detection. Results inform retraining priorities.

13.8.3 Risk-Tiered Application

Property	Critical	Standard	Low
Timing	Atomic buffered, synchronous	Post-hoc audit monitoring	Sampled post-hoc audit
Streaming	Disabled (fully buffered)	Permitted	Permitted
Deterministic checks	All applied, short-circuit on fail	All applied	All applied
Classifier checks	All applied, synchronous	All applied, async	Sampled (min floor enforced)
Fail posture	Fail-closed (block on timeout)	Fail-open-flagged	Fail-open
Fallback on block	Safe response + redaction (with re-eval)	N/A (detection only)	N/A
Audit evidence	Deterministic: hard. Classifier: scored ($\models \tau$)	Same	Sampled
PHI handling	PHI-first policy: block/redact/human-review	Flag + alert	Flag
Control type	PREVENTATIVE	DETECTIVE (post-hoc)	DETECTIVE (sampled)

Critical distinction: Only Critical tier provides true preventative controls on S_{delivery} . Standard and Low tiers provide detection and audit evidence — valuable for incident response and trend analysis, but the output has already been delivered before violations are detected. This distinction must be communicated clearly to compliance teams and regulators.

13.8.4 Latency Budget

Real-time blocking evaluation adds latency to every Critical-tier interaction. The latency budget must be explicitly managed:

- Deterministic evaluation: target < 50ms (regex, blocklist, structural checks)
- Classifier evaluation: target < 500ms per classifier (depends on model complexity)
- Total pipeline: target < 2 seconds for Critical-tier full evaluation
- **Timeout behavior for Critical tier is non-negotiable:** if classifiers exceed the latency budget, the timeout is treated as a failure. ROC defaults to block + safe response. Delivering an unevaluated output on timeout is never permitted for Critical workloads. This is the only defensible posture under HIPAA, NERC CIP, and SOX.
- For Standard/Low tiers: timeout simply means the async evaluation was delayed; the output was already delivered.

13.8.5 Resource Planning

Classifier governance at scale requires dedicated investment. Organizations should plan for:

- **Data engineering:** maintaining benchmark datasets and regression test suites for each classifier. Expect 0.5-1 FTE per critical threat domain (PHI, financial advice, safety).
- **MLOps:** classifier retraining, deployment, and drift monitoring infrastructure. Shared across classifiers but non-trivial.
- **Security data science:** red-team testing and adversarial evaluation. Expect 0.25-0.5 FTE per classifier for quarterly testing cadence.
- **Centralization recommendation:** organizations should centralize classifier governance rather than having each business unit maintain independent classifier stacks. A central AI Safety/Governance team owns classifier lifecycle; business units consume classifiers as governed services.
- **Scope management:** if an organization cannot sustain the validation cadence for all classifiers, it must scope ROC to fewer, truly critical threats rather than deploy poorly-maintained classifiers that provide false confidence.

13.9 Limitations

ROC governs the output surface. This section states explicitly what ROC does and does not guarantee.

What ROC guarantees (formal commitments):

- Evaluation completeness: every governed output in Critical and Standard tiers is evaluated against every applicable output constraint, with a tamper-evident, version-stamped audit record. Low-tier sampled interactions have evaluation records only for sampled outputs.
- Deterministic evaluation reliability: deterministic output constraints produce stable, repeatable results
- Audit chain participation: ROC's audit records compose with EPG and MDR records via shared interaction identifier

What ROC does not guarantee:

- Output correctness: ROC checks for constraint violations, not factual accuracy. An output that passes all ROC evaluations may still be incorrect, misleading, or unhelpful.
- Classifier infallibility: classifier-based evaluations are probabilistic. False negatives (missed violations) are possible at any threshold. The meta-framework's Theorem 4 (Irreducible Residual Risk) applies.
- Complete threat coverage: new attack techniques (novel jailbreaks, novel injection patterns) may evade current classifiers until retraining occurs.
- Model behavior modification: ROC filters outputs; it does not change how the model generates them. A model that consistently produces non-compliant outputs requires prompt governance (EPG) or model-level intervention, not just output filtering.

What ROC provides as governance-process tools (operationally valuable, not formally proven):

- Classifier-based detection: probabilistic scoring that catches most violations but cannot guarantee zero false negatives
- Injection and jailbreak detection: pattern-based and classifier-based detection that covers known techniques but is subject to evasion by novel methods
- Cross-tenant leakage detection: limited to detectable patterns in output content; does not address model-level contamination

The formal model appendix uses the same two-tier structure throughout: formal guarantees for deterministic evaluations, scored evidence for classifier-based evaluations. This distinction is not a weakness to be fixed — it is an honest representation of what is and is not achievable in output governance.

13.10 Key takeaways

- A governed prompt does not guarantee a governed output. Because models are stochastic, residual risk on the output surface is irreducible (Chapter 11, Theorem 4); ROC exists to detect, contain, and trace it.
- ROC evaluates outputs in two tiers with honestly different evidence: deterministic checks are hard evidence, classifier scores are scored evidence. Neither is promoted to the other's standard (§2, §3).
- Timing is a safety property. Critical workloads get real-time blocking; standard and low-risk workloads get asynchronous evaluation, trading depth for latency so governance does not push teams toward ungoverned tools (§5).
- ROC picks up the output-side threats EPG defers — injection, jailbreaking, PII/PHI leakage, cross-tenant isolation (§6) — and, as `g_ROC` with `R_hard = ∅`, is independently deployable while composing with EPG and MDR (§7).

Prompt governance and output controls govern the two sides of a single interaction. What neither can see is the system over time: whether the controls are running at all, whether their evaluations are drifting, whether an interaction slipped past them. That is monitoring's subject. ROC's formal model, worked examples, and classifier specifications are collected in the formal-model, worked-examples and templates appendices.

14 Seeing the Whole Picture (MDR)

EPG and ROC governed the two sides of a single interaction: the prompt going in, the output coming out. But some risks are invisible at the level of any one interaction — a slow drift in constraint quality, an adversary probing across many sessions, a governance component silently degrading, an interaction that bypassed governance entirely. This chapter watches the whole system over time through Monitoring, Detection & Response (MDR), the component `g_MDR`, which governs the remaining surfaces — `S_input` and `S_config` — and correlates the evidence the other two components produce.

By the end of the chapter you will be able to:

- position MDR as a detective, cross-cutting layer rather than a fourth preventative control, and state what it can and cannot promise;
- monitor the input and configuration surfaces and the health of the audit chain, including detection of governance bypass (“ghost” interactions);
- classify incidents and drive the response workflow — contain, investigate, remediate, report — that MDR orchestrates;
- describe the audit and integration contracts through which MDR consumes EPG’s and ROC’s records.

14.1 Purpose and Scope

EPG governs what goes into the model (prompts). ROC governs what comes out (outputs). MDR governs the operational health of the entire governance system and detects threats that neither EPG nor ROC can catch in isolation.

MDR’s three functions:

Monitor — continuously observe AI interactions, governance component health, constraint drift, and cross-interaction patterns.

Detect — identify anomalies, policy violations, adversarial probing, governance degradation, and emerging threat patterns that span multiple interactions or components.

Respond — orchestrate incident response: alert, escalate, contain, remediate, and report.

MDR is the only component that operates across all control surfaces. EPG sees prompts. ROC sees outputs. MDR sees the full interaction pipeline — prompt, input, config, output, delivery — through the audit records produced by EPG and ROC, plus its own direct monitoring of `S_input` and `S_config`.

14.1.1 What MDR Is Not

MDR is not a replacement for EPG or ROC. It does not evaluate prompts against constraints (EPG does that). It does not filter outputs (ROC does that). MDR operates at the meta-level: it monitors whether EPG and ROC are functioning correctly, detects patterns that individual interactions don’t reveal, and coordinates response when things go wrong.

MDR is primarily a **detective** framework, not a **preventative** one. Its containment capabilities (§5.1) can stop ongoing incidents, but it cannot prevent the first occurrence of a violation

— that is EPG’s and ROC’s role. This distinction matters for regulatory positioning: MDR provides detection and response evidence, not prevention evidence.

14.1.2 Architectural Assumptions

The following assumptions must hold for MDR to deliver its promised capabilities. If any assumption is violated, the corresponding detection claims are degraded.

A1 (Instrumented gateway). All AI model invocations traverse an instrumented API gateway or network proxy that produces interaction records independently of EPG, ROC, and the GIL. This gateway operates outside the trust domain of the governance components. Without this, ghost detection (§3.1) cannot detect interactions that bypass the entire governance stack.

A2 (Governed CI/CD for configuration). All governance-relevant configuration changes (model selection, inference parameters, constraint modifications, threshold changes) flow through a governed CI/CD path that produces immutable change records. Without this, configuration drift detection (§2.2) cannot detect out-of-band changes.

A3 (Identity binding). Identity binding between interactions, users, and tenants is enforced by EA1-EA4 from the meta-framework and supplemented by Identity Provider (IdP) integration. Detections are enriched with employee metadata (role, department, location) before reaching the SIEM. Without this, cross-interaction adversarial detection (§3.3) produces unactionable alerts.

A4 (EPG/ROC audit feeds for full coverage). MDR has a minimal independent function (input monitoring, config validation, infrastructure-level invocation logging). Full threat coverage for governance-related incidents — constraint drift, classifier degradation, decomposition aging, cross-component correlation — is contingent on receiving structured audit records from EPG and ROC. Without these feeds, MDR degenerates to generic monitoring with significantly reduced detection capability.

14.2 Control Surfaces

14.2.1 S_input: User Input Monitoring

S_input has governability $\gamma = \text{partial}$. MDR can observe and validate input form (length, format, encoding, injection patterns) but cannot fully constrain input content — users can type anything.

Input monitoring capabilities:

- **Injection pattern detection:** scan user inputs for known prompt injection patterns before they reach the prompt assembly process. This is a defense-in-depth layer — EPG constrains the prompt, ROC filters the output, and MDR catches injection attempts at the input boundary.
- **Rate and volume monitoring:** detect anomalous input patterns — sudden volume spikes, repetitive probing inputs, systematic boundary-testing sequences — that suggest adversarial behavior.
- **Input classification:** categorize inputs by risk level (routine query, edge case, potentially adversarial) to inform downstream processing. High-risk inputs may trigger enhanced ROC evaluation even for Standard-tier workloads.

- **Session-level analysis:** detect multi-turn attack patterns where individual inputs are benign but the sequence is adversarial (progressive jailbreak attempts, incremental data extraction).

14.2.2 S_config: Model and Configuration Governance

S_config has mixed governability: model selection and inference parameters (θ) are fully governable; model internals are external.

Configuration monitoring capabilities:

- **Model selection policy enforcement:** verify that each interaction uses a model approved for its risk tier and use case. A PHI-handling workload must use a model approved for healthcare; a financial advisory tool must use a model approved for financial services.
- **Inference parameter validation:** verify that θ (temperature, top-p, sampling parameters) falls within governed bounds. A safety-critical workload with temperature > 0.3 may violate configuration governed policy.
- **Model version tracking:** detect when model endpoints are updated by providers (Axiom 5 — temporal identity). If the model behind an endpoint changes, MDR flags all workloads using that endpoint for re-evaluation.
- **Configuration drift detection:** compare current deployment configurations against governed baselines. Unauthorized changes to model selection, parameters, or routing are detected and alerted.

14.3 Cross-Component Monitoring

MDR's highest-value function is cross-component pattern detection — finding issues that no single interaction reveals.

14.3.1 Audit Chain Health

MDR continuously monitors the health of the audit chain using two independent data sources:

- **Infrastructure-level invocation log (ground truth).** Per assumption A1, an API gateway or network proxy outside the governance trust domain produces independent records of all model invocations. This is MDR's ground truth — it cannot be suppressed by compromising EPG, ROC, or the GIL. MDR compares this infra log against GIL registrations and audit chain records. Discrepancies indicate either a governance bypass (interaction hit the model but never entered the governance pipeline) or a GIL failure.
- **GIL-to-chain comparison.** GIL entries without corresponding EPG + ROC audit records are ghosts — either component failures or partial bypasses that entered the GIL but were not processed by all components.
- **Chain integrity:** periodically verify hash chains and signatures across EPG and ROC audit records (AI2-AI3). Tampering detection.
- **Component health:** monitor EPG and ROC availability and response times. Degraded components are flagged before they fail entirely.

- **Degraded record ratio:** track the ratio of process records (supervisor-signed) to evaluation records (component-signed). Rising degraded ratios indicate component reliability issues.

Without assumption A1 (independent infra log), ghost detection can only identify bypasses that entered the GIL but were not fully processed. Interactions that bypass the entire governance stack — including GIL — are invisible. This limitation must be explicitly communicated to regulators.

14.3.2 Constraint Drift and Governance Decay

- **Constraint modification velocity:** track the rate of constraint changes. Sudden spikes (many constraints modified in a short window) suggest unauthorized bulk changes or compromise.
- **Threshold drift:** monitor ROC classifier thresholds over time. Gradual threshold increases (loosening) without corresponding classifier improvements is a governance decay signal.
- **Evaluation failure rates:** track the percentage of prompts/outputs that fail constraint evaluation. Rising failure rates may indicate model drift, prompt quality degradation, or changing user behavior.
- **Decomposition aging:** flag EPG constraint decompositions that haven't been re-validated within their required cadence (Critical: annually, Standard: per cycle).

14.3.3 Adversarial Pattern Detection

Individual interaction monitoring catches overt attacks. Cross-interaction analysis detects patterns across many interactions, but with important scope limitations:

Coarse-grained detection (reliable at scale):

- **Aggregate statistical shifts:** per-tenant, per-model, or per-use-case shifts in ROC block rates, flag rates, and classifier score distributions. These are computationally feasible at enterprise scale and detect systemic changes (model drift, configuration errors, broad adversarial campaigns).
- **Output pattern anomalies:** stable baselines for each workload, with alerts on statistically significant deviations.

Fine-grained detection (best-effort, depends on identity and feature engineering):

- **Progressive probing:** sequences of inputs that systematically test constraint boundaries. Requires session-level identity binding (assumption A3) and is effective only when sessions map reliably to individuals.
- **Constraint boundary mapping:** users whose inputs consistently score just below classifier thresholds ($\tau - \epsilon$). Requires per-user tracking and produces meaningful signals only with strong identity resolution.

- **Distributed attacks:** adversarial inputs spread across sessions/users. Detection depends on identity resolution across devices/VPNs, which MDR does not solve — it requires enterprise IdP integration per assumption A3.

Honest limitation: Fine-grained user-level attack pattern detection is best-effort. It depends on identity binding quality, feature engineering investment, and analyst capacity that this architecture does not provide. Sophisticated adversaries who vary patterns and use multiple identities may evade fine-grained detection. Coarse-grained aggregate monitoring is the reliable baseline.

14.4 Incident Detection and Classification

14.4.1 Alert Taxonomy

MDR generates three categories of alerts:

Governance alerts — issues with the governance system itself:

- Component unavailability (EPG, ROC, GIL, Supervisor down)
- Ghost chain detected (GIL entry without audit chain)
- Audit integrity failure (hash chain or signature verification failed)
- Unauthorized constraint modification detected
- Configuration drift from governed baseline

Compliance alerts — potential regulatory violations:

- PHI detected in output (from ROC) with confirmed delivery to user
- Financial advice generated for unauthorized audience
- Safety-critical constraint violated in production
- Constraint decomposition overdue for re-validation
- Residual gap compensating control not executing (HITL rate below required minimum)

Adversarial alerts — potential attacks or abuse:

- Prompt injection pattern detected in input
- Jailbreak probing sequence detected across sessions
- Classifier threshold boundary probing detected
- Anomalous interaction volume from single user/tenant
- Cross-tenant leakage signal (from ROC monitoring feed)

14.4.2 Alert Severity and Escalation

Severity	Definition	Response SLA	Escalation Path
P1 — Critical	Active PHI/PII exposure, governance system compromise, safety violation in production	15 minutes	SOC → CISO → Incident Commander
P2 — High	Confirmed constraint violation, component failure, adversarial campaign detected	1 hour	SOC → AI Governance team
P3 — Medium	Anomalous patterns, drift detected, threshold probing	4 hours	AI Governance team
P4 — Low	Informational: decomposition aging, minor configuration drift	Next business day	AI Operations

14.5 Incident Response Workflows

14.5.1 Containment

When a compliance or adversarial alert fires, MDR’s first action is containment. Containment operates through two paths to avoid circular dependency:

Primary: Out-of-band infrastructure containment. Direct integration with cloud provider IAM, service mesh, or API gateway to sever model connectivity without relying on EPG or ROC. This path is used when the governance stack itself may be compromised or malfunctioning:

- **Model endpoint disablement:** revoke model API credentials or update service mesh routing to block traffic to the affected endpoint. This operates at infrastructure level, below the governance stack.
- **Workload isolation:** network-level isolation of the affected project or tenant’s AI traffic.
- **User-level blocking:** rate-limit or block specific users via the API gateway or identity provider.

Secondary: Governance-level containment. Used when EPG and ROC are trusted (the incident is in the AI output or user behavior, not in the governance system):

- **Constraint-level containment:** MDR triggers EPG break-glass to add emergency constraints (tightening only, TTL-bound per Chapter 12, §4.5).
- **Threshold override:** MDR triggers ROC to lower classifier thresholds temporarily (tightening, increasing sensitivity).

MDR privilege model. MDR’s service account requires least-privilege access scoped to containment actions: API gateway route modification, IAM credential revocation for model endpoints, EPG break-glass trigger, ROC threshold override trigger. MDR does NOT have write access to constraints, classifier models, or audit records. This limits the blast radius if MDR itself is compromised. All MDR containment actions are logged in an independent audit trail (not MDR’s own audit trail — a separate infra-level log).

14.5.2 Investigation

MDR provides the data for root cause analysis:

- **Full audit chain retrieval:** for any flagged interaction, retrieve the complete chain: EPG prompt evaluation → ROC output evaluation → MDR monitoring signals. Every constraint version, evaluation result, classifier score, and governance decision is available.
- **Cross-interaction correlation:** link the flagged interaction to related interactions — same user, same prompt template, same model, same time window — to determine if the issue is isolated or systemic.
- **Timeline reconstruction:** using Axiom 5 (temporal identity), reconstruct the exact state of all governed elements at the time of the incident: which constraints were active, which classifier versions were running, what thresholds were configured.

14.5.3 Remediation

Based on root cause analysis:

- **Prompt deficiency:** EPG constraint gap identified → add or modify constraints through standard authorship (Chapter 12, §4)
- **Classifier gap:** ROC classifier failed to detect the violation → retrain classifier, adjust threshold, add test case to regression suite
- **Configuration error:** wrong model or parameters for the workload → correct configuration, add configuration policy to prevent recurrence
- **Adversarial technique:** new attack pattern identified → add detection pattern to MDR, add mechanical check to ROC, update red-team corpus
- **Model behavior drift:** model producing different output patterns than baseline → flag for model re-evaluation or replacement
- **External provider incident:** model provider reports a security issue or MDR detects anomalies attributable to provider-side changes. Escalate to vendor with SLA requirements. Apply stricter monitoring and containment on affected endpoints until vendor resolution is confirmed. MDR cannot remediate provider-side issues directly.
- **Bulk-job incident:** a batch processing job (document analysis, report generation) has already processed thousands of records before a violation is detected. Containment: cancel the running job immediately. Impact analysis: identify all records processed since the last known-good evaluation. Notify affected data subjects per regulatory requirements. Bulk incidents require a distinct response workflow because containment cannot un-process already-completed records.

14.5.4 Reporting

Regulated industries require formal incident reporting:

- **Internal reporting:** incident summary, root cause, containment actions, remediation timeline. Feeds the cross-domain governance body and CISO office.

- **Regulatory reporting:** for PHI breaches (HIPAA 60-day notification), financial violations (SEC/FINRA), or safety incidents (NERC CIP). MDR produces the evidence; legal/compliance produce the report. Note: MDR only supplies evidence. Regulatory readiness requires mapped reporting playbooks per regime (HIPAA, SOX, GDPR, NERC CIP) with different timelines, evidence standards, and notification requirements. Multi-jurisdictional incidents (GDPR + HIPAA + SOX) require parallel reporting with potentially conflicting requirements — legal counsel drives these decisions, not MDR.
- **Trend reporting:** monthly/quarterly governance health reports: component uptime, constraint coverage, violation rates, classifier performance, adversarial activity trends.

Forensic Evidence Locker. Upon a P1 or P2 alert, MDR automatically triggers a forensic collection workflow that “freezes” the relevant governance state into a separate, immutable evidence store:

- Constraint versions active at the time of incident (from EPG audit records)
- Classifier versions, thresholds, and calibration data active at the time (from ROC audit records)
- The full audit chain for the flagged interaction(s)
- Infrastructure logs from the independent API gateway feed
- MDR’s own detection signals and alert chain

This evidence store is write-once, tamper-evident (WORM storage, per meta-framework AI1), and separate from operational audit storage. It provides the chain-of-custody evidence that regulators require: a snapshot of the exact governance state at the time of the incident, preserved independently of ongoing system operation.

14.6 Integration Architecture

14.6.1 Data Flows

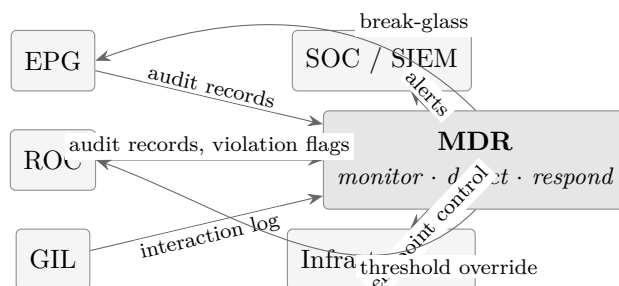


Figure 14.1: MDR data flows. EPG, ROC and the Governance Interaction Log supply evidence; MDR emits alerts and endpoint controls, and closes the loop by acting back on the layers it observes.

14.6.2 SIEM/SOAR Integration

MDR's alerts integrate with existing Security Information and Event Management (SIEM) and Security Orchestration, Automation, and Response (SOAR) platforms:

- **SIEM ingestion:** MDR pushes structured alert data (JSON) to the enterprise SIEM. AI governance alerts appear alongside traditional security events, enabling correlation (e.g., AI jailbreak attempt from the same IP as a failed VPN login).
- **SOAR playbooks:** MDR triggers SOAR playbooks for automated response — rate limiting, user lockout, model endpoint rotation. Pre-built playbooks for common scenarios (PHI exposure, jailbreak campaign, component failure).
- **Dashboard integration:** MDR feeds governance dashboards showing real-time component health, constraint evaluation rates, classifier performance, and incident status.

14.6.3 Meta-Framework Compliance

MDR instantiates `g_MDR` from the meta-framework:

- **Surfaces:** $S = \{S_input, S_config\}$
- **Monitoring scope:** extends to all surfaces via audit record consumption
- **R_hard:** $\emptyset$ — independently deployable
- **R_soft:** audit records from EPG and ROC (enhanced effectiveness, not required for guarantees)
- **Failure posture:** MDR itself is fail-open — if MDR goes down, EPG and ROC continue to function (Axiom 3 guarantee independence). MDR's detection guarantees (ghost detection, drift monitoring, adversarial pattern detection) apply ONLY when MDR is operational. During MDR outage, these capabilities are suspended and the outage period is logged as an “ungoverned monitoring window” in the infrastructure audit trail. EPG and ROC guarantees are unaffected by MDR status.
- **Minimal vs full function:** MDR has a minimal independent function (input monitoring via assumption A1, config validation via assumption A2) that operates without EPG/ROC audit feeds. Full detection capability — constraint drift, classifier degradation, decomposition aging, cross-component correlation — requires EPG and ROC audit feeds (assumption A4). MDR is independently deployable per $R_hard = \emptyset$, but its advertised coverage for governance-related incidents is contingent on those feeds.

14.7 Operational Requirements

14.7.1 Staffing

MDR operations require a three-tier staffing model:

- **Tier 1: SOC analysts (AI-trained):** 2-3 analysts with AI governance training for initial alert triage. These are existing SOC staff with additional training — not dedicated

headcount. They handle P3/P4 alerts and perform initial triage on P1/P2 before escalation. A dedicated AI Governance Runbook (knowledge base) translates ML-specific alerts (classifier threshold probing, score distribution shifts) into actionable security steps that non-ML-expert analysts can execute.

- **Tier 2: AI governance engineers:** 1-2 engineers maintaining MDR detection rules, tuning alert thresholds, managing SIEM/SOAR integration, and handling P3 investigations.
- **Tier 3: AI security specialist (on-call):** At least one ML-trained security specialist available for P1/P2 escalation. This role requires understanding of classifier behavior, adversarial ML techniques, and model internals. Cannot be replaced by Tier 1 training alone — alerts like “classifier threshold boundary probing” or “cross-tenant score distribution anomaly” require ML engineering judgment for accurate disposition.
- **Incident response capability:** on-call rotation for P1 alerts (15-minute SLA). Tier 3 specialist must be reachable within the P1 SLA window. Can share rotation with existing security incident response if the specialist is cross-trained.

Runbook requirement. Before MDR goes operational, a comprehensive AI Governance Runbook must be developed covering: alert-to-action mappings for each alert type in §4.1, escalation decision trees, containment action authorization (who approves model endpoint disablement), evidence preservation checklists for P1/P2, and regulatory notification triggers per regime.

14.7.2 Infrastructure

- **Audit data storage:** MDR consumes audit records from EPG, ROC, and GIL. Storage requirements scale with interaction volume. Retention per regulatory requirements (AI5 from meta-framework).
- **Real-time processing:** input monitoring (§2.1) and cross-component health checks (§3.1) require real-time stream processing capability.
- **Batch analytics:** cross-interaction pattern detection (§3.3) and trend reporting (§5.4) run as batch analytics on accumulated data.
- **Alert infrastructure:** integration with enterprise alerting (PagerDuty, ServiceNow, or equivalent) for P1/P2 escalation.

14.7.3 Phased Deployment

Phase 1 (align with EPG Phase 1): Deploy audit chain health monitoring and GIL ghost detection. SIEM integration for governance alerts. This provides immediate value: “is the governance system working?”

Phase 2 (align with ROC Phase 1): Add ROC audit consumption. Deploy compliance alert generation. Input monitoring for injection patterns. Configuration drift detection.

Phase 3 (align with EPG/ROC Phase 2+): Add cross-interaction pattern detection. Adversarial behavior analysis. Automated containment playbooks via SOAR. Full trend reporting.

14.8 Limitations

MDR is primarily a **detective and responsive** framework, not a **preventative** one. This distinction is critical for regulatory positioning.

- **S_input:** MDR can detect injection patterns but cannot prevent users from typing adversarial inputs. Prevention is achieved through EPG (prompt hardening) and ROC (output filtering). MDR’s input monitoring is defense-in-depth, not primary defense.
- **S_config:** MDR enforces configuration policies but cannot prevent a provider from updating a model behind an endpoint. It can only detect the change after the fact.
- **Cross-interaction detection:** coarse-grained aggregate monitoring (per-tenant, per-model statistical shifts) is reliable at scale. Fine-grained user-level adversarial detection is best-effort and depends on identity resolution, feature engineering, and analyst capacity not specified in this architecture. Sophisticated adversaries who vary patterns across identities may evade fine-grained detection.
- **Dependency on audit data (assumption A4):** MDR’s full detection capability depends on receiving complete audit records from EPG and ROC. Without these feeds, MDR provides only minimal independent monitoring (input validation, config checks, infra-level invocation logging). Governance-specific detection (constraint drift, classifier degradation, decomposition aging) requires the feeds.
- **Dependency on independent infra feed (assumption A1):** ghost detection relies on an API gateway or network proxy log outside the governance trust domain. Without this, interactions that bypass the entire governance stack are invisible to MDR.
- **Standard-tier detection is not prevention:** for Standard-tier workloads where ROC operates in post-hoc audit mode, MDR’s detection of violations occurs after the output has been delivered. This is notification-of-incident, not prevention. For sensitive-but-non-Critical workloads, organizations should consider rate-limiting output delivery to match MDR processing lag, ensuring detection occurs within a manageable exposure window.
- **Human-in-the-loop:** P1 and P2 incident response requires human judgment, including a Tier 3 AI security specialist for ML-specific alerts. MDR automates detection and initial infrastructure-level containment but cannot fully automate remediation for complex incidents.
- **MDR compromise:** if MDR itself is compromised, an attacker can suppress alerts and disable detection. Compensating controls: MDR containment actions are logged in an independent infra-level audit trail (not MDR’s own); MDR health is monitored by the enterprise SIEM independently of MDR’s own reporting.

MDR is the “nervous system” of the governance solution — it senses, alerts, and initiates response. It depends on EPG and ROC as the “muscles” that enforce governance at the prompt and output levels. Without the muscles, the nervous system can detect but not prevent. Without the nervous system, the muscles operate blind to cross-interaction patterns and system-level degradation.

14.9 Key takeaways

- MDR is a detective, cross-cutting layer, not a fourth preventative control. It cannot stop a first occurrence — that is EPG's and ROC's role — but it detects, contains, investigates, and reports what the other layers cannot see in isolation (§1).
- It governs the remaining surfaces, S_input and S_config, and monitors the health of the audit chain itself — including ghost detection, which surfaces interactions that bypassed governance entirely (§2–§3).
- Its value is cross-interaction: drift, adversarial probing across sessions, and component degradation are visible only in aggregate (§3–§4). Response is an orchestrated workflow — contain, investigate, remediate, report (§5).
- MDR consumes EPG's and ROC's audit records through defined contracts; without those feeds it degrades to generic monitoring (§6).

This completes Part IV: the three components now govern every control surface of an interaction, and the system that runs them. How those controls line up against the regulatory frameworks an enterprise must answer to is set out in the regulatory-mapping appendix.

A Worked Examples

End-to-end examples for the control layers of Part IV.

A.1 Prompt Governance (EPG)

A.1.1 Healthcare: Patient-Facing Chatbot

Enterprise constraints:

- $O(\text{hipaa_disclaimer})$ — every prompt must include HIPAA-compliant disclaimer language
- $F(\text{pii_in_logs})$ — prompts must never include instructions that would cause PII to appear in system logs
- $F(\text{medical_diagnosis})$ — prompts must not instruct the model to diagnose conditions
- $\setminus\{\}Permitted(\text{clinical_terminology})$ — enterprise has considered clinical terminology and imposes no restriction

Department constraints (Patient Services):

- Inherits $O(\text{hipaa_disclaimer})$ and $F(\text{pii_in_logs})$ — cannot change
- Adds $F(\text{clinical_terminology})$ — patients are not clinicians; layman terms required. This is a valid tightening of P_meta (which imposes no restriction). Flagged for tension review because P_meta existed at enterprise level; review confirms department rationale is sound.
- Adds $O(\text{empathetic_tone})$ — patient-facing communications require empathetic framing
- Adds $O(\text{redirect_to_provider})$ — prompts must instruct the model to redirect medical questions to qualified providers

Project constraints (Patient FAQ Chatbot):

- Inherits all enterprise and department constraints (5 total)
- Adds $O(\text{scope_to_faq_topics})$ — prompt must limit the model to answering questions within the FAQ knowledge base
- Adds $O(\text{cite_sources})$ — prompt must instruct the model to cite specific FAQ entries

Effective constraint set at project level: $C^*(\text{project}) = \{O(\text{hipaa_disclaimer}), F(\text{pii_in_logs}), F(\text{medical_diagnosis}), F(\text{clinical_terminology}), O(\text{empathetic_tone}), O(\text{redirect_to_provider}), O(\text{scope_to_faq_topics}), O(\text{cite_sources})\}$

Decomposition example: $F(\text{medical_diagnosis})$

Semantic intent: “The AI system must not produce output that could be construed as diagnosing a medical condition.”

Mechanical components (C_m):

- Pattern scan for diagnostic language: regex and NER-based detection of diagnostic phrases (“you have,” “this indicates,” “the symptoms suggest,” “diagnosis:”)
- Required disclaimer presence: prompt must include explicit instruction “Do not diagnose conditions or suggest specific diagnoses”
- Scope limitation: prompt must include instruction to refuse requests for diagnosis

Procedural components (C_p):

- Medical affairs review: a qualified medical affairs officer reviews the prompt and attests that its framing adequately prevents diagnostic behavior
- Adversarial testing: red-team testing with prompts designed to elicit diagnostic language; results reviewed and attested by medical affairs

Residual gap: mechanical checks cover an estimated 85% of diagnostic language patterns based on regression testing against a corpus of known diagnostic phrases. The remaining 15% includes novel or indirect diagnostic formulations. Compensating control: 15% HITL review rate on chatbot outputs, managed by medical affairs, with monthly reporting.

Conflict example: O(empathetic_tone) vs F(promissory_language)

The department’s obligation for empathetic tone (authored by Patient Experience domain) creates tension with Legal’s prohibition on promissory language. Empathetic language like “we will make sure you get the care you need” could be construed as a promissory commitment.

Phase 1 resolution: cross-domain review between Patient Experience and Legal. Resolution: add a precedence rule to the Enterprise Precedence Table — F(promissory_language) takes precedence over O(empathetic_tone). The project team must find empathetic language that does not make promissory commitments. Documented rationale: legal liability outweighs tone preference.

A.1.2 Financial Services: Investment Research Assistant

Enterprise constraints:

- O(sox_audit_trail) — all AI-generated content used in financial reporting must have complete audit trail
- F(investment_advice_to_retail) — prompts must not instruct the model to provide personalized investment advice to retail investors
- O(data_source_citation) — prompts must instruct the model to cite data sources for all quantitative claims

Department constraints (Institutional Research):

- Inherits all enterprise constraints
- $\{\}$ Permitted(investment_thesis_language) — the department has considered investment thesis language and does not restrict it for professional audiences
- O(model_uncertainty_disclosure) — prompts must instruct the model to disclose uncertainty and confidence levels

Project constraints (Analyst Tool):

- Inherits all enterprise and department constraints
- `F(forward_looking_statements_without_disclaimer)` — prompts must prohibit forward-looking statements unless accompanied by required regulatory disclaimers

Decomposition example: `F(investment_advice_to_retail)`

Semantic intent: “The AI system must not provide personalized investment recommendations to retail investors.”

Mechanical components: audience detection (prompt must include explicit audience scoping: “this tool is for institutional investors only”), keyword patterns for personalized advice language (“you should buy,” “I recommend,” “based on your portfolio”), required disclaimer presence.

Procedural components: compliance officer review, quarterly re-attestation, FINRA suitability framework review.

Residual gap: mechanical checks cannot distinguish between general market commentary (permitted) and implicit investment guidance that sophisticated readers would recognize as advice but retail investors might act on differently. Compensating control: 25% HITL review by compliance, with escalation protocol for borderline cases.

A.1.3 Energy: Grid Operations Decision Support**Enterprise constraints:**

- `O(nerc_cip_compliance_notice)` — all AI interactions involving grid operations must include NERC CIP compliance framing
- `F(operational_commands_without_human_confirmation)` — prompts must never instruct the model to issue operational commands; all recommendations require human confirmation
- `O(audit_all_recommendations)` — every AI recommendation must be fully auditable with complete constraint evaluation

Department constraints (Grid Operations):

- `O(cite_sensor_data_source)` — prompts must instruct the model to cite specific sensor data sources for all recommendations
- `F(recommendations_outside_trained_scenarios)` — prompts must restrict the model to scenarios covered by validated training data

Conflict example: efficiency vs safety

An efficiency optimization constraint `O(minimize_response_time)` from the Operations Efficiency domain creates tension with `F(operational_commands_without_human_confirmation)` from the Safety domain. Faster response times could pressure the system toward fewer confirmation steps.

Resolution: Enterprise Precedence Table specifies Safety > Efficiency as the default domain priority. The safety prohibition takes absolute precedence. The efficiency constraint is modified to: `O(minimize_response_time_within_safety_constraints)` — a new atomic property that explicitly acknowledges the safety boundary.

Break-glass scenario: During an active grid stability event, an enterprise-level actor invokes break-glass to add `F(non_essential_ai_queries)` — temporarily prohibiting all non-essential AI interactions to preserve computational resources for critical grid operations. TTL: 24 hours (energy sector default). The constraint is tightening-only (adds a prohibition). Immediate CONTRADICTION scan confirms no conflicts with existing constraints. Post-hoc review within 24 hours formalizes the constraint through standard dual-control if the operational condition persists, or allows automatic reversion if the event resolves.

A.2 Runtime Output Controls (ROC)

A.2.1 Healthcare: Patient-Facing Chatbot (continued)

The EPG example above established constraints for a Patient FAQ Chatbot. ROC extends governance to the output:

Output constraints applied:

Constraint	Type	Tier	Action on Violation
SSN/MRN pattern detection	O_d	Deterministic	Block + redact
PHI NER classifier ($\tau = 0.75$)	O_c	Classifier	Block (Critical)
Medical advice classifier ($\tau = 0.80$)	O_c	Classifier	Block (Critical)
Disclaimer presence check	O_d	Deterministic	Block + retry with reinforced prompt
Composite PHI check (regex + NER)	O_x	Composite (any_flag)	Block + redact

Scenario: Normal operation. Patient asks “What are your visiting hours?” Model responds with visiting hours and a HIPAA disclaimer. ROC pipeline: deterministic checks pass (no PII patterns, disclaimer present), classifiers score below thresholds (no medical advice, no PHI). Decision: PASS. Output delivered. Full audit record produced.

Scenario: Jailbreak attempt. Patient crafts input designed to elicit medical diagnosis. Model partially complies despite EPG’s `F(medical_diagnosis)` prompt constraint. ROC pipeline: medical advice classifier scores 0.91 (above 0.80 threshold). Decision: BLOCK. Fallback: safe response substitution (“I’m not able to provide medical advice. Please contact your health-care provider.”). Audit record documents: classifier score, threshold, block decision, fallback delivered.

Scenario: Incidental PII. Model response references “your appointment with Dr. Smith on March 15” — incidental PHI from conversation context. ROC pipeline: PHI NER classifier scores 0.82 (above 0.75). Composite PHI check (O_x, any_flag) triggers. Decision: BLOCK with redaction. Delivered output: “your appointment with [PROVIDER] on [DATE].” Audit record documents redaction with specific PII types detected.

A.2.2 Financial Services: Investment Research Assistant

Output constraints applied:

Scenario: Borderline advice. Analyst asks for sector analysis. Model produces market commentary that edges toward a buy recommendation. Financial advice classifier scores 0.68 (below 0.70 threshold). Decision: BELOW_THRESHOLD ($\neq \tau$). This is a tuning decision —

Constraint	Type	Tier	Action on Violation
Account number pattern detection	O_d	Deterministic	Block + redact
Financial advice classifier ($\tau = 0.70$)	O_c	Classifier	Flag (Standard)
Forward-looking statement detector ($\tau = 0.75$)	O_c	Classifier	Flag + append disclaimer
Data source citation check	O_d	Deterministic	Block + retry

the threshold represents the organization’s risk appetite. The audit record captures the score, threshold version, and classifier version for post-hoc review if the output is later challenged.

Important scope note: This example assumes the tool is for internal expert use only with explicit policy walls preventing retail exposure. If the tool’s outputs could reach retail investors, the workload must be reclassified as Critical with synchronous blocking on high financial-advice scores. FINRA/SEC scrutiny applies regardless of the intended audience if the tool is accessible beyond qualified institutional users.

Scenario: Missing citation. Model produces quantitative claims without citing data sources. Citation check (O_d) fails. Decision: BLOCK + retry with modified prompt reinforcing O(data_source_citation). Retry succeeds — model produces the same analysis with source citations. Second evaluation: PASS. Audit record documents both attempts.

A.2.3 Energy: Grid Operations Decision Support

Output constraints applied:

Constraint	Type	Tier	Action on Violation
Operational command detection	O_d	Deterministic	Block + escalate
Recommendation confidence scorer ($\tau = 0.60$)	O_c	Classifier	Flag if low confidence
NERC CIP compliance notice check	O_d	Deterministic	Block + retry

Scenario: Model generates operational command. Despite EPG’s F(operational_commands_without_human_confirmation), model outputs “Execute load shedding on circuit 7.” Deterministic command detection catches the imperative structure. Decision: BLOCK + escalation to human operator. Output is NOT delivered. Audit record documents the attempted command, the block, and the escalation. This is a safety-critical catch — ROC prevents a potential grid operations violation that EPG’s prompt constraint failed to prevent at the model behavior level.

B Meeting Regulators

Parts I and II built the framework and its three control layers. This final chapter turns the framework outward: it maps Clad’s controls to the regulatory frameworks and industry standards an enterprise in a regulated sector must answer to — NIST AI RMF, the EU AI Act, ISO/IEC 42001, HIPAA, SOX, GLBA, and NERC CIP.

By the end of the chapter you will be able to:

- locate, for a given regulatory obligation, the Clad control that addresses it and the chapter where it is defined;
- distinguish what the framework provides from what remains an organizational responsibility;
- use the crosswalk to prepare audit evidence without overclaiming coverage.

A necessary disclaimer. This mapping shows where the framework provides relevant controls. It does **not** claim that deploying the framework achieves full compliance with any regulation. Compliance requires organizational controls, policies, training, and regulatory engagement beyond what any technical framework provides. The coverage ratings reflect architectural design intent; as noted in the Preface, the framework is operationally unverified, and effectiveness must be confirmed empirically.

B.1 NIST AI Risk Management Framework (AI RMF 1.0)

The NIST AI RMF organizes AI risk management into four functions: Govern, Map, Measure, Manage. The governance solution addresses elements of each.

B.1.1 Govern Function

B.1.2 Map Function

B.1.3 Measure Function

B.1.4 Manage Function

NIST AI RMF	Solution Component	Coverage Notes	
GOV-1: Policies for AI risk management	EPG §3-§4 (constraint hierarchy, RBAC)	Strong	Hierarchical constraints implement AI-specific policy. Dual control and cross-domain review enforce policy governance.
GOV-2: Accountability structures	EPG §4 (RBAC, dual control), §7 (cross-domain body)	Strong	Role-based authorship with orthogonal approval. Cross-domain governance body provides oversight.
GOV-3: Workforce diversity and training	MDR §7.1 (staffing, runbook)	Partial	MDR defines staffing model and runbook requirement. Broader workforce training is organizational, not framework-provided.
GOV-4: Organizational commitments	Meta-framework §2 (threat model scope)	Partial	Threat model explicitly scopes in-scope vs out-of-scope risks. Organizational commitment to governance is assumed, not enforced.
GOV-5: Processes for risk management	EPG §5 (decomposition), §6 (conflict resolution), ROC §5 (pipeline)	Strong	Structured processes for constraint development, conflict resolution, and output evaluation.
GOV-6: Policies for third-party AI	EPG §9.4 (constraint libraries, local verification)	Moderate	Library verification gate addresses third-party constraint adoption. Model provider governance is out of scope (meta-framework §14.2).

NIST AI RMF	Solution Component	Coverage Notes	
MAP-1: Intended purpose and context	EPG §3 (constraint model per use case)	Strong	Constraints are scoped per project with specific use-case context.
MAP-2: Interdependencies	Meta-framework §8 (interface contracts)	Strong	Formal interface contracts define component dependencies.
MAP-3: AI-specific risks	Meta-framework §2 (threat model T1-T11)	Strong	Explicit threat enumeration with in-scope/out-of-scope classification.
MAP-5: Impact assessment	EPG §5.6 (residual gap controls), ROC §6 (threat-specific controls)	Moderate	Residual gap documentation and compensating controls address impact. Full impact assessment is organizational.

NIST AI RMF	Solution Component	Coverage Notes	
MEA-1: Risk metrics	ROC §8.2 (classifier governance), MDR §3.2 (drift detection)	Moderate	Classifier precision/recall/ECE, constraint violation rates, drift metrics. Not a complete risk measurement framework.
MEA-2: AI system performance	ROC §4.2 (classifier profiles), MDR §3.3 (pattern detection)	Moderate	Classifier performance tracked. Model performance per se is out of scope.
MEA-3: Trustworthy characteristics	EPG §9.6 (limitations), ROC §9 (limitations)	Partial	Limitations sections honestly scope what is and isn't guaranteed. Fairness, explainability not directly addressed.

NIST AI RMF	Solution Component	Coverage Notes	
MAN-1: Risk treatment	EPG (prompt constraints), ROC (output controls), MDR (monitoring)	Strong	Three-component governance pipeline treats prompt, output, and monitoring risks.
MAN-2: Risk response	MDR §5 (incident response workflows)	Strong	Containment, investigation, remediation, reporting workflows defined.
MAN-3: Risk communication	MDR §5.4 (reporting), EPG §9.6 (limitations)	Moderate	Internal and regulatory reporting. Stakeholder communication is organizational.
MAN-4: Risk monitoring	MDR §3 (cross-component monitoring), §4 (alert taxonomy)	Strong	Continuous monitoring with tiered alerting and escalation.

B.2 EU AI Act (High-Risk AI Systems)

The EU AI Act requires specific obligations for providers and deployers of high-risk AI systems. The governance solution addresses several but not all.

EU AI Act Requirement	Article	Solution Component	Coverage	Notes
Risk management system	Art. 9	Meta-framework (risk model §10), EPG, ROC, MDR	Strong	Three-component system with formal risk model, residual risk documentation, and continuous monitoring.
Data and data governance	Art. 10	Out of scope	None	Data quality, training data governance, and bias testing are not addressed by this framework. Model internals are external (meta-framework §14.2).
Technical documentation	Art. 11	All chapters (meta-framework, EPG, ROC, MDR)	Strong	Comprehensive technical documentation of governance architecture, formal models, and operational procedures.
Record-keeping and logging	Art. 12	Meta-framework §9 (audit chain), EPG Section E.1.13, ROC Section E.2.4, MDR §3.1	Strong	Tamper-evident, version-stamped audit records with full chain composition across components. GIL provides interaction-level traceability.
Transparency	Art. 13	EPG §9.6 (limitations), ROC §9 (limitations), MDR §8 (limitations)	Moderate	Each component explicitly states what it guarantees and what it does not. User-facing transparency (informing users they are interacting with AI) is organizational, not framework-provided.
Human oversight	Art. 14	EPG §4.5 (break-glass), §5.6 (HITL trigger), ROC §5.4 (escalation), MDR §5 (incident response)	Moderate	Human-in-the-loop review for residual gaps, escalation for blocked outputs, human incident response. Routine human oversight of AI decisions is not mandated by the framework.
Accuracy, robustness, cybersecurity	Art. 15	ROC §4.2 (classifier governance), §8.2 (validation), MDR §3.2 (drift detection)	Partial	Classifier accuracy tracked. Robustness via red-team testing. Cybersecurity of the AI system itself is infrastructure-level, partially addressed by MDR containment. Model accuracy is out of scope.
Quality management system	Art. 17	EPG §4.4 (change control), §9.3 (change management)	Moderate	Constraint change management with CI/CD pipelines, staging, rollback. Broader QMS is organizational.
Post-market monitoring	Art. 72	MDR §3 (monitoring), §5.4 (trend reporting)	Strong	Continuous monitoring, drift detection, trend reporting.

Key gap: Art. 10 (data governance) is entirely out of scope. Organizations deploying this framework for EU AI Act compliance must complement it with a separate data governance program.

B.3 ISO/IEC 42001 (AI Management System)

ISO 42001 defines requirements for an AI management system (AIMS). The governance solution provides technical controls that underpin several AIMS requirements.

ISO 42001 Clause	Solution Component	Coverage	Notes
4. Context of the organization	Meta-framework §2 (threat model), §4 (control surfaces)	Moderate	Threat model and control surface decomposition define the AI governance context. Organizational context (stakeholders, scope boundaries) is broader.
5. Leadership	EPG §4.1 (RBAC), §7.4 (cross-domain body)	Partial	Governance roles defined. Leadership commitment is organizational.
6. Planning (risk assessment)	Meta-framework §10 (risk model), EPG §5.6 (residual gaps)	Strong	Formal risk model with attribution across surfaces. Residual gap documentation with compensating controls.
7. Support (resources, competence)	MDR §7.1 (staffing), EPG §9.5 (tooling)	Moderate	Staffing model and tooling recommendations defined. Training and awareness are organizational.
8. Operation (AI system lifecycle)	EPG §9.1 (phased adoption), §9.3 (change management), ROC §8.1 (phased adoption)	Strong	Lifecycle management from constraint development through deployment, change management, and retirement.
9. Performance evaluation	MDR §3 (monitoring), §5.4 (reporting), ROC §8.2 (classifier governance)	Strong	Continuous monitoring, trend reporting, classifier performance tracking.
10. Improvement	MDR §5.3 (remediation), EPG §5.7 (decomposition verification)	Moderate	Remediation workflows and periodic re-validation. Continual improvement process is organizational.

B.4 HIPAA (Health Insurance Portability and Accountability Act)

HIPAA compliance for AI systems requires controls across the Privacy Rule, Security Rule, and Breach Notification Rule.

HIPAA Re-requirement	Rule/Section	Solution Component	Coverage	Notes
Minimum necessary standard	Privacy §164.502(b)	EPG: O(data_minimization_instruction)	Strong	Enforceable as a prompt constraint with mechanical evaluation.
Notice of privacy practices	Privacy §164.520	EPG: O(hipaa_disclaimer_present)	Strong	Enforceable as a prompt constraint with mechanical evaluation.
PHI protection in outputs	Privacy §164.502(a)	ROC: composite PHI check (O_d + O_c), PHI-first policy (§6.3)	Moderate	Deterministic pattern matching + NER classifier with conservative threshold and post-redaction re-evaluation. Classifier false negatives are possible (ROC Theorem 4: irreducible residual risk). Best-available detection, not guaranteed prevention.
Access controls	Security §164.312(a)	EPG §4 (RBAC), Meta-framework EA2 (identity binding)	Strong	Role-based constraint authorship. Identity-bound model invocations.
Audit controls	Security §164.312(b)	Meta-framework §9 (audit chain), AI1-AI6 (integrity)	Strong	Tamper-evident, version-stamped, independently verifiable audit records.
Integrity controls	Security §164.312(c)	Meta-framework AI2-AI3 (hash chain, signing)	Strong	Cryptographic chain integrity with KMS-managed signing.
Transmission security	Security §164.312(e)	Out of scope	None	Encryption in transit is infrastructure-level, not framework-provided.
Breach notification	Breach §164.404	MDR §5.4 (regulatory reporting), forensic evidence locker	Moderate	MDR provides detection and evidence. 60-day notification compliance requires organizational reporting processes.
Risk assessment	Security §164.308(a)(4)	Meta-framework §10 (risk model), EPG §5.6 (residual gaps)	Strong	Formal risk model with residual gap documentation and compensating controls.
PHI in logs/audit records	Security §164.308(a)(4)	EPG: F(pii_logs), Meta-framework AI5 (PII minimization in audit)	Moderate	Constraint prohibits PHI in logs. AI5 requires PII minimization in audit records. Full implementation is deployment-specific.

B.5 SOX (Sarbanes-Oxley Act, Section 404)

SOX §404 requires reliable internal controls over financial reporting (ICFR). AI systems used in financial reporting must demonstrate controlled, auditable operation.

SOX Requirement	Solution Component	Coverage	Notes
Internal controls documentation	All chapters (meta-framework, EPG, ROC, MDR)	Strong	Comprehensive documentation of governance controls, formal models, and operational procedures.
Segregation of duties	EPG §4.2 (orthogonal dual control)	Moderate	Author $\neq$ approver with different reporting lines for Critical constraints. Central non-business-unit approver required. Mitigates single-actor and same-unit abuse; does NOT prevent cross-unit collusion (EPG §4.2 collusion limitation).
Change management	EPG §4.4, §9.3 (constraint change management)	Strong	Versioned, signed, dual-controlled changes with CI/CD pipelines, staging, and rollback.
Audit trail	Meta-framework §9 (audit chain), AI1-AI4	Strong	Tamper-evident, immutable audit records with cryptographic integrity and independent verification.
Management assessment of controls	MDR §5.4 (trend reporting), ROC §8.2 (classifier governance)	Moderate	Governance health metrics, classifier performance tracking, and trend reporting support management assessment. Formal control testing is organizational.
Content accuracy of AI-generated reports	Out of scope	None	The framework governs the governance process, not content accuracy. AI-generated financial content requires separate validation controls (human review, data reconciliation).

Key gap: SOX requires that AI-generated content used in financial reporting be materially accurate. This framework ensures the governance process is auditable but does not validate content correctness. Organizations must maintain human review and validation workflows for any AI output used in ICFR.

B.6 GLBA (Gramm-Leach-Bliley Act)

GLBA Requirement	Solution Component	Coverage	Notes
Safeguards Rule — information security program	Meta-framework (governance architecture), EPG, ROC, MDR	Strong	Three-component governance with formal controls, monitoring, and incident response.
Customer data protection	ROC §6.3 (PII filtering), EPG: F(pii constraints)	Strong	Prompt-level and output-level PII protection with deterministic and classifier-based checks.
Risk assessment	Meta-framework §10 (risk model)	Strong	Formal risk model with surface-level attribution.
Incident response	MDR §5 (incident response)	Strong	Containment, investigation, remediation, reporting workflows with forensic evidence locker.
Service provider oversight	EPG §9.4 (constraint libraries, local verification gate)	Moderate	Library verification requires local attestation. Model provider oversight is deployment-specific.

B.7 NERC CIP (Critical Infrastructure Protection)

NERC CIP Standard	Solution Component	Coverage Notes	
CIP-003: Security manage- ment controls	EPG §4 (RBAC, dual control), Meta-framework EA1-EA4	Strong	Role-based access, change control, enforcement architecture.
CIP-004: Person- nel and training	MDR §7.1 (staffing, runbook)	Partial	Staffing model and runbook defined. Formal personnel security screening is organizational.
CIP-005: Elec- tronic security perime- ter	Meta-framework EA1 (chokepoint enforcement)	Moderate	All AI interactions must pass through governed enforcement points. Network perimeter security is infrastructure-level.
CIP-007: System security manage- ment	EPG §9.3 (change management), MDR §3.2 (config drift)	Strong	Change management with staging and rollback. Configuration drift detection.
CIP-008: Incident reporting and response	MDR §5 (incident response), §5.4 (forensic evidence locker)	Strong	Incident detection, containment, investigation, and reporting workflows with evidence preservation.
CIP-011: Informa- tion protec- tion	ROC §6.3 (PII/PHI filtering), EPG: F(operational_ commands_ without_ confirmation)	Moderate	Output filtering for sensitive data (classifier-based, probabilistic). Safety constraints preventing unauthorized operational commands (deterministic). Cross-tenant isolation (T11) is monitoring-only, not a preventative control (ROC §6.4).

B.8 Cross-Framework Coverage Summary

Important caveat: Coverage ratings below reflect each component’s CONTRIBUTION to the regulatory requirement, not full satisfaction. Multiple components may contribute to the same requirement without collectively satisfying it. The framework guarantees audit completeness and risk reduction within its scope — not full regulatory compliance, which requires organizational controls, policies, training, and processes beyond any technical framework. Risk coverage across surfaces is modeled as additive for attribution purposes (meta-framework §10.1) but this is a simplification — real risks interact across surfaces.

B.8.1 Key Gaps Across All Frameworks

Governance Solution Component	NIST AI RMF	EU AI Act	ISO 42001	HIPAA	SOX	GLBA	NERC CIP
Meta-Framework	Govern, Map	Art. 9, 11, 12	Cl. 4, 6	§164.308, 312	Audit trail	Safeguards	CIP-003
EPG (Chapter 12)	Govern, Manage	Art. 9, 14, 17	Cl. 5, 6, 8	§164.502, 520	Controls, SoD	Data protection	CIP-003, 007
ROC (Chapter 13)	Measure, Manage	Art. 9, 15	Cl. 9	§164.502(a)	—	Data protection	CIP-011
MDR (Chapter 14)	Manage	Art. 72	Cl. 9, 10	§164.404	Assessment	Incident response	CIP-008

Gap	Affects	Mitigation
Data and training data governance	EU AI Act Art. 10, ISO 42001 Cl. 8	Requires separate data governance program
Model accuracy and fairness	NIST MEA-3, EU AI Act Art. 15	Requires model evaluation and bias testing programs
User-facing transparency	EU AI Act Art. 13	Requires organizational disclosure processes
Content accuracy for financial reporting	SOX §404	Requires human review and data reconciliation
Transmission security and encryption	HIPAA §164.312(e), GLBA	Infrastructure-level controls
Personnel security screening	NERC CIP-004, ISO 42001 Cl. 7	Organizational HR processes

B.9 Key takeaways

- The framework provides *relevant controls* against every standard surveyed, but coverage is uneven: strongest on record-keeping, risk treatment, and monitoring; partial or out of scope on data governance, model internals, and anything that is fundamentally an organizational responsibility.
- The recurring boundary is technical vs. organizational. Clad can produce the evidence a regulator asks for; it cannot substitute for the policies, training, and human processes that surround it.
- Used honestly — as a map from obligation to control to chapter — the crosswalk shortens audit preparation without overclaiming compliance.

This is where the book’s argument lands: governance that is *provable* gives an enterprise something to show a regulator. The formal model behind these claims, the worked examples, the reusable templates, and a glossary of terms and notation follow in the appendices.

C A Validation Benchmark: Corpus and Case Design

The chapter on limitations concedes that the framework’s claims are unmeasured. This appendix specifies the corpus and case design that would measure them, at the level of detail needed for someone else to build it. The sources below were checked for the properties the design requires; the design has not been run, and no results are reported anywhere in this book.

C.1 Why a real corpus rather than a synthetic one

A synthetic corpus would let the author construct both the cases and the notion of applicability they test, which is precisely the circularity that would make the results worthless. The design therefore uses public regulatory sources that supply the ground truth independently: supersession flagged by the issuing authority, version histories published by the registry, and divergent indications recorded by two regulators.

C.2 Sources

Source	Supplies	Verified property
openFDA drug label API	Current US labels: indications, dosage, population sections, with set_id , version , effective_time	Serves <i>only</i> the current version per set_id ; superseded versions are not retrievable
DailyMed SPL services	Per-label version history and superseded label text	History endpoint returns all versions with publication dates; superseded text is retrievable by version
EMA medicines register	EU authorized medicines: therapeutic indication, active substance, revision number, authorization, withdrawal, suspension dates	Register is of <i>centrally</i> authorized medicines, so coverage skews toward biologics and specialist products

The division of labor between the first two is forced rather than chosen: openFDA supplies structured current metadata and cannot supply history, so the valid-time and supersession families depend on DailyMed.

C.3 Case families

1. **Valid time.** A question whose correct answer differs by effective date, against a label with multiple versions. Ground truth is the version history.
2. **Supersession.** A question answerable from a superseded label version, where the superseding version differs. Ground truth is the registry’s own supersession record; the EU side additionally supplies withdrawal, revocation, and suspension dates.
3. **Transaction versus valid time.** Cases where the SPL effective time and the publication date differ, so that a corpus current to one is not current to the other. This family exists because the two are separate recorded fields rather than a distinction the author imposed.

4. **Population.** A question about a paediatric case answered from adult dosing or indication text. Ground truth is the label’s own age-based distinction.
5. **Indication applicability.** A question about an indication the label does not carry — off-label use. Authentic, authoritative, correctly citable evidence that does not govern the case, which is MISAPPLIED in the sense of Section 3.10.
6. **Jurisdiction.** A question answered from a US label where the EU authorization for the same active substance carries a different indication, or the reverse. Neither source governs the other’s territory.

A seventh family that may not be sourceable. Conflicting authority of equal standing requires two sources that diverge and that the institution’s precedence order does not separate. Same-substance labels from different manufacturers do diverge substantially, but the divergence appears to track prescription versus over-the-counter status — a distinction precedence *can* resolve, which makes it the resolved case rather than the contested one. Whether genuine equal-standing conflict is available within a single regulatory class is an open question, and if the data does not supply it the family should be reported as unsourceable rather than constructed.

C.4 Configurations and measures

Three configurations over the same corpus: generation without retrieval; retrieval by semantic similarity alone; and governed retrieval with applicability filtering, coverage checking, and the defeater search of Section 7. Measures as specified in Section 10: accuracy, citation faithfulness, the ungrounded assertion rate decomposed by cause, the misapplied rate, abstention rate with coverage, form divergence, and repeatability. The prediction the framework makes is specific and falsifiable: the three configurations should separate sharply on the ungrounded and misapplied rates while separating much less, or not at all, on accuracy and citation faithfulness.

C.5 Reuse conditions

US federal sources are public domain as works of the United States government. The EMA download page states no license or reuse terms, and that should be resolved before publishing derived data. Both APIs are rate-limited; the fetchers written for this design cache every response and throttle below the documented limits.

D Theorems, Lemmas, and Proofs

The meta-framework chapter states the control model: the surfaces an interaction presents, the component that governs one, and the algebra by which components compose. This appendix carries the results proved over that model, and the assumptions every one of them rests on.

They are collected here rather than in the chapter because their audience is different. A reader deciding whether to adopt the framework needs the model and the guarantees in prose. A reader building it, or auditing someone else’s build, needs the statements, the preconditions, and the proofs — and needs them in one place rather than scattered across a chapter.

Read the assumption table at the end first if you are evaluating a deployment. Every result below is conditional on it.

D.0.1 Surface Partition Theorem

Theorem 1 (Surface Completeness). *Every element of every AI interaction is assigned to exactly one control surface, and every control surface has a defined governability class.*

Proof. By Axiom 1, every interaction is described by (x, u, M, θ, o) . o' is derived from o by definition of the pipeline. By the Governance Surface Partition definition, the surfaces partition $\{p, u, m, \theta, o, o'\}$ exhaustively and disjointly. By the Governability classification, each surface has one. By Axiom 4, governability requires observability; surfaces classified as full or partial are observable; surfaces classified as external are explicitly outside the governance boundary. Therefore, no interaction element lacks surface assignment, no element is multiply-assigned, and all governance boundaries are explicit. □

D.0.2 Component Independence

Lemma 1 (Component Independence). *Given Axiom 3 preconditions P1-P3 hold: If components g_1 and g_2 govern non-overlapping surfaces, then:*

$\text{guar}(g_1)$ holds regardless of whether g_2 is deployed
 $\text{guar}(g_2)$ holds regardless of whether g_1 is deployed

Proof. By Axiom 3 (Guarantee Independence, Conditional), given P1-P3, $\text{guar}(g)$ holds in $\text{deployment}(\{g\})$ iff $\text{guar}(g)$ holds in $\text{deployment}(G')$. P1 ensures adding g_2 doesn’t degrade infrastructure for g_1 . P2 ensures g_2 doesn’t prevent g_1 from receiving its artifacts. P3 ensures no shared mutable state between g_1 and g_2 . This is a direct consequence of the conditional axiom.

If P1-P3 are violated, independence is not guaranteed and must be verified empirically for the specific deployment. □

Corollary (Graceful Degradation). *Given P1-P3, deploying a subset of governance components provides the guarantees of that subset. No guarantee is weakened by the absence of other components.*

$\forall G' \subseteq G : \forall g \in G' : \text{guar}(g) \text{ holds in } \text{deployment}(G')$

This enables phased adoption: an organization can deploy EPG (prompt governance) alone and receive its full guarantees, then add ROC (output controls) later for additional coverage.

Caveat (Soft Dependency Effectiveness). A component's *GUARANTEE* (Φ) holds without soft dependencies (R_soft). A component's *EFFECTIVENESS* may be reduced without them. Example: ROC's guarantee (evaluate every output, produce audit record) holds without EPG prompt context. But ROC's evaluation quality may be lower because it lacks the constraint context EPG would provide. The guarantee is about process completeness, not evaluation quality. Phased adoption is safe for guarantees; evaluation effectiveness improves as more components are deployed.

D.0.3 Contract Satisfaction Theorem

Theorem 2 (Contract Composability). *Given: Axiom 3 preconditions P1-P3, Axiom 4 (observability), Axiom 5 (temporal identity), Audit Linkability requirement.*

If components g_1 and g_2 each satisfy their component guarantees ($\Phi(g_1)$, $\Phi(g_2)$), and the interface contract $K(g_1, g_2)$ is satisfied, then the composed system (g_1, g_2) provides:

$\text{guar}(g_1) \wedge \text{guar}(g_2)$	- both individual guarantees hold
$\wedge \text{audit_chain}(g_1, g_2) \text{ is complete}$	- audit records compose
$\wedge \text{coverage}(g_1, g_2) = S_{g_1} \cup S_{g_2}$	- governed surface expands

Proof. $\text{guar}(g_1)$ holds by hypothesis.

$\text{guar}(g_2)$ holds by hypothesis.

By Lemma 1 (given P1-P3), guarantees are independent.

Contract satisfaction ensures handoff data is available, including

interaction_id for chain linking (Design Requirement).

Handoff includes version_manifest (Axiom 5), preserving temporal audit.

Audit integrity (§6.4, AI1-AI4) ensures records are tamper-evident.

Surface coverage is the union by definition.

□

D.0.4 Audit Completeness Theorem

Theorem 3 (Audit Chain Completeness). *Given: Axiom 3 (P1-P3), Axiom 4 (observability), Axiom 5 (temporal identity), Audit Linkability requirement, Enforcement (EA1-EA2), Audit Integrity (AI1-AI4).*

For a deployed component set $G' \subseteq G$ with satisfied interface contracts, the audit chain for any interaction i covers every governed surface, bounded by the observability frontier and enforcement boundary.

$$\forall i \in I_{\text{governed}} : \bigcup \{A_g.\text{surface} \mid A_g \in \text{chain}(i)\} = \bigcup \{S_g \mid g \in G'\}$$

where I_{governed} = interactions that pass through enforcement points (EA1) and A_g may be either a normal record or a degraded record (§7.2).

Preconditions:

- All elements in $\bigcup \{S_g \mid g \in G'\}$ are observable (Axiom 4).
- All interactions pass through enforcement points (EA1-EA2).
- Audit records are tamper-evident (AI1-AI4).

Elements beyond the observability boundary, and interactions that bypass enforcement, are not covered. Both contribute ungoverned risk that must be documented per Tier 2 classification (Axiom 1) and detected per EA3 (bypass detection).

Proof. For governed interactions (enforcement ensures they reach components): Each deployed component $g \in G'$ produces either $\text{Audit}_g(i, t)$ (normal) or $A_g_degraded(i, t)$ (failure) by

the guarantee under failure (§7.3). Each record covers surface S_g by definition. Each record includes `version_manifest` by Axiom 5. Chain linking via shared `interaction_id` is ensured by Design Requirement. Record integrity is ensured by AI1-AI4. Observability is ensured by Axiom 4 and component precondition (§5.1). Therefore, the chain covers the union of all deployed component surfaces for all governed interactions, with explicit degraded-state markers for any component failures.

□

Corollary (CISO Audit Property). *Given the premises of Theorem 3:*

For any interaction i at any time t , the audit chain provides:

- *Which constraints were in effect: $\bigcup\{C_g \mid g \in G'\}$ with $ver(c, t)$*
- *Whether each constraint was satisfied: $eval(c, artifact(i))$*
- *Who authored each constraint: $owner(c)$*
- *The artifact as it existed: verifiable via `artifact_hash`*
- *Which model version processed the interaction: $ver(m, t)$ (Axiom 5)*
- *What inference parameters were used: θ at time t (Axiom 5)*
- *What was NOT observable: `observability_note` per record (Axiom 4)*

This is the formal guarantee behind “show me exactly which rules were in effect when this AI produced that output” — with the added rigor that the system explicitly declares what it could and could not observe.

Important Distinction (Record Types in the Chain).

Audit records in `chain(i)` are one of two types with different evidentiary strength:

Evaluation Record (component-signed, §9.1). *Produced by the governance component itself during normal operation. Contains actual constraint evaluations ($eval(c, artifact(i))$). Signed by the component’s KMS key. Evidentiary value: STRONG — proves constraints were evaluated.*

Process Record (supervisor-signed, §7.4). *Produced by the Governance Supervisor on behalf of a failed component. Contains failure metadata but NO constraint evaluations. Signed by the Supervisor’s KMS key. Evidentiary value: MODERATE — proves the component failed and the failure was recorded, but does NOT prove constraints were evaluated.*

Consumers of audit records (regulators, auditors, incident responders) MUST distinguish between these types. A chain containing process records has unbroken integrity but incomplete evaluation coverage. The presence of process records should trigger investigation, not be treated as equivalent to full evaluation.

D.0.5 Monotonic Risk Reduction Lemma

Lemma 2 (Monotonic Risk Reduction). *Given: Axiom 3 preconditions P1-P3, and that all components are correctly implemented (no implementation defects that introduce new vulnerabilities).*

Deploying additional governance components cannot increase total compliance risk AS MEASURED BY $R(i)$.

$$\forall G' \subseteq G'' \subseteq G : R(i \mid G'' \text{ deployed}) \leq R(i \mid G' \text{ deployed})$$

Scope, and it is a real restriction. $R(i)$ is the probability that the DELIVERED OUTPUT violates a requirement. Governance creates compliance risk of a different kind, which this measure cannot represent: an audit chain that records prompts and outputs verbatim is a new repository of regulated data, subject to minimization, retention, access control and breach exposure in its own right. The regulatory mapping treats this as a live obligation, not a hypothetical — PHI in logs and audit records maps to Security §164.308(a)(4), and AI1-AI6 exist partly to discharge it.

So the honest statement of what governance does is not that it lowers risk monotonically. It TRADES output risk for custody risk, deliberately, and the trade is worth making because output risk is unbounded and undetectable while custody risk is bounded, located, and controllable by established means. A privacy officer is right to raise this, and the trade is the answer, not the lemma.

Proof. Let $g \in G'' \setminus \{ \} G'$ (a component in the larger set but not the smaller).

By the Risk Reduction definition, deploying g reduces R_Sg : $R_Sg(i|g) \leq R_Sg(i|\neg g)$.

By Axiom 3 (given P1-P3), deploying g does not affect the guarantees of any $g' \in G'$. Therefore, deploying g does not increase R_S_k for any $k \neq g$'s surface.

Total risk $R(i) = \sum R_S_k$. One term decreases, others are unchanged.

Therefore $R(i)$ is non-increasing.

Caveat (Implementation Risk). This lemma assumes correct implementation. In practice, deploying a new component may introduce implementation defects (bugs, misconfigs, new attack surface) that increase risk outside the formal model. Implementation risk is addressed by testing, security review, and staged rollout — not by the governance framework itself. The lemma holds for the GOVERNANCE risk model; it does not claim that total OPERATIONAL risk is monotonically non-increasing.

□

Corollary (Incremental Value). Given P1-P3:

Each component deployed provides non-negative marginal risk reduction: $\Delta R(g) = R(i \mid G' \text{ deployed}) - R(i \mid G' \cup \{g\} \text{ deployed}) \geq 0$

This formally justifies phased adoption: every component added improves the risk posture. There is no deployment order that makes things worse.

D.0.6 Residual Risk Theorem

Theorem 4 (Irreducible Residual Risk). Even with all governance components deployed, residual compliance risk is non-zero.

$$\forall G \text{ fully deployed} : R(i \mid G) > 0$$

Proof. By Axiom 2 (Non-Determinism), model output is sampled from a distribution: $o \sim M(x, u, \theta)$. For any non-degenerate distribution, $\exists o$ in the support such that o violates a

compliance requirement. Output filtering (S_delivery) operates on generated content, not on the distribution — it cannot prevent generation, only intercept. Novel violation patterns not covered by existing filters remain possible. By Axiom 4 (Observability), elements beyond the observability boundary contribute ungoverned risk that no component can reduce. Therefore, $R(i | G) > 0$ for any finite governance system. $\square$

Corollary (Risk Management, Not Elimination). *Given the premises of Theorem 4:*

The goal of the governance solution is to minimize $R(i)$ to an acceptable level and ensure that all residual risk is: (a) documented — known and acknowledged (b) monitored — detectable when it materializes (c) remediable — traceable to root cause via the audit chain (d) bounded — the observability frontier is declared, so ungoverned risk is explicitly identified, not silently ignored (Axiom 4)

D.0.7 Full Chain Audit

Theorem 5 (Full Solution Audit Completeness). *Given: Axiom 3 (P1-P3), Axiom 4, Axiom 5, enforcement (EA1-EA2), audit integrity (AI1-AI6), GIL (GIL1-GIL4).*

When all three components are deployed with satisfied interface contracts, the composed audit chain covers every Tier 1 element of every governed interaction within the observability boundary.

$\forall i \in I_{\text{governed}} :$

$\text{chain}(i) = [\text{Audit_EPG}(i, t), \text{Audit_ROC}(i, t), \text{Audit_MDR}(i, t)]$

$\text{covers: } \sigma_{\text{prompt}} \cup \sigma_{\text{output}} \cup \sigma_{\text{delivery}} \cup \sigma_{\text{input}} \cup \sigma_{\text{config}}$
 $= \{p, o, o', u, m, \theta\}$
 $= \text{all Tier 1 interaction elements}$

NOT covered by this theorem:

- *Tier 2 elements (known but unmodeled — documented as ungoverned)*
- *Tier 3 elements (unknown — undocumented residual risk)*
- *Model internals ($\gamma = \text{external}$)*
- *Ungoverned interactions that bypass enforcement (detectable via GIL)*
- *Threats T7-T11 (out of scope per §2.4)*

Proof. For governed interactions (registered in GIL, passing through EA1): g_EPG covers S_prompt = {p}. g_ROC covers S_output $\cup$ S_delivery = {o, o'}. g_MDR covers S_input $\cup$ S_config = {u, m, θ }. By Theorem 1, these surfaces are exhaustive over Tier 1 elements. By Theorem 2 (given P1-P3), the chain composes correctly. By Theorem 3, every governed surface has audit records. Model internals are documented as ungoverned (Axiom 4). Tier 2 elements are documented per Axiom 1 three-tier classification. Therefore, the chain covers all Tier 1 elements of governed interactions, and explicitly declares what it does not cover. $\square$

D.0.8 Algebraic Properties

Lemma 3 (Composition Properties). *Given components with non-overlapping surfaces ($S_{g_1} \cap S_{g_2} = \emptyset$):*

(a) Associativity:

$$(g_1 \oplus g_2) \oplus g_3 = g_1 \oplus (g_2 \oplus g_3)$$

Proof: Surface union is associative. Constraint union is associative. Audit composition via shared `interaction_id` is associative (chain order is determined by pipeline causality, not composition order).

(b) Commutativity:

$$g_1 \oplus g_2 = g_2 \oplus g_1$$

Proof: Surface union and constraint union are commutative, which gives equality of the *S*, *C* and *E* components directly. (Axiom 3 is not needed here and does not belong in this proof: it concerns preservation of Φ under composition, which is a different claim from equality of the composed tuple.) The audit component is excluded — its order is fixed by pipeline causality, not by composition.

(c) Identity: $\exists g_{\emptyset} = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$ such that $g \oplus g_{\emptyset} = g$

The null component governs nothing, produces no audit records, and does not affect any other component.

Theorem 6 (Governance Partial Commutative Monoid — Abstract Components).

Given: Axiom 3 preconditions P1-P3, and components satisfying P3’s isolation constraints.

*The abstract governance components form a commutative monoid under $\oplus$ that is PARTIAL, and the claim holds for the (*S*, *C*, *E*) components of the tuple rather than for the audit component.*

($G_{\text{abstract}}, \oplus$) satisfies: associativity, identity, commutativity, on the domain where $\oplus$ is defined.

Two qualifications, both load-bearing:

Closure fails, and $\oplus$ is partial. Non-overlap is a relation between a pair of components, not a property of one, so “components with non-overlapping surfaces” does not pick out a set closed under $\oplus$. Concretely: $g_1 \oplus g_2$ governs $S_1 \cup S_2$, so $(g_1 \oplus g_2) \oplus g_1$ has overlapping surfaces and is undefined. Formally, $\oplus$ is a total function into an error-or-result sum: $g_1 \oplus g_2$ yields either a composed component (when $S_{g_1} \cap S_{g_2} = \emptyset$) or a `ComposeError` carrying the overlapping surfaces — matching the Lean (`Except ComposeError ComponentSpec`) and Scala (`Either[ComposeError, ComponentSpec]`) implementations. This is a partial commutative monoid — a real structure, and the right one, but not a monoid.

Audit composition is not a binary operation on components. The definition of $\oplus$ sets $A = \text{compose}(A_{g_1}, A_{g_2})$ via shared `interaction_id`, and chain order is fixed by pipeline causality, which is a parameter of neither operand. Because AI2 chains each record to its predecessor, `compose(A1, A2)` and `compose(A2, A1)` differ at the byte level. Commutativity and associativity therefore hold for surface and constraint union, which is all the phased-deployment argument needs; they do not hold for the audit component, and no claim is made that they do.

APPLICATION TO CONCRETE COMPONENTS (EPG, ROC, MDR): The concrete solution components have read-only cross-surface data flows (*R_{soft}*) permitted by P3. These flows do not affect guarantee composition (Lemma 1), so the monoid properties hold for guarantees. However, the read-only flows mean that EVALUATION EFFECTIVENESS (not guarantees)

depends on deployment composition — ROC is more effective with EPG’s context than without it.

Therefore: guarantee-level composition is commutative and associative. Effectiveness-level composition is not — deployment order matters for quality, even though it does not matter for formal guarantees.

This distinction is the mathematical basis for phased deployment: guarantees are safe in any order; effectiveness improves as more components provide cross-surface context.

D.1 Syllogistic Arguments for Solution Validity

These connect the formal model to the business case using classical deductive form.

D.1.1 Syllogism 2 (Necessity of Prompt Governance)

Major Premise:

Model outputs are conditioned on prompts (Axiom 2, pipeline definition).

Minor Premise:

Ungoverned prompts have no guaranteed constraint satisfaction,
therefore no auditable compliance posture for the prompt surface.

Conclusion:

Without prompt governance, the prompt surface contributes unmanaged,
unauditable compliance risk.

Contrapositive:

If compliance risk on the prompt surface is managed and auditable,
then prompt governance (or a functional equivalent) is deployed.

D.1.2 Syllogism 3 (Necessity of Output Controls)

Major Premise:

Prompt governance cannot guarantee output compliance
(Axiom 2, Theorem 4).

Minor Premise:

Output compliance is a regulatory requirement in governed industries.

Conclusion:

Prompt governance alone is insufficient; output-level controls are
necessary to address the residual risk on the output surface.

D.1.3 Syllogism 4 (Sufficiency of the Composed Solution)

Major Premise:

The composed solution covers all observable interaction surfaces
(Theorem 5), given Axiom 3 preconditions P1-P3, enforcement
architecture EA1-EA2, and audit integrity AI1-AI4.

Minor Premise:

Each component provides deterministic, immutable, version-stamped
audit records for its surfaces (Component Guarantee, Axiom 5),
including degraded-state records during component failures (§7).

Conclusion:

The composed solution provides an auditable compliance posture
across the governed portion of the AI interaction pipeline,

within the framework’s threat model (T1-T6).

Scope of “auditable compliance posture” — what this DOES mean:

- Every governed interaction has a tamper-evident audit chain
- Every applicable constraint is evaluated with version stamps
- Every component failure is recorded with declared posture
- Observability gaps are explicitly documented

What this does NOT mean:

- Every output is compliant (Theorem 4: residual risk > 0)
- Ungoverned interactions are covered (EA3 detects, doesn’t prevent)
- Threats T7-T11 are addressed (requires ROC, MDR)
- Tier 2 and Tier 3 elements are governed (Axiom 1)
- Implementation defects are absent (Lemma 2 caveat)

D.1.4 Syllogism 5 (Observability as Governance Precondition)

Major Premise:

Governance requires evaluation of governed elements against constraints (Component Definition, §5.1).

Minor Premise:

Evaluation requires observation of the element’s state
(Axiom 4: governable $\rightarrow$ observable).

Conclusion:

Any interaction element that is not observable cannot be governed.
The governance boundary is determined by the observability boundary.

Implication:

Extending governance coverage requires extending observability first.
Deploying a governance component without ensuring observability of its surfaces produces a guarantee that is formally vacuous.

D.2 What must be true

Every theorem in this chapter is conditional. The conditions are named in twenty-two places across the chapter, which is twenty-one too many for anyone who has to sign an attestation. They are collected here.

The rightmost column is the one that matters. An assumption that fails silently is worse than one that fails loudly, because the theorems keep reading as though they hold.

Assumption	What it requires	How it fails in practice
Axiom 1 (Interaction Model)	The interaction is described by ($\mathbf{x}$, $\mathbf{u}$, $\mathbf{M}$, θ , $\mathbf{o}$)	Tier 2 elements — retrieval index state, tool behavior, routing — influence output and are not in the tuple
Axiom 2 (Non-Determinism)	Governance assumes non-singleton output support	Rarely fails; the risk is a deployment that quietly assumes determinism at temperature 0
Axiom 3 (Guarantee Independence)	P1-P3 below	See P1-P3
Axiom 4 (Observability)	Governed elements are observable	A vendor endpoint that does not return the resolved prompt, or logs it lossily

Assumption	What it requires	How it fails in practice
Axiom 5 (Temporal Identity)	Every artifact carries the version in force at interaction time	A constraint edited between prompt assembly and output delivery, stamped once
P1 (Infras- tructure Stability)	Shared storage, network and compute keep their guarantees as components are added	Loudly, under load — and this is the assumption most likely to be false in a real deployment
P2 (Pipeline Non- Interference)	No component silently drops or redirects an interaction	A retry or circuit-breaker that swallows an interaction without a record
P3 (Surface Isolation)	No shared mutable evaluation state between components	A shared cache or feature store introduced for performance
AI1 (Immutable Storage)	Records cannot be modified or deleted for the retention period	A lifecycle policy, a misconfigured Object Lock, an administrator with bucket-level rights. Silently
AI2 (Chain Integrity)	Each record hashes its predecessor	Rarely fails once built; detects modification, not deletion
AI3 (Record Signing)	Components sign via an independent KMS and hold no keys	A component with a local key, added for latency
AI4 (Independent Verification)	An external party can verify without any governance component	Verification tooling that reads through the system it is verifying
AI5 (Chain Truncation Detection)	Per-tier expected component sets are recorded and compared	Not implemented — the check has to exist, and its absence is invisible
AI6 (Retention and Jurisdiction)	Records retained per regulation, residency respected, regulated data protected	The audit store becomes the largest unreviewed PII repository in the estate
GIL1 (Pre- Component Registration)	Every governed interaction registers before processing	Registration inside a component rather than at the chokepoint
GIL2 (Independent Operation)	The GIL survives any component failure, and fails closed	A GIL that shares infrastructure with what it observes
GIL3 (Com- pleteness Verification)	GIL entries are reconcilable against chain records	Reconciliation that is possible but never run
GIL4 (GIL Integrity)	The GIL carries AI1-AI6	The GIL treated as telemetry rather than as evidence
EA1 (Chokepoint Enforcement)	No path to the model bypasses EPG; none to the user bypasses ROC	One unmanaged credential. Silently
EA2 (Identity Binding)	Every invocation binds to a governed project identity	Service accounts shared across projects
EA3 (Bypass Detection)	Ungoverned interactions are detected with probability at least a stated threshold	The threshold is never stated, so the requirement is unfalsifiable
EA4 (Constraint Authorship)	Constraint changes require dual control and audit	Break-glass used as routine

Assumption	What it requires	How it fails in practice
Lemma 2 side condition	Components are correctly implemented, introducing no new vulnerabilities	Always partly false; it is an engineering assumption, not a provable one

Three of these deserve to be read twice, because they fail silently and every audit-completeness result rests on them: **AI1**, **EA1**, and **AI5**. A deployment that cannot evidence all three does not have the guarantees this chapter proves, whatever its documentation says.

D.3 Theorems stated with the requirements they justify

These three are proved over requirements rather than over the surface or component model: the Global Interaction Log, the enforcement architecture, and the audit integrity properties respectively. The chapter states what each requirement buys; the results are here.

Requirement (Global Interaction Log — GIL). A Global Interaction Log records the existence of every AI interaction BEFORE any governance component processes it. The GIL is independent of all governance components.

Properties: GIL1 (Pre-Component Registration): Every interaction i receives an `interaction_id` that is registered in the GIL before the interaction enters the governance pipeline. This registration occurs at the enforcement chokepoint (EA1), not within any governance component.

$\forall i \in I_{\text{governed}} : \exists \text{gil_entry}(i) \text{ with timestamp } t_{\text{registered}}$
 where $t_{\text{registered}} < t_{\text{first_component_evaluation}}$

GIL2 (Independent Operation): The GIL operates independently of EPG, ROC, and MDR. A failure in any governance component does not affect GIL availability. The GIL has its own fail-closed posture: if the GIL is unavailable, no interaction proceeds.

GIL3 (Completeness Verification): For any time window $[t_1, t_2]$, it is possible to compare:

- The set of `interaction_ids` registered in the GIL
- The set of `interaction_ids` present in chain records

Any GIL entry without a corresponding chain is a “ghost” — an interaction that was initiated but not fully governed. Ghosts **MUST** be investigated and classified as either: (a) component failure (degraded record should exist) (b) enforcement bypass (security incident — T4) (c) in-flight interaction (not yet completed)

GIL4 (Integrity): The GIL is subject to the same integrity properties as audit records (AI1-AI6): immutable storage, signed entries, independent verification.

Theorem 3b (Ghost Detection). *Given GIL properties GIL1-GIL4 and Audit Integrity AI1-AI5:*

Every interaction that enters the governance pipeline is either: (a) fully governed with a complete audit chain, OR (b) partially governed with degraded records in the chain, OR (c) detectable as a “ghost” via GIL completeness verification

No interaction that enters the pipeline can be silently ungoverned.

This is deliberately narrower than “no interaction can be silently ungoverned,” which the GIL cannot support. An interaction that never reaches the enforcement chokepoint — a developer’s own API credential, a model deployed outside the governed platform — receives no GIL entry

and produces no chain record. Absent from both sets, it is invisible to GIL3's comparison, which detects interactions that entered and then vanished. Interactions that never entered are the subject of EA3 (§6), where detection is probabilistic, deployment-specific, and depends on network egress controls and credential management rather than on anything the GIL can prove.

Proof. By GIL1, every governed interaction is registered before processing.

By GIL2, registration is independent of component health.

Case (a): All components operational $\rightarrow$ full chain (Theorem 3).

Case (b): Component failure $\rightarrow$ degraded records (§7.2) in chain.

Case (c): Neither full nor degraded records exist $\rightarrow$ GIL entry without chain match $\rightarrow$ ghost detected by GIL3.

By GIL4, the GIL itself is tamper-evident.

Therefore, silent ungoverned processing is impossible given GIL availability.

Note: If the GIL itself is unavailable, GIL2 mandates fail-closed: no interactions proceed. This makes the GIL a critical-path dependency — a deliberate architectural choice that prioritizes governance completeness over availability for governed workloads.

□

Theorem 1a (Conditional Full Governability). $\text{gov}(\sigma_{\text{prompt}}) = \text{full}$ IFF enforcement architecture EA1-EA2 holds.

If EA1 is violated (interactions bypass EPG): $\text{gov}(\sigma_{\text{prompt}}) = \text{partial}$ — some prompts are governed, others are not.

If EA2 is violated (anonymous model calls exist): $\text{gov}(\sigma_{\text{prompt}}) = \text{partial}$ — ungoverned interactions exist.

Proof. $\gamma = \text{full}$ requires that ALL elements of S_{prompt} are deterministically constrained prior to execution (Definition, §4.3). Without EA1, some interactions bypass constraint evaluation. Without EA2, some interactions are unattributable to governed projects. In either case, $\exists i$ where S_{prompt} is unconstrained $\rightarrow \gamma \neq \text{full}$.

□

Theorem 3a (Tamper-Evident Audit Chain). Given properties AI2-AI4, and supposing AI1 has been defeated for a single record, any modification to any audit record in $\text{chain}(i)$ is detectable by an independent verifier.

Note on the premises: AI1 (immutable storage) is the control this theorem backstops, not one of its premises. Assuming AI1 holds makes the theorem vacuous — modification would be impossible by hypothesis. The useful claim is the one an auditor asks for: if immutability is bypassed for some record, whether by a misconfigured retention policy or an administrator with lifecycle rights, the chain and the signatures still reveal it.

Proof. Suppose record A_g_k is modified after writing. By AI3, the signature on A_g_k was computed over its original content with a key held only by the KMS; the modified record therefore fails signature verification on its own, independent of the chain. By AI2, $A_g(k+1).\text{chain_hash}$ was computed from the original A_g_k , so the modified A_g_k also breaks the chain at position $k+1$. Repairing the chain requires re-signing every subsequent record, which AI3 prevents

without KMS compromise. By AI4, an independent verifier can recompute both and detect the discrepancies without relying on any governance component. The tail record is covered: signature verification does not depend on a successor existing.

Scope — deletion is a different attack. The chain detects modification. Removing the tail of a chain leaves every remaining link valid and every remaining signature verifying, so truncation is not detected cryptographically. It is prevented by AI1 and detected by AI4's completeness check against `interaction_id` generation logs. See the Chain Truncation requirement below for the partial case.

□

E The EPG and ROC Formal Models

This appendix collects the formal models underpinning the control layers of Part IV: the deontic constraint model of Enterprise Prompt Governance (Chapter 12) and the output-evaluation model of Runtime Output Controls (Chapter 13). The chapters draw on these results without requiring the reader to work through them in full.

E.1 Prompt Governance (EPG)

This appendix contains the complete EPG formal model. It uses symbolic logic throughout; §3 gives business-language explanations of the same concepts.

E.1.1 Two Layers: Logical Core and Governance Meta-Layer

The model has two layers, and keeping them apart is what makes the results composable. The **logical core** is the satisfaction relation for a given $C^*(\text{level})$ and prompt x , the inheritance rules, and the evaluability classification with its evaluation functions. The **governance meta-layer** — **author**, **approver**, **scope**, **decomp**, **CONTRADICTION**, **shared_scope**, **precedence** — governs who may create a constraint and how it is checked.

The meta-layer does not reach into the core. It changes neither the satisfaction relation, nor the inheritance rules, nor the evaluability classification. Audit Completeness and No Evaluability Gap are therefore theorems over the core alone, while Contradiction Freedom, Scope Isolation and Decomposition Coverage are theorems over the meta-layer and hold independently of it. A deployment can change its authorship controls without disturbing either set.

The logical core uses a two-operator deontic fragment $\{O, F\}$ over atomic properties (§3.2a). P_meta is a governance annotation outside this fragment. The standard axiom $O(\phi) \rightarrow P(\phi)$ does not apply because P_meta is not a deontic operator.

E.1.2 Primitive Sets and Domains

$L = \{\text{enterprise, department, project}\}$	- governance levels
$C = C_m \uplus C_p$	- operational constraints (disjoint union)
C_s	- semantic constraints (intent records, NOT part of C , documentation only)
$X = \text{set of all prompts}$	- the governed artifacts
$A = \text{set of all agents}$	- authors/owners of constraints
$D = \text{set of all domains}$	- legal, security, data_privacy, etc.
$T = \text{totally ordered set of time points}$	- for versioning and audit
$V = \text{set of audit records}$	- evaluation evidence
$\Phi = \text{controlled vocabulary of atomic identifiers}$	- enterprise-governed property
property identifiers	

E.1.3 Ordering on Governance Levels

enterprise $\succ$ department $\succ$ project
($\succ$ denotes “governs over”)
($L, \succ$) is a strict total order.

E.1.4 Core Functions

Base functions:

$\text{level} : C \rightarrow L$ - assigns constraint to governance level
 $\text{owner} : C \rightarrow A$ - who authored the constraint
 $\text{domain} : C \rightarrow D$ - which domain the constraint belongs to
 $\text{ver} : C \times T \rightarrow C_t$ - version of constraint c active at time t

RBAC and governance meta-layer:

$\text{scope} : A \rightarrow \mathcal{P}(D)$ - domains agent may author constraints for
 $\text{approve} : C \rightarrow A$ - who approved the constraint
 $\text{emergency} : C \rightarrow \{\text{true}, \text{false}\}$ - break-glass flag

Dual control invariant:

$\forall c \in C : \neg \text{emergency}(c) \rightarrow \text{author}(c) \neq \text{approver}(c)$

Orthogonal approval (Critical tier):

$\forall c \in C : \text{risk_tier}(c) = \text{critical} \wedge \neg \text{emergency}(c)$
 $\rightarrow \text{org_unit}(\text{author}(c)) \neq \text{org_unit}(\text{approver}(c))$
 $\wedge \exists a \in \text{approvers}(c) : \text{org_unit}(a) \in \{\text{CISO}, \text{central_risk}, \text{compliance}\}$

Break-glass bounds:

$\forall c \in C : \text{emergency}(c)$
 $\rightarrow \text{tightening_only}(c) \wedge \text{ttl}(c) \leq \text{max_ttl}(\text{industry}(c))$

Cross-domain review:

$\text{cross_domain_reviewed} : C \rightarrow \{\text{true}, \text{false}\}$

Enterprise Precedence Table:

$\text{precedence} : C \times C \rightarrow \{c_1_wins, c_2_wins, \text{unresolved}\}$

E.1.5 Constraint Taxonomy

Evaluability classes:

C_m = mechanically evaluable constraints
 C_p = procedurally evidenced constraints
 $C = C_m \uplus C_p$ (operational constraints, disjoint)

Semantic constraints and decomposition mapping:

C_s = semantic constraints (intent records, NOT part of C)

$\text{decomp} : C_s \rightarrow (\mathcal{P}(C_m) \times \mathcal{P}(C_p))$

Maps each intent record to its operational decomposition.

$\models_s$: conceptual satisfaction relation for semantic constraints

Used in soundness claims only, NOT in evaluation machinery.

E.1.6 Hierarchical Inheritance

$C^*(\text{enterprise}) = C(\text{enterprise})$
 $C^*(\text{department}) = C(\text{enterprise}) \cup C(\text{department})$
 $C^*(\text{project}) = C(\text{enterprise}) \cup C(\text{department}) \cup C(\text{project})$

Generally: $C^*(l) = \bigcup \{C(l') \mid l' \succeq l\}$

Monotonicity (inviolability):

$\forall l_1, l_2 \in L : l_1 \succ l_2 \rightarrow C^*(l_1) \subseteq C^*(l_2)$

Strengthening rules:

$\forall l_1 \succ l_2 :$
 $O(\phi) \in C^*(l_1) \rightarrow O(\phi) \in C^*(l_2)$ - obligations propagate

$F(\phi) \in C^*(l_1) \rightarrow F(\phi) \in C^*(l_2)$ - prohibitions propagate
 $\neg \exists c \in C(l_2)$ that negates any $c' \in C^*(l_1)$
 Only 0 and F constraints participate in inheritance.
 P_meta annotations are NOT part of $C^*(level)$.

E.1.7 Permission Semantics

P_meta(ϕ) is a governance annotation, NOT a deontic operator.

Properties:

- NO inheritance effect
- NO evaluation effect
- NO role in CONTRADICTION detection
- For satisfaction and inheritance: equivalent to absence
- For meta-governance (audit, review triggers): NOT equivalent to absence — signals “considered”

The system is NOT permissive-by-default:

$\neg F(\phi)$ does NOT entail P_meta(ϕ) or any form of permission.

E.1.8 Lateral Authority Scoping

Authorship constraint:

$\forall c \in C : \text{domain}(c) \in \text{scope}(\text{owner}(c))$

Direct overlap:

$\text{shared_scope}(D_1, D_2) = \text{scope}(D_1) \cap \text{scope}(D_2)$

Joint authorship for shared scope:

$\forall c : \text{property}(c) \in \text{shared_scope}(D_1, D_2)$
 $\rightarrow \text{approved_by}(c, D_1) \wedge \text{approved_by}(c, D_2)$

Indirect interaction (advisory, not formal):

affects(D_1, D_2) — captures known systemic couplings

Detected via impact analysis (§7.3), NOT by scope intersection

E.1.9 Conflict Detection and Resolution

CONTRADICTION predicate (formal, complete for atomic fragment):

$\text{CONTRADICTION}(c_1, c_2) \equiv c_1 = O(\phi) \wedge c_2 = F(\phi)$
 for some atomic $\phi \in \Phi$

Completeness: pairwise check is sufficient for independent atomic properties. Joint satisfiability of the full set follows from pairwise satisfiability in this fragment.

TENSION classification (advisory, NOT formal):

Heuristic, not sound or complete.

No theorem depends on TENSION.

Enterprise Precedence Table:

$\text{precedence} : C \times C \rightarrow \{c_1_wins, c_2_wins, unresolved\}$
 $\text{default_priority} : D \rightarrow \mathbb{N}$ (domain priority ordering)

Resolution protocol:

Phase 1: authoring-time prevention (CONTRADICTION blocks, TENSION triggers review)

Phase 2: automated precedence $\rightarrow$ default priority $\rightarrow$ arbiter

Anti-circularity: arbiter resolutions must produce permanent structural fixes

- conflict_detection_results : CONTRADICTION scan outcome
- precedence_rule_applied : reference to precedence table entry (if any)
- decomposition_version : ver(decomp(c_s), t) for each c derived from C_s
- emergency_flag : emergency(c) for any break-glass constraints

E.1.14 Theorems

Theorems over the logical core:

Theorem (Audit Completeness). $\forall x \in X_{\text{executed}} : \text{audit}(x, t)$ is total over $C^*(\text{project})$
 $\wedge \text{audit}(x, t)$ is deterministic
 $\wedge \text{audit}(x, t)$ is immutable

Theorem (No Evaluability Gap). $\forall c \in C^*(\text{project}) : c \in C_m \vee c \in C_p$

Theorems over the governance meta-layer, with explicit preconditions and scope:

Theorem (Contradiction Freedom — Pairwise, Atomic Fragment). *After Phase 1 authoring review, no pair of constraints in $C^*(\text{level})$ is contradictory per the CONTRADICTION predicate over atomic properties $\phi \in \Phi$.*

Scope: complete for the atomic fragment. Multi-constraint and semantic interactions are handled by governance process (tension detection, cross-domain review), not formal guarantee.

Theorem (Scope Isolation — Direct, Under Fixed Definitions). *No agent can create, modify, or delete constraints outside their authorized domain scope, given fixed domain definitions.*

Scope: does NOT cover indirect effects (addressed by impact analysis §7.3), scope redefinition by the cross-domain governance body, or constraints authored via break-glass.

Theorem (Decomposition Coverage — Existence, Not Quality). Every semantic intent in C_s has a registered decomposition in $\text{decomp}(c_s)$.

Scope: guarantees existence, not soundness or adequacy. Soundness is approximated empirically (§5.7). Coverage quality for Critical-tier constraints is governed by residual gap controls (§5.6).

E.1.15 Limitations of the Formal Model

The formal model guarantees properties of the governance process: who can author what, contradictions are detected, audit is complete, scope is isolated. It does not guarantee properties of the governed artifacts beyond what the constraints specify — content correctness, semantic completeness, or model behavior are outside scope.

Mechanisms that are governance-process tools — tension detection, impact analysis, residual gap review, collusion detection — are operationally valuable but not mathematically proven. They are best-effort controls, not formal guarantees.

This distinction is critical for the target audience: formal guarantees are hard commitments that can be verified and audited. Governance-process tools are organizational controls that improve outcomes but cannot be reduced to logical proofs.

E.2 Runtime Output Controls (ROC)

E.2.1 Primitive Sets

O_{raw}	= set of all raw model outputs	- generated by model, before filtering
O_{del}	= set of all delivered outputs	- after ROC processing
K	= set of all classifiers	- versioned ML models for output scoring
R	= set of all deterministic rules	- pattern, blocklist, structural checks
τ	= $K \rightarrow [0, 1]$	- threshold function mapping
classifiers		to their governance thresholds

ROC also uses the shared sets from the meta-framework: I (interactions), T (time), V (audit records), Φ (property vocabulary).

Applicability

`applicable` is used in the pipeline definition and in the Output Evaluation Completeness theorem, and it carries more weight than its notation suggests. The theorem states that the audit record is *total over* `applicable(C_ROC, i)`. Left undefined, that claim is vacuous: an implementation could define `applicable` to return $\emptyset$ for every interaction and satisfy the theorem while evaluating nothing. The definition is therefore part of the guarantee, not a detail beneath it.

`scope` : $C_ROC \rightarrow (I \rightarrow \{\text{true}, \text{false}\})$ — each constraint declares the interactions it governs

`applicable` : $\mathcal{P}(C_ROC) \times I \rightarrow \mathcal{P}(C_ROC)$
`applicable(X, i) = { c  $\in$  X | scope(c)(i) }`

Resolved against the interaction’s context — the same four dimensions the grounding chapters index evidence to, named here rather than symbolized, because in this appendix ‘c’ is a constraint:

jurisdiction
 valid time (which version of the rule was in force)
 data classification / regulated form
 the requesting role

Three requirements make the definition load-bearing rather than decorative:

- **Totality.** `scope(c)` is total over I . There is no interaction for which a constraint’s applicability is undefined; a constraint either governs an interaction or it does not.
- **Recorded exclusion.** `A_ROC(i, t)` records `C_ROC \setminus \{ \}` `applicable(C_ROC, i)` together with the `scope` predicate version that excluded each constraint. A constraint deemed inapplicable is an auditable decision, not an absence. This is what closes the vacuity: narrowing applicability does not shrink the audit record, it changes what the record must justify.
- **Governed change.** `scope` is a versioned governed artifact under the same dual-control and version-stamping rules as τ (§Section E.2.6). Narrowing a constraint’s applicability is a governance action with an author, an approver, and a timestamp.

Without recorded exclusion, evaluation completeness and evaluation avoidance are indistinguishable in the audit record — the failure mode the meta-framework’s evidence-surface dis-

cussion identifies, where a control that is never invoked looks identical to a control that always passes.

E.2.2 Output Constraint Taxonomy

$C_ROC = O_d \uplus O_c \uplus O_x$ (disjoint union)

O_d = deterministic output constraints

$eval_d : O_raw \times R \rightarrow \{\text{pass}, \text{fail}\}$

Properties: total, deterministic, repeatable

O_c = classifier-based output constraints

$eval_c : O_raw \times K \rightarrow [0, 1]$

Governance decision: $eval_c(o, k) \geq \tau(k) \rightarrow \text{flag}$; else $\rightarrow \text{below_threshold}$

Quantized satisfaction: $o \models_{\tau} c$ (weaker than deterministic $o \models c$)

Properties: total, probabilistic, threshold-dependent

Threshold monotonicity invariant:

$\forall l_2 \prec l_1 : \tau(k, l_2) \leq \tau(k, l_1)$

Lower levels can only tighten (lower) thresholds, never loosen.

Threshold policy bounds (Critical tier):

$\forall k, \text{risk_tier} = \text{critical} : \tau(k) \leq \tau_max(k)$

where τ_max is policy-defined and cannot be exceeded without exceptional process.

O_x = composite output constraints

$eval_x : O_raw \times (\mathcal{P}(O_d) \times \mathcal{P}(O_c)) \times \text{logic} \rightarrow \{\text{pass}, \text{flag}, \text{block}\}$

where $\text{logic} \in \{\text{any_flag}, \text{all_flag}, \text{weighted}\}$

Properties: deterministic given component evaluations

E.2.3 Pipeline Stages

Definition (ROC Pipeline). For interaction i with raw output $o \in O_raw$:

Stage 1: $D_results = \{ (r, eval_d(o, r)) \mid r \in \text{applicable}(O_d, i) \}$ Stage 2: $C_results = \{ (k, eval_c(o, k), \tau(k)) \mid k \in \text{applicable}(O_c, i) \}$ Stage 3: $X_results = \{ (x, eval_x(o, x, D_results, C_results)) \mid x \in \text{applicable}(O_x, i) \}$

Short-circuit (Critical tier): if $\exists r \in D_results : eval_d = \text{fail} \rightarrow \text{Decision} = \text{BLOCK}$ immediately (do not await Stage 2/3)

Decision(o) =

BLOCK if $\exists r \in D_results : eval_d = \text{fail} \wedge \text{risk_tier} = \text{critical}$

$\vee \exists x \in X_results : eval_x = \text{block}$

$\vee \text{risk_tier} = \text{critical} \wedge \text{classifier_timeout}$

FLAG if $\exists k \in C_results : eval_c \geq \tau(k)$

$\vee \exists x \in X_results : eval_x = \text{flag}$

BELOW_THRESHOLD otherwise

(Note: BELOW_THRESHOLD, not PASS — for classifier-evaluated outputs)

Delivery(o) = Critical: atomic buffered — o' emitted only after Decision is terminal Standard: $o' = o$ delivered immediately; evaluation async Low: $o' = o$ delivered; evaluation sampled async

if BLOCK: $o' = \text{fallback}(o, \text{decision_reason})$

if FLAG (Critical): $o' = \text{fallback}$ or human review

if FLAG (Standard/Low): $o' = o$ (delivered, flag in audit)

E.2.4 Audit Record

```

Audit_ROC(i, t) = {
  interaction_id  : identifier(i)
  timestamp      : t
  raw_output_hash : hash(o)
  delivered_hash  : hash(o')      - differs from raw if redaction occurred
  d_evaluations  : D_results      - deterministic: (rule, pass/fail)
  c_evaluations  : C_results      - classifier: (classifier_ver, score,  $\tau$ , decision)
  x_evaluations  : X_results      - composite: (constraint, decision, logic_applied)
  pipeline_decision : {PASS, FLAG, BLOCK}
  fallback_action : action taken if BLOCK (substitution, redaction, retry, escalation)
  timing_mode    : {synchronous, asynchronous, sampled}
  epg_context    : pointer(Audit_EPG(i, t)) |  $\perp$ 
  predecessor    : pointer(Audit_EPG(i, t)) |  $\perp$ 
}

```

E.2.5 Formal Properties

Theorem (Output Evaluation Completeness — Critical and Standard Tiers). $\forall i$

$\in I_{\text{governed}}$ where $\text{timing_mode} \in \{\text{synchronous}, \text{asynchronous}\}$:

Audit_ROC(i, t) is total over applicable(C_ROC, i)

$\wedge$ Audit_ROC(i, t) is immutable

$\wedge$ Audit_ROC(i, t) contains interaction_id

Scope: applies to Critical and Standard tier interactions within ROC's enforcement boundary.

Low-tier sampled interactions have evaluation records only when sampled. Outputs that bypass ROC entirely are detected via GIL.

Theorem (Deterministic Evaluation Reliability). $\forall o, r : \text{eval_d}(o, r, t_1) = \text{eval_d}(o, r, t_2)$

given $\text{ver}(r, t_1) = \text{ver}(r, t_2)$

Deterministic evaluations produce identical results for identical inputs and rule versions. This is the same evidentiary standard as EPG's mechanical evaluation.

Property (Classifier-Based Evaluation — Scored, Not Guaranteed). eval_c is probabilistic. The following are NOT theorems:

- eval_c is deterministic (it is not — classifier non-determinism)
- eval_c has zero false negatives (it does not at any threshold)
- eval_c captures all violations (novel patterns evade classifiers)

Classifier evaluations are SCORED EVIDENCE, not formal guarantees. The audit record captures the score, threshold, and classifier version, enabling post-hoc assessment of evaluation quality.

E.2.6 Threshold Governance

Thresholds are governed artifacts:

$\forall k \in K : \tau(k)$ is subject to:

- Dual-control modification (author $\neq$ approver)
- Version tracking: $\text{ver}(\tau(k), t)$ recorded in audit

- Risk-tier defaults: $\tau_{\text{critical}} < \tau_{\text{standard}} < \tau_{\text{low}}$
(lower threshold = more conservative = more flags)

E.2.7 Limitations

The formal model provides guarantees for the deterministic tier and scored evidence for the classifier tier. It does not provide:

- Soundness or completeness guarantees for classifiers
- Coverage guarantees for novel threat patterns
- Guarantees about model behavior (only about output evaluation)

This two-tier evidentiary structure is intentional. Merging probabilistic evaluations with deterministic ones would either overstate classifier reliability or understate deterministic reliability. Keeping them separate enables honest, tier-appropriate audit.

F Templates & Classifier Specifications

Reusable artifacts for putting the framework into practice.

F.1 EPG Constraint Library Template

F.1.1 Library Structure

A constraint library package contains:

Library metadata:

- Name and version (semantic versioning: major.minor.patch)
- Target industry and regulatory basis (e.g., “HIPAA Privacy Rule §164.502-§164.514”)
- Maintainer organization and contact
- Last review date and next scheduled review
- Applicable governance levels (recommended)

Constraint entries: each containing:

- Deontic modality (O or F)
- Atomic property identifier from the controlled vocabulary
- Recommended governance level (enterprise, department, or project)
- Domain assignment
- Evaluability class (C_m or C_p)
- Rationale and regulatory reference

Pre-attested decompositions: for each semantic intent underlying the constraints:

- The intent record (C_s) with full semantic description
- The mechanical components (C_m) with implementation specifications
- The procedural components (C_p) with attestation requirements
- The decomposition attestation record (who attested, when, qualifications)

Known residual gaps: documented per constraint where soundness cannot be fully demonstrated, including compensating control recommendations.

Regression test suite: known-bad inputs that mechanical checks must catch, plus expected evaluation outcomes. This suite is the minimum test set that must pass during local verification (§9.4).

F.1.2 Library Lifecycle

Libraries follow semantic versioning:

- **Major version** (e.g., 1.0 → 2.0): new constraints added, constraints removed, or constraint semantics materially changed. Requires full re-attestation by adopting organizations.
- **Minor version** (e.g., 1.0 → 1.1): modified decompositions, updated mechanical checks, expanded test suites. Requires regression test re-execution by adopting organizations.
- **Patch version** (e.g., 1.0.0 → 1.0.1): documentation updates, test case additions, editorial corrections. No action required by adopting organizations beyond acknowledgment.

Each version requires domain expert attestation from the library maintainer. Organizations that have adopted a library version receive update notifications when new versions are released, including a changelog and impact assessment.

F.1.3 Example: HIPAA Prompt Constraints v1.0 (Skeleton)

Library metadata:

- Name: HIPAA Prompt Constraints
- Version: 1.0.0
- Industry: Healthcare / Life Sciences
- Regulatory basis: HIPAA Privacy Rule (45 CFR §164.502-§164.514), HIPAA Security Rule (45 CFR §164.302-§164.318)
- Recommended level: Enterprise

Constraint entries (8 constraints):

#	Modality	Property	Class	Regulatory Reference
1	F	phi_disclosure_in_output	C_m	§164.502(a) — Minimum Necessary
2	O	hipaa_disclaimer_present	C_m	§164.520 — Notice of Privacy Practices
3	F	pii_in_system_logs	C_m	§164.312(a) — Access Control
4	O	data_minimization_instruction	C_m	§164.502(b) — Minimum Necessary Standard
5	F	medical_diagnosis_instruction	C_m + C_p	§164.502 — Uses and Disclosures
6	O	patient_redirect_to_provider	C_m	§164.502 — Treatment Exception
7	F	phi_in_test_environments	C_m	§164.308(a)(4) — Information Access Management
8	O	breach_notification_instruction	C_p	§164.404 — Notification to Individuals

Decomposition example (Constraint #1: F(phi_disclosure_in_output)):

Intent record: “The AI system must not include protected health information in its outputs unless the disclosure is explicitly authorized under a HIPAA-permitted use or disclosure.”

Mechanical components:

- NER-based PHI detection in prompt instructions (names, dates, MRNs, SSNs, addresses, phone numbers)
- Pattern-based detection of prompt instructions that request PHI inclusion
- Required presence of explicit “Do not include patient-identifying information” instruction

Procedural components:

- Privacy officer review and attestation that the prompt’s PHI protections are adequate for the intended use case
- Annual re-attestation aligned with HIPAA risk assessment cycle

Residual gap: mechanical NER detection covers approximately 90% of standard PHI patterns. Novel identifiers, contextual PHI (e.g., “the patient in room 3”), and inference-based re-identification are not fully captured. Compensating control: 10% HITL review of outputs by trained privacy staff.

Regression test suite (partial):

- 50 known-PHI test prompts containing explicit patient identifiers → mechanical checks must flag all 50
- 20 boundary-case prompts with contextual identifiers → mechanical checks should flag at least 15
- 30 clean prompts with no PHI → mechanical checks must pass all 30 (false positive baseline)

F.2 ROC Output Classifier Specifications

F.2.1 Classifier Governance Profile Template

Every classifier deployed in ROC must have a documented governance profile:

Field	Description
Classifier ID	Unique identifier
Version	Semantic version (major.minor.patch)
Purpose	What threat/constraint it evaluates
Output	O_c or component of O_x
constraint class	
Training data summary	Source, size, composition (no PII in the profile)
Benchmark metrics	Precision, recall, F1 at configured threshold
Known limitations	Classes of inputs where performance degrades
Expected Calibration Error (ECE)	Calibration metric for the current version — required for threshold governance. A threshold is meaningless without knowing the classifier’s calibration curve.
Reliability diagram	Visual calibration curve for the current version

Field	Description
Threshold (τ)	Current governance threshold, with risk-tier variants. Only governable when ECE is known.
Threshold owner	Which governance domain owns the threshold
Validation cadence	How often retested against updated benchmarks
Last validation date	When last validated
Red-team results	Summary of last adversarial testing
Retraining trigger	Conditions that mandate classifier retraining

F.2.2 Minimum Classifier Requirements by Threat

Threat	Minimum Classifier Type	Recommended Benchmark
T7: Prompt injection	Sequence classifier on output patterns	Injection benchmark dataset (updated quarterly)
T8: Jailbreak	Safety boundary classifier	Red-team jailbreak corpus (updated quarterly)
T10: PII/PHI	NER + regex composite	HIPAA PHI pattern corpus, PCI test data
T10: Memorization	Similarity scorer against training corpus	Memorization benchmark (model-specific)
T11: Cross-tenant	Tenant-specific terminology detector	Tenant vocabulary cross-reference

F.2.3 Threshold Calibration Guidance

Threshold selection involves a precision-recall tradeoff with regulatory implications:

Conservative (low τ , ~0.5-0.7): More flags, fewer misses. Higher false-positive rate increases review burden. Appropriate for Critical-tier PHI/safety constraints where a missed violation has severe regulatory or safety consequences.

Balanced (medium τ , ~0.7-0.85): Moderate flags and misses. Appropriate for Standard-tier constraints where the cost of false positives and false negatives is roughly symmetric.

Permissive (high τ , ~0.85-0.95): Fewer flags, more misses. Lower review burden but higher miss rate. Appropriate for Low-tier or informational constraints where violations are not safety-critical.

Threshold calibration is not a one-time event. As models change, user populations shift, and adversarial techniques evolve, the operating point on the precision-recall curve shifts. Periodic recalibration (aligned with classifier validation cadence) ensures thresholds remain appropriate.

Conclusion

The book makes two connected claims and draws one practical conclusion.

The first claim is a redefinition. Hallucination, as ordinarily defined, is a false or fabricated output, and that definition is wrong in both directions. It convicts a system that faithfully reports an applicable source which happens to be wrong—an error, whose fault lies in the evidence—and it acquits a system that asserts something true while resting on nothing, which is the same defect as fabrication with a luckier outcome. Defined instead as ungrounded assertion, with no truth condition, hallucination becomes a property of the relation between an assertion and an evidential base. That is not a terminological preference. The truth-conditional definition can only be evaluated against reference answers, offline, on questions already answered; the redefinition can be evaluated from the system’s own execution trace, at the point of assertion, and therefore used as a gate.

From the redefinition, what a system can assert falls into five categories, not two. Crossing grounding against truth separates a right answer reached for a reason from a right answer reached for none, and a faithful report of a bad source from a fabrication. Requiring grounding to be applicable at a resolved context $c = \{j, t_v, f, u\}$ separates a third case from both: an assertion resting on evidence that is authentic, versioned, and authoritative, and that does not govern the decision at hand. These five categories reduce to three remedies with three different owners, and a pooled hallucination rate identifies none of them.

The failures are structural. The architecture maps token sequences to distributions over token sequences, with no stage at which two encodings of one claim are brought to a common form. The pretraining objective contains no term that is a function of truth, evidence, provenance, or context, and none minimized by declining to answer. In-context learning makes the prompt the entire control surface, and the prompt is a string. Preference optimization fits a reward to comparisons between outputs, so grounding is not visible to it, and the pressure runs toward assertion and away from abstention. And there is a statistical floor: a calibrated model must assert some rare facts incorrectly, and enterprise questions live where facts are rare.

The second claim—and it is the step that decides what to measure—is that there is no canonical form. Nothing constructs the map to be constant on classes of materially equivalent requests, and nothing in the objective penalizes divergence between them. So a passing test case establishes that one string produced an acceptable output on one occasion, and not that a materially equivalent string will, nor that the same string will later in the same session. The equivalence classes are unbounded, so coverage is not a remedy; the failures are silent, so nothing alerts; and repeatability fails too, so the guarantee does not securely cover even the tested case. Semantic normalization is not the fix, because material equivalence is context-sensitive, depends on domain ontology, and turns on distinctions that regulated language uses deliberately. Accuracy is a property of a sample, and the sample does not generalize in the direction validation needs of it. A model’s latent representation is therefore not a candidate system of record: it has no stable identity, no inspectable normalized form, no declared dependencies, and no reproducible derivation.

Grounding does not have this defect. Checking correctness requires knowing the answer, so it can happen only offline, on questions nobody needed to ask. Checking grounding requires knowing what evidence the system used and whether it governs the case—both properties of the system’s own execution, available at runtime, per assertion, on live traffic. That is the difference

between a metric and a gate, and blocking, escalating, qualifying, and logging a reconstructable basis all require a gate. Proposition 3.1 shows that neither figure identifies the other: no accuracy figure constrains the rate at which a system asserts beyond its evidence, and an intervention that raises accuracy by encouraging confident guessing lowers that rate while improving the headline number.

The practical conclusion is a requirement on architecture rather than a recommendation about metrics. The grounding rate is well defined for any system and measurable only where the evidential base is explicit at inference time—identified, versioned, scope-filtered, and resolved to a context, at claim granularity, with verification of the support relation and an abstention path that is a branch rather than a sentence. The nineteen invariants of Section 5 separate what the model will not supply from what the system must, and the division of that work across data, policy, and platform functions is itself a principal cause of failure: the operational unit of failure is the ungoverned handoff, and the grounding rate is the right cross-functional metric precisely because no single function can move it alone.

None of this is an argument against using these systems. I build them, and they work. It is an argument about what to measure, in what order, and what a passing test entitles an organization to conclude. A language model may be useful, fluent, and often accurate. But in a regulated enterprise, a proposition becomes fit for consequential reliance only when its context is resolved, its evidence is authentic and applicable, its support relation is verified, its production path is reconstructable, and its use is authorized within an accountable institutional process.

Limitations and Threats to Validity

The framework has costs and unresolved edges, and a governance argument that does not state its own is asking for a trust it has not earned. Eight, in descending order of how much they would change the conclusions.

No empirical validation of the framework’s central claims. This is a conceptual and architectural book. The failure shapes it describes are drawn from deployment experience and are offered as illustrations rather than as measurements; the prevalence claims are correspondingly weak, and where the text says a failure is common I mean that I have encountered it repeatedly, not that I have counted it. Section 10 specifies the measurements that would settle these questions, and Appendix C specifies a corpus and case design that would support them, but neither has been run. In particular: the claim that faithfulness metrics score misapplied evidence as clean is an analytic consequence of how those metrics are defined, and has not been demonstrated by running them.

The sensitivity condition is estimated, not established. Section 10.3 sets out a controlled design and is explicit that it yields a dependence estimate with error in both directions — false positives from residual form sensitivity, false negatives from redundant supporting evidence. Grounding as defined therefore has one conjunct that is, in practice, measured with noise. A program reporting a grounding rate should report the protocol and its error properties alongside, and a rate quoted without them is not comparable across systems.

The defeater search is relative to a search. By Section 3.4, NoDefeater is bounded to a governed query rather than quantified over the evidential base. That makes it checkable and it means an undetected defeater outside the search is not a violation of the definition. Whether the search is adequate is a corpus-coverage question that the definition cannot answer, and a system can satisfy the condition while a contrary authority sits in a repository nobody indexed.

Authority ordering is assumed to exist. The precedence order standing is treated as an institutional artifact. Real authority is issue-specific, jurisdiction-specific, temporally unstable, and sometimes genuinely incomparable; it is better described as a maintained decision procedure than as a partial order, and an institution that has not built one cannot evaluate the condition. The framework converts that absence into a visible missing deliverable, which is progress, but it does not supply the deliverable.

The framework is silent on how much control is enough. Section 10.7 derives thresholds by anchoring to the incumbent manual standard, which presumes such a standard exists and is defensible. Where the manual process was never held to an evidentiary standard — which is common — the anchor is missing and the framework offers no substitute.

The proportionality argument could license under-control. Tiering exists because the full control set is not economic for all traffic. The same argument can be used to classify consequential work into a lower tier to avoid the cost, and nothing in this book prevents that;

tier assignment is exactly where a governance regime is gamed. The dimensions in Section 9 constrain the argument without settling it.

Applicability may be less crisp than the formalism suggests. c is treated as resolvable into jurisdiction, valid time, material facts, and intended use. Real decisions include cases where the governing regime is itself contested, where the material facts are what is under determination, and where the intended use changes after the assertion is made. The formalism handles unresolved context by declining to answer, which is correct and is also a cost that a system facing many such cases will pay heavily.

Generalization beyond the systems in scope is unaddressed. Section 2.1 restricts the argument to autoregressive transformer language models emitting natural-language assertion. Section 5.3.2 argues the structural properties survive the developments visible now, and that argument is about current architectures rather than a claim about what is possible. An architecture that maintained content-level representations, closed its asserted set, and gated on evidence would satisfy the invariants of Section 5, and the framework would then be measuring something that holds rather than something that fails.

Related Work

This book draws on nine bodies of work and departs from each in the same way: it treats the object of interest as institutional reliance on a *particular* assertion rather than as an aggregate property of a model or a distribution. The contribution is integration and operationalization, not discovery in any of these fields, and the sections below say specifically what is taken and where the departure lies.

Attribution, faithfulness, and citation verifiability. The problem of determining whether generated text is attributable to identified sources has a careful operational treatment, and the framework of Rashkin et al. [17] is what the audit procedure of Section 10.2 adopts. The distinction between consistency with a source and correspondence with the world originates in work on summarization [13] and is established as standard by surveys of the area [7]; this book claims no priority for it. Liu et al. [12] measure how often statements in generative search output are actually supported by the citations attached to them, which is the closest existing measurement to the one proposed here.

The departure is that attribution is necessary and not sufficient. A citation can be present, accurate, and authoritative while still failing temporal validity, applicability to *c*, standing for the decision class, or authorization for reliance. Attribution establishes I12. Warrant requires I8–I19.

Data quality dimensions, evidence doctrine, and records management. Six of the seven dimensions of Section 3.13.2 are established terms of art, and Section 3.13.2 attests them individually. Three vocabularies supply them. The data quality literature has provided dimensional frameworks for three decades — believability, reputation, relevancy, completeness, currency and the rest — codified in enterprise data management bodies of knowledge and in the corresponding ISO standards. Evidence doctrine supplies authentication, relevance and materiality, the best-evidence preference, weight, sufficiency, and chain of custody. Records management and archival science supply authenticity, reliability, integrity and usability as the qualities a record must have to be relied upon; evidence-grading frameworks in medicine supply indirectness, which is applicability under another name.

The contribution is the partition rather than the vocabulary. Data quality dimensions are assessed by profiling a corpus and are properties of it. Evidence doctrine is settled by a tribunal, after the fact, about an item offered for a purpose. Records management governs the artifact, not the inference drawn from it. None of the three is evaluated at the moment a machine emits a sentence, which is the only moment at which a control can act — and reorganizing the dimensions around that moment is what separates document properties from relations to a resolved context, and shows the second group to be where retrieval-based systems fail and where corpus-level assurance cannot reach.

Provenance, lineage, and temporal data management. The requirement that a derived value carry the record of what produced it is not new; it is the subject of a substantial database literature, surveyed by Cheney et al. [5] under the why/how/where taxonomy of provenance. The valid-time and transaction-time distinction this book leans on in Section 1.1.2 is standard bitemporal modeling [22], and none of it is offered as novel.

What this book does is apply those requirements to machine-generated assertion, where they are routinely absent. The observation that an enterprise would not accept a warehouse fact without lineage, and does accept a model assertion without it, is the whole of the argument in Section 4.

Retrieval-augmented generation and grounded generation. Retrieval conditioning [10] is the architectural ancestor of Section 7, and it does improve factuality and supply provenance of a kind. Section 3.13.4 states the departure precisely rather than dismissively: standard retrieval supplies authenticity where the corpus is controlled and traceability where the event is logged, and supplies none of the four conditions that are relations to c or to the evidential base as a whole.

Uncertainty, selective prediction, and abstention. A substantial literature addresses whether models know what they know and how uncertainty can be surfaced [8, 11], and conformal methods give distribution-free coverage guarantees under stated exchangeability assumptions [1]. The abstention design of Section 10.4 borrows the risk-coverage framing directly.

The departure is the one recorded at I5 and generalized in Section 3.12: these are properties of a distribution, and reliance decisions are made about individual assertions. A calibrated probability of correctness is not an audit trail, does not identify the evidence relied upon, and does not establish applicability to the case at hand. Conformal coverage is the closest thing to a per-instance guarantee in this family, and it guarantees coverage of a label set, not that the asserted content rests on evidence that governs the decision.

Robustness, paraphrase consistency, and behavioral testing. The observation that model behavior moves under semantically inert input variation is well established: format and prompt-design sensitivity [19], paraphrase-consistency measurement [6], insensitivity to prompt meaning [24], and framing effects under user preference cues [20]. Behavioral and capability-based testing beyond aggregate accuracy is the program of Ribeiro et al. [18], and the form-invariance audit of Section 10.5 is a specialization of it.

The departure is what the instability is taken to *mean*. That literature treats paraphrase sensitivity primarily as a robustness defect to be reduced. This book treats it as the reason a validation inference fails, and therefore as an argument for measuring a different property rather than for hardening the same one.

Assurance cases and model risk management. The argument here belongs to a tradition older than the technology: structuring a claim, the evidence for it, the argument connecting them, the controls applied, and the accountable party. Safety-case and assurance-case practice in regulated engineering supplies the shape of Table 5.2, and independent model validation as a second-line function supplies the fourth row of Table 9.2.

Legal, temporal, and jurisdiction-aware reasoning. The phenomena this book indexes into c are each recognized in their own literatures — temporal validity in time-sensitive question answering, jurisdiction in legal information retrieval, population scope in clinical decision support. What appears to be absent, and what Section 3.7 claims, is their integration into a single condition on grounded assertion together with the observation that faithfulness metrics cannot express it.

Human factors and automation bias. That operators over-trust automated output, and under-verify it when it appears authoritative, is long established [16, 21]. This bears directly on the argument of Section 3.9: a fluent assertion with an accurate citation is precisely the presentation most likely to suppress the verification it requires, which is why cell (ii) is dangerous rather than merely mislabelled. It also bears on Section 7.9.4: rendering a resolved request back for confirmation is a control that automation-bias research predicts will be under-used, and it should be designed accordingly.

Causal attribution and the limits of ablation. The sensitivity condition of Definition 3.3 is a counterfactual claim about the production of an assertion, and estimating it inherits every difficulty of attribution in generative systems. Section 10.3 sets out the controlled design this book requires and is explicit that the protocol *estimates* rather than establishes dependence.

Alignment and output shaping. Preference optimization [15] and rule-conditioned training [2] shape output distributions and improve compliance behavior; Section 2.6 treats the first as part of the object of analysis rather than as a competing proposal. The departure follows from the structure of (2.5): the reward is a function of the output, so unless the training and deployment architecture explicitly represents and verifies the evidence relation, these methods cannot establish a source-specific, version-specific, context-specific evidence path for a live assertion.

Glossary

Definitions of the terms used throughout the book. Symbols and notation are collected separately in *Notation & Conventions* at the front of the book.

Attestation (procedural). Human sign-off ($\models_{\text{p}}$) used to satisfy a constraint that cannot be checked mechanically, recorded as auditable evidence. See *decomposition*.

Invariants (I1–I19). The properties an assertion-emitting system does not supply on its own (I1–I7, *behavioral*) and those it must be built to supply (I8–I19, *system*). Both sets are tabulated in the invariants chapter; they are listed here because they are referred to by number throughout.

	Name	What it requires
I1	Form invariance	Materially equivalent requests give equivalent decision-relevant outcomes
I2	Closure	Asserting $\mathbf{p}$ and $\mathbf{p} \rightarrow \mathbf{q}$ implies asserting $\mathbf{q}$
I3	Decomposition	Asserting a conjunction implies asserting its parts
I4	Consistency	Not both $\mathbf{p}$ and $\mathbf{not\ p}$
I5	Self-report fidelity	Claims about what it knows track what it asserts
I6	Correctness	What is asserted is true
I7	Repeatability	Same input, same session, same assertion
I8	Source identity	Every cited source has stable identity and established authority
I9	Temporal validity	Source version and effective date are recorded and compared to the decision date
I10	Context binding	The resolved context is explicit before assertion, or explicitly marked unresolved
I11	Applicability	Evidence is filtered on scope, not on similarity alone
I12	Traceability	Each consequential claim links to the evidence supporting it
I13	Support verification	Cited evidence is checked to support the claim at the strength asserted
I14	Reproducibility	The assertion can be re-derived from recorded inputs and versions
I15	Authorization	Reliance is permitted at the applicable tier, with duties assigned
I16	Reconstructability	The full decision path is retained for the required period
I17	Controlled failure	Unmet conditions produce qualification, refusal, or escalation
I18	Sufficiency	The evidence set is checked for coverage of every condition the claim requires
I19	Defeater search	A scoped search for contrary or undermining authority of comparable standing

I1–I7 are the ones that do *not* hold; naming them is how the book states what a system cannot be assumed to do. I8–I19 are requirements, each tied to a control.

Audit chain. The composed, hash-linked sequence of audit records produced for a single interaction by the components that processed it. Written `chain(i)`.

Break-glass. An emergency mechanism (Chapter 12, §4.5) for adding constraints under time pressure. Break-glass constraints may only *tighten*, are time-bound (TTL), and are subject to the same audit requirements as any constraint. MDR can trigger break-glass as a containment action.

Classifier evaluation (soft). ROC’s probabilistic tier ($\models_{\text{c}}$): a model scores an output for a property (e.g., “medical advice”) and the score is compared with a governed threshold. Produces *scored evidence*, distinct from the *hard evidence* of deterministic checks.

Component (governance component). A unit of governance modeled as the tuple $g = (S, C, E, A, R)$: the surfaces it governs, its constraints, its evaluation procedure, its audit output, and its requirements on other components. The three components in this book are EPG, ROC, and MDR.

Composite constraint. An output constraint (O_{x}) built by combining deterministic and classifier evaluations with explicit logic — for example, a PHI check that is satisfied only if both a pattern match and a classifier agree.

Composition algebra. The rules (Chapter 11, §12) under which components combine. Components compose under the operator $\oplus$ and preserve each other’s guarantees when deployed together.

Conflict resolution. The defined precedence (Chapter 12, §6) that determines the outcome when two constraints apply to the same artifact and disagree; upstream (enterprise) protections are never overridden by downstream additions.

Constraint. A governed requirement on an artifact, expressed as an obligation $O(\phi)$ or a prohibition $F(\phi)$ over a property ϕ .

Constraint hierarchy. EPG’s model in which constraints flow enterprise $\succ$ department $\succ$ project (Chapter 12, §3). Each level may add specificity and tighten, but never remove protections established upstream (*inheritance monotonicity*).

Control surface. A point in an AI interaction at which governance can be applied. The five surfaces are S_{prompt} , S_{input} , S_{config} , S_{output} , and S_{delivery} .

Decomposition (mandatory). The requirement (Chapter 12, §5) that a semantic constraint be broken into mechanically evaluable checks plus procedural attestation, so that it becomes auditable rather than a matter of trust.

Deontic modality. The logical category of a constraint — obligatory O , forbidden F , or permitted P — used by EPG to express what a prompt must, must not, or may do.

Deterministic evaluation (hard). ROC’s exact tier ($\models_{\text{d}}$) and EPG’s mechanical checks: a rule that either matches or does not (e.g., an SSN regex). Produces *hard evidence*.

Domain isolation. The property (Chapter 12, §7) that a department or project cannot weaken another domain’s protections; lateral authority is bounded to a domain’s own scope.

Dual control. A separation-of-duties requirement (Chapter 12, §4) that the author of a constraint is not its approver, with orthogonal reporting lines for critical constraints.

Enforcement architecture (EA1–EA4). The conditions under which a surface is *fully* governable — chokepoint enforcement (EA1), identity binding (EA2), and related requirements. Without them, a component’s guarantees apply only to interactions that voluntarily pass through it.

EPG (Enterprise Prompt Governance). The component that governs S_{prompt} : the instructions sent to a model before inference. Component g_{EPG} . See Chapter 12.

Evaluation record vs. process record. An *evaluation record* is component-signed and carries the actual evaluation result; a *process record* is produced by the Governance Supervisor during a component failure and documents the failure without evaluation content — a weaker form of evidence.

Fail-closed / fail-open. A component’s failure posture. Fail-closed blocks the interaction when the component is unavailable; fail-open lets it proceed (fail-open-*flagged* records it as

ungoverned). Posture is assigned per risk tier and is itself a governed constraint.

Ghost interaction (ghost chain). An interaction that reached the model without being recorded by the governance pipeline. The Global Interaction Log and MDR exist to make ghosts detectable.

GIL (Global Interaction Log). An independent log that records the existence of every interaction before any component processes it, so bypassed or failed interactions remain detectable (Chapter 11, design requirement).

Governability. The degree to which a control surface can be constrained, written $\backslash\{\}gov(\backslash\{\}sigma)$: *full*, *partial*, or *external*. It bounds what any component governing that surface can promise.

Governance Supervisor. The independent element that maintains audit-chain continuity when a component fails, emitting supervisor-signed *process records*.

Hard vs. soft requirements. A component's R_hard are the other components it requires to function; R_soft are ones that improve it but are not required. All three components in this book have $R_hard = \emptyset$ — each is independently deployable.

Interaction. A single, non-agentic exchange with an AI model, modeled as the tuple (x, u, M, θ, o) — prompt, user input, model, inference configuration, and output (Chapter 11, Axiom 1).

Interface contract. The formal agreement $K(g_1, g_2)$ describing what one component provides to another (Chapter 11, §8). Contracts carry EPG context to ROC and both components' records to MDR.

Mechanical evaluation. EPG's exact tier ($\models_m$): keyword, regular-expression, and structural checkers that decide constraint satisfaction deterministically.

MDR (Monitoring, Detection & Response). The cross-cutting component that governs S_input and S_config and correlates the audit records of EPG and ROC to detect and respond to system-level issues. Component g_MDR . See Chapter 14.

Pass / Flag / Block. The three decisions of ROC's output pipeline (Chapter 13, §5): deliver as-is, deliver but record a violation, or withhold and substitute a fallback.

Property (ϕ) and property vocabulary (Φ). An atomic, named characteristic a constraint ranges over (ϕ), drawn from a controlled vocabulary (Φ) shared across components so their records compose.

Redaction / fallback. ROC's handling of a blocked or non-compliant output — removing offending content, or delivering a safe substitute response in place of the model's output.

Residual risk. The compliance risk that remains after governance is applied. By the Irreducible Residual Risk theorem (Chapter 11, Theorem 4) it is non-zero even with all components deployed; governance reduces and evidences it rather than eliminating it.

Risk tier. The classification of a workload as *Critical*, *Standard*, or *Low*, which selects evaluation timing (real-time blocking vs. asynchronous) and failure posture.

ROC (Runtime Output Controls). The component that governs S_output and $S_delivery$: model output before it reaches a user. Component g_ROC . See Chapter 13.

Tamper-evident. A property of the audit records whereby any modification is detectable, via hash-chaining and independent signing (Chapter 11, Theorem 3a).

Threat model (T1–T11). The enumerated threats (Chapter 11, §2), classified as in-scope for the meta-framework (T1–T6) or deferred to ROC and MDR (T7–T11).

Threshold (τ) and threshold monotonicity. The cutoff a classifier score is compared against. Thresholds obey the same tightening-only inheritance as constraints: a lower level may

lower a threshold (more flags) but never raise it above the level above.

Version stamp / version manifest. The recorded version of every governed element at interaction time (Chapter 11, Axiom 5); the manifest is the set of these stamps included in an audit record so “which rules were in effect” is answerable.

Bibliography

- [1] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 2023.
- [2] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint*, 2022.
- [3] Lukas Berglund, Meg Tong, Maximilian Kaufmann, Mikita Balesni, Asa Cooper Stickland, Tomasz Korbak, and Owain Evans. The reversal curse: LLMs trained on “A is B” fail to learn “B is A”. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [5] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 2009.
- [6] Yanai Elazar, Nora Kassner, Shauli Ravfogel, Abhilasha Ravichander, Eduard Hovy, Hinrich Schütze, and Yoav Goldberg. Measuring and improving consistency in pretrained language models. *Transactions of the Association for Computational Linguistics*, 2021.
- [7] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.
- [8] Saurav Kadavath, Tom Conerly, Amanda Askell, et al. Language models (mostly) know what they know. *arXiv preprint*, 2022.
- [9] Adam Tauman Kalai and Santosh S. Vempala. Calibrated language models must hallucinate. In *Proceedings of the 56th Annual ACM Symposium on Theory of Computing (STOC)*, 2024.
- [10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [11] Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022.
- [12] Nelson F. Liu, Tianyi Zhang, and Percy Liang. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- [13] Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. On faithfulness and factuality in abstractive summarization. In *Proceedings of ACL*, 2020.
- [14] Niels Mündler, Jingxuan He, Slobodan Jenko, and Martin Vechev. Self-contradictory hallucinations of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

- [15] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [16] Raja Parasuraman and Victor Riley. Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2):230–253, 1997.
- [17] Hannah Rashkin, Vitaly Nikolaev, Matthew Lamm, Lora Aroyo, Michael Collins, Dipanjan Das, Slav Petrov, Gaurav Singh Tomar, Iulia Turc, and David Reitter. Measuring attribution in natural language generation models. *Computational Linguistics*, 2023.
- [18] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of ACL*, 2020.
- [19] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design. In *International Conference on Learning Representations (ICLR)*, 2024.
- [20] Mrinank Sharma, Meg Tong, Tomasz Korbak, David Duvenaud, and Ethan Perez. Towards understanding sycophancy in language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [21] Linda J. Skitka, Kathleen L. Mosier, and Mark Burdick. Does automation bias decision-making? *International Journal of Human-Computer Studies*, 1999.
- [22] Richard T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 1999.
- [23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [24] Albert Webson and Ellie Pavlick. Do prompt-based models really understand the meaning of their prompts? In *Proceedings of NAACL-HLT*, 2022.

About the Author

David Callaghan is a data and AI engineer who builds the governance, quality, and trust infrastructure that production data systems depend on.

His work focuses on the practical problems that arise when data and AI systems must operate reliably under real-world constraints: changing requirements, sensitive information, distributed pipelines, and accountable stakeholders.

CLAD grew out of that practice. Every concept in this book was developed against real systems, real constraints, and real delivery environments.

This book is free. If you found it useful, connect with David:

LinkedIn <https://www.linkedin.com/in/mrdavidcallaghan>

ORCID <https://orcid.org/0009-0006-4855-7774>

GitHub <https://github.com/davecallaghan>

This book <https://github.com/davecallaghan/clad>

Email mr.david.callaghan@gmail.com